



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Final Year Project

Assessment Weightage & Type

40% Final Year Project - Final Report

ThreatQuill Analyzer

AY 2025 2026

Credit: 30 Credit Year Long Module

Student Name: Binni Shrestha

London Met ID: 23047391

College ID: NPO1NT4A230091

Assignment Due Date: Wednesday, April 22, 2026

Assignment Submission Date: Wednesday, April 22, 2026

Internal Supervisor: Shashwot Singh Shahi

External Supervisor: Aman Thakur

Word Count:8432

One Drive Link: [23047391_Binni Shrestha \[ThreatQuill Analyzer\]](#)

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Abstract

The growing reliance on digital infrastructure, networks, and web applications has substantially increased the level of risk of cybersecurity threats. Various vulnerability scanning tools are commonly used in organizations to detect security vulnerabilities, these include: Nmap, Nikto, OpenVAS and Nessus. Nevertheless, such tools are independent and, therefore, the scanning, analysis and management of the results remains complex, time-consuming and hard to interpret, particularly to non-experts.

This project proposes the design of ThreatQuill Analyzer, a combined vulnerability assessment platform which tries to simplify and automate the vulnerability identification and analyser. The system integrates various scanning instruments, utilizing one platform and performs scans over a centralized dashboard. It can collect, process and analyse results of scans with a backend written in Python and presents users with a graphical interface that is easy to use to monitor scan activities in real-time.

The system offers the following features: structured visualization of the results, recording of the activities, risk prioritization (based on severity), and the ability to generate reports automatically. New vulnerability reports are generated using the concise and systematic format, which is easy to understand and prioritize vulnerability reference to the users. The system also facilitates safe communications with external scanning environment via SSH.

The system is developed in the Agile way with the Dynamic Systems Development Method (DSDM) that enables continuous development and improvement through iterations. The verification and testing have shown that the system has been able to integrate numerous scanners, enhance efficiency, and decrease the manual efforts in vulnerability testing.

Overall, ThreatQuill Analyzer offers a feasible and easy to use tool to carry out vulnerability assessment and therefore, cybersecurity analysis is more accessible, efficient and effective to non-technical users.

Acknowledgement

I would like to express my sincere gratitude to my external supervisor, Mr. Aman Thakur, for his valuable guidance, support, and encouragement throughout the development of this project. His experiences, ideas and positive feedback have greatly contributed to the work direction. His constant encouragement enabled me to keep myself focused and motivated especially at difficult points in the project.

I would also like to express my sincere gratitude to my internal supervisor, Mr. Shashwot Singh Shahi, who guided, supervised, and provided me with academic support in this project. His advice and prompt feedback was instrumental in the quality of my work and making sure that the project was accomplished within the stipulated period.

Another thing that I am grateful about is that the resources, learning environment and technical support I needed to execute this project were provided by my college and members of the faculty. They have played a significant role in improving my knowledge and skills, during my course of study.

Finally, I would like to express my gratitude to my family and friends for their constant support, encouragement, and understanding. They have been motivated which has played an important role in encouraging me to accomplish this project.

Table of Contents

Chapter 1: Introduction	1
1.1 Project Description	2
1.2 Current Scenario	3
1.3 Problem Domain and Project as a Solution	4
1.3.1 Problem Domain	4
1.3.2 Project as a Solution	4
1.4 Aim and Objectives	5
1.4.1 Aim	5
1.4.2 Objectives	5
1.5 Report Structure	6
1.5.1 Background	6
1.5.2 Development	6
1.5.3 Testing and Analysis	6
1.5.4 Conclusion	6
Chapter 2: Background	7
2.1 Targeted User Type	8
2.2 Understanding the Solution	8
2.3 Similar Projects	10
2.3.1 Multi Tool Network Vulnerability Scanner	10
2.3.2 Python Based Vulnerability Scanning Framework	11
2.3.3 OpenVAS Dashboard & Reporting System	12
2.4 Comparison table	13
2.4.1 Analysis of Conclusion of Comparison	14
Chapter 3: Development	15
3.1 Considered Methodologies	16
3.1.1 Agile Methodology	16
3.1.2 Prototype Model	17
3.1.3 Waterfall Model	18
3.1.4 Spiral Model	19

3.1.5 RUP Methodology	20
3.2 Selected Methodology	21
3.2.1 Agile Methodology – DSDM	21
3.2.1 Phase of Methodology	22
1. Pre-Project Phase:	22
2. Feasibility Phase	23
3. Foundations Phase	23
4. Evolutionary Development Phase	23
5. Deployment Phase	23
6. Post-Project Phase:	23
3.3 Survey Results	24
3.3.1 Pre-survey Results	24
3.3.2 Post-survey Results	25
3.4 Requirement Analysis	26
3.4.1 Hardware Requirements	26
3.4.2 Software Requirements	26
3.5 Design	28
3.5.1 Pre Planning	28
3.5.1.1 Wireframes	28
3.5.1.2 UI Design	30
3.5.2 Diagrams	33
3.5.2.1 Logo	33
3.5.2.2 System Architecture Diagram	33
3.5.2.3 Flowchart	34
3.5.2.4 ERD	35
3.5.2.5 Use Case Diagram	36
3.5.2.6 UML Diagram	38
3.5.2.7 DFD	39
3.6 Implementation	40
3.6.1 Implementation Using DSDM Methodology	40
1. Pre-Project Phase	40

2. Feasibility Phase	40
3. Foundations Phase	42
4. Evolutionary Development Phase	42
4.1 Nmap Implementation	42
4.2 Nikto Implementation	47
4.3 Nessus Implementation.....	50
4.4 OpenVAS Implementation	58
3.6.2 Backend Development	79
3.6.3 Frontend Development.....	112
3.6.4 Sustainability	133
Chapter 4: Testing and Analysis	134
4.1 Test Plan	135
4.1.1 Unit Testing Test Plan.....	135
4.1.2 System Testing Test Plan	136
4.1.3 Security Testing Test Plan	137
4.2 Unit Testing	138
4.2.1 Test Case 1	138
4.2.2 Test Case 2	140
4.2.4 Test Case 4	142
4.2.5 Test Case 5	143
4.2.6 Test Case 6	144
4.2.7 Test Case 7	145
4.2.8 Test Case 8	146
4.2.9 Test Case 9	148
4.2.10 Test Case 10	149
4.3 System Testing	151
4.3.1 Test Case 1	151
4.3.3 Test case 3	155
4.3.4 Test Case 4	157
4.3.5 Test case 5	159
4.3.6 Test case 6	160

4.4 Security Testing	161
4.4.1 Test Case 1	161
4.4.2 Test Case 2	162
4.4.3 Test Case3	163
4.4.4 Test Case 4	164
4.4.5 Test Case 5	165
4.5 Critical Analysis.....	166
4.5.1 Strengths.....	166
4.5.2 Weaknesses.....	166
4.5.3 Opportunities.....	166
4.5.4 Threats	167
Chapter 5: Conclusion.....	168
5.1 Legal, Social and Ethical Issues	169
5.1.1 Legal Issues	169
5.1.2 Social Issues	170
5.1.3 Ethical Issues	170
5.2 Advantages	171
5.3 Limitations	171
5.4 Future Work	172
5.4.1 Cloud-Based Scanning System.....	172
5.4.2 Data Visualization Data Dashboard	172
5.4.3 Automated Scan Timetable System	173
5.4.4 Enhanced Reporting and VAPT Format	173
5.4.5 Integration of Additional Scanners.....	174
6.References.....	175
7. Appendix	183
7.1 Background	183
7.1.1 Literature Review	183
7.2 Appendix A: Pre-survey	185
7.2.1 Pre-Survey Form.....	185
7.2.2 Sample of Filled Pre-Survey Forms	189

7.3.3 Pre-Survey Result	193
7.3 Appendix B: Post-survey.....	200
7.3.1 Post-survey Form	200
7.3.2 Sample of Filled Post-Survey Forms	206
7.3.3 Post-survey Result	212
7.4 Appendix C: Codes	223
7.4.1 Backend Code.....	223
7.4.2 Frontend Development.....	257
7.5 Appendix D: Designs	278
7.5.1 Work Breakdown Structure	278
7.5.2 Gantt Chart.....	279
7.5.3 Milestone.....	280
7.5.4 Mind Map	281
7.6 Appendix E: Screenshot of the System.....	282

Table of Figures

Figure 1: Increase in global cyberattack cases	2
Figure 2: Multi Scan Overview	10
Figure 3: Python Vulnerability Scanner	11
Figure 4: OpenVas Scanning.....	12
Figure 5: Agile Methodology	16
Figure 6: Prototype Model	17
Figure 7: Waterfall Model	18
Figure 8: Spiral Model	19
Figure 9: RUP Methodology	20
Figure 10: DSDM Principles	21
Figure 11:DSDM Steps.....	22
Figure 12: Wireframe of Login Page	28
Figure 13: Wireframe of Dashboard	29
Figure 14: Wireframe of dashboard after scan	29
Figure 15: Login Page	30
Figure 16: Dashboard page.....	30
Figure 17: Dashboard after scan	31
Figure 18:Generated report with vulnerability summary	31
Figure 19:Scan summary showing vulnerability distribution	32
Figure 20: Email interface for sending final report.....	32
Figure 21: ThreatQuill Analyzer system logo	33
Figure 22: System architecture showing interaction between dashboard, backend, and scanners.....	33
Figure 23: Flowchart representing the scanning and reporting process	34
Figure 24: Entity Relationship Diagram showing data structure of the system.....	35
Figure 25:Use case diagram	36
Figure 26:Use case diagram admin part	37
Figure 27: UML diagram representing system structure and component relationships .	38
Figure 28: DFD Level 0 showing overall data flow in the system	39
Figure 29: DFD Level 1 showing detailed data flow between system components	39

Figure 30: SSH connection established and system environment prepared for scanner setup	41
Figure 31: Nmap installation verified successfully in Kali Linux environment	41
Figure 32: Nikto installation and system update completed successfully	42
Figure 33: Nmap installation verified successfully in the Kali Linux environment	43
Figure 34: Nmap scan executed on the target system to identify open ports and services	43
Figure 35: Nmap scan performed on local system to detect active ports and services ..	44
Figure 36: Scan results generated showing open ports and running services	44
Figure 37: Nmap scan results saved in XML format for further processing	45
Figure 38: Scan output stored in the system directory for backend integration	45
Figure 39: Nmap results prepared for integration into the ThreatQuill Analyzer system	46
Figure 40: Nmap scan results displayed on dashboard showing detected ports and services	46
Figure 41: Nikto scanner installed and configured for web vulnerability assessment	47
Figure 42: Nikto scan executed on target web server to identify vulnerabilities	47
Figure 43: Nikto scan results generated showing detected web vulnerabilities	48
Figure 44: Nikto scan results saved in XML format for further processing	48
Figure 45: Nikto scan results processed and prepared for system integration	48
Figure 46: Nikto scan results displayed on dashboard with detected vulnerabilities	49
Figure 47: Nessus service started and verified successfully	50
Figure 48: Python environment configured for Nessus API integration	50
Figure 49: Nessus API configured with connection details and authentication	51
Figure 50: Successful connection established with Nessus API	51
Figure 51: Nessus scan configured and initiated for target system	52
Figure 52: Nessus vulnerability scan in progress on target system	52
Figure 53: Nessus API script configured for authentication and scan execution	53
Figure 54: Nessus interface accessed showing scan management dashboard	53
Figure 55: Nessus scan initiated and processing in progress	54
Figure 56: Scan results showing detected vulnerabilities for target host	54
Figure 57: Detailed vulnerability list displayed with severity classification	55

Figure 58: Scan history showing completed and previous scan results	55
Figure 59: API requests used to retrieve Nessus scan details	56
Figure 60: Nessus scan results fetched using Python script with severity summary	56
Figure 61: Nessus scanner integrated and executed within ThreatQuill system	57
Figure 62: Nessus scan results integrated into dashboard with detected vulnerabilities	57
Figure 63: OpenVAS services started successfully for vulnerability scanning	58
Figure 64: OpenVAS setup verified ensuring all required services are configured correctly	58
Figure 65: OpenVAS vulnerability feeds synchronized for updated scan data	59
Figure 66: OpenVAS services verified and confirmed to be running successfully	59
Figure 67: System packages updated and dependencies installed for OpenVAS setup	60
Figure 68: System cleaned and updated to ensure a stable environment for deployment	60
Figure 69: Existing Docker packages removed to avoid conflicts during installation	61
Figure 70: Docker repository configured for installation of required components	61
Figure 71: Docker engine and related tools installed successfully	62
Figure 72: Docker service enabled and verified using test container execution	62
Figure 73: Greenbone Community Edition files downloaded for OpenVAS setup	63
Figure 74: OpenVAS Docker images pulled successfully from repository	64
Figure 75: OpenVAS Docker containers created and initialized successfully	64
Figure 76: OpenVAS container status showing running and healthy services	65
Figure 77: Docker container logs showing OpenVAS service initialization details	65
Figure 78: OpenVAS services restarted to resolve configuration issues	66
Figure 79: Docker containers verified after restart showing active services	66
Figure 80: OpenVAS container status showing running and healthy services	67
Figure 81: Final OpenVAS container status showing all services running successfully.	67
Figure 82: OpenVAS dashboard showing feed synchronization in progress	68
Figure 83: OpenVAS processes verified to ensure services are running correctly	68
Figure 84: Scan configuration section accessed for setting up vulnerability scans	69
Figure 85: OpenVAS dashboard indicating ongoing feed synchronization	69
Figure 86: New scan configuration created for vulnerability assessment	70

Figure 87: Target system configured for OpenVAS vulnerability scanning	70
Figure 88: Scan task created and configured for the selected target	71
Figure 89: OpenVAS task created and ready for execution.....	71
Figure 90: Scan task queued and waiting for execution	72
Figure 91: OpenVAS scan task in progress on target system	72
Figure 92: OpenVAS scan completed successfully for the target system.....	73
Figure 93: Generated report showing scan details and execution summary	73
Figure 94: Scan results displaying host information and detected services	74
Figure 95: OpenVAS scan results integrated into ThreatQuill Analyzer dashboard	74
Figure 96: Login interface allowing users to securely enter credentials	75
Figure 97: Dashboard displaying system overview	75
Figure 98: Final system execution showing integrated scan results on dashboard	76
Figure 99: Final report generated displaying vulnerability summary and analysis.....	77
Figure 100: Scan summary showing severity distribution of detected vulnerabilities	77
Figure 101: Generated report sent successfully using email functionality	78
Figure 102: Scanner controller module initialized to manage scan execution	79
Figure 103: Functions defined to coordinate execution of multiple scanner	80
Figure 104: Logic implemented to trigger Nmap, Nikto, Nessus, and OpenVAS scans.	81
Figure 105: Scan results collected from different scanners during execution.....	82
Figure 106: Combined scan results processed for dashboard display and report generation	83
Figure 107: Nmap scanner module implemented for detecting open ports and network services	87
Figure 108: Nikto scanner module implemented for identifying web vulnerabilities in target systems	90
Figure 109: Nessus scanner integrated using API for automated vulnerability scanning and result retrieval.....	92
Figure 110: OpenVAS scanner module implemented for performing vulnerability scans on target systems.....	94
Figure 111: OpenVAS scan results parsed and structured	95
Figure 112: SSH module implemented to execute scans	96

Figure 113:Database module developed to store and manage scan.....	97
Figure 114: Report generation module implemented.....	109
Figure 115: Email module implemented	111
Figure 116: Login module implemented to authenticate users	113
Figure 117: Dashboard module developed to display scan results	122
Figure 118: Report view module implemented to display detailed vulnerability reports	126
Figure 119: Summary view module implemented to display an overview of scan results	132
Figure 120: Main module implemented to initialize and run the ThreatQuill Analyzer system.....	132
Figure 121: Dashboard with empty target	139
Figure 122: Message shown to enter a target	139
Figure 123: Dashboard in idle state before scan	140
Figure 124: Invalid target entered	141
Figure 125: SSH connection established successfully	142
Figure 126: SSH connection failed due to connection error	143
Figure 127: Nmap output parsed successfully	144
Figure 128: Risk levels calculated for scanned ports	145
Figure 129: Scan execution.....	146
Figure 130: After scan Clear result implementation.....	147
Figure 131: Scan completed with results displayed	148
Figure 132: Scan Completed for report generation	149
Figure 133: Generated report showing vulnerability summary	150
Figure 134: Complete scan executed successfully	152
Figure 135: Dashboard showing real time output.....	153
Figure 136: Activity log showing real-time update	154
Figure 137: Executive Summary	156
Figure 138: Scan summary updated after completion	156
Figure 139: Nmap and Nikto scan result displayed in table.....	157
Figure 140: Nessus Scan result	158

Figure 141: OpenVAS scan result	158
Figure 142: Multiple scanners executed together	159
Figure 143: Report generated and opened successfully	160
Figure 144: Invalid login credentials rejected	161
Figure 145: Password input hidden securely	162
Figure 146: Invalid Credential	163
Figure 147: Invalid Target	164
Figure 148: Secure connection established with scanners	165
Figure 149: Pre-survey Form	188
Figure 150: Pre-survey form Filled	192
Figure 151: Weak network security impact	193
Figure 152: Network security risk experience	193
Figure 153: Use of security tools	194
Figure 154: Effectiveness of vulnerability scanning	194
Figure 155: Importance of network security	195
Figure 156: High-risk prioritization	195
Figure 157: Challenges in using security tools	196
Figure 158: Security tools reducing attacks	196
Figure 159: Frequency of vulnerability scanning	197
Figure 160: Expected outcomes of system	197
Figure 161: Preferred report format	198
Figure 162: Useful system features	198
Figure 163: Importance of risk prioritization	199
Figure 164: Suggested system improvements	199
Figure 165: Post-survey Form	205
Figure 166: Filled Post-survey Form	211
Figure 167: Filled Emails	212
Figure 168: Dashboard scan execution effectiveness	212
Figure 169: Visibility of scan progress	213
Figure 170: Activity log clarity	213
Figure 171: Presentation of scan results	214

Figure 172: Navigation across modules	214
Figure 173: Multi-scanner integration effectiveness:	215
Figure 174: System stability during scans	215
Figure 175: Scan execution performance.....	216
Figure 176: Consistency of scan results.....	216
Figure 177: Handling of duplicate findings	217
Figure 178: Accuracy of detected services and ports	217
Figure 179: Correlation of vulnerabilities with services.....	218
Figure 180: Effectiveness of risk prioritization	218
Figure 181: Understanding risk through severity levels.....	219
Figure 182: Consolidation of multi-scanner results	219
Figure 183: Clarity of vulnerability descriptions	220
Figure 184: Usefulness of scan summary	220
Figure 185: Overall report quality	221
Figure 186: Suitability for real-world use	221
Figure 187: Areas for system improvement.....	222
Figure 188: User suggestions and feedback.....	222
Figure 189: Controls and coordinates execution of all scanners	227
Figure 190: Performs network scanning and detects open ports	231
Figure 191: Scans web servers for vulnerabilities	234
Figure 192: Integrates Nessus for automated vulnerability scanning	236
Figure 193: Executes OpenVAS scans on target systems	238
Figure 194: Parses OpenVAS results into structured data	239
Figure 195: Manages secure SSH communication with Kali Linux	240
Figure 196: Stores and manages scan results in the database.....	241
Figure 197: Generates structured vulnerability reports.....	253
Figure 198: Sends generated reports via email.....	256
Figure 199: Handles user authentication and login interface.....	258
Figure 200: Displays scan results, activity logs, and system status	267
Figure 201: Shows detailed vulnerability reports to the user	271
Figure 202: Displays summary of scan results and severity levels	277

Figure 203: Initializes and runs the ThreatQuill Analyzer system	277
Figure 204: WBS	278
Figure 205: Gantt Chart.....	279
Figure 206: Project Milestone.....	280
Figure 207: Mind Map	281
Figure 208: Login Page	282
Figure 209: login after Dashboard Page.....	282
Figure 210: Dashboard after scanning completed.....	283
Figure 211: Report opened.....	283
Figure 212: Scan Summary.....	284
Figure 213: Email sender	284

Table of Table

Table 1: Comparison Table	13
Table 2: Unit Test Plan.....	135
Table 3: System Test Plan	136
Table 4: Security Test Plan	137
Table 5: Unit Testing Test Case 1	138
Table 6: Unit Testing Test Case 2	140
Table 7: Unit Testing Test Case 3	141
Table 8: Unit Testing Test Case 4	142
Table 9: Unit Testing Test Case 5	143
Table 10: Unit Testing Test Case 6	144
Table 11: Unit Testing Test Case 7	145
Table 12: Unit Testing Test Case 8	146
Table 13: Unit Testing Test Case 9	148
Table 14: Unit Testing Test Case 10	149
Table 15: System Testing Test Case 1.....	151
Table 16: System Testing Test Case 2.....	153
Table 17: System Testing Test Case 3.....	155
Table 18: System Testing Test Case 4.....	157
Table 19: System Testing Test Case 5.....	159
Table 20: System Testing Test Case 6.....	160
Table 21: Security Testing Test Case 1	161
Table 22: Security Testing Test Case 2.....	162
Table 23: Security Testing Test Case 3.....	163
Table 24: Security Testing Test Case 4.....	164
Table 25: Security Testing Test Case 5.....	165

List of Abbreviation

VAPT: Vulnerability Assessment and Penetration Testing

CVE: Common Vulnerabilities and Exposures

CVSS: Common Vulnerability Scoring System

DSDM: Dynamic Systems Development Method

GUI: Graphical User Interface

API: Application Programming Interface

SSH: Secure Shell

HTTP: Hyper Text Transfer Protocol

HTTPS: Hyper Text Transfer Protocol Secure

TCP: Transmission Control Protocol

IP: Internet Protocol

IDS: Intrusion Detection System

PCAP: Packet Capture

GVM: Greenbone Vulnerability Management

OS: Operating System

JSON: JavaScript Object Notation

XML: Extensible Markup Language

IDE: Integrated Development Environment

UI: User Interface

DB: Database

Chapter 1: Introduction

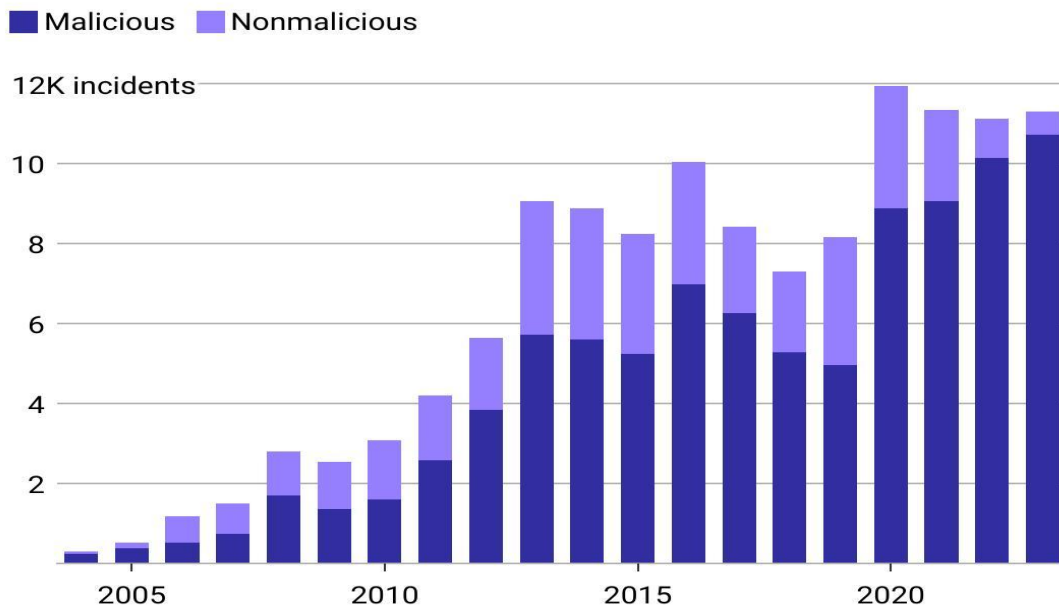
1.1 Project Description

Organizations are now depending on interconnected networks, web applications, and cloud infrastructures for their business operations. Although this improves efficiency in operations, it raises cybersecurity concerns as there are many sources from which an organization might be attacked. For instance, the IT vulnerabilities are a significant reason for cybersecurity breaches leading to risks such as loss of funds and assets (Behl & Behl, 2017).

Attackers identify vulnerable spots in an organization's infrastructure and exploit them to carry out malicious attacks. Conventional approaches used to assess vulnerabilities have been manual, time-consuming, and technical-oriented. Different tools used lead to unstructured data and outputs making it hard to make conclusive determination and action. As shown below, cyberattack cases are on an increasing trend (Sharma, 2020).

Cyberattacks are on the rise across the world

Computer security breaches spiked during the pandemic, hitting a peak of nearly 2,000 confirmed cyber events in 2020.



Data source: Advisen, International Monetary Fund

Figure 1: Increase in global cyberattack cases

ThreatQuill Analyzer is designed as a complete vulnerability assessment system in response to these issues. The system integrates the use of various software such as Nmap, Nikto, OpenVAS, and Nessus. The system streamlines the scanning process, analyses vulnerabilities through output collection and analysis in the Python backend (Alhassan, 2021)

Such a system ensures that there is a proper dashboard, live updates, and automatic generation of reports. This system simplifies the management of vulnerability assessments by ensuring that scans are managed in an integrated manner and prioritized based on risks (Scarfone, 2007).

1.2 Current Scenario

Nowadays vulnerability testing is usually conducted individually with the help of such tools as Nmap, OpenVAS, and Nikto that are popular network scanners and security assessment programs. The tools are effective, but they work independently and need to be configured, executed and results analyzed separately. In practice, security specialists may frequently be required to combine the output of many tools to gather a holistic picture of the weakness of the system. Nevertheless, it is multifaceted and time-consuming, with the tools producing various formats of the results, which must be correlated and interpreted manually (Scarfone, 2007). Also, this process is manual which adds the possibility of human error and decreases overall efficiency, particularly in large or dynamic environments.

The second significant shortcoming is that the current methods are not automated. Scanning, consolidation of results, generating reports and alerting are usually done manually thus adding more workload and reducing response time (Sommestad, et al., 2013). Moreover, the reports about vulnerabilities generated by numerous vulnerability scanners are often very technical and cannot be easily read by non-expert users, which restricts their application and accessibility (Sharma & al., 2020). Recent research also notes that the use of one tool might not be able to fully cover vulnerabilities, and there is a need to have integrated solutions that incorporate the use of multiple tools into a single system to have a more accurate and effective vulnerability assessment (Alhassan & al., 2021).

1.3 Problem Domain and Project as a Solution

1.3.1 Problem Domain

In today's rapidly evolving digital environment, organizations depend heavily on interconnected networks, web applications, and cloud-based systems. As these systems grow and complexity, the chances of security vulnerabilities also increase. Many new vulnerabilities are reported every year by the MITRE Corporation, making it difficult for organizations to continuously monitor and secure their systems. Many organizations also struggle with maintaining proper visibility of their assets, which leads to unnoticed vulnerabilities and increased security risks.

In addition, commonly used tools such as Nmap, OpenVAS, and Nikto operate independently and produce complex technical outputs. This makes it difficult for users to analyze results, correlate findings, and identify critical vulnerabilities efficiently. The lack of integration and automation increases manual effort and may result in delayed responses or overlooked security threats.

1.3.2 Project as a Solution

The ThreatQuill Analyzer addresses these challenges by providing an integrated and automated vulnerability assessment system with the following features:

- A unified platform that integrates multiple vulnerability scanning tools into a single system.
- Simultaneous execution of multiple scans to improve efficiency and coverage.
- Automated collection, processing, and analysis of scan results using a Python based backend.
- Severity-based prioritization of vulnerabilities to focus on critical risks first.
- Generation of structured and easy-to-understand reports for better decision making.
- Automated email alert system to notify users about detected vulnerabilities in real time.

Overall, the system simplifies the vulnerability assessment process, reduces manual effort, and enhances the ability to detect and respond to security threats effectively.

1.4 Aim and Objectives

1.4.1 Aim

The aim of the ThreatQuill Analyzer project is to develop an integrated and automated vulnerability assessment tool that simplifies the process of identifying, analyzing, and managing security vulnerabilities in network environments, thereby helping users improve their overall security posture and respond effectively to cyber threats.

1.4.2 Objectives

- Design a centralized system whereby various vulnerability scanning tools are incorporated in one system since most users were using different tools to scan.
- Introduce a single dashboard which will be easy to manage and view scan results, as it was challenging to integrate the output of various scanners.
- Mechanizing the scanning and analysis procedure to save time, which is an issue that has been identified by users, i.e., scanning vulnerability is very time consuming.
- Use risk-based prioritization with the categorization of vulnerabilities into levels of severity (High, Medium, Low) since many users were reported to struggle to identify and prioritize critical threats.
- Facilitate real-time scan results and activity monitoring, where users were greatly interested in having systems that will give them real-time updates and monitoring.
- Produce clear and structured reports to enhance understanding, bearing in mind that scan results were difficult to interpret by the users.
- Enhance usability of the system by having a simple and user-friendly interface, since not all users may be technologically savvy in the use of security tools.
- The pre-survey results reveal that 64.8% of the users have experience using security tools and therefore a system that accommodates both experienced and novice users are required.

1.5 Report Structure

This report is organized into different chapters to provide a clear and structured understanding of the ThreatQuill Analyzer project.

1.5.1 Background

The background section explains the overall concept of the project and highlights the importance of vulnerability assessment in network security. It also discusses existing tools and similar systems, providing a comparison to show the need for the proposed solution.

1.5.2 Development

This section describes the methodology used for developing the project. It includes planning, system design, and implementation details. Key components such as system architecture, diagrams, and technologies used in the development process are also explained.

1.5.3 Testing and Analysis

The testing and analysis section presents how the system was tested to ensure its functionality and reliability. It includes different testing methods and evaluates the performance of the system based on the project objectives.

1.5.4 Conclusion

This section summarizes the overall outcomes of the project, highlighting the achievements and effectiveness of the system. It also discusses the limitations and suggests possible improvements for future development.

Chapter 2: Background

2.1 Targeted User Type

The ThreatQuill Analyzer is designed for a range of users who require an efficient and simplified approach to vulnerability assessment. The primary users of this system include students, cybersecurity beginners, and small to medium-sized organizations that may not have advanced technical expertise or dedicated security teams.

Students and academic users can utilize the system as a learning tool to understand how vulnerability scanning works in real-world environments. By integrating tools such as Nmap, OpenVAS, and Nikto, the platform provides practical exposure to commonly used security tools in a simplified manner.

In addition, small organizations and individual users can benefit from the system as it reduces the need for complex configurations and manual analysis. The user-friendly interface, automated scanning process, and clear reporting features make it easier for non-expert users to identify and understand vulnerabilities without requiring deep technical knowledge.

Overall, the system is targeted at users who need a cost-effective, easy-to-use, and automated solution for managing network security vulnerabilities.

2.2 Understanding the Solution

The ThreatQuill Analyzer is based on several important cybersecurity concepts that help in identifying and managing system vulnerabilities. Understanding these technical terms is essential to clearly understand how the system operates.

Vulnerability assessment refers to the process of identifying security weaknesses in networks, systems, or web applications that could be exploited by attackers. It is a proactive approach used to improve system security before any real attack occurs. To support this process, network scanning is performed to detect active devices, open ports, and running services within a network. Tools such as Nmap are commonly used for this purpose.

A key part of scanning is port scanning, which identifies open ports on a system. Open ports often indicate active services, and if these services are not properly

secured, they may become entry points for attackers. In addition to this, vulnerability scanners such as OpenVAS and Nikto are used to detect known vulnerabilities in systems and web servers.

To standardize vulnerability identification, the Common Vulnerabilities and Exposures (CVE) system is used. Managed by the MITRE Corporation, CVE provides a unique identifier for each known vulnerability, making it easier to track and manage security issues.

Once vulnerabilities are identified, they are categorized into different severity levels such as High, Medium, and Low. This helps users prioritize critical issues first. The ThreatQuill Analyzer automates this entire process, including scanning, analysis, and report generation. The generated reports present findings in a clear and structured format, allowing users to understand vulnerabilities easily and take appropriate actions.

2.3 Similar Projects

2.3.1 Multi Tool Network Vulnerability Scanner

A Multi-Tool Network Vulnerability Scanner is a network vulnerability scanner that is multi-tool in nature is a combination of several security tools such as Nmap, OpenVAS as well as other scanners to automate and expand vulnerability checking across a network. These are providing a unified view of discovered services and potential risks to improve coverage and visibility of network exposures (LabEx, 2025).

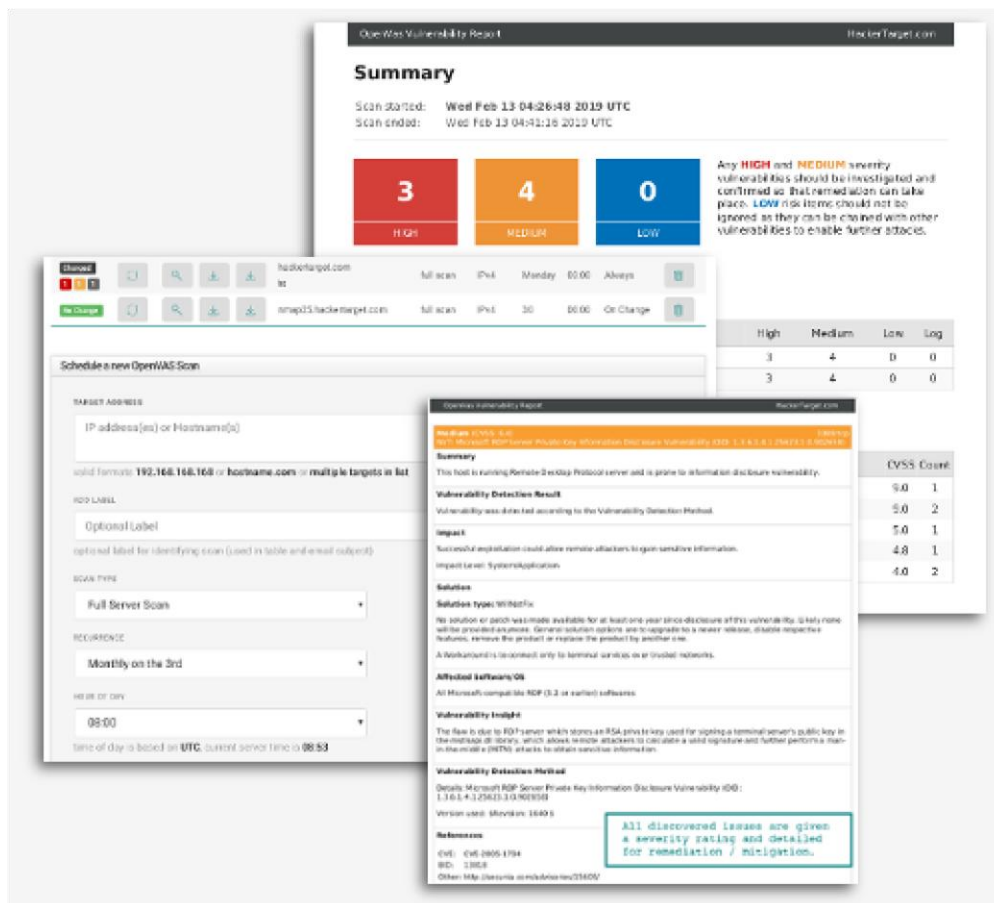


Figure 2: Multi Scan Overview

(HackerTarget, 2019)

2.3.2 Python Based Vulnerability Scanning Framework

A Python-Based Vulnerability Scanning Framework A Python-based vulnerability scanning frame is the use of Python scripts and libraries to automate the process of vulnerability detection. These tools normally use some type of Nmap to identify hosts and services and then match the findings to vulnerability information to find known security flaws. Such frameworks make security assessment easier and can prioritize the vulnerabilities with the severity information (Carvajaldz9, 2025).

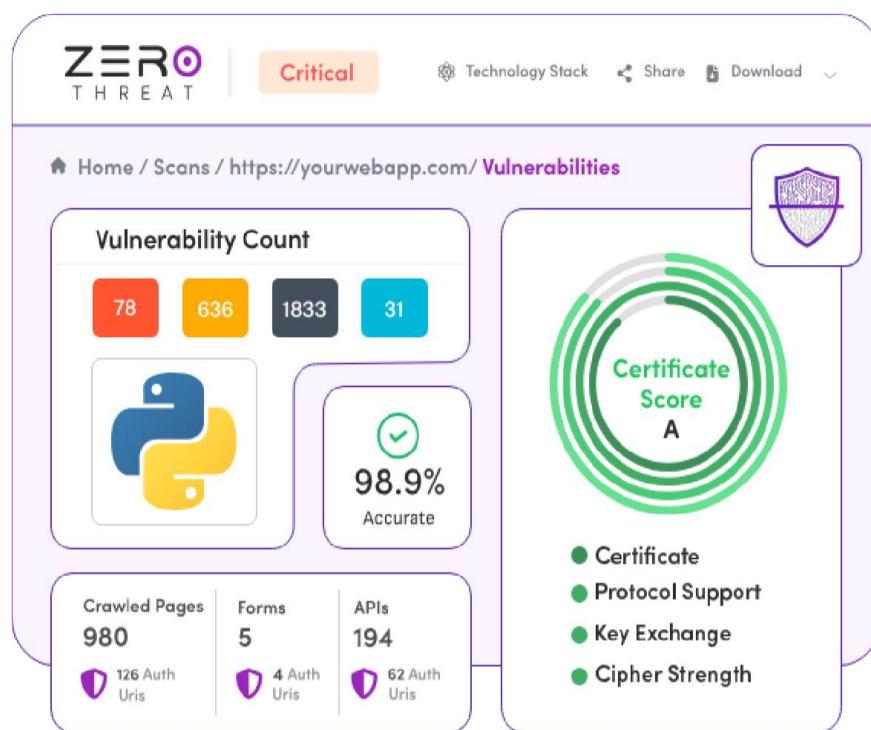


Figure 3: Python Vulnerability Scanner

(Carvajaldz9, 2025)

2.3.3 OpenVAS Dashboard & Reporting System

OpenVAS (Open Vulnerability Assessment Scanner) is an open-source vulnerability management tool that allows you to scan networks and services with a vast number of tests and writes extensive reports on the security issues identified. It is part of the GVM suite and includes dashboards to help prioritise the risks and provide visualization of the results and planning of remediation steps (Networks, 2023).

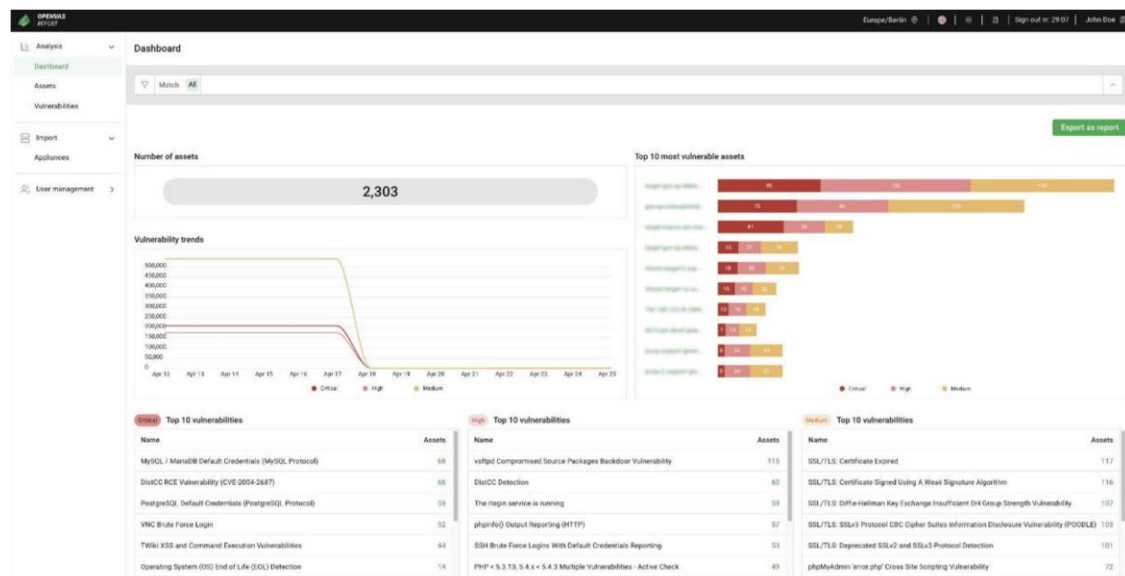


Figure 4: OpenVas Scanning

(Networks, 2023)

2.4 Comparison table

Features	Multi_Tool Network Vulnerability Scanner	Python_Based Vulnerability Scanning Framework	OpenVAS Dashboard & Reporting System	ThreatQuill Analyzer
Network vulnerability scanning	✓	✓	✓	✓
Web server vulnerability scanning	✓	✓	✓	✓
Integration of multiple scanners	✓	✗	✗	✓
Automated result parsing and analysis	✗	✓	✓	✓
Severity-based vulnerability prioritization	✗	✓	✓	✓
Professional report generation	✗	✓	✓	✓
Python-based backend processing	✗	✓	✗	✓
Automated email alerts	✗	✗	✗	✓
Beginner-friendly user interface	✗	✗	✗	✓
Cost-effective for academic use	✓	✓	✓	✓

Table 1: Comparison Table

2.4.1 Analysis of Conclusion of Comparison

The comparison of the multi-tool network vulnerability scanner, Python based vulnerability scanning framework and OpenVAS dashboard and reporting system shows that although all three intend to detect and analyze the Network Vulnerabilities, they all have different strengths. Multi-Tool Scanners excel in combining multiple results in parsers and results consolidation, Python frameworks offer flexibility, automation and customization when working with research and development incapacity. OpenVAS is focused on enterprise-level reports and prioritization of vulnerabilities based on severity. Compared to these with ThreatQuill Analyzer would have the advantages of having multi-scanner integration, Python based automation, user-friendly interfaces, automated alerts and detailed reporting together make it a more comprehensive and accessible solution for academic and practical applications.

Chapter 3: Development

3.1 Considered Methodologies

Before beginning the work of this project, various software development methodologies were analysed to establish the solution that could be used best to achieve the objectives of the project.

3.1.1 Agile Methodology

Agile methodology is an iterative approach to software development characterized by its division into smaller cycles called sprints. Agile emphasizes the need to deliver functional software periodically, obtain feedback from the client, and accommodate change (GeeksforGeeks, 2025)



Figure 5: Agile Methodology

(Nvisia, 2020)

3.1.2 Prototype Model

Prototype model is to develop an initial, uncomplicated version of the system which is meant to solicit user feedback. The model assists to sharpen the determination of the requirements of the system, because the user will have a chance to work with the prototype, and the design is subject to a few modifications, which are done iteratively, prior to building the actual system (GeeksforGeeks, 2025).

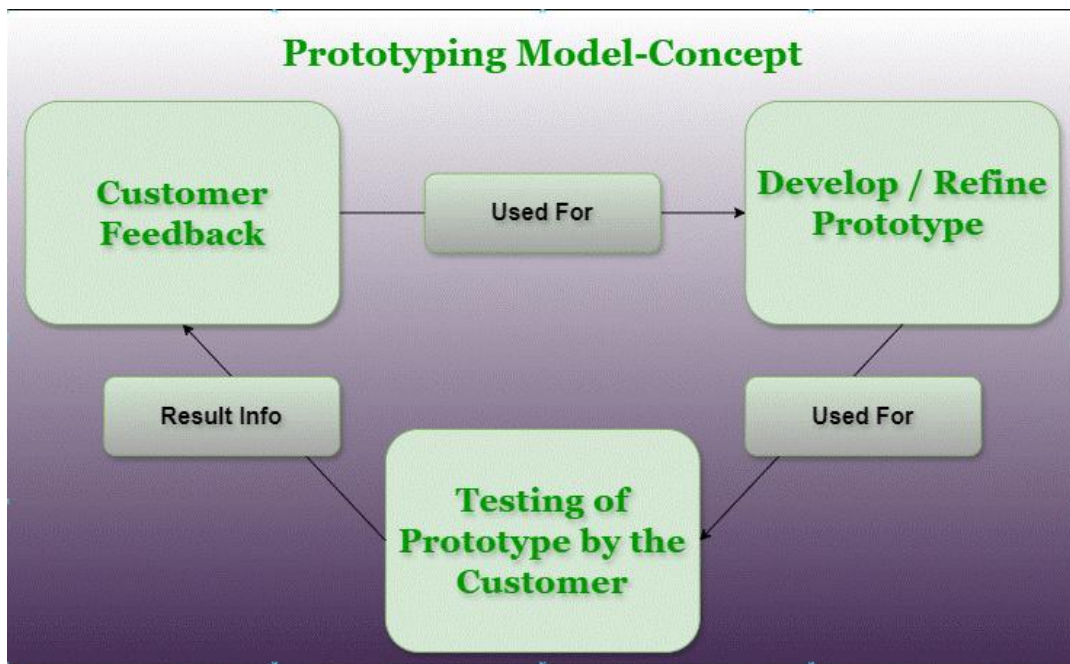


Figure 6: Prototype Model

(Geeksforgeeks, 2025)

3.1.3 Waterfall Model

Waterfall Model is a sequential or step-by-step model of software development. In this model, each stage (requirements analysis, system design, coding, testing, and deployment) must be completed in full before moving on to the next. While this structure is clear and organized, it has little room for change as soon as a phase is completed. (TutorialsCampus, 2025)

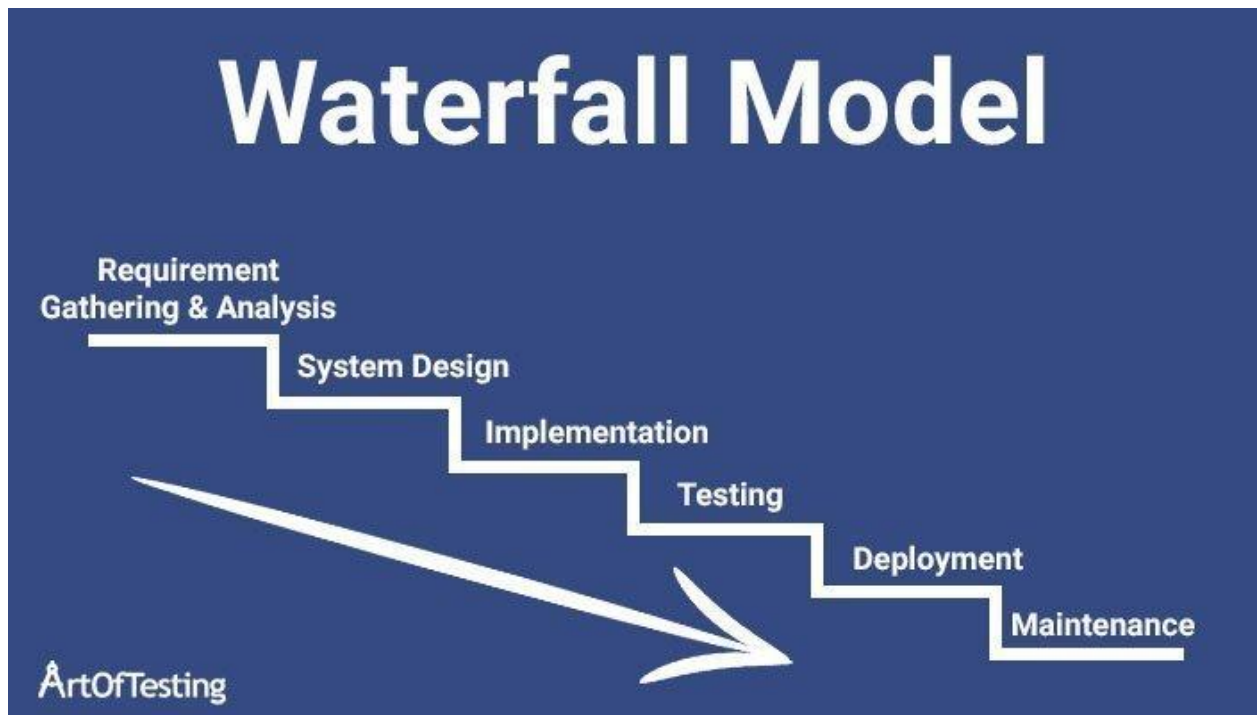


Figure 7: Waterfall Model

(ArtOfTesting, 2023)

3.1.4 Spiral Model

Spiral Model is the integration of iterative and risk management process. Each spiral involves planning, risk identification, designing, building, testing, and evaluating. This model provides an opportunity for gradual improvement through identifying possible risks within the system. It is ideal for huge and complex projects (Hashemi-Pour, 2025).

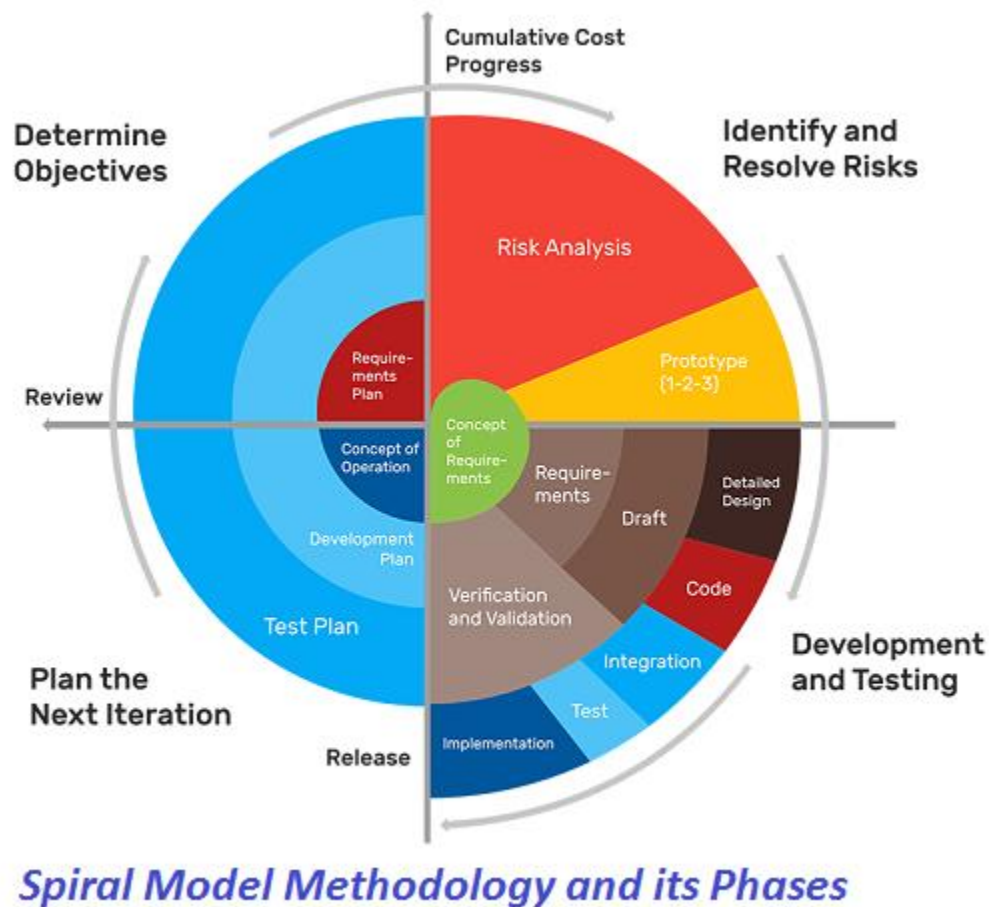


Figure 8: Spiral Model

(Sovereign, 2025)

3.1.5 RUP Methodology

RUP is an iterative and incremental software development framework, which is both use-cases driven and architecture-centric. It categorizes development into four phases: Inception, Elaboration, Construction, and Transition with several iterations in each phase. RUP focuses on early identification of risks, constant testing and incremental delivery of useful software and as such is effective for complex or big-scale projects (GeeksforGeeks, 2024).

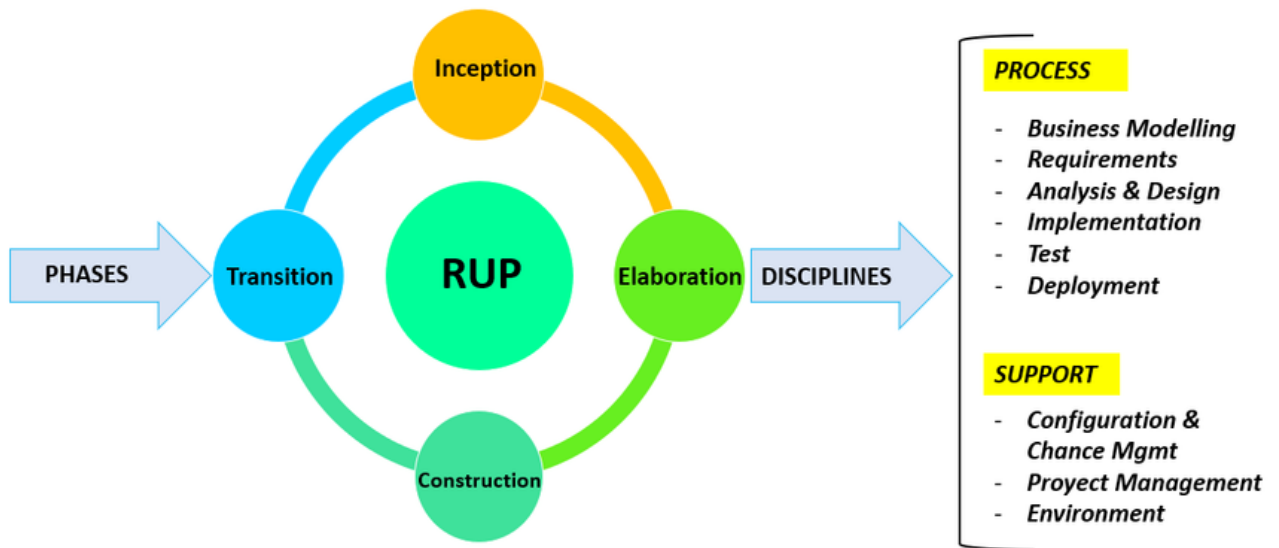


Figure 9: RUP Methodology

(ResearchGate, 2022)

3.2 Selected Methodology

Among the various software development methodologies, the **DSDM** a part of Agile methodology has been selected for this project.

3.2.1 Agile Methodology – DSDM

DSDM is an Agile software development methodology that focuses on delivering applications in an iterative and incremental way and ensures that working solutions are delivered in a short time and are flexible to change. DSDM is chosen because it allows early delivery of working components, active participation of users, and an ability to change the system in response to evolving requirements or new knowledge acquired during the development process. DSDM follows a structured lifecycle comprising six phases: Pre-Project, Feasibility, Foundations, Evolutionary Development, Deployment and Post-Project which together ensures that the project is aligned to business goals and feasible from a technical perspective and continuously improved as it is being developed (Consortium, 2025)

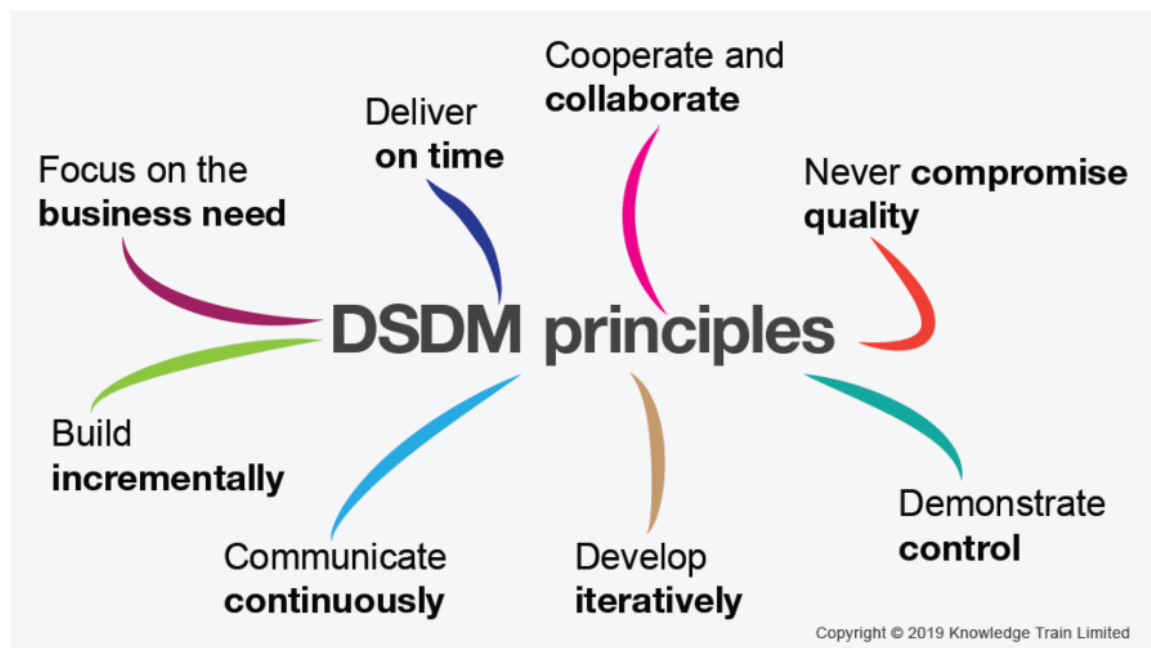


Figure 10: DSDM Principles

(Buehring, 2026)

3.2.1 Phase of Methodology

The DSDM is an agile framework that focuses on delivering systems quickly through iterative development and continuous user involvement. It ensures that projects meet business requirements while maintaining quality and time constraints. DSDM consists of six phases, each guiding the project from initial idea to final deployment and improvement.

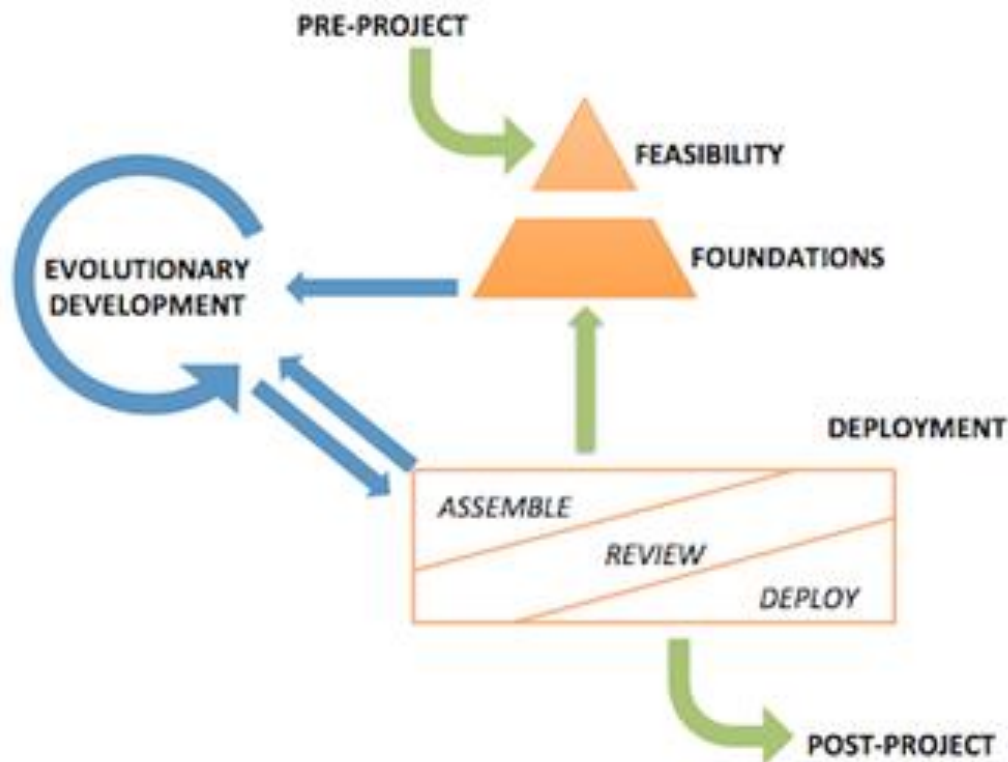


Figure 11:DSDM Steps

(Study.com, 2025)

- 1. Pre-Project Phase:** This phase focuses on identifying the project idea and ensuring that it aligns with business objectives and stakeholder needs. It defines the overall scope of the project at a high level and determines whether the project is worth pursuing. Proper planning in this stage helps avoid unnecessary risks and ensures a clear project direction.

2. Feasibility Phase: In this phase, the project is evaluated to determine whether it is technically and economically feasible. Different tools, technologies, and possible approaches are analyzed, and potential risks are identified. This phase ensures that the project can be successfully implemented within the available time, cost, and resource constraints.

3. Foundations Phase: This phase establishes a strong base for system development by defining both functional and non-functional requirements. It includes designing the system architecture, preparing UML diagrams, and planning the development approach. A clear foundation helps reduce confusion during development and ensures all stakeholders have a shared understanding of the system.

4. Evolutionary Development Phase: This is the core phase of DSDM where the system is developed in an iterative and incremental manner. Features are implemented step-by-step, and continuous testing is performed alongside development. Feedback from users is incorporated regularly, allowing improvements and refinements to be made throughout the process.

5. Deployment Phase: In this phase, the completed system is thoroughly tested and prepared for final delivery. The system is deployed to the user environment, and necessary documentation and user training are provided. This ensures that users can effectively operate the system and that it meets all required standards.

6. Post-Project Phase: This phase focuses on maintaining the system after deployment and evaluating its performance. Any issues identified are resolved, and future enhancements are planned. It ensures the system remains effective, up-to-date, and adaptable to changing user requirements.

3.3 Survey Results

3.3.1 Pre-survey Results

The pre-survey was done to learn what users experience and the difficulties in vulnerability scanning and security analysis. The results are as follows:

- Network security was rated to be very significant by most of the respondents, and approximately 85% of the users were quite aware of the dangers of cybersecurity.
- A big percentage of users, about 75% indicated that they use various tools like Nmap and Nikto to do vulnerability scanning rather than using a single integrated system.
- One of the main challenges reported was the control and integration of the results of various tools, and almost three-quarters of the participants indicated that they found it challenging to work with fragmented results.
- Approximately 68-percent of users pointed out the lack of a centralized system to display scan results in a clear and systematic format.
- Another significant problem was the interpretation of technical scan results, with about 72% of respondents struggling to derive meaningful information out of raw data.
- Vulnerability prioritization was also found to be among the major issues, with nearly 65 percent of the users unable to find which vulnerabilities are critical and needed urgent action.
- The entire process of scanning and analysis was seen to be time-consuming, and almost three-quarters of the respondents stated that it involves a lot of technical expertise.
- Many users, around 80%, reported that automated analysis was necessary to save time and effort in the analysis.

Lastly, most of the respondents who are approximately 85 percent expressed great interest in a system that offers real-time results, enhanced visualization, and user-friendly dashboard. Pre-survey Result [7.3.3](#)

3.3.2 Post-survey Results

The purpose of the post-survey was to check how well and convenient the use of ThreatQuill Analyzer system was. The results are as follows:

- The dashboard was considered useful in executing scans by most users, and 35.8% of them said that it worked seamlessly.
- Most participants, 50.9% scanned progress clearly with real-time tracking and 47.2% Activity Log informative.
- The overall structure of scan results was found to be easy to use, however some had trouble interpreting and navigating with 32.1% reporting a smooth flow.
- There was a moderate level of multiple scanner integration of 28.3% and general stability of the system by 35.8% of users.
- Scan's performance was quick to a third of the users but at 37.7 others reported delays and inconsistencies.
- Duplicate handling was functional with 43.4 and detection effectiveness and vulnerability correlation were mostly good.
- Prioritization of risks 37.7% and classification of risk severity 41.5% assisted the users in understanding better the risks in the system.
- Quick analysis and decision-making were supported by report clarity (30.2) and scan summary usefulness (35.8).
- The system was found to be acceptable by 34 percent although this would need a further change so that it can use it in practice.
- Faster scanning 35.8%, better filtering, better visualization 24.5% and better risk prioritization were the proposed improvements suggested by the users.

Overall, the findings indicate that ThreatQuill Analyzer is viable and user-friendly, and it can be improved in terms of performance, accuracy, and usability.

Post-survey Result [7.2.4](#)

3.4 Requirement Analysis

3.4.1 Hardware Requirements

The system development and testing were carried out on a Dell Inspiron 5493 with a minimum of 8 GB RAM, Intel i5 processor, 256 GB storage, and a stable internet connection.

3.4.2 Software Requirements

Category	Software / Tool	Purpose / Notes
Operating System	Windows 11 & Kali Linux	Windows used for dashboard development; Kali Linux used for running vulnerability scanners like Nmap, Nikto, OpenVAS, and Nessus.
Development Tools	Visual Studio Code (v1.80)	Used for writing and managing Python code for backend and GUI development.
Programming Language	Python	Used for backend logic, integrating scanners, parsing results, and generating reports.
Vulnerability Scanners	Nmap, Nikto, OpenVAS, Nessus	Used to perform network scanning and detect vulnerabilities.
Communication Library	Paramiko	Used to enable SSH communication between Windows

		dashboard and Kali Linux backend.
Version Control	GitHub (Git 2.42)	Used for source code management and version tracking.
Backup Storage	OneDrive / GitHub	Used for secure storage and backup of project files.
Project Management Tool	Trello	Used to manage tasks, milestones, and project progress.
Documentation	MS Word	Used to prepare project reports and documentation.
Gantt Chart	Team Gantt	Used for planning and tracking project timelines.

3.5 Design

3.5.1 Pre Planning

Pre-planning design was done to understand the system layout and user interaction before development. It helped in organizing the interface and reducing design errors.

3.5.1.1 Wireframes

Wireframes were created to show the basic structure of the system interface before implementation.

The login page includes fields for username and password with a login button for user access.

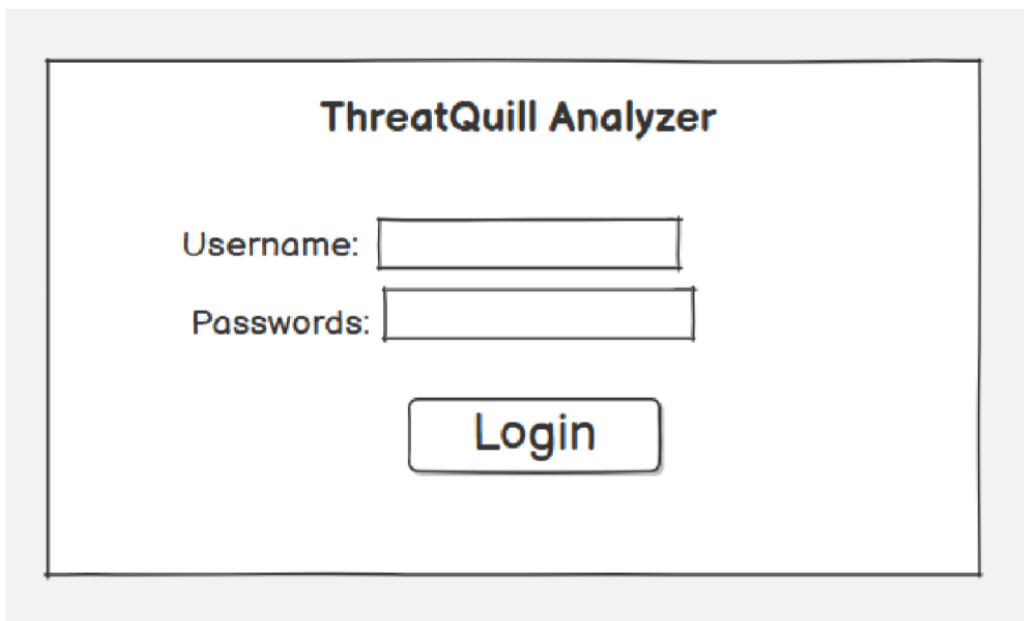


Figure 12: Wireframe of Login Page

The main page shows the dashboard layout with navigation options and sections for scan results and activity logs.

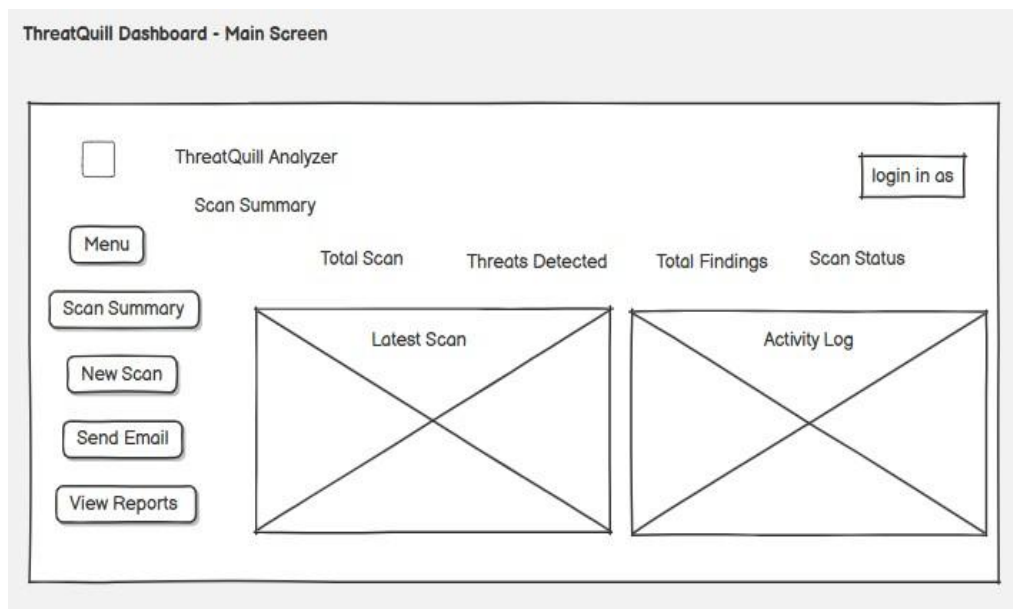


Figure 13: Wireframe of Dashboard

After the start scan button, all the inputs data vulnerabilities with its severity are shown.

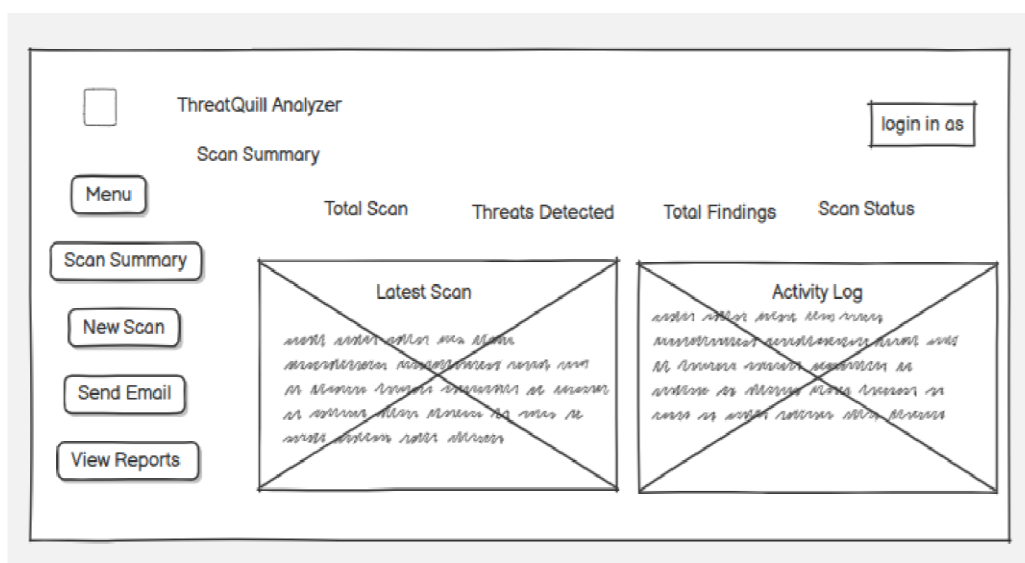


Figure 14: Wireframe of dashboard after scan

3.5.1.2 UI Design

The user interface of the ThreatQuill Analyzer was developed based on wireframes. It provides a simple and user-friendly environment for performing scans and viewing results. The interface was implemented using Tkinter to ensure smooth interaction and clear display of information.

The login interface allows users to securely enter credentials and access the system.

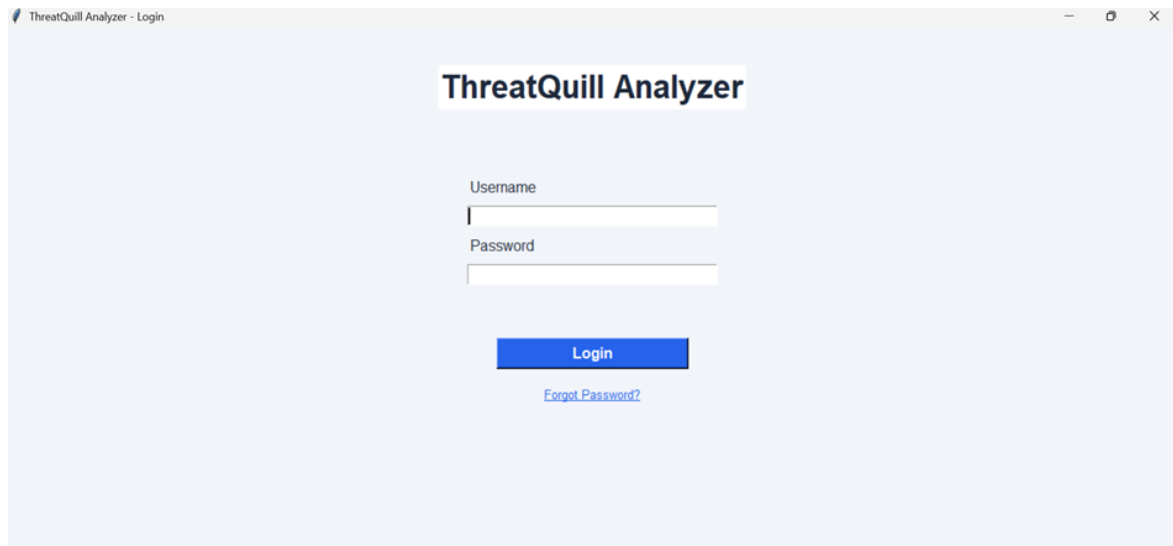


Figure 15: Login Page

The dashboard provides options to start scans, view results, and monitor activity logs in real time.

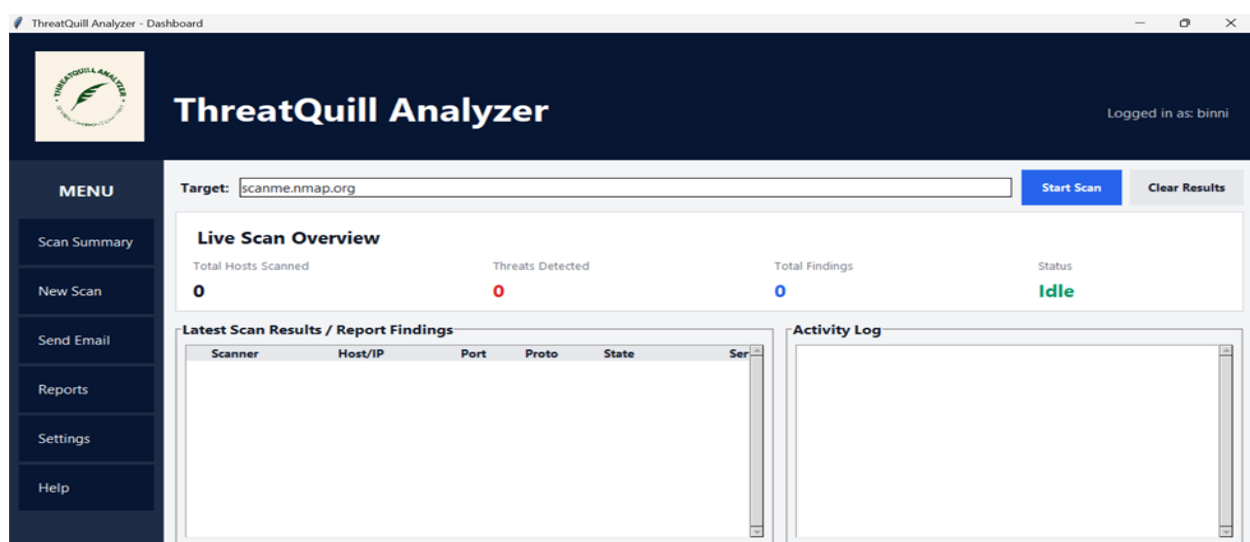


Figure 16: Dashboard page

After scanning all the scan results and log are shown in dashboard.

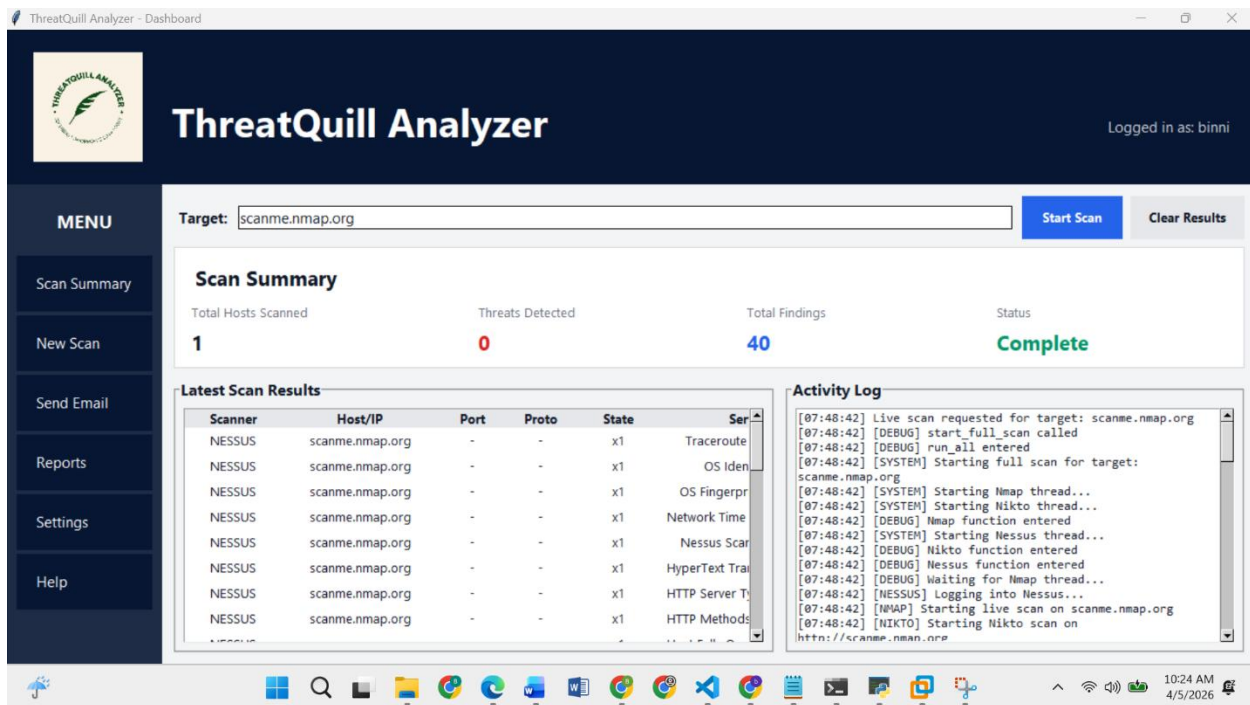


Figure 17: Dashboard after scan

The report interface presents detailed vulnerability results in a structured format for analysis.

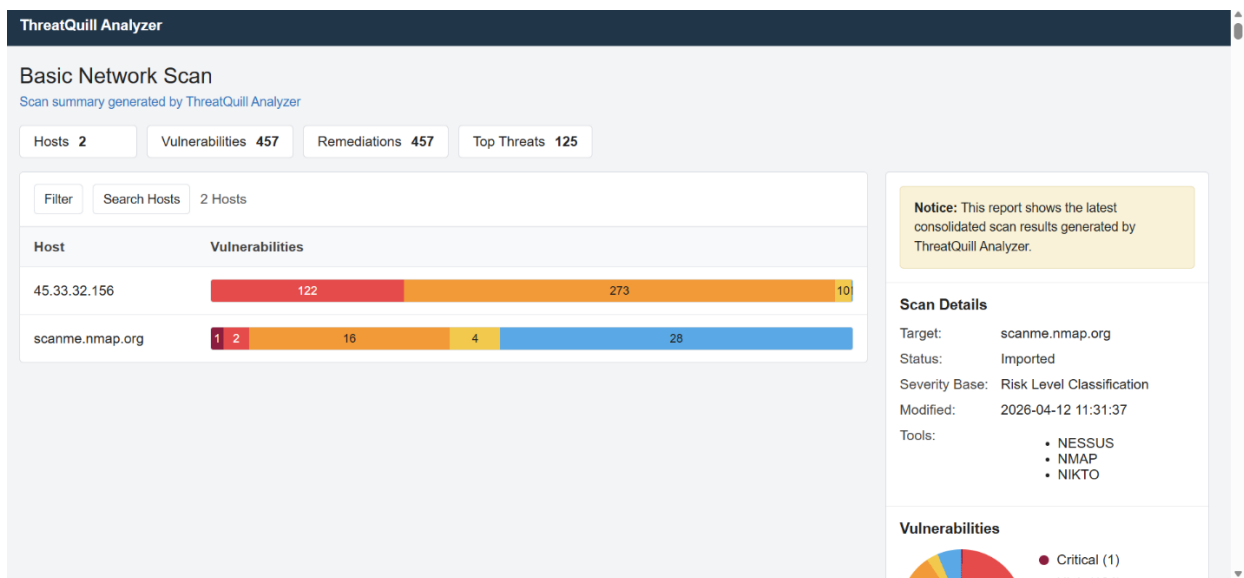


Figure 18: Generated report with vulnerability summary

The scan summary displays an overview of scan results including threats, findings, and severity levels.

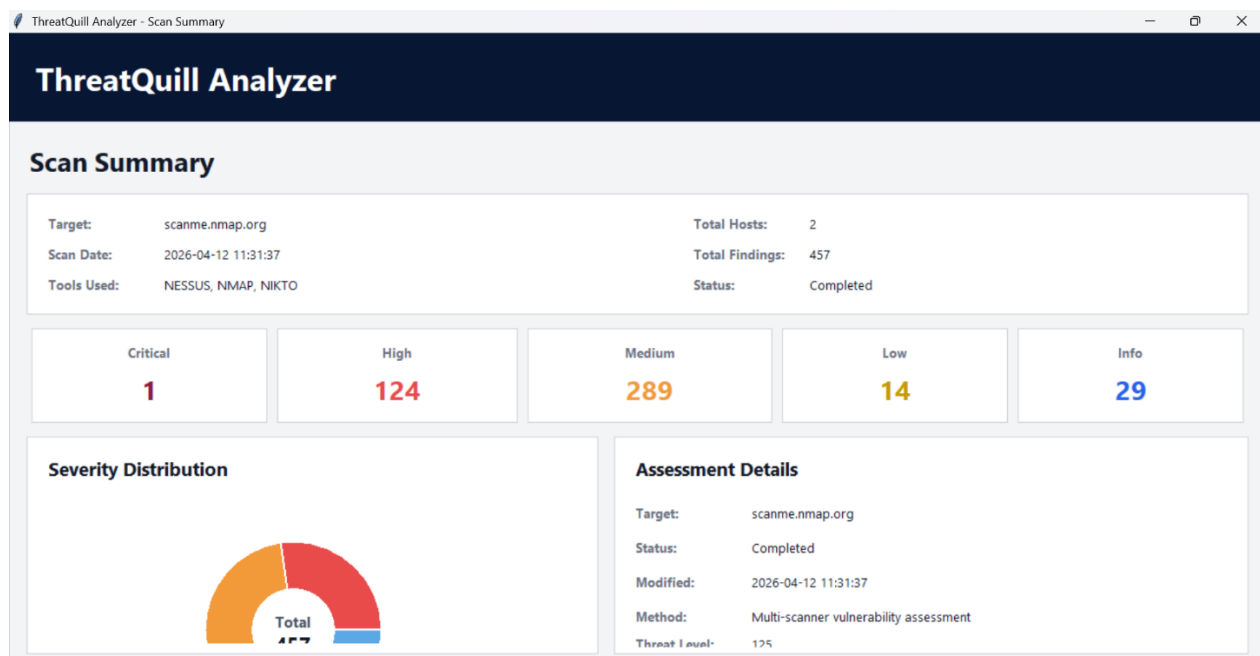


Figure 19: Scan summary showing vulnerability distribution

The email interface allows users to send the generated report directly to a recipient.

The screenshot shows the 'Send Final Report' window. The title bar includes the application name and standard window controls. The main header is 'Send Final Report'.

Your Gmail:

App Password:

Recipient Email:

Figure 20: Email interface for sending final report

3.5.2 Diagrams

3.5.2.1 Logo



Figure 21: ThreatQuill Analyzer system logo

3.5.2.2 System Architecture Diagram

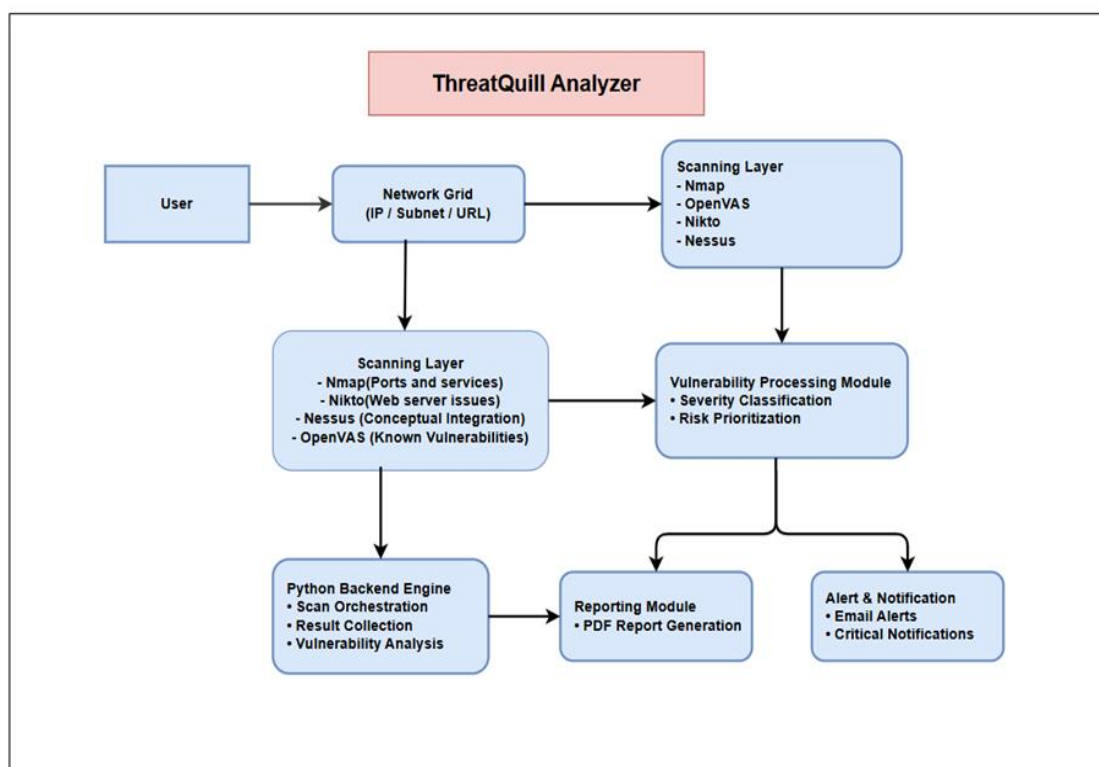


Figure 22: System architecture showing interaction between dashboard, backend, and scanners

3.5.2.3 Flowchart

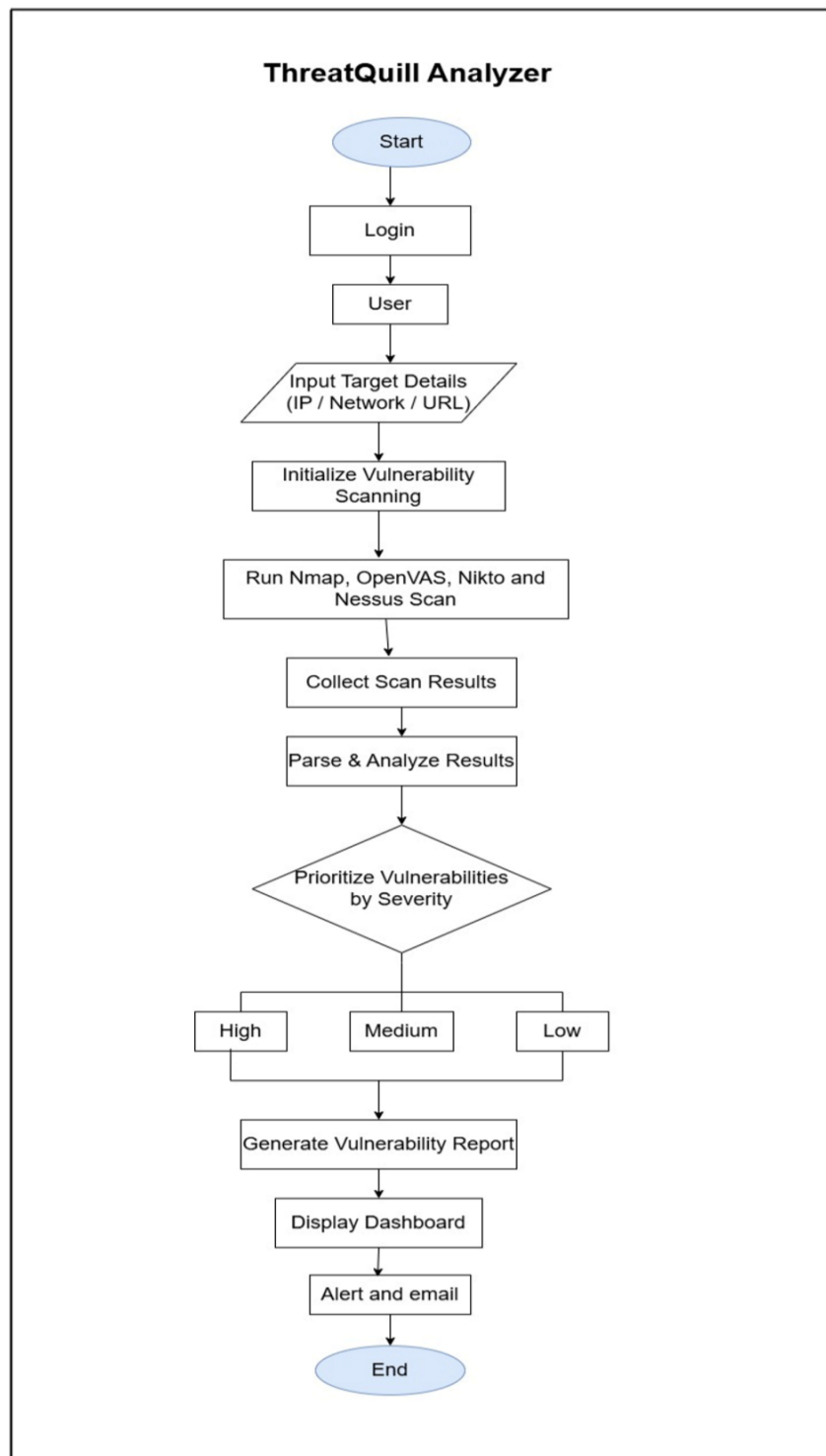


Figure 23: Flowchart representing the scanning and reporting process

3.5.2.4 ERD

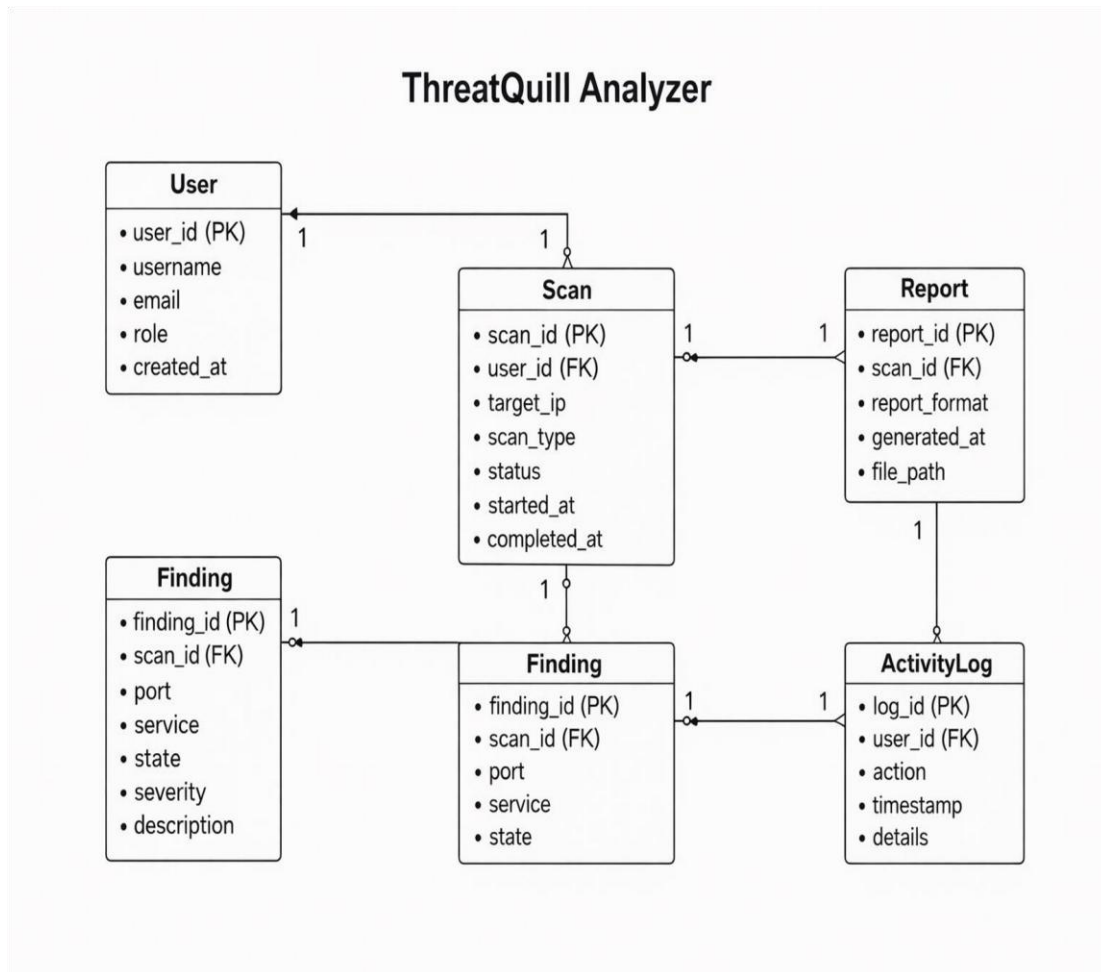


Figure 24: Entity Relationship Diagram showing data structure of the system

3.5.2.5 Use Case Diagram

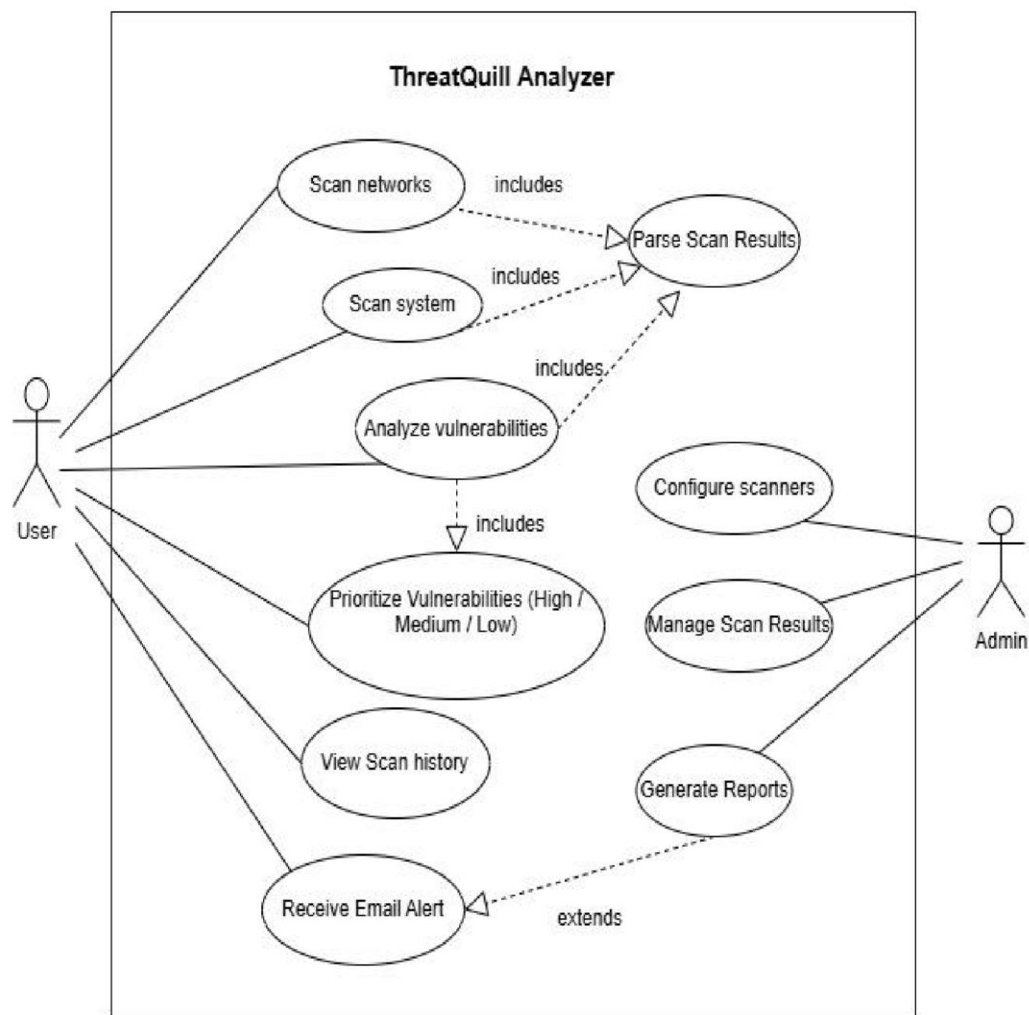


Figure 25: Use case diagram

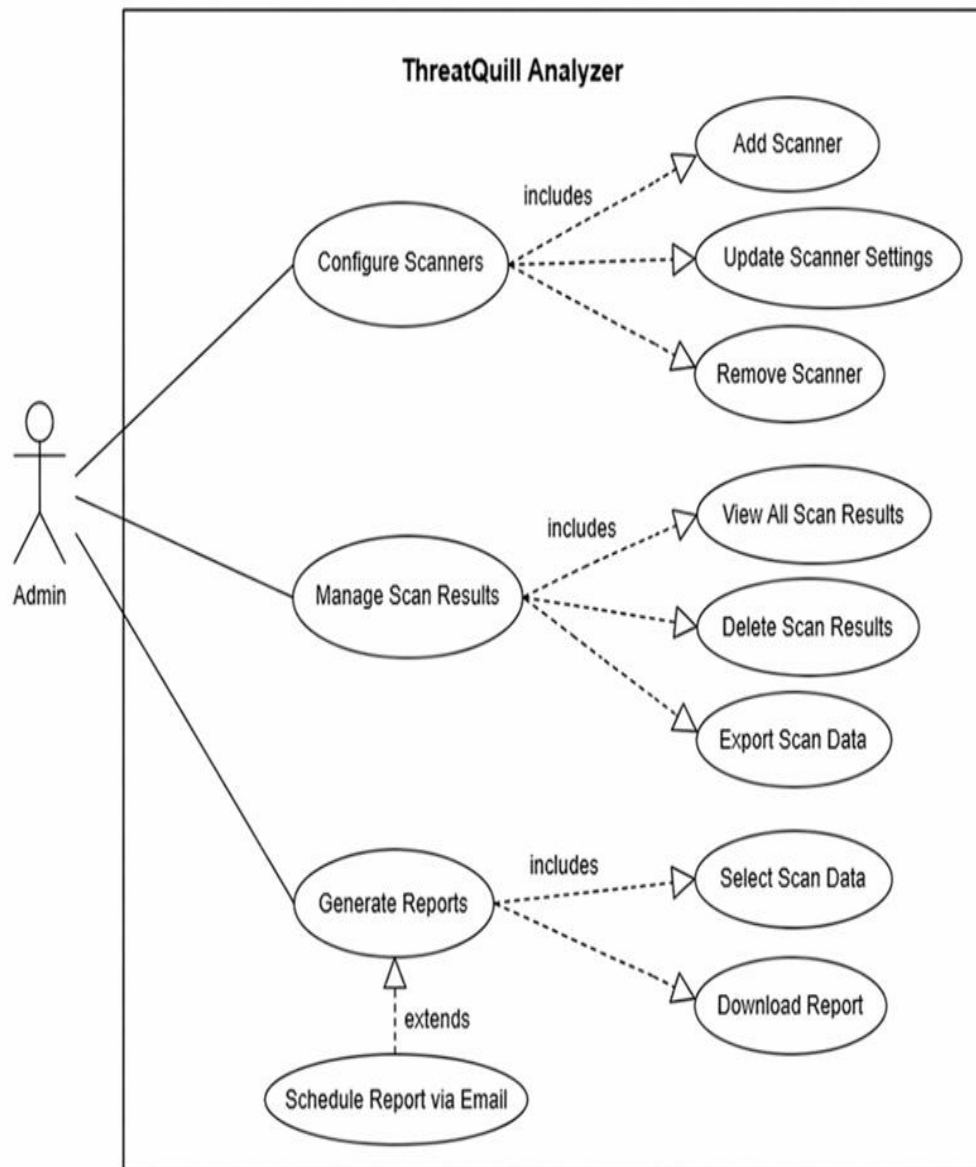


Figure 26: Use case diagram admin part

3.5.2.6 UML Diagram

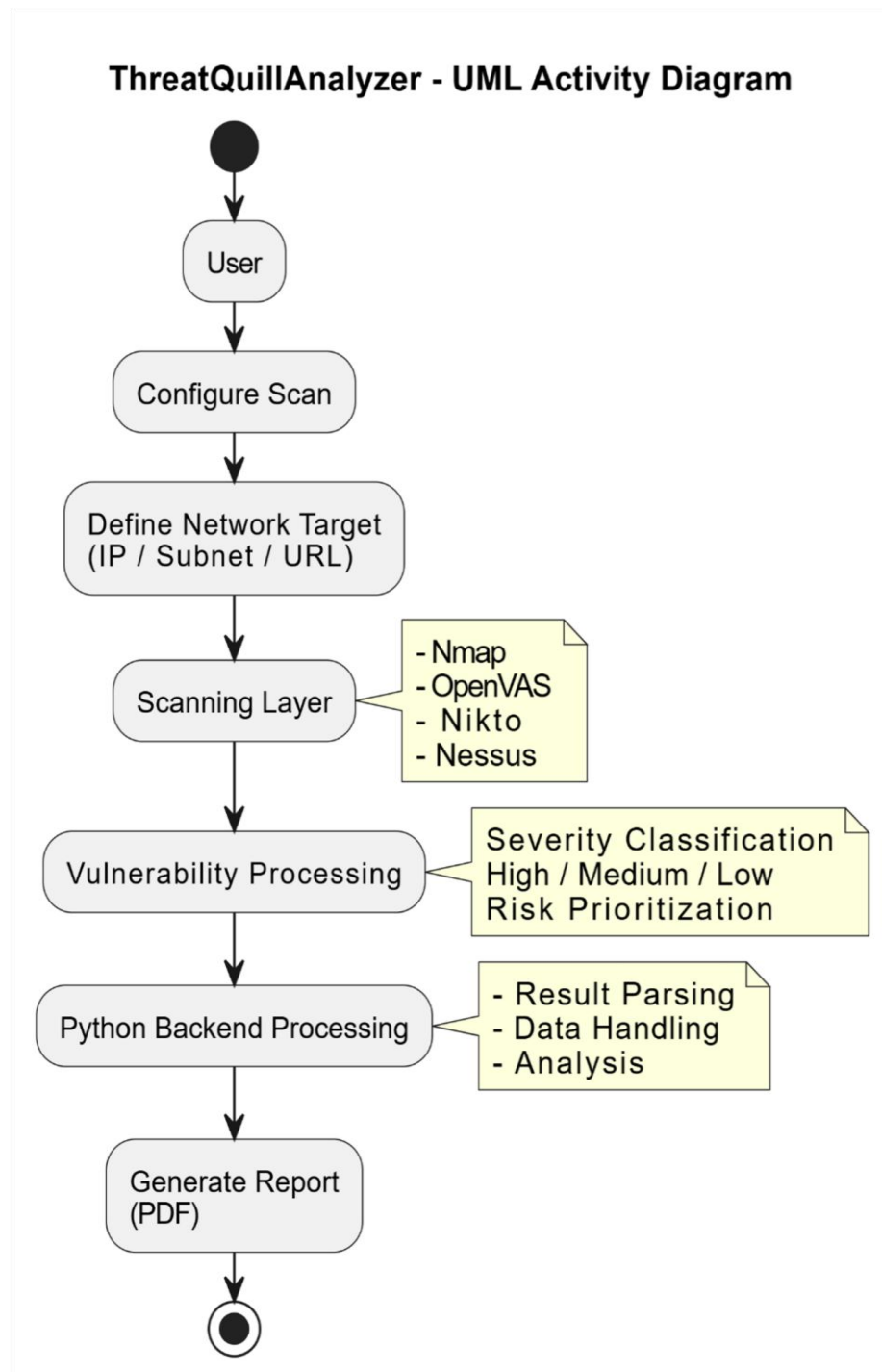


Figure 27: UML diagram representing system structure and component relationships

3.5.2.7 DFD

DFD Level 0

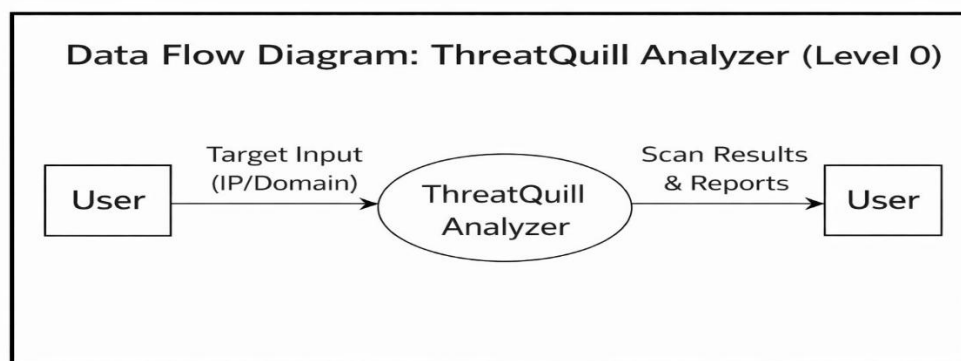


Figure 28: DFD Level 0 showing overall data flow in the system

DFD Level 1

ThreatQuill Analyzer - Level 1 Data Flow Diagram

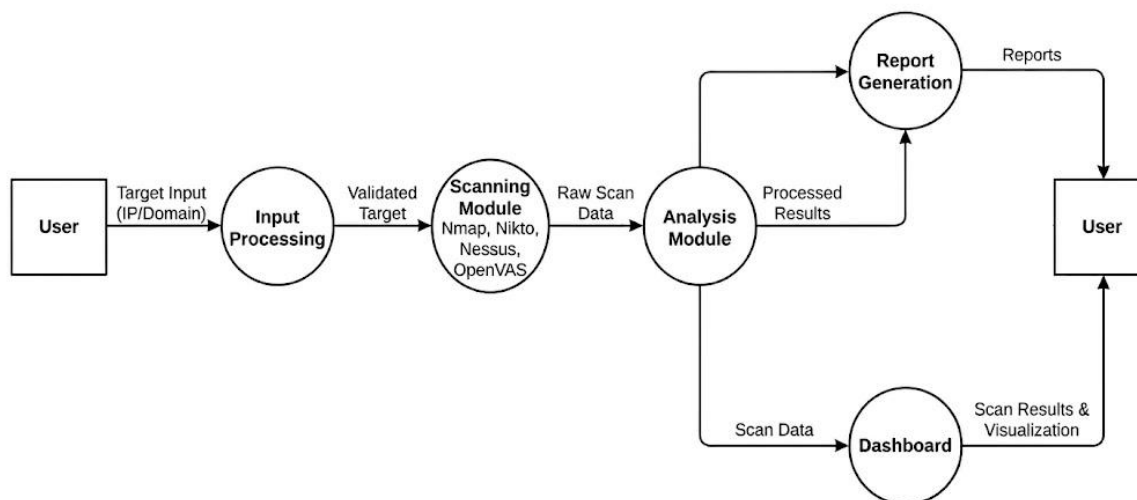


Figure 29: DFD Level 1 showing detailed data flow between system components

3.6 Implementation

3.6.1 Implementation Using DSDM Methodology

The development of ThreatQuill Analyzer used the DSDM approach where the system was developed step by step through its different phases.

1. Pre-Project Phase

The concept of creating the ThreatQuill Analyzer had been found during the Pre-Project phase. The current vulnerability scanning tools were noted to be utilized individually, i.e. Nmap, Nikto, OpenVAS and Nessus, and thus the process is complicated and time consuming.

To use the results, users are required to use several tools separately and analyze the results manually, which adds additional work and risks of making a mistake. To address this issue, the idea of a system that is integrated was introduced.

The project scope was defined as coming up with a single platform which will be able to scan, analyze vulnerabilities and produce structured reports. The system scope and targets were also well determined during this phase.

2. Feasibility Phase

The Feasibility phase involved analysis of the project to find out whether it is possible to develop it successfully with the help of resources and tools available. Different technologies such as Python, Tkinter, Kali Linux, and vulnerability scanners like Nmap, Nikto, OpenVAS, and Nessus were evaluated.

```

PS D:\ThreatQuill Analyzer> ssh "kali@192.168.153.143"
>>
the exact distribution terms for each program are described in t
he
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
[Message from Kali developers]
y tools. Learn how:
| => https://www.kali.org/docs/troubleshooting/common-minimum-set
up/
L(Run: "touch ~/.hushlogin" to hide this message)
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ mkdir -p /home/kali/scans

```

Figure 30: SSH connection established and system environment prepared for scanner setup

Kali Linux machine via SSH from Windows.

```

(kali@kali)-[~]
$ sudo apt install nmap
[sudo] password for kali:
nmap is already the newest version (7.98+dfsg-1kali1).
The following packages were automatically installed and are no longer required:
gir1.2-girepository-2.0  liboauth0
gir1.2-peas-1.0          libokular6core3
gir1.2-totem-1.0         libpackagekitqt6-1
gir1.2-totemlparser-1.0 libpeas-1.0-0
grilo-plugins-0.3        libpeas-common
libavfilter10            libpocketsphinx3
libavformat61            libportal-gtk3-1
libcolorcorrect6         libpostproc58
libconfig-inifiles-perl  libppsddocument-4.0-5
libdisplay-info2         libppsview-4.0-4
libdmapsharing-4.0-3t64  libsindutf27
libgirepository-1.0-1    libsphinxbase3t64

```

Figure 31: Nmap installation verified successfully in Kali Linux environment

```
(kali㉿kali)-[~]
└─$ sudo apt update
sudo apt install nikto
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.5 MB]
Fetched 73.5 MB in 29s (2,510 kB/s)
6 packages can be upgraded. Run 'apt list --upgradable' to see them.
nikto is already the newest version (1:2.5.0+git20230114.90ff645-0kali1).
The following packages were automatically installed and are no longer required:
  gir1.2-girepository-2.0  liboauth0
  gir1.2-peas-1.0          libokular6core3
  gir1.2-totem-1.0         libpackagekitqt6-1
  gir1.2-totemlparser-1.0  libpeas-1.0-0
  grilo-plugins-0.3        libpeas-common
  libavfilter10            libpocketsphinx3
  libavformat61            libportal-gtk3-1
  libcolorcorrect6         libpostproc58
  libconfig-inifiles-perl  libppsdocument-4.0-5
  libdisplay-info2         libppsview-4.0-4
  libdmasharing-4.0-3t64  libsimdutf27
  libgirepository-1.0-1    libsphinxbase3t64
```

Figure 32: Nikto installation and system update completed successfully

3. Foundations Phase

During the Foundations stage, the general workflow and framework of the ThreatQuill Analyzer was created. The system architecture was laid out, including interaction of the user with the dashboard, how the data is processed by the backend and how various scanners are incorporated.

Pragmatic needs like scanning, parsing of results, risk prioritization and report generation were established. Non-functional requirements, including performance and usability, were also taken into consideration.

4. Evolutionary Development Phase

4.1 Nmap Implementation

Kali Linux terminal first run `sudo apt full-upgrade` to update the system packages and then `sudo apt install nmap`, which confirms that Nmap is already installed and up to date, proving that the scanning environment for the backend is fully prepared.


```

(kali@kali)-[~]
└─$ sudo apt install nmap
[sudo] password for kali:
nmap is already the newest version (7.98+dfsg-1kali1).
The following packages were automatically installed and are no longer required:
  gir1.2-girepository-2.0  liboauth0
  gir1.2-peas-1.0          libokular6core3
  gir1.2-totem-1.0         libpackagekitqt6-1
  gir1.2-totemlparser-1.0  libpeas-1.0-0
  grilo-plugins-0.3        libpeas-common
  libavfilter10            libpocketsphinx3
  libavformat61            libportal-gtk3-1
  libcolorcorrect6         libpostproc58
  libconfig-inifiles-perl  libppsdocument-4.0-5
  libdisplay-info2         libppsview-4.0-4
  libdmapsharing-4.0-3t64  libsimutf27
  libgirepository-1.0-1    libspnbase3t64

```

Figure 33: Nmap installation verified successfully in the Kali Linux environment

```

(kali@kali)-[~]
└─$ sudo apt full-upgrade
The following packages were automatically installed and are no longer required:
  gir1.2-girepository-2.0  liboauth0
  gir1.2-peas-1.0          libokular6core3
  gir1.2-totem-1.0         libpackagekitqt6-1
  gir1.2-totemlparser-1.0  libpeas-1.0-0
  grilo-plugins-0.3        libpeas-common
  libavfilter10            libpocketsphinx3
  libavformat61            libportal-gtk3-1
  libcolorcorrect6         libpostproc58
  libconfig-inifiles-perl  libppsdocument-4.0-5
  libdisplay-info2         libppsview-4.0-4
  libdmapsharing-4.0-3t64  libsimutf27
  libgirepository-1.0-1    libspnbase3t64
  libgnome-rr-4-2t64       libswscale8
  libgom-1.0-0t64         libtotem0
  libgpgme11t64           pocketsphinx-en-us
  libgpgmepp6t64          qml6-module-org-kde-notifications
  libgrilo-0.3-0          totem

```

Figure 34: Nmap scan executed on the target system to identify open ports and services

```

(kali@kali)-[~]
$ nmap -sV -oX /home/kali/scans_test.xml 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-29 14:27 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000017s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache httpd 2.4.65 ((Debian))
3389/tcp  open  ms-wbt-server Microsoft Terminal Service
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.03 seconds

(kali@kali)-[~]
$ ls /home/kali
Desktop    Music      Pictures    Templates    Videos
Documents  nmap_sample.xml  Public      thinclient_drives
Downloads  nmap_scan.xml   scans_test.xml  ThreatQuill

```

Figure 35: Nmap scan performed on local system to detect active ports and services

The resulting scan output saved as an XML file in the /home/kali directory, proving that the backend can successfully execute a scan and store its results for later parsing.

```

(kali@kali)-[~]
$ cat scans_test.xml | head
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE nmaprun>
<?xml-stylesheet href="file:///usr/share/nmap/nmap.xsl" type="text/xsl"?>
<!-- Nmap 7.98 scan initiated Mon Dec 29 14:27:38 2025 as: /usr/lib/nmap/nmap -sV -oX /home/kali/scans_test.xml 127.0.0.1 -->
<nmaprun scanner="nmap" args="/usr/lib/nmap/nmap -sV -oX /home/kali/scans_test.xml 127.0.0.1" start="1767036458" startstr="Mon Dec 29 14:27:38 2025" version="7.98" xmloutputversion="1.05">
<scaninfo type="syn" protocol="tcp" numservices="1000" services="1,3-4,6-7,9,13,17,19-26,30,32-33,37,42-43,49,53,70,79-85,88-90,99-100,106,109-111,113,119,125,135,139,143-144,146,161,163,179,199,211-212,222,254-256,259,264,280,301,306,311,340,366,389,406-407,416-417,425,427,443-445,458,464-465,481,497,500,512-515,524,541,543-545,548,554-555,563,587,593,616-617,625,631,636,646,648,666-668,683,687,691,700,705,711,714,720,722,726,749,765,777,783,787,800-801,808,843,873,880,888,898,900-903,911-912,981,987,990,992-993,995,999-1002,1007,1009-1011,1021-1100,1102,1104-1108,1110-1114,1117,1119,1121-1124,1126,1130-1132,1137-1138,1141,1145,1147-1149,1151-1152,1154,1163-1166,1169,1174-1175,1183,1185-1187,1192,1198-1199,1201,1213,1216-1218,1233-1234,1236,1244,1247-1248,1259,1271-1272,1277,1287,1296,1300-1301,1309-1311,1322,1328,1334,1352,1417,1433-1434,1443,1455,1461,1494,1500-1501,1503,1521,1524,1533,1556,1580,1583,1594,1600,1641,1658,1666,1687-1688,1700,1711,1714,1715,1716,1717,1718,1719,1720,1721,1722,1723,1724,1725,1726,1727,1728,1729,1730,1731,1732,1733,1734,1735,1736,1737,1738,1739,1740,1741,1742,1743,1744,1745,1746,1747,1748,1749,1750,1751,1752,1753,1754,1755,1756,1757,1758,1759,1760,1761,1762,1763,1764,1765,1766,1767,1768,1769,1770,1771,1772,1773,1774,1775,1776,1777,1778,1779,1780,1781,1782,1783,1784,1785,1786,1787,1788,1789,1790,1791,1792,1793,1794,1795,1796,1797,1798,1799,1800,1801,1802,1803,1804,1805,1806,1807,1808,1809,1810,1811,1812,1813,1814,1815,1816,1817,1818,1819,1820,1821,1822,1823,1824,1825,1826,1827,1828,1829,1830,1831,1832,1833,1834,1835,1836,1837,1838,1839,1840,1841,1842,1843,1844,1845,1846,1847,1848,1849,1850,1851,1852,1853,1854,1855,1856,1857,1858,1859,1860,1861,1862,1863,1864,1865,1866,1867,1868,1869,1870,1871,1872,1873,1874,1875,1876,1877,1878,1879,1880,1881,1882,1883,1884,1885,1886,1887,1888,1889,1890,1891,1892,1893,1894,1895,1896,1897,1898,1899,1900,1901,1902,1903,1904,1905,1906,1907,1908,1909,1910,1911,1912,1913,1914,1915,1916,1917,1918,1919,1920,1921,1922,1923,1924,1925,1926,1927,1928,1929,1930,1931,1932,1933,1934,1935,1936,1937,1938,1939,1940,1941,1942,1943,1944,1945,1946,1947,1948,1949,1950,1951,1952,1953,1954,1955,1956,1957,1958,1959,1960,1961,1962,1963,1964,1965,1966,1967,1968,1969,1970,1971,1972,1973,1974,1975,1976,1977,1978,1979,1980,1981,1982,1983,1984,1985,1986,1987,1988,1989,1990,1991,1992,1993,1994,1995,1996,1997,1998,1999,2000">

```

Figure 36: Scan results generated showing open ports and running services

```

PS D:\ThreatQuill Analyzer> ssh "kali@192.168.153.143"
>>
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
└─(Message from Kali developers)
y tools. Learn how:
|  ⇒ https://www.kali.org/docs/troubleshooting/common-minimum-set
up/
└─(Run: "touch ~/.hushlogin" to hide this message)
zsh: corrupt history file /home/kali/.zsh_history
└─(kali@kali)-[~]
└─$ mkdir -p /home/kali/scans

```

Figure 37: Nmap scan results saved in XML format for further processing

Kali Linux machine via SSH from Windows and then creating a new folder called scans in the Kali home directory.

```

└─(kali@kali)-[~]
└─$ nmap -sV -oX /home/kali/scans/test.xml 127.0.0.1
ls /home/kali/scans
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-29 14:43 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000050s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 10.2p1 Debian 3 (protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.65 ((Debian))
3389/tcp   open  ms-wbt-server Microsoft Terminal Service
Service Info: OSs: Linux, Windows; CPE: cpe:/o:linux:linux_kernel, cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results
at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.14 seconds
test.xml

```

Figure 38: Scan output stored in the system directory for backend integration

Nmap scan on the local machine (127.0.0.1) to find open ports and services and saving the scan results to a file named test.xml.

```

PS D:\ThreatQuill Analyzer> cd "D:\ThreatQuill Analyzer"
PS D:\ThreatQuill Analyzer> mkdir backend

Directory: D:\ThreatQuill Analyzer

Mode                LastWriteTime         Length Name
----                -
d-----          12/30/2025   1:33 AM             backend

PS D:\ThreatQuill Analyzer> cd backend
PS D:\ThreatQuill Analyzer\backend> python -m venv venv
PS D:\ThreatQuill Analyzer\backend> .\venv\Scripts\activate
(venv) PS D:\ThreatQuill Analyzer\backend> pip install fastapi u
vicorn paramiko lxml
Collecting fastapi
  Downloading fastapi-0.128.0-py3-none-any.whl.metadata (30 kB)
Collecting uvicorn
  Downloading uvicorn-0.40.0-py3-none-any.whl.metadata (6.7 kB)
Collecting paramiko
  Downloading paramiko-4.0.0-py3-none-any.whl.metadata (3.9 kB)
Collecting lxml
  Downloading lxml-5.3.0-py3-none-any.whl.metadata (8.1 kB)

```

Figure 39: Nmap results prepared for integration into the ThreatQuill Analyzer system

Setting up a Python backend by creating a folder, creating and activating a virtual environment, and installing required packages.

ThreatQuill Analyzer - Dashboard

Target: [Scan] [Clear Results]

Scan Summary

Total Hosts Scanned	Threats Detected	Total Findings	Status
1	1	3	Complete

Latest Scan Results

Scanner	Host/IP	Port	Proto	State	Service
NIKTO	scanme.nmap.org	80	tcp	open	http
NIKTO	scanme.nmap.org	80	tcp	open	http
NIKTO	scanme.nmap.org	80	tcp	open	http
NMAP	45.33.32.156	22	tcp	filtered	ssh
NMAP	45.33.32.156	25	tcp	filtered	smtp
NMAP	45.33.32.156	80	tcp	open	http
NMAP	45.33.32.156	9929	tcp	open	nping-echo
NMAP	45.33.32.156	31337	tcp	open	tcpwr

Activity Log

```

[07:37:50] [NMAP] Not shown: 995 closed tcp ports (reset)
[07:37:50] [NMAP] PORT STATE SERVICE VERSION
[07:37:50] [NMAP] 22/tcp filtered ssh
[07:37:50] [NMAP] 25/tcp filtered smtp
[07:37:50] [NMAP] 80/tcp open http Apache httpd
2.4.7 ((Ubuntu))
[07:37:50] [NMAP] 9929/tcp open nping-echo Nping echo
[07:37:50] [NMAP] 31337/tcp open tcpwrapped
[07:37:50] [NMAP] Service detection performed. Please report
any incorrect results at https://nmap.org/submit/ .
[07:37:50] [NMAP] Nmap done: 1 IP address (1 host up) scanned
in 20.66 seconds
[07:37:50] [NMAP] Scan completed.
[07:37:50] [DEBUG] Nmap function finished
[07:37:50] [DEBUG] Waiting for Nikto thread...

```

Figure 40: Nmap scan results displayed on dashboard showing detected ports and services

4.2 Nikto Implementation

```
(kali@kali)-[~]
└─$ sudo apt update
sudo apt install nikto
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.5 MB]
Fetched 73.5 MB in 29s (2,510 kB/s)
6 packages can be upgraded. Run 'apt list --upgradable' to see them.
nikto is already the newest version (1:2.5.0+git20230114.90ff645-0kali1).
The following packages were automatically installed and are no longer required:
  gir1.2-girepository-2.0  liboauth0
  gir1.2-peas-1.0          libokular6core3
  gir1.2-totem-1.0         libpackagekitqt6-1
  gir1.2-totemlparser-1.0  libpeas-1.0-0
  grilo-plugins-0.3        libpeas-common
  libavfilter10            libpocketsphinx3
  libavformat61            libportal-gtk3-1
  libcolorcorrect6         libpostproc58
  libconfig-inifiles-perl  libppsddocument-4.0-5
  libdisplay-info2         libppsview-4.0-4
  libdmapsharing-4.0-3t64  libsindutf27
  libgirepository-1.0-1    libsphinxbase3t64
```

Figure 41: Nikto scanner installed and configured for web vulnerability assessment

Updating the Kali Linux system and installing the Nikto web vulnerability scanning tool.

```
(kali@kali)-[~]
└─$ nikto -h http://127.0.0.1 -o /home/kali/scans/nikto_test.xml -Format xml
- Nikto v2.5.0

-----
+ Target IP:      127.0.0.1
+ Target Hostname: 127.0.0.1
+ Target Port:    80
+ Start Time:     2025-12-29 15:34:56 (GMT-5)
-----

+ Server: Apache/2.4.65 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Server may leak inodes via ETags, header found with file /, inode: 29cf, size: 642d4b7df44cc, mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
```

Figure 42: Nikto scan executed on target web server to identify vulnerabilities

Nikto web vulnerability scan on the local web server (127.0.0.1) and saving the scan results in an XML file.

```
(kali@kali)-[~]
$ ls /home/kali/scans
nikto_test.xml      scan_1767039877.xml  scan_1767039928.xml  test.xml
scan_1767038400.xml scan_1767039890.xml  scan_1767039941.xml
scan_1767039083.xml scan_1767039903.xml  scan_1767039953.xml
scan_1767039528.xml scan_1767039916.xml  scan_1767039965.xml

(kali@kali)-[~]
$
```

Figure 43: Nikto scan results generated showing detected web vulnerabilities

All the saved scan result XML files (from Nmap and Nikto) inside the /home/kali/scans folder.

```
PS D:\ThreatQuill Analyzer> cd "D:\ThreatQuill Analyzer\backend"
PS D:\ThreatQuill Analyzer\backend> .\venv\Scripts\activate
(venv) PS D:\ThreatQuill Analyzer\backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['D:\ThreatQuill Analyzer\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [21588] using StatReload
INFO: Started server process [20992]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 44: Nikto scan results saved in XML format for further processing

FastAPI backend server being started successfully using Uvicorn on http://127.0.0.1:8000 in development mode.

```
ptography>=3.3->paramiko) (2.0.0)
Requirement already satisfied: pycparser in d:\threatquill analyzer\backend\venv\lib\site-p
=2.0.0->cryptography>=3.3->paramiko) (2.23)
(venv) PS D:\ThreatQuill Analyzer\backend> python test_nikto.py
```

Figure 45: Nikto scan results processed and prepared for system integration

ThreatQuill Analyzer - Dashboard

Logged in as: binni

MENU

- Scan Summary
- New Scan
- Send Email
- Reports
- Settings
- Help

Target: scanme.nmap.org **Start Scan** **Clear Results**

Scan Summary

Total Hosts Scanned	Threats Detected	Total Findings	Status
1	23	23	Running

Latest Scan Results

Scanner	Host/IP	Port	Proto	State	Ser
NIKTO	http://scanme.nmap.org	-	-	issue	[999100] /inde
NIKTO	http://scanme.nmap.org	-	-	issue	[999965] /inde
NIKTO	http://scanme.nmap.org	-	-	issue	[999990] OPTI
NIKTO	http://scanme.nmap.org	-	-	issue	ERROR: *** Err
NIKTO	http://scanme.nmap.org	-	-	issue	ERROR: *** Co
NIKTO	http://scanme.nmap.org	-	-	issue	Scan terminate
NIKTO	http://scanme.nmap.org	-	-	issue	End Time:
NIKTO	http://scanme.nmap.org	-	-	issue	1 host(s)

Activity Log

```
[22:02:43] Live scan requested for target: scanme.nmap.org
[22:02:43] [Nessus] Logging into Nessus...
[22:02:43] [NMAP] Starting full port scan on scanme.nmap.org
[22:02:43] Starting Nikto scan on http://scanme.nmap.org
[22:02:44] Starting Nmap 7.98 ( https://nmap.org ) at
2026-04-04 12:17 -0400
[22:02:44] [Nessus] Login successful.
[22:02:45] [Nessus] Scan complete. Hosts: 1, Findings: 17
[22:02:45] [Nessus] Critical: 1, High: 0, Medium: 0, Low: 0,
Info: 16
[22:02:46] - Nikto v2.6.0
[22:02:46]
-----
[22:02:50] + Your Nikto installation is out of date.
[22:02:54] + Target IP: 45.33.32.156
```

Figure 46: Nikto scan results displayed on dashboard with detected vulnerabilities

4.3 Nessus Implementation

The Nessus service was successfully started and its status was verified, confirming that the vulnerability scanner is active and running on the system.

```
(kali@kali)-[~]
└─$ sudo systemctl start nessusd
sudo systemctl status nessusd
[sudo] password for kali:
● nessusd.service - The Nessus Vulnerability Scanner
   Loaded: loaded (/usr/lib/systemd/system/nessusd.service; enabled; preset: >
   Active: active (running) since Sat 2026-04-04 06:38:55 EDT; 1h 10min ago
 Invocation: c0a69c915fa54c588cc6d90c00783f04
    Main PID: 1054 (nessus-service)
       Tasks: 22 (limit: 9243)
      Memory: 2.8G (peak: 2.8G)
         CPU: 38min 59.040s
        CGroup: /system.slice/nessusd.service
                └─1054 /opt/nessus/sbin/nessus-service -q
                  └─1063 nessusd -q

Apr 04 06:38:55 kali systemd[1]: Started nessusd.service - The Nessus Vulnerabi>
Apr 04 06:38:56 kali nessus-service[1054]: nessus-service [1054][INFO] : Nessus>
lines 1-14/14 (END)
```

Figure 47: Nessus service started and verified successfully

The Paramiko library and its dependencies were installed using pip to enable SSH communication between the dashboard and the Kali Linux backend.

```
PS D:\ThreatQuill> python -m pip install paramiko
Collecting paramiko
  Using cached paramiko-4.0.0-py3-none-any.whl.metadata (3.9 kB)
Collecting bcrypt>=3.2 (from paramiko)
  Using cached bcrypt-5.0.0-cp39-abi3-win_amd64.whl.metadata (10 kB)
Collecting cryptography>=3.3 (from paramiko)
  Downloading cryptography-46.0.6-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting invoke>=2.0 (from paramiko)
  Using cached invoke-2.2.1-py3-none-any.whl.metadata (3.3 kB)
Collecting pynacl>=1.5 (from paramiko)
  Downloading pynacl-1.6.2-cp38-abi3-win_amd64.whl.metadata (10 kB)
Collecting cffi>=2.0.0 (from cryptography>=3.3->paramiko)
  Using cached cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography>=3.3->paramiko)
```

Figure 48: Python environment configured for Nessus API integration

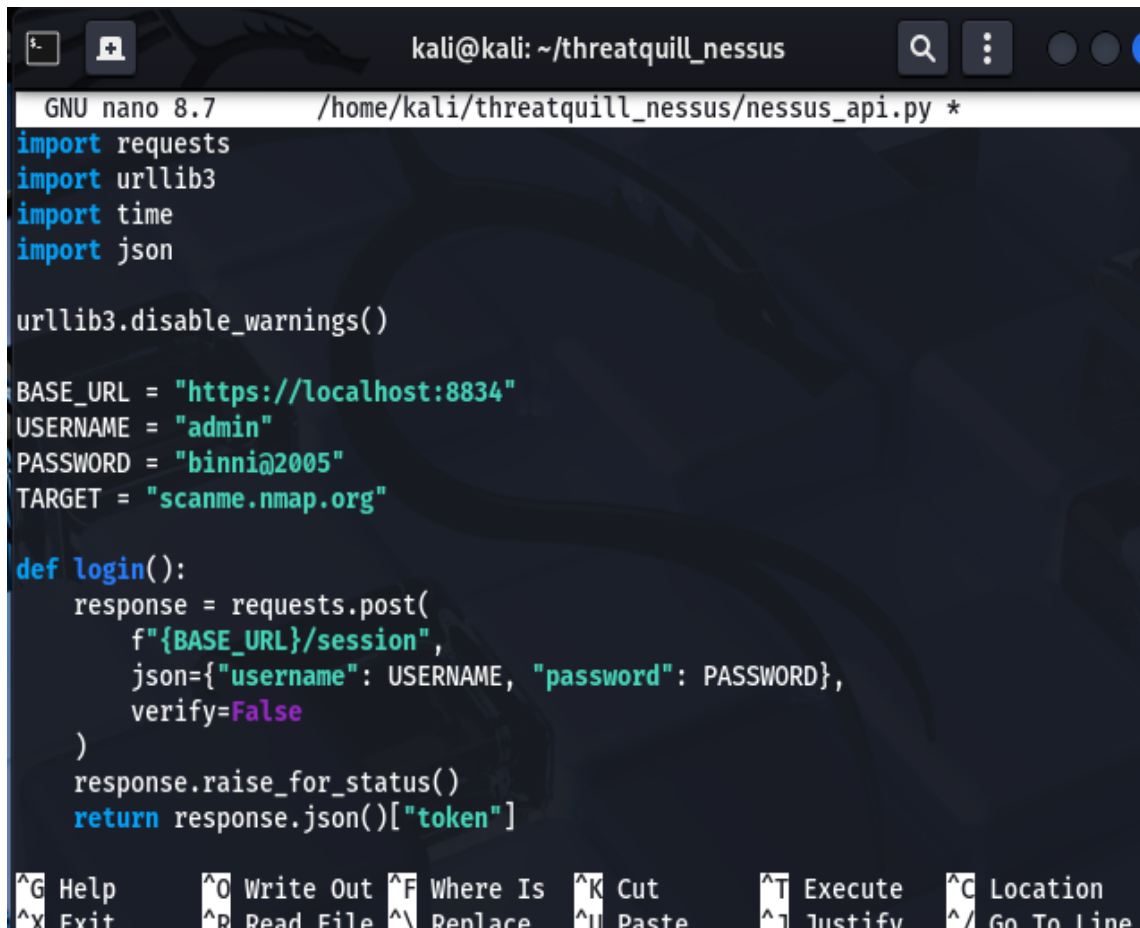
A project directory was created, and a Python script file was initialized to develop the Nessus API integration for automated scanning.

```
(kali@kali)-[~]
└─$ mkdir -p /home/kali/threatquill_nessus
└─cd /home/kali/threatquill_nessus
└─pwd
/home/kali/threatquill_nessus

(kali@kali)-[~/threatquill_nessus]
└─$ nano /home/kali/threatquill_nessus/nessus_api.py
```

Figure 49: Nessus API configured with connection details and authentication

The Nessus API script was developed by importing required libraries and configuring connection details such as URL, credentials, and target for automated scanning.



```
GNU nano 8.7 /home/kali/threatquill_nessus/nessus_api.py *
import requests
import urllib3
import time
import json

urllib3.disable_warnings()

BASE_URL = "https://localhost:8834"
USERNAME = "admin"
PASSWORD = "binni@2005"
TARGET = "scanme.nmap.org"

def login():
    response = requests.post(
        f"{BASE_URL}/session",
        json={"username": USERNAME, "password": PASSWORD},
        verify=False
    )
    response.raise_for_status()
    return response.json()["token"]

^G Help      ^O Write Out ^F Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste    ^J Justify  ^_ Go To Line
```

Figure 50: Successful connection established with Nessus API

The Python virtual environment package was installed on Kali Linux to manage project dependencies in an isolated environment.

```
(kali㉿kali)-[~/threatquill_nessus]
└─$ sudo apt install python3-venv -y
The following packages were automatically installed and are no longer required:
gir1.2-girepository-2.0      libokular6core3
gir1.2-peas-1.0             libpackagekitqt6-1
gir1.2-totem-1.0            libpocketsphinx3
gir1.2-totemlparser-1.0     libportal-gtk3-1
grilo-plugins-0.3           libpostproc58
libavfilter10               libppsddocument-4.0-5
libavformat61               libppsview-4.0-4
libcolorcorrect6            libsimdutf27
libconfig-inifiles-perl     libsphinxbase3t64
libdisplay-info2            libswscale8
libdmapsharing-4.0-3t64     libtotem0
libgirepository-1.0-1       libxml2
libgnome-rr-4-2t64          pocketsphinx-en-us
libgom-1.0-0t64             postgresql-17
libgpgme11t64               postgresql-17-pg-gvm
libgpgmepp6t64              postgresql-client-17
libgrilo-0.3-0               qml6-module-org-kde-notifications
libmad0                      totem
```

Figure 51: Nessus scan configured and initiated for target system

A Python virtual environment was created and activated and required libraries such as requests were installed for handling API communication.

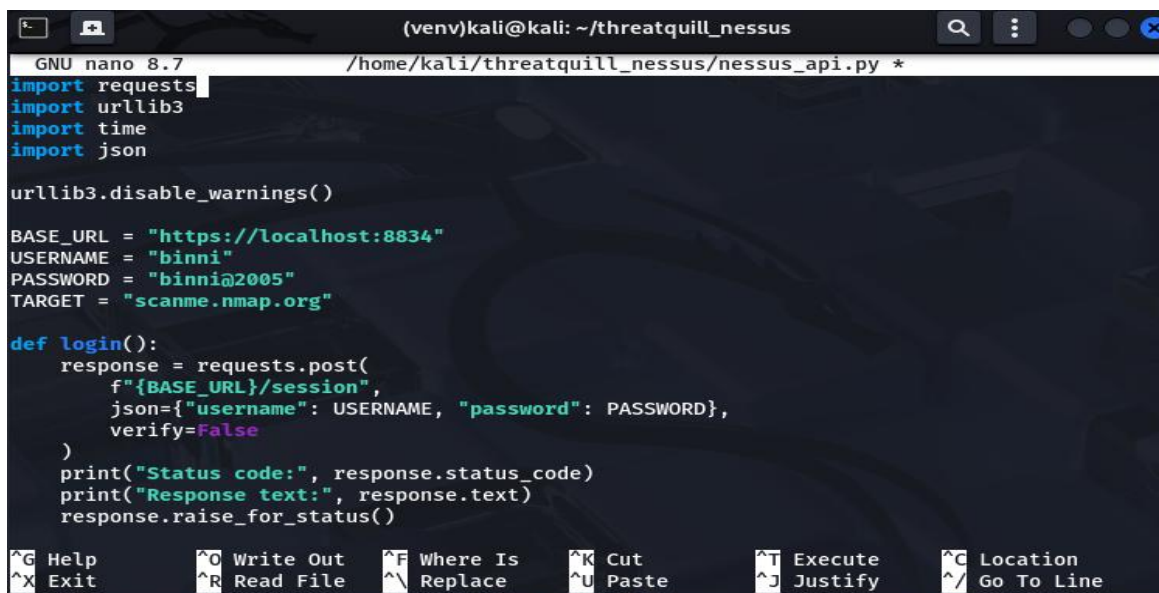
```
(kali㉿kali)-[~/threatquill_nessus]
└─$ cd ~/threatquill_nessus
python3 -m venv venv

(kali㉿kali)-[~/threatquill_nessus]
└─$ source venv/bin/activate

(venv)-(kali㉿kali)-[~/threatquill_nessus]
└─$ pip install requests
Collecting requests
  Downloading requests-2.33.1-py3-none-any.whl.metadata (4.8 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinu
x_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
```

Figure 52: Nessus vulnerability scan in progress on target system

The Nessus API script was updated to include login functionality and response handling for retrieving authentication tokens and verifying connectivity.



```

(venv)kali@kali: ~/threatquill_nessus
GNU nano 8.7 /home/kali/threatquill_nessus/nessus_api.py *
import requests
import urllib3
import time
import json

urllib3.disable_warnings()

BASE_URL = "https://localhost:8834"
USERNAME = "binni"
PASSWORD = "binni@2005"
TARGET = "scanme.nmap.org"

def login():
    response = requests.post(
        f"{BASE_URL}/session",
        json={"username": USERNAME, "password": PASSWORD},
        verify=False
    )
    print("Status code:", response.status_code)
    print("Response text:", response.text)
    response.raise_for_status()
  
```

Figure 53: Nessus API script configured for authentication and scan execution

The Nessus “My Scans” dashboard was opened, showing the workspace where new vulnerability scans can be created and managed.

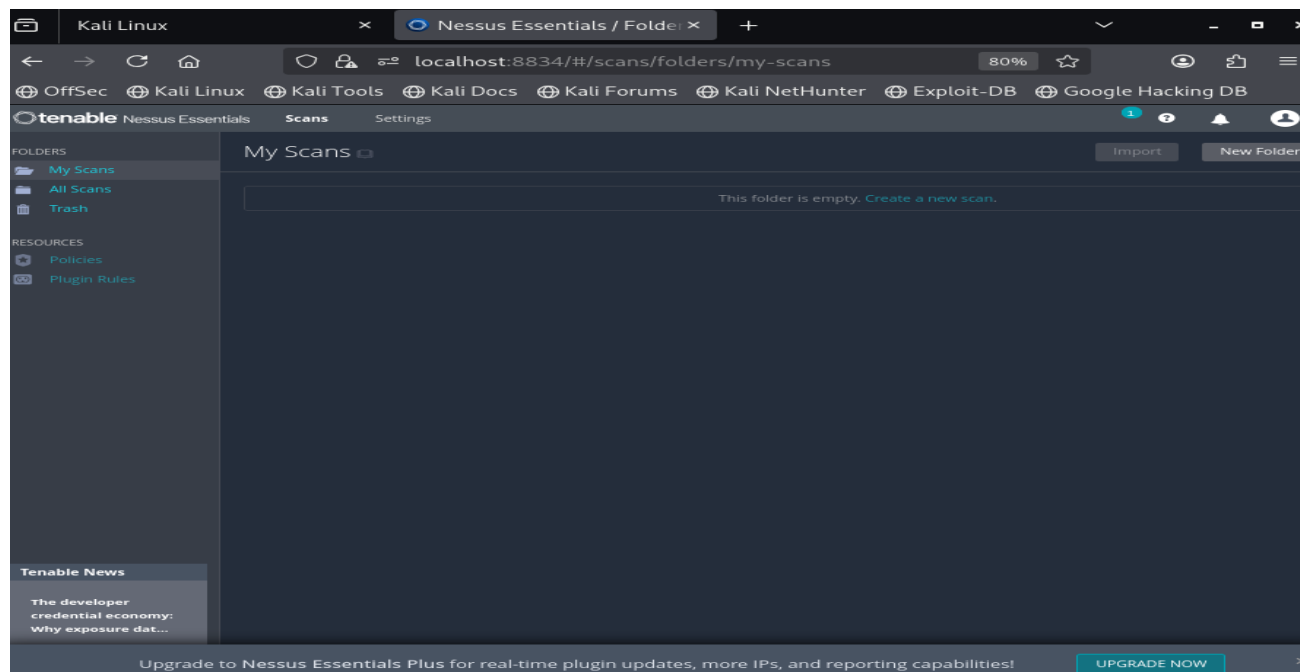


Figure 54: Nessus interface accessed showing scan management dashboard

A Nessus scan was initiated and is currently in processing state, indicating that the vulnerability assessment has started.

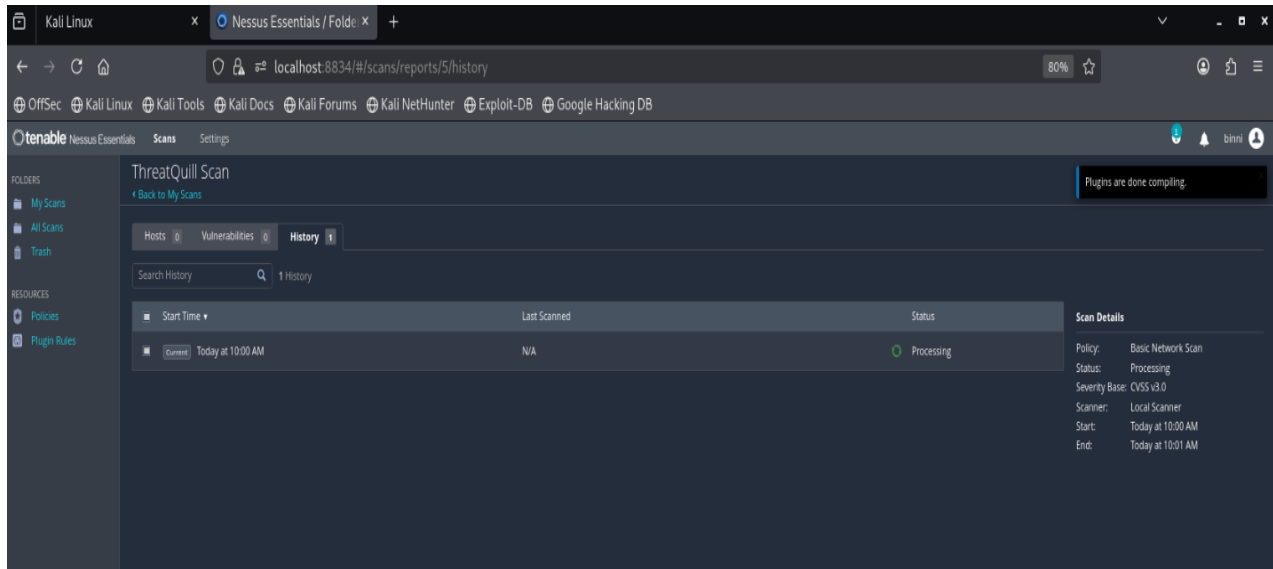


Figure 55: Nessus scan initiated and processing in progress

The scan results displayed detected hosts, and vulnerability counts for the target system.

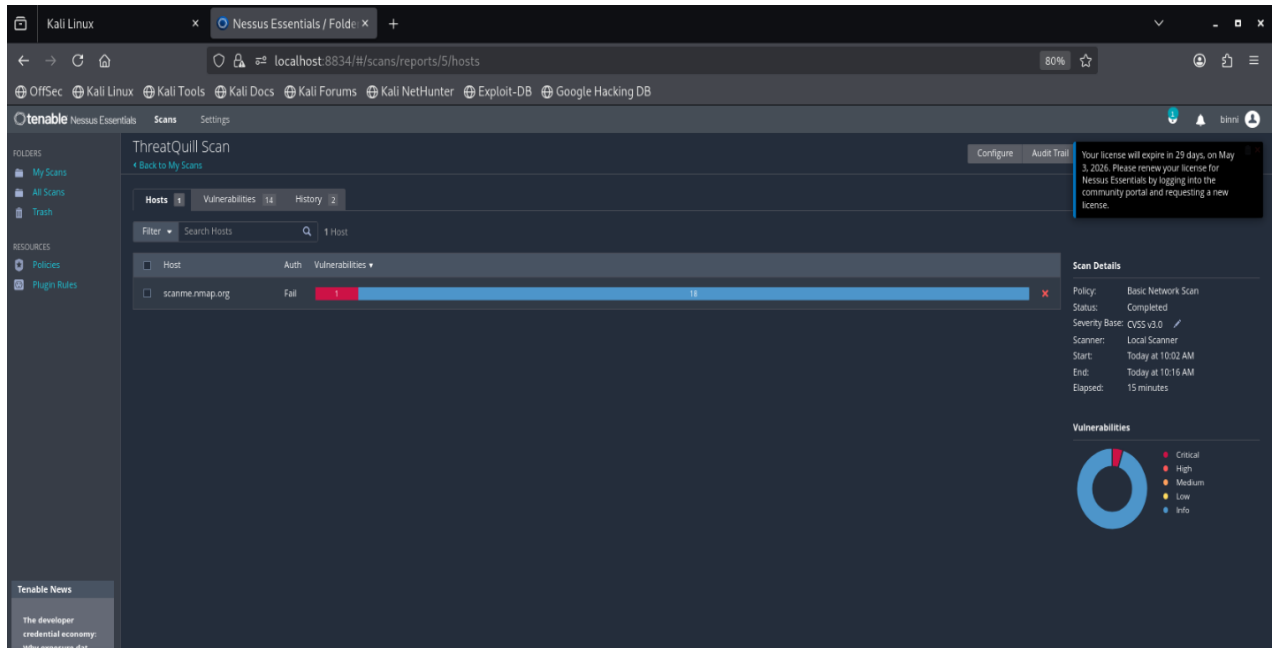


Figure 56: Scan results showing detected vulnerabilities for target host

Detailed vulnerability results were shown, listing identified issues along with severity levels and affected services.

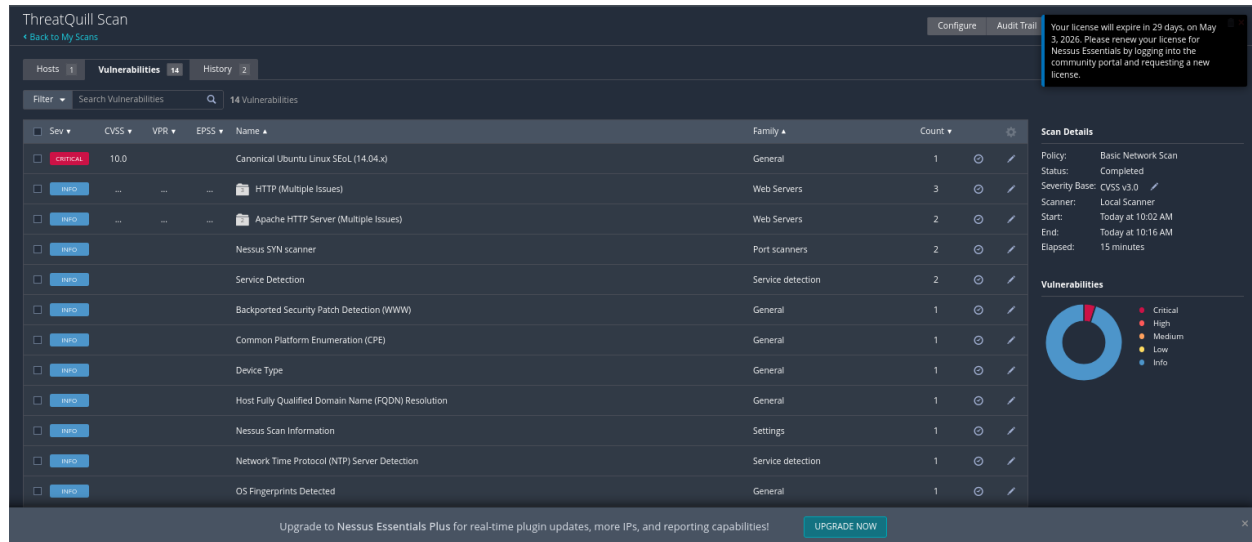


Figure 57: Detailed vulnerability list displayed with severity classification

The scan history was displayed, showing completed and cancelled scans along with their status and timestamps.

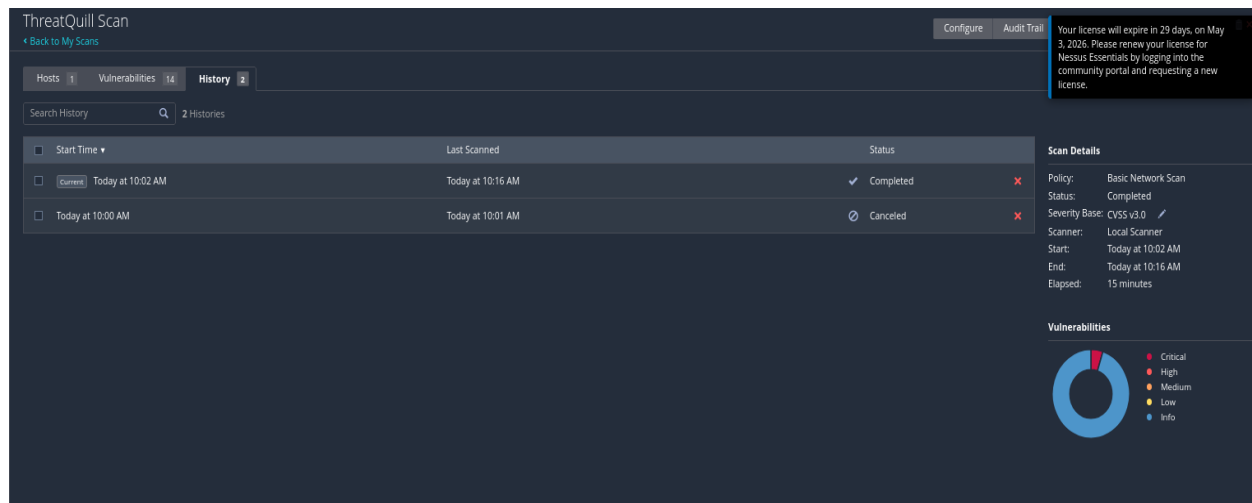


Figure 58: Scan history showing completed and previous scan results

Nessus API endpoints were tested and configured to retrieve scan details dynamically using the scan ID.

```
(venv)-(kali@kali)-[~/threatquill_nessus]
$ nano /home/kali/threatquill_nessus/nessus_api.py

(venv)-(kali@kali)-[~/threatquill_nessus]
$ GET /scans/{scan_id}

(venv)-(kali@kali)-[~/threatquill_nessus]
$ GET /scans/{scan_id}

(venv)-(kali@kali)-[~/threatquill_nessus]
$
```

Figure 59: API requests used to retrieve Nessus scan details

The Nessus API script was executed successfully, displaying total vulnerabilities and categorized severity results

```
(venv)-(kali@kali)-[~/threatquill_nessus]
$ nano nessus_api.py

(venv)-(kali@kali)-[~/threatquill_nessus]
$ python /home/kali/threatquill_nessus/nessus_api.py
Scan Name: Nessus Scan
Total Vulnerabilities: 17
Summary: {'Critical': 1, 'High': 0, 'Medium': 0, 'Low': 0, 'Info': 16}

Vulnerabilities:
{'name': 'Traceroute Information', 'severity': 'Info', 'count': 1, 'plugin_id': 10287}
{'name': 'OS Identification', 'severity': 'Info', 'count': 1, 'plugin_id': 11936}
{'name': 'OS Fingerprints Detected', 'severity': 'Info', 'count': 1, 'plugin_id': 20965}
{'name': 'Network Time Protocol (NTP) Server Detection', 'severity': 'Info', 'count': 1, 'plugin_id': 10884}
{'name': 'Nessus Scan Information', 'severity': 'Info', 'count': 1, 'plugin_id': 19506}
{'name': 'HyperText Transfer Protocol (HTTP) Information', 'severity': 'Info', 'count': 1, 'plugin_id': 24260}
{'name': 'HTTP Server Type and Version', 'severity': 'Info', 'count': 1, 'plugin_id': 10107}
{'name': 'HTTP Methods Allowed (per directory)', 'severity': 'Info', 'count': 1, 'plugin_id': 43111}
{'name': 'Host Fully Qualified Domain Name (FQDN) Resolution', 'severity': 'Info', 'count': 1, 'plugin_id': 12053}
{'name': 'Device Type', 'severity': 'Info', 'count': 1, 'plugin_id': 54615}
{'name': 'Common Platform Enumeration (CPE)', 'severity': 'Info', 'count': 1, 'plugin_id': 45590}
{'name': 'Backported Security Patch Detection (WWW)', 'severity': 'Info', 'count': 1, 'plugin_id': 10107}
```

Figure 60: Nessus scan results fetched using Python script with severity summary

A custom Nessus scanner module was developed and executed to fetch detailed scan results and integrate them into the ThreatQuill system.

```
(venv)-(kali@kali)-[~/ThreatQuill]
└─$ mkdir scanners
└─nano scanners/nessus_scanner.py

(venv)-(kali@kali)-[~/ThreatQuill]
└─$ python scanners/nessus_scanner.py
[NESSUS] Logging into Nessus...
[NESSUS] Login successful.
[NESSUS] Fetching scan details for scan ID 5...
SUMMARY: 1 1 17 Running
[NESSUS] Scan name: Nessus Scan
[NESSUS] Scan status: completed
[NESSUS] Hosts found: 1
[NESSUS] Vulnerabilities found: 17
[NESSUS] Severity summary -> Critical: 1, High: 0, Medium: 0, Low: 0, Info: 16
[NESSUS] Fetching detailed host results for host: scanme.nmap.org
Nessus scanme.nmap.org - - open Traceroute Information Info
[NESSUS] scanme.nmap.org | port=- | proto=- | severity=Info | Traceroute Information
Nessus scanme.nmap.org - - open OS Identification Info
[NESSUS] scanme.nmap.org | port=- | proto=- | severity=Info | OS Identification
Nessus scanme.nmap.org - - open OS Fingerprints Detected Info
[NESSUS] scanme.nmap.org | port=- | proto=- | severity=Info | OS Fingerprints Detecte
d
Nessus scanme.nmap.org - - open Network Time Protocol (NTP) Server Detection Info
[NESSUS] scanme.nmap.org | port=- | proto=- | severity=Info | Network Time Protocol (
NTP) Server Detection
Nessus scanme.nmap.org - - open Nessus Scan Information Info
```

Figure 61: Nessus scanner integrated and executed within ThreatQuill system

The ThreatQuill Analyzer dashboard showed scan results, including summary, detected threats, and logs from different scanners.

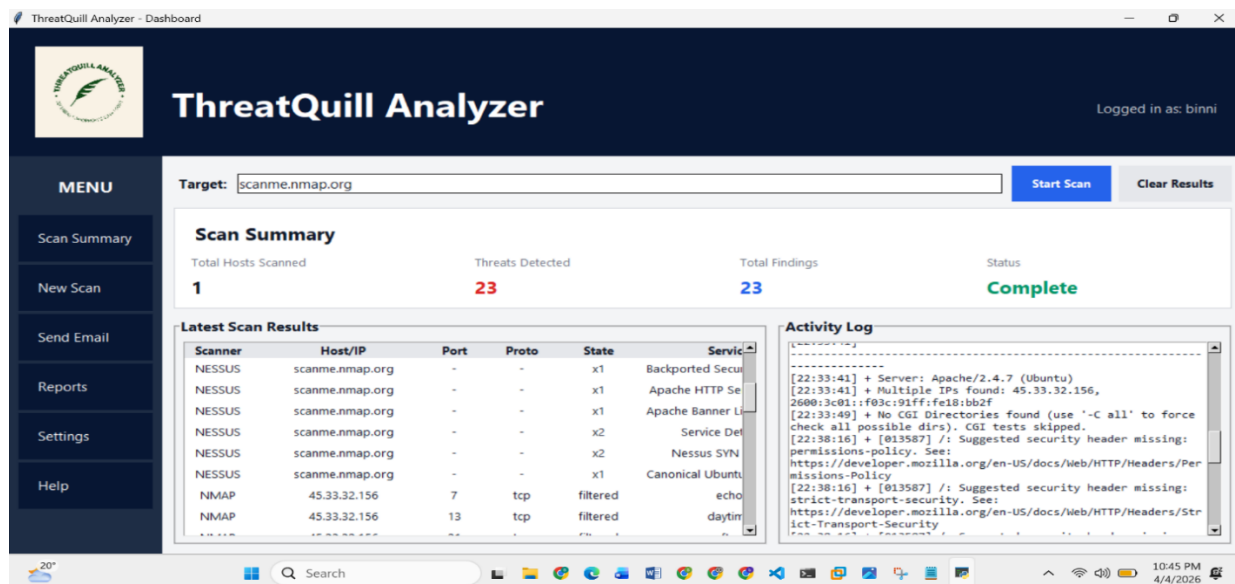
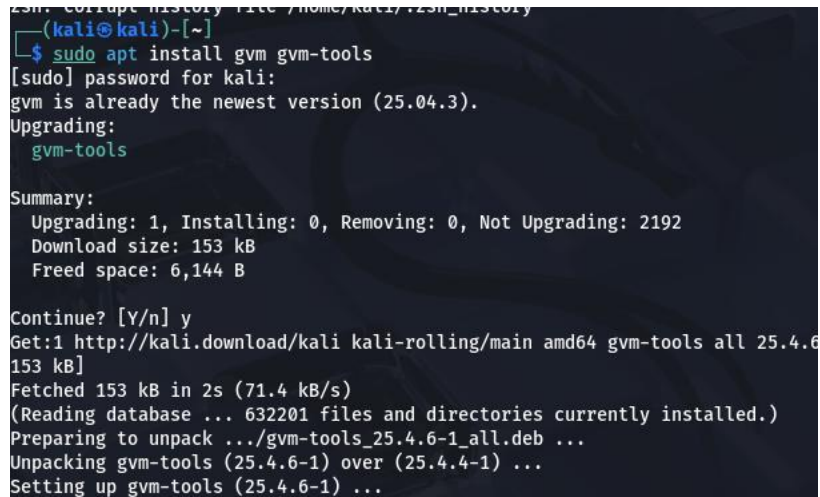


Figure 62: Nessus scan results integrated into dashboard with detected vulnerabilities

4.4 OpenVAS Implementation

An OpenVAS (GVM) scanner was configured and executed to perform vulnerability scanning and integrate the results into the ThreatQuill system.

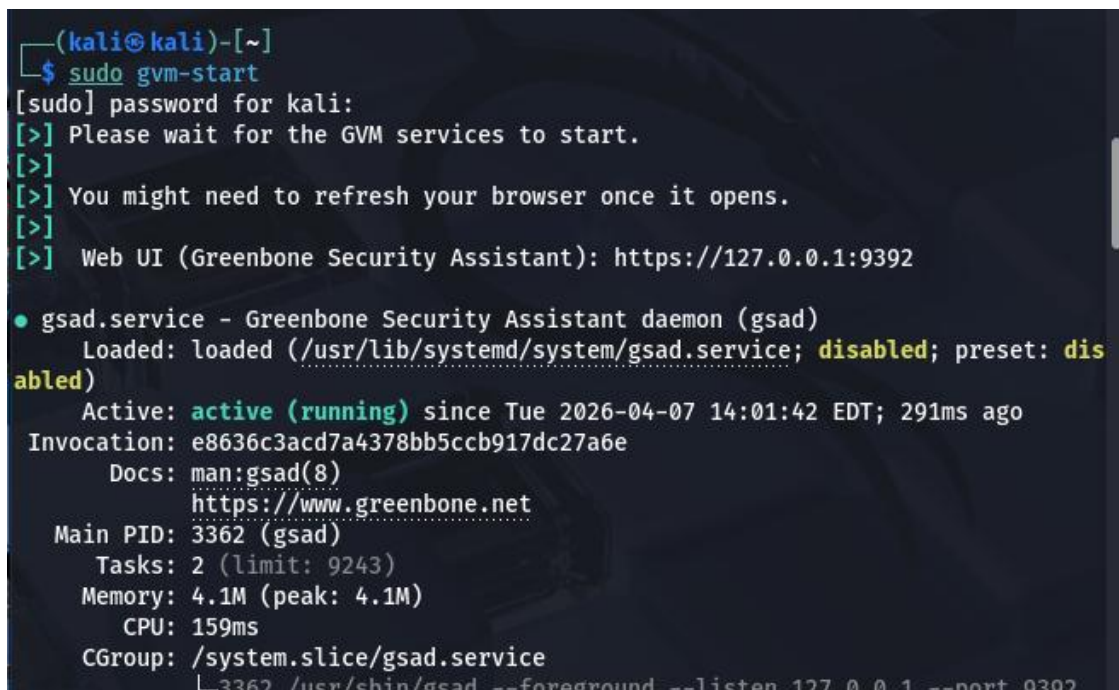


```

kali: Corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
└─$ sudo apt install gvm gvm-tools
[sudo] password for kali:
gvm is already the newest version (25.04.3).
Upgrading:
  gvm-tools
Summary:
  Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 2192
  Download size: 153 kB
  Freed space: 6,144 B
Continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 gvm-tools all 25.4.6-1 [153 kB]
Fetched 153 kB in 2s (71.4 kB/s)
(Reading database ... 632201 files and directories currently installed.)
Preparing to unpack .../gvm-tools_25.4.6-1_all.deb ...
Unpacking gvm-tools (25.4.6-1) over (25.4.4-1) ...
Setting up gvm-tools (25.4.6-1) ...
  
```

Figure 63: OpenVAS services started successfully for vulnerability scanning

The OpenVAS (GVM) services were successfully started and the web interface was launched to enable vulnerability scanning within the ThreatQuill system.



```

(kali@kali)-[~]
└─$ sudo gvm-start
[sudo] password for kali:
[>] Please wait for the GVM services to start.
[>]
[>] You might need to refresh your browser once it opens.
[>]
[>] Web UI (Greenbone Security Assistant): https://127.0.0.1:9392

● gsad.service - Greenbone Security Assistant daemon (gsad)
   Loaded: loaded (/usr/lib/systemd/system/gsad.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-04-07 14:01:42 EDT; 291ms ago
 Invocation: e8636c3acd7a4378bb5ccb917dc27a6e
    Docs: man:gsad(8)
          https://www.greenbone.net
   Main PID: 3362 (gsad)
     Tasks: 2 (limit: 9243)
    Memory: 4.1M (peak: 4.1M)
       CPU: 159ms
    CGroup: /system.slice/gsad.service
            └─3362 /usr/sbin/gsad --foreground --listen 127.0.0.1 --port 9392
  
```

Figure 64: OpenVAS setup verified ensuring all required services are configured correctly

The OpenVAS setup was verified using gvm-check-setup to ensure all components and services were properly configured and ready for vulnerability scanning.

```
(kali㉿kali)-[~]  
$ sudo gvm-check-setup  
gvm-check-setup 25.04.0  
This script is provided and maintained by Debian and Kali.  
Test completeness and readiness of GVM-25.04.0  
Step 1: Checking OpenVAS (Scanner)...  
OK: OpenVAS Scanner is present in version 23.35.3.  
OK: Notus Scanner is present in version 22.7.2.  
OK: Server CA Certificate is present as /var/lib/gvm/CA/servercert.pem.  
Checking permissions of /var/lib/openvas/gnupg/*  
OK: redis-server is present.  
OK: scanner (db_address setting) is configured properly using the redis-  
server socket: /var/run/redis-openvas/redis-server.sock  
OK: the mqtt_server_uri is defined in /etc/openvas/openvas.conf  
OK: _gvm owns all files in /var/lib/openvas/plugins  
OK: NVT collection in /var/lib/openvas/plugins contains 95086 NVTs.  
OK: The notus directory /var/lib/notus/products contains 512 NVTs.  
Checking that the obsolete redis database has been removed  
OK: No old Redis DB  
OK: ospd-openvas service is active.
```

Figure 65: OpenVAS vulnerability feeds synchronized for updated scan data

The OpenVAS (GVM) services started using the gvm-start command, successfully launching the scanner and web interface for vulnerability assessment.

```
(kali㉿kali)-[~]  
$ sudo gvm-start  
[i] GVM services are already running  
  
(kali㉿kali)-[~]  
$
```

Figure 66: OpenVAS services verified and confirmed to be running successfully

System packages were updated and required dependencies, including Docker, were installed and configured to prepare the environment for OpenVAS (GVM) deployment.

```
(kali㉿kali)-[~]
└─$ sudo apt update
sudo apt install ca-certificates curl gnupg docker.io docker-compose-plugin -y
sudo usermod -aG docker $USER
newgrp docker
Get:1 http://mirror.nju.edu.cn/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirror.nju.edu.cn/kali kali-rolling/main amd64 Packages [21.1 MB]
Get:3 http://mirror.nju.edu.cn/kali kali-rolling/main amd64 Contents (deb) [53.6 MB]
Get:4 http://mirror.nju.edu.cn/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:5 http://mirror.nju.edu.cn/kali kali-rolling/contrib amd64 Contents (deb) [277 kB]
Get:6 http://mirror.nju.edu.cn/kali kali-rolling/non-free amd64 Packages [185 kB]
Ign:6 http://mirror.nju.edu.cn/kali kali-rolling/non-free amd64 Packages
Get:7 http://mirror.nju.edu.cn/kali kali-rolling/non-free amd64 Contents (deb) [905 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [185 kB]
```

Figure 67: System packages updated and dependencies installed for OpenVAS setup

APT cache was cleared and package lists were refreshed to ensure a clean and up to date environment for OpenVAS setup.

```
(kali㉿kali)-[~]
└─$ sudo rm -rf /var/lib/apt/lists/*
sudo apt clean
sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.1 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.6 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [277 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [185 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [905 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [14.3 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [33.8 kB]
Fetched 76.3 MB in 1min 10s (1,096 kB/s)
629 packages can be upgraded. Run 'apt list --upgradable' to see them.
(kali㉿kali)-[~]
└─$
```

Figure 68: System cleaned and updated to ensure a stable environment for deployment

Unnecessary Docker-related packages were removed to clean the system before setting up OpenVAS.

```
(kali@kali)-[~]
$ sudo apt remove -y docker.io docker-compose docker-doc podman-docker contain
erd runc
Package 'docker-doc' is not installed, so not removed
Package 'podman-docker' is not installed, so not removed
The following packages were automatically installed and are no longer required:
  binutils-mingw-w64-base          liblua5.1-common
  binutils-mingw-w64-i686         libmodule-find-perl
  binutils-mingw-w64-x86_64       libmupdf25.1
  criu                             libnids1.21t64
  docker-buildx                   liborcus-0.20-0
  docker-cli                       liborcus-parser-0.20-0
  dsniff                           libplacebo351
  ettercap-common                  libproc-processtable-perl
  ettercap-graphical               librubberband2
  gcc-mingw-w64-base               libsframe2
  gcc-mingw-w64-i686-win32         libsnmp40t64
  gcc-mingw-w64-i686-win32-runtime libsort-naturally-perl
  gcc-mingw-w64-x86_64-win32       libtsk19t64
  gcc-mingw-w64-x86_64-win32-runtime libvdpau-vdgl
  ibus-gtk                         linux-image-6.12.38+kali-amd64
  libaio                           mesa-vdpau-drivers
  libaudio2                        mingw-w64-common
  libboost-filesystem1.83.0        mingw-w64-i686-dev
```

Figure 69: Existing Docker packages removed to avoid conflicts during installation

Docker repository and security key were added to the system to allow installation.

```
(kali@kali)-[~]
$ sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg
] https://download.docker.com/linux/debian bookworm stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

(kali@kali)-[~]
$
```

Figure 70: Docker repository configured for installation of required components

Docker and its required components were installed successfully.

```
(kali@kali)-[~]
$ sudo apt update
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin d
ocker-compose-plugin
Get:1 https://download.docker.com/linux/debian bookworm InRelease [46.6 kB]
Get:2 https://download.docker.com/linux/debian bookworm/stable amd64 Packages [6
7.3 kB]
Get:3 https://download.docker.com/linux/debian bookworm/stable amd64 Contents (d
eb) [1,719 B]
Hit:4 http://http.kali.org/kali kali-rolling InRelease
Fetched 116 kB in 1s (142 kB/s)
628 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
binutils-mingw-w64-base libmodule-find-perl
binutils-mingw-w64-i686 libmupdf25.1
binutils-mingw-w64-x86-64 libnids1.21t64
criu liborcus-0.20-0
dsniff liborcus-parser-0.20-0
ettercap-common libplacebo351
ettercap-graphical libproc-processtable-perl
gcc-mingw-w64-base librubberband2
gcc-mingw-w64-i686-win32 libsframe2
gcc-mingw-w64-i686-win32-runtime libsnmp40t64
```

Figure 71: Docker engine and related tools installed successfully

Docker service was started and verified by running a test container.

```
(kali@kali)-[~]
$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/sys
temd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

(kali@kali)-[~]
$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
```

Figure 72: Docker service enabled and verified using test container execution

The OpenVAS setup file was downloaded, and containers started to begin the installation process.

```
(kali㉿kali)-[~]
$ docker compose version
Docker Compose version v5.1.3

(kali㉿kali)-[~]
$ mkdir -p ~/greenbone-ce
cd ~/greenbone-ce
curl -f -L https://greenbone.github.io/docs/latest/_static/docker-compose.yml -o compose
.yaml
docker compose -f compose.yaml up -d
```

% Total	% Received	% Xferd	Average Dload	Speed Upload	Time Total	Time Spent	Time Left	Current Speed
100	8670	100	8670	0	0	16155	0	0

```
[+] up 2/156
:: Image registry.community.greenbone.net/commu... [..      ] Pulling 48.7s
:: Image registry.community.greenbone.net/community/gpg-data [  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community/... [::  ] Pulling 48.7s
:: Image registry.community.greenbone.net/communi... [:::  ] Pulling 48.7s
:: Image registry.community.greenbone.net/communit... [:::  ] Pulling 48.7s
:: Image registry.community.greenbone.net/communit... [  ] Pulling 48.7s
:: Image registry.community.greenbone.... [ ] Pulling 48.7s
:: Image registry.community.greenbone.net/communit... [  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community... [  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community... [.... ] Pulling 48.7s
:: Image registry.community.greenbone.net... [  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community/... [::  ] Pulling 48.7s
:: Image registry.community.greenbone.net... [.... ] Pulling 48.7s
:: Image registry.community.greenbone.net/community... [::  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community/vulne... [::  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community/g... [  ] Pulling 48.7s
:: Image registry.community.greenbone.net/community/gvm... [..  ] Pulling 48.7s
:: Image registry.community.greenbone.net/commu... [::: ] 11.79MB / 1.273GB Pulling 48.7s
:: Image registry.community.greenbone.net/commun... [::  ] Pulling 48.7s
```

Figure 73: Greenbone Community Edition files downloaded for OpenVAS setup

All required OpenVAS components were downloaded as Docker images for the system setup.

```

+] up 188/190
✓ Image registry.community.greenbone.net/community/gvm-tools Pulled 983.2s
✓ Image registry.community.greenbone.net/community/gpg-data Pulled 271.3s
✓ Image registry.community.greenbone.net/community/cert-bund-data Pulled 875.9s
✓ Image registry.community.greenbone.net/community/gvmd:stable Pulled 1410.3s
✓ Image registry.community.greenbone.net/community/data-objects Pulled 169.4s
✓ Image registry.community.greenbone.net/community/nginx:latest Pulled 1057.7s
✓ Image registry.community.greenbone.net/community/ospd-openvas:stable Pulled 755.9s
✓ Image registry.community.greenbone.net/community/redis-server Pulled 733.6s
✓ Image registry.community.greenbone.net/community/pg-gvm:stable Pulled 375.6s
✓ Image registry.community.greenbone.net/community/gsad:stable Pulled 283.2s
✓ Image registry.community.greenbone.net/community/pg-gvm-migrator:s... Pulled 1003.8s
✓ Image registry.community.greenbone.net/community/report-formats Pulled 822.6s
✓ Image registry.community.greenbone.net/community/openvas-scanner:s... Pulled 167.5s
✓ Image registry.community.greenbone.net/community/dfn-cert-data Pulled 920.1s
✓ Image registry.community.greenbone.net/community/vulnerability-tests Pulled 1978.2s
✓ Image registry.community.greenbone.net/community/gsa:stable-slim Pulled 810.6s
✓ Image registry.community.greenbone.net/community/gvm-config:latest Pulled 376.8s
✓ Image registry.community.greenbone.net/community/scap-data Pulled 1630.3s
✓ Image registry.community.greenbone.net/community/notus-data Pulled 341.9s
✓ Network greenbone-community-edition_default Created 1.1s
✓ Volume greenbone-community-edition_vt_data_vol Created 0.1s
✓ Volume greenbone-community-edition_gsa_data_vol Created 0.0s
✓ Volume greenbone-community-edition_mysql_socket_vol Created 0.1s
✓ Volume greenbone-community-edition_nginx_certificates_vol Created 0.1s
✓ Volume greenbone-community-edition_openvas_log_data_vol Created 0.0s
✓ Volume greenbone-community-edition_ospd_openvas_socket_vol Created 0.1s
✓ Volume greenbone-community-edition_gpg_data_vol Created 0.0s
✓ Volume greenbone-community-edition_gvmd_socket_vol Created 0.2s
✓ Volume greenbone-community-edition_mysql_data_vol Created 0.3s
✓ Volume greenbone-community-edition_nginx_config_vol Created 0.0s
✓ Volume greenbone-community-edition_notus_data_vol Created 0.0s
✓ Volume greenbone-community-edition_data_objects_vol Created 0.1s
... 21 more

```

Figure 74: OpenVAS Docker images pulled successfully from repository

Necessary OpenVAS service images were pulled from the repository to prepare for running the system.

```

—(kali@kali)-[~/greenbone-ce]
$ curl -f -L https://greenbone.github.io/docs/latest/_static/docker-compose.yml -o
compose.yaml
% Total % Received % Xferd Average Speed Time Time Time Current
00 8670 100 8670 0 0 10754 0 0 0

—(kali@kali)-[~/greenbone-ce]
$ docker compose -f compose.yaml pull
+] pull 31/41
✓ Image registry.community.greenbone.net/communi... Pulled 9.7s
✓ Image registry.community.greenbone.net/... Pulled 9.6s
✓ Image registry.community.greenbone.net/com... Pulled 9.6s
✓ Image registry.community.greenbone.net/comm... Pulled 9.6s
: Image registry.community.greenbone.net/co... [■] Pulling 187.5s
✓ Image registry.community.greenbone.net/commu... Pulled 119.8s
✓ Image registry.community.greenbone.net/com... Pulled 24.3s
: Image registry.community.greenbone.net/... [■] 44.92MB / 228.1MB Pulling 187.5s
: Image registry.community.greenbone.net/c... [■] 71.85MB / 1.271GB Pulling 187.5s
✓ Image registry.community.greenbone.net/community... Pulled 9.3s
✓ Image registry.community.greenbone.net/commu... Pulled 7.7s
✓ Image registry.community.greenbone.net/com... Pulled 9.3s
✓ Image registry.community.greenbone.net/community/gpg-data Pulled 9.9s
✓ Image registry.community.greenbone.net/comm... Pulled 9.6s
✓ Image registry.community.greenbone.net/community/gvm-tools Pulled 9.7s
✓ Image registry.community.greenbone.net/com... Pulled 9.7s
✓ Image registry.community.greenbone.net/com... Pulled 8.0s
✓ Image registry.community.greenbone.net/commun... Pulled 126.8s
✓ Image registry.community.greenbone.net/commun... Pulled 76.2s

```

Figure 75: OpenVAS Docker containers created and initialized successfully

Core data services like SCAP and CERT were started to support vulnerability scanning.

```
(kali@kali)-[~/greenbone-ce]
$ docker compose -f ~/greenbone-ce/compose.yaml up -d scap-data dfn-cert-data
[+] up 4/4
✓ Network greenbone-community-edition_default          Created           0.2s
✓ Container greenbone-community-edition-scap-data-1    Started           1.3s
✓ Container greenbone-community-edition-cert-bund-data-1 Healthy           6.8s
✓ Container greenbone-community-edition-dfn-cert-data-1 Started           8.8s

(kali@kali)-[~/greenbone-ce]
$ docker compose -f ~/greenbone-ce/compose.yaml ps
NAME                                COMMAND          SERVICE          CREATED          STATUS
greenbone-community-edition-cert-bund-data-1 registry.community.greenbone.net/community/cert-bund-data "/bin/init.sh" cert-bund-data 24 seconds ago Up 23 seconds (healthy)
greenbone-community-edition-dfn-cert-data-1 registry.community.greenbone.net/community/dfn-cert-data "/bin/init.sh" dfn-cert-data 24 seconds ago Up 16 seconds (healthy)
greenbone-community-edition-scap-data-1 registry.community.greenbone.net/community/scap-data "/bin/init.sh" scap-data 24 seconds ago Up 23 seconds (health: starting)
```

Figure 76: OpenVAS container status showing running and healthy services

Some OpenVAS components failed to start initially due to dependency issues in the setup.

```
(kali@kali)-[~/greenbone-ce]
$ docker compose -f ~/greenbone-ce/compose.yaml up -d
[+] up 21/21
✓ Container greenbone-community-edition-gvm-config-1    Exited           110.2s
✗ Container greenbone-community-edition-vulnerability-tests-1 Error depe... 726.9s
✓ Container greenbone-community-edition-notus-data-1    Healthy          82.9s
✓ Container greenbone-community-edition-redis-server-1  Started          65.8s
✓ Container greenbone-community-edition-scap-data-1     Healthy          79.2s
✓ Container greenbone-community-edition-data-objects-1  Healthy          79.2s
✓ Container greenbone-community-edition-cert-bund-data-1 Healthy          79.2s
✓ Container greenbone-community-edition-dfn-cert-data-1 Healthy          79.3s
✓ Container greenbone-community-edition-pg-gvm-migrator-1 Exited           76.8s
✓ Container greenbone-community-edition-configure-openvas-1 Exited           72.7s
✓ Container greenbone-community-edition-gsa-1           Healthy          110.2s
✓ Container greenbone-community-edition-gpg-data-1     Exited           71.5s
✓ Container greenbone-community-edition-report-formats-1 Healthy          51.9s
✓ Container greenbone-community-edition-openvas-1       Started          21.6s
✓ Container greenbone-community-edition-openvasd-1     Created          0.4s
✓ Container greenbone-community-edition-pg-gvm-1       Started          22.3s
✓ Container greenbone-community-edition-osspd-openvas-1 Created          5.2s
✓ Container greenbone-community-edition-gvmd-1         Started          50.1s
✓ Container greenbone-community-edition-gsad-1         Started          47.8s
✓ Container greenbone-community-edition-gvm-tools-1    Created          0.7s
✓ Container greenbone-community-edition-nginx-1        Started          49.5s
dependency failed to start: container greenbone-community-edition-vulnerability-tests-1 is unhealthy

(kali@kali)-[~/greenbone-ce]
```

Figure 77: Docker container logs showing OpenVAS service initialization details

Running containers were checked to verify which OpenVAS services were active and which were not.

```
(kali@kali)-[~]
$ cd ~/greenbone-ce

(kali@kali)-[~/greenbone-ce]
$ docker compose -f compose.yaml ps
```

NAME	COMMAND	IMAGE	SERVICE	CREATED	STATUS
greenbone-community-edition-gsad-1	registry.community.greenbone.net/communi	gsad	2 hours ago	Up 9 minutes	
ty/gsad:stable	"/usr/local/bin/entr..."				
greenbone-community-edition-gvmd-1	registry.community.greenbone.net/communi	gvmd	2 hours ago	Up 9 minutes	
ty/gvmd:stable	"/usr/local/bin/entr..."				
(unhealthy)					
greenbone-community-edition-nginx-1	registry.community.greenbone.net/communi	nginx	2 hours ago	Up 9 minutes	
ty/nginx:latest	"/docker-entrypoint..."				
	127.0.0.1:443->443/tcp, 80/tcp, 127.0.0.1:9392->9392/tcp				
greenbone-community-edition-pg-gvm-1	registry.community.greenbone.net/communi	pg-gvm	2 hours ago	Up 18 second	
ty/pg-gvm:stable	"/usr/local/bin/entr..."				
s (health: starting)					
greenbone-community-edition-redis-server-1	registry.community.greenbone.net/communi	redis-server	2 hours ago	Up 9 minutes	
ty/redis-server	"docker-entrypoint.s..."				
	6379/tcp				

```
(kali@kali)-[~/greenbone-ce]
$
```

Figure 78: OpenVAS services restarted to resolve configuration issues

System logs were reviewed to identify errors related to database and service initialization.

```
(kali@kali)-[~/greenbone-ce]
$ docker compose -f ~/greenbone-ce/compose.yaml logs gvmd
```

```
gvmd-1 | connection
gvmd-1 | -----
gvmd-1 | connected
gvmd-1 | (1 row)
gvmd-1 |
gvmd-1 | md main:MESSAGE:2026-04-18 06h55.37 utc:21: Greenbone Vulnerability Ma
nager version 26.23.0 (DB revision 273)
gvmd-1 | md main: INFO:2026-04-18 06h55.37 utc:21: Migrating database.
gvmd-1 | md main:WARNING:2026-04-18 06h55.37 utc:21: manage_migrate: no task table
s yet, so no need to migrate them
gvmd-1 | md main:MESSAGE:2026-04-18 06h55.37 utc:21: No SCAP database found for mi
gration
gvmd-1 | md main:MESSAGE:2026-04-18 06h55.37 utc:21: No CERT database found for mi
gration
gvmd-1 | md main: INFO:2026-04-18 06h55.37 utc:21: gvmd: databases are already a
t the supported version
gvmd-1 | md main:MESSAGE:2026-04-18 06h55.37 utc:30: Greenbone Vulnerability Ma
nager version 26.23.0 (DB revision 273)
gvmd-1 | md manage: INFO:2026-04-18 06h55.37 utc:30: Creating user.
gvmd-1 | md manage:WARNING:2026-04-18 06h55.37 utc:30: sql_exec_internal: PQexec fai
led: ERROR: relation "public.meta" does not exist
gvmd-1 | LINE 1: SELECT value FROM public.meta WHERE name = 'database version...
```

Figure 79: Docker containers verified after restart showing active services

The OpenVAS manager service was restarted to fix initialization and connection issues.

```
(kali@kali)-[~/greenbone-ce]
$ docker compose -f ~/greenbone-ce/compose.yaml restart gvmd
[+] restart 0/1
  Container greenbone-community-edition-gvmd-1 Restarting 11.9s

(kali@kali)-[~/greenbone-ce]
$ docker compose -f ~/greenbone-ce/compose.yaml ps
NAME                                IMAGE                                SERVICE    CREATED        STATUS
greenbone-community-edition-gsad-1  registry.community.greenbone.net/communi  gsad       2 hours ago    Up 27 minute
ty/gsad:stable                       "/usr/local/bin/entr..."
s
greenbone-community-edition-gvmd-1  registry.community.greenbone.net/communi  gvmd       2 hours ago    Up 14 minute
ty/gvmd:stable                       "/usr/local/bin/entr..."
s (healthy)
greenbone-community-edition-nginx-1  registry.community.greenbone.net/communi  nginx      2 hours ago    Up 27 minute
ty/nginx:latest                     "/docker-entrypoint..."
s 127.0.0.1:443->443/tcp, 80/tcp, 127.0.0.1:9392->9392/tcp
greenbone-community-edition-pg-gvm-1  registry.community.greenbone.net/communi  pg-gvm     2 hours ago    Up 14 minute
ty/pg-gvm:stable                    "/usr/local/bin/entr..."
s (healthy)
greenbone-community-edition-redis-server-1  registry.community.greenbone.net/communi  redis-server 2 hours ago    Up 27 minute
ty/redis-server                    "docker-entrypoint.s..."
s 6379/tcp
```

Figure 80: OpenVAS container status showing running and healthy services

All required OpenVAS containers were successfully running and the system became ready for use.

```
(kali@kali)-[~/greenbone-ce]
$ docker compose -f compose.yaml ps
NAME                                IMAGE                                SERVICE    CREATED        STATUS
greenbone-community-edition-cert-bund-data-1  registry.community.greenbone  cert-bund-data 8 hours ago    Up 37 minutes (healthy)
.net/community/cert-bund-data                "/bin/init.sh"
greenbone-community-edition-data-objects-1  registry.community.greenbone  data-objects   8 hours ago    Up 37 minutes (healthy)
.net/community/data-objects                  "/bin/init.sh"
greenbone-community-edition-dfn-cert-data-1  registry.community.greenbone  dfn-cert-data  8 hours ago    Up 37 minutes (healthy)
.net/community/dfn-cert-data                 "/bin/init.sh"
greenbone-community-edition-gsa-1            registry.community.greenbone  gsa            8 hours ago    Up 37 minutes (healthy)
.net/community/gsa:stable-slim               "/usr/local/bin/entr..."
greenbone-community-edition-gsad-1           registry.community.greenbone  gsad           8 hours ago    Up 41 minutes
.net/community/gsad:stable                   "/usr/local/bin/entr..."
greenbone-community-edition-gvmd-1           registry.community.greenbone  gvmd           8 hours ago    Up 41 minutes (healthy)
.net/community/gvmd:stable                   "/usr/local/bin/entr..."
greenbone-community-edition-nginx-1          registry.community.greenbone
```

Figure 81: Final OpenVAS container status showing all services running successfully

The OpenVAS dashboard was accessed via the web interface, where feed synchronization was in progress, preparing the system for vulnerability scanning.

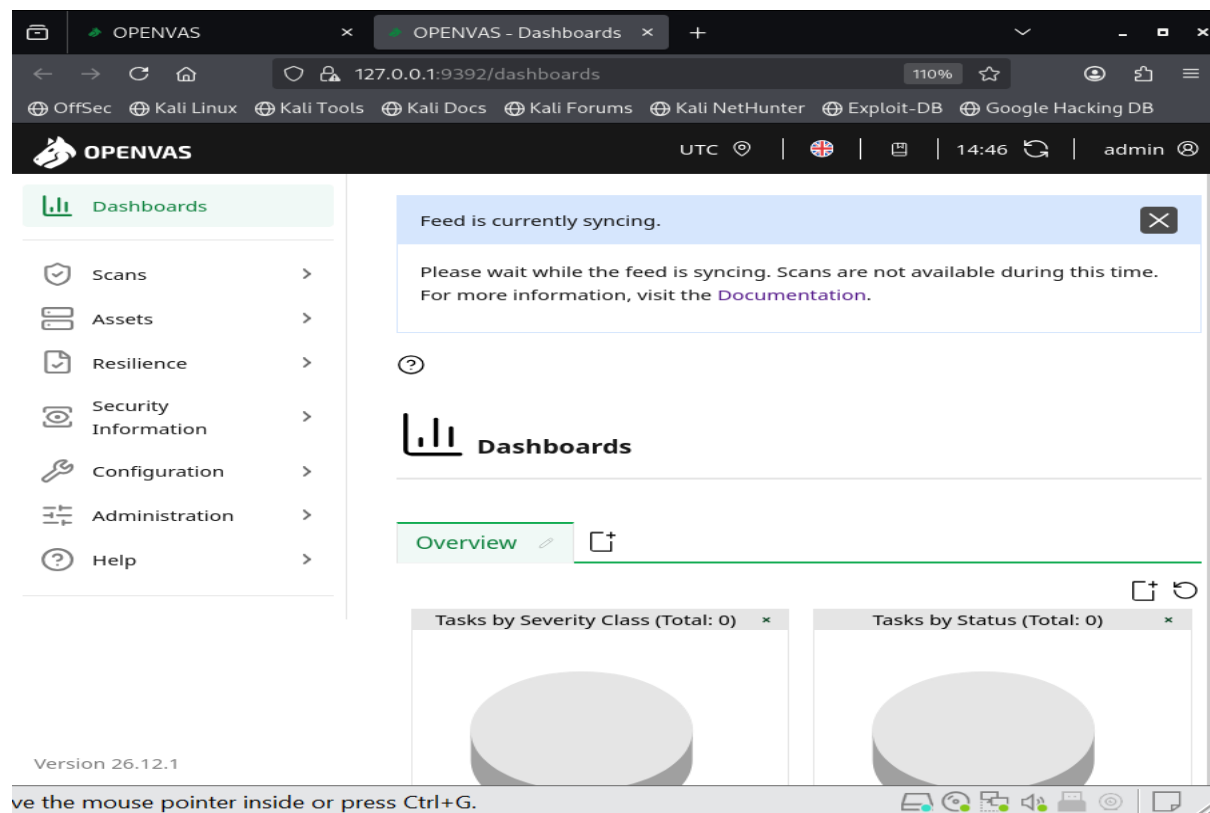


Figure 82: OpenVAS dashboard showing feed synchronization in progress

System processes were checked using `ps` command to verify whether OpenVAS and feed services were running correctly.

```
(kali@kali)-[~]
$ ps aux | grep greenbone
kali      5148  0.0  0.0  6748  2448 pts/0    S+   14:19   0:00 grep --color=
auto      greenbone

(kali@kali)-[~]
$ ps aux | grep feed
kali      5151  0.0  0.0  6748  2404 pts/0    S+   14:19   0:00 grep --color=
auto      feed

(kali@kali)-[~]
$
```

Figure 83: OpenVAS processes verified to ensure services are running correctly

The Scan Configurations section was accessed, showing that no scan configurations were available due to incomplete feed synchronization.

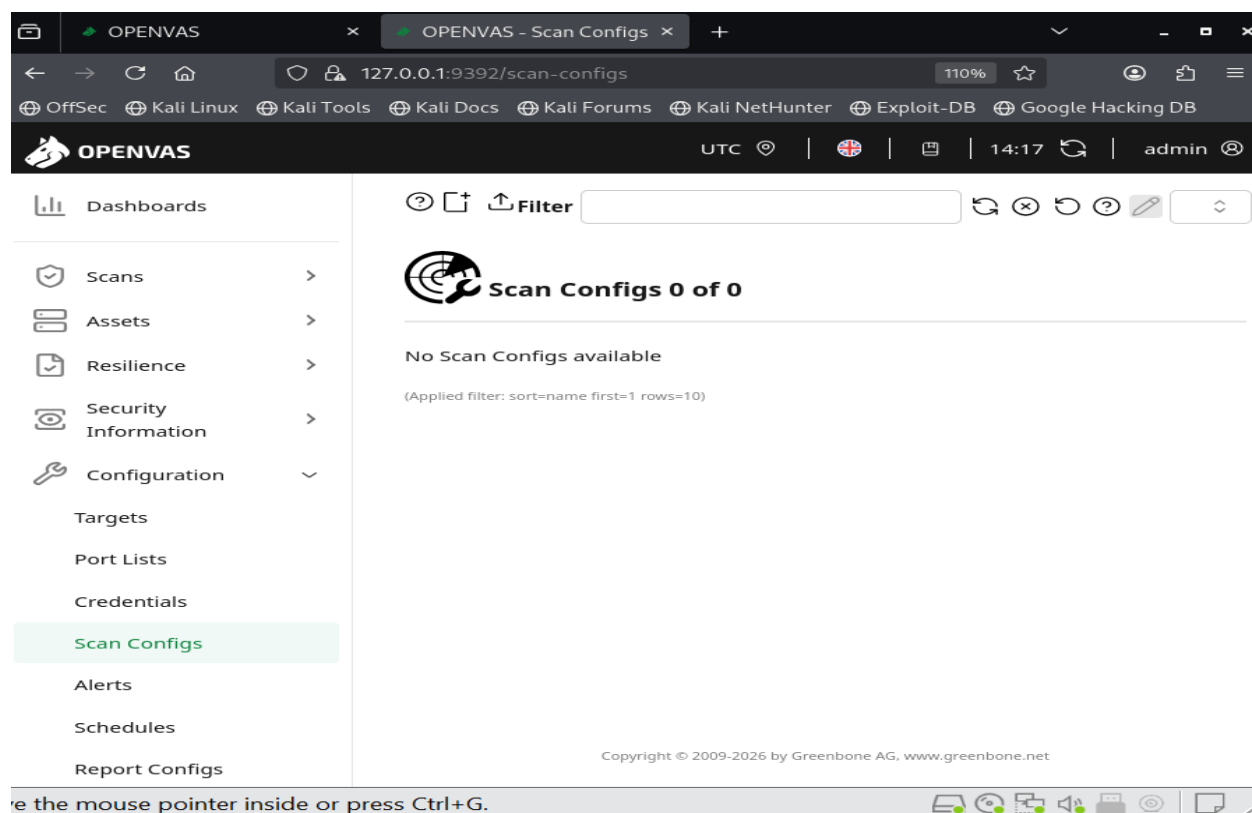


Figure 84: Scan configuration section accessed for setting up vulnerability scans

The OpenVAS dashboard indicated that feed synchronization was in progress, temporarily disabling scan functionalities.

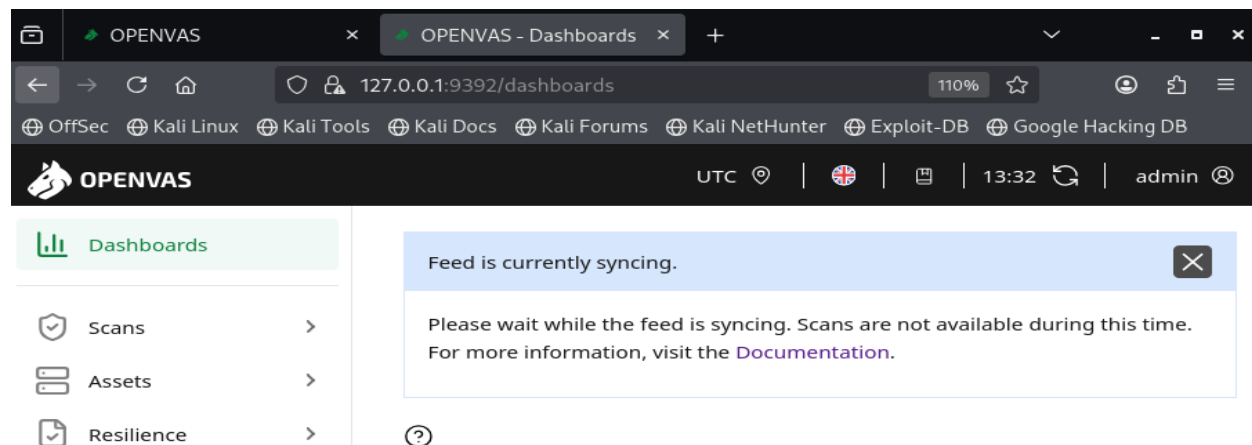


Figure 85: OpenVAS dashboard indicating ongoing feed synchronization

A new scan configuration was initiated by selecting scan options and preparing the system for vulnerability scanning.

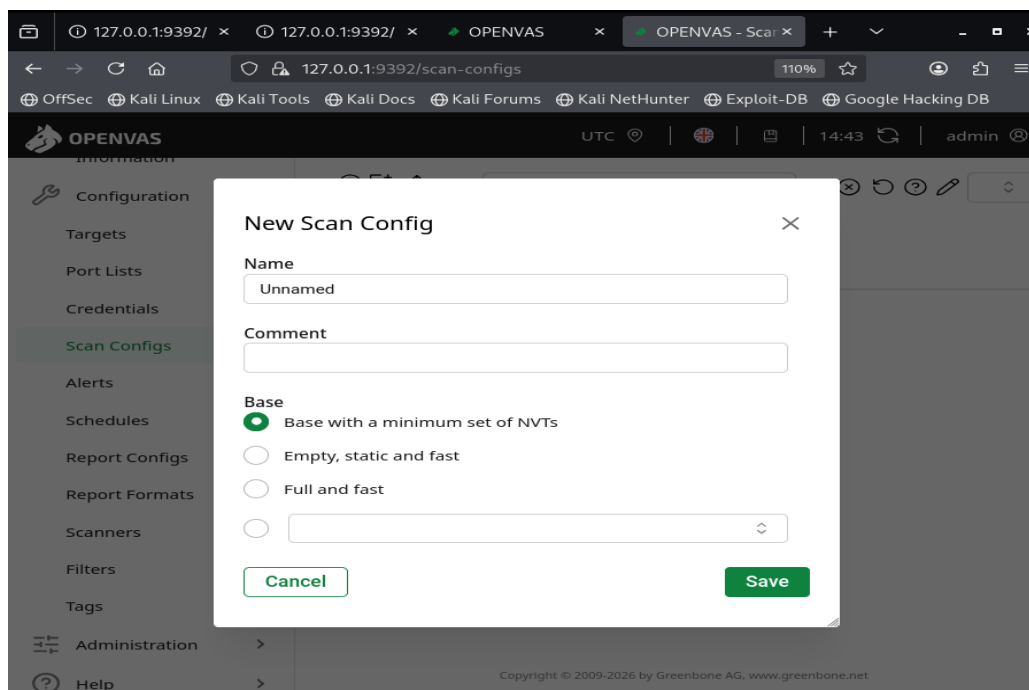


Figure 86: New scan configuration created for vulnerability assessment

A target was successfully created in OpenVAS by specifying the host details for vulnerability scanning.

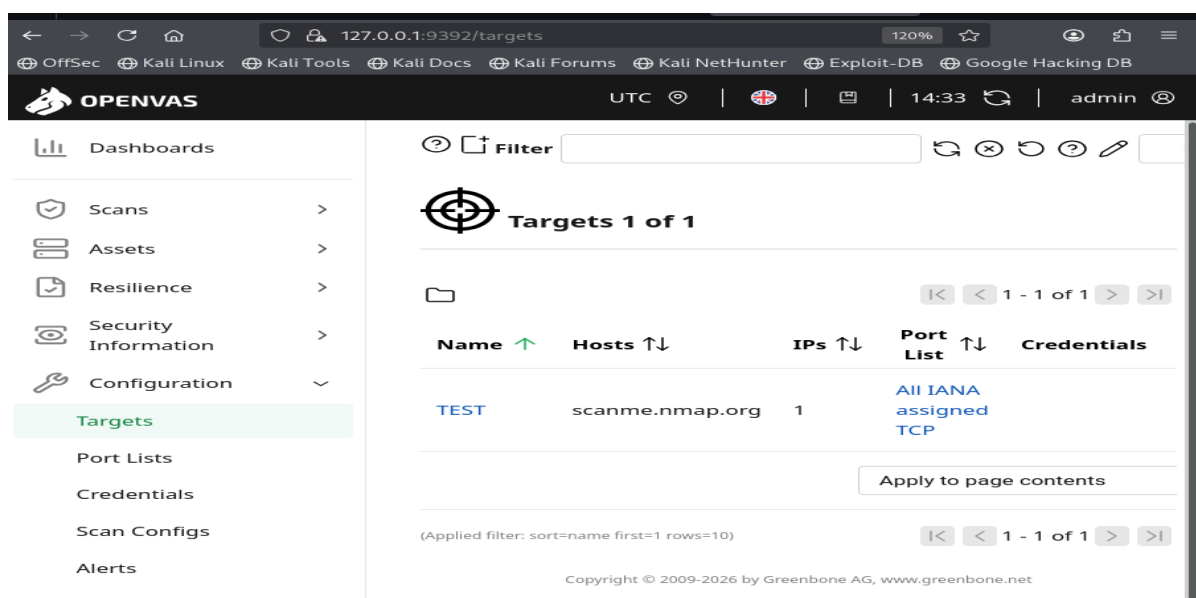


Figure 87: Target system configured for OpenVAS vulnerability scanning

A new scan task was configured by selecting the OpenVAS scanner and defining scanning parameters.

Figure 88: Scan task created and configured for the selected target

An OpenVAS scan task named ThreatQuill Analyzer was successfully configured, indicating the system is ready to perform vulnerability scanning.

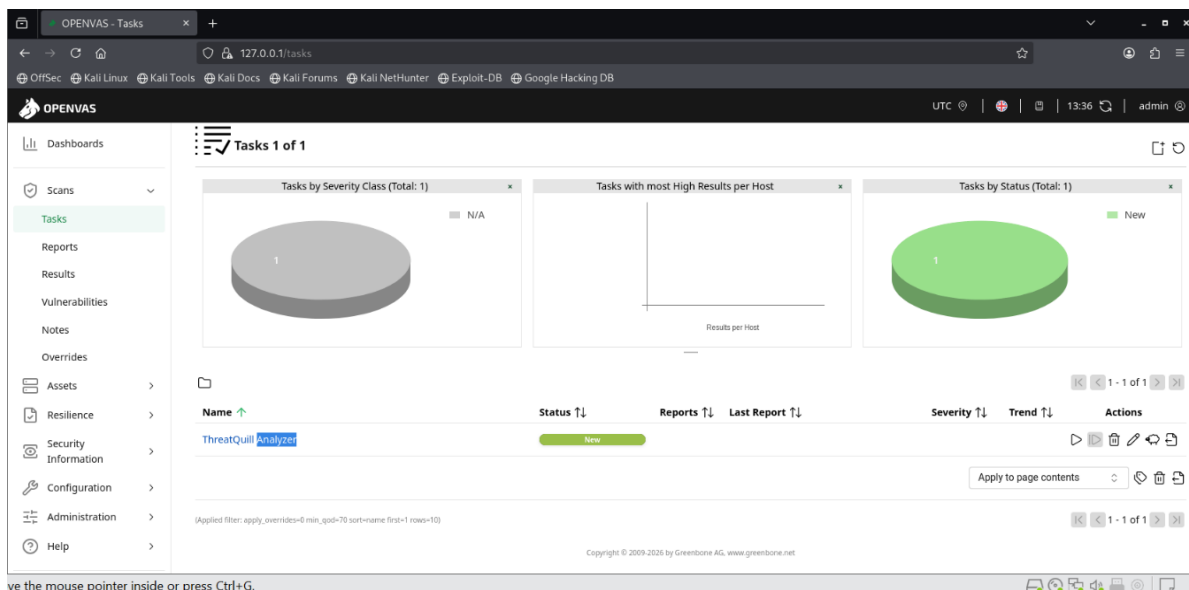


Figure 89: OpenVAS task created and ready for execution

The scan task was successfully initiated and is waiting to be processed by the OpenVAS scanner.

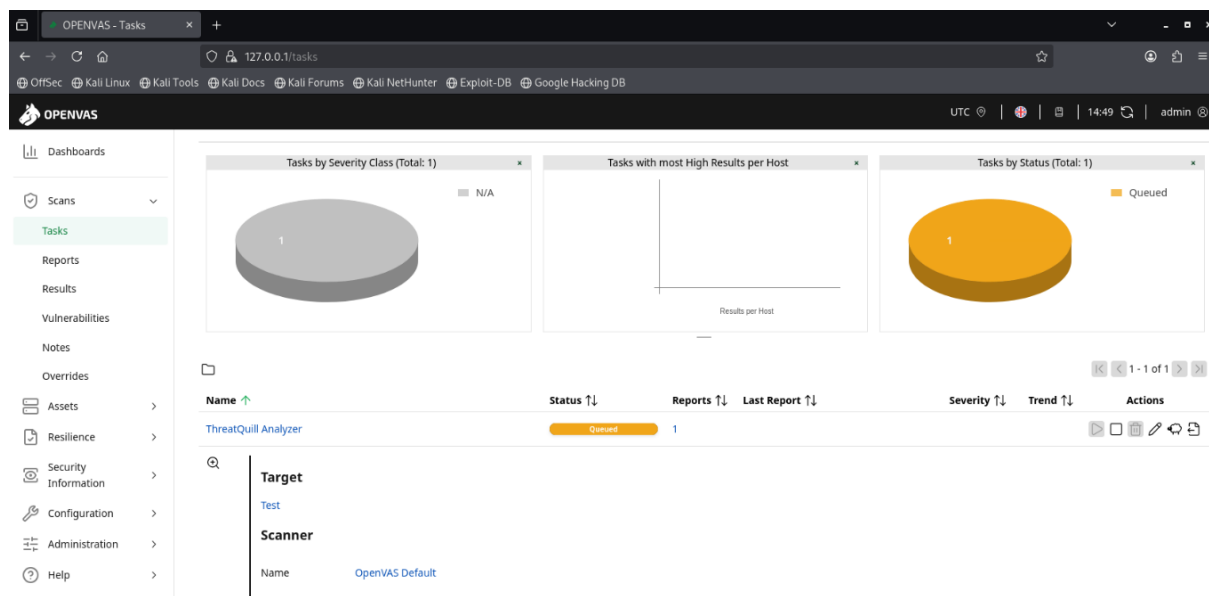


Figure 90: Scan task queued and waiting for execution

The OpenVAS scanner is actively performing vulnerability assessment on the target system.

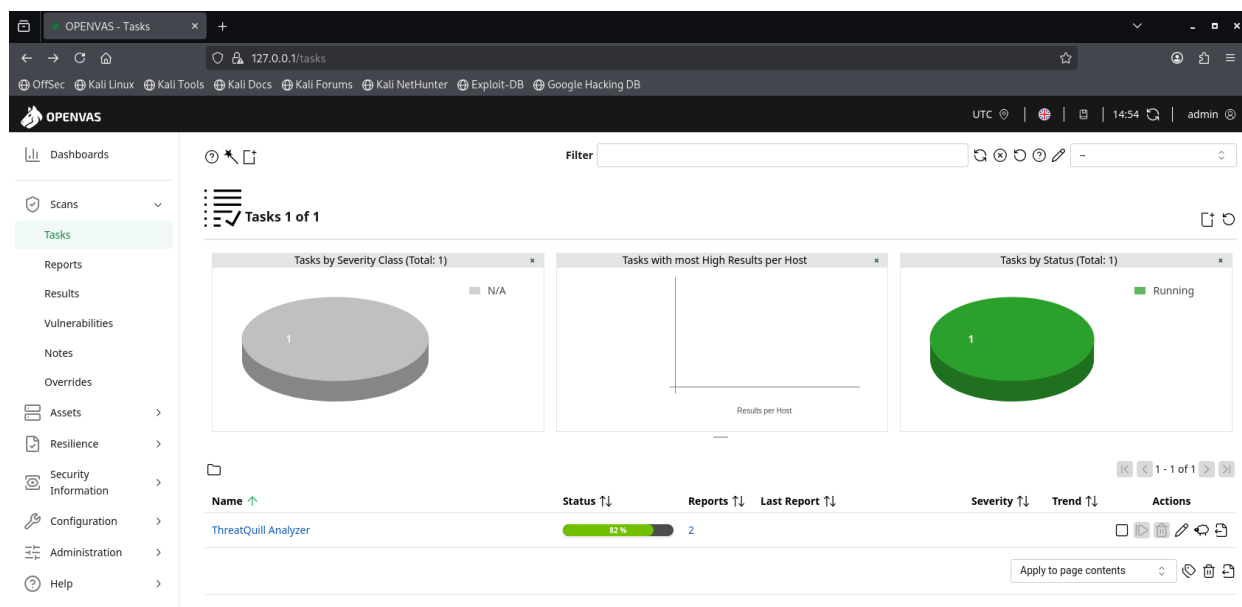


Figure 91: OpenVAS scan task in progress on target system

The vulnerability scan has been completed successfully, and results are available for analysis.

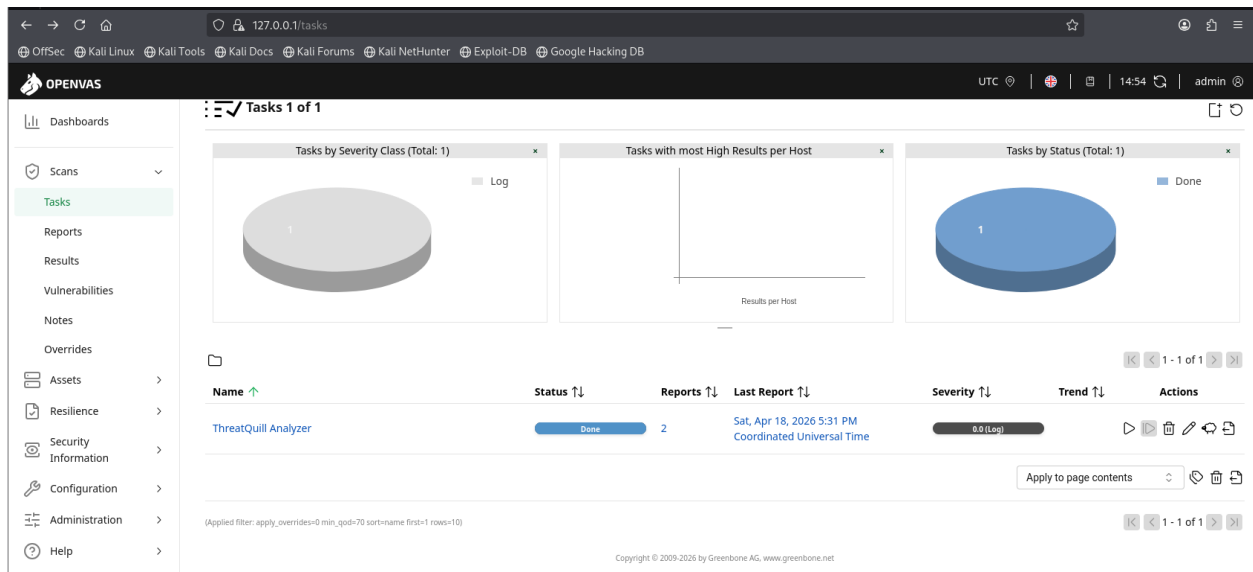


Figure 92: OpenVAS scan completed successfully for the target system

The scan was completed successfully, showing overall scan details such as duration, status, and number of hosts scanned.

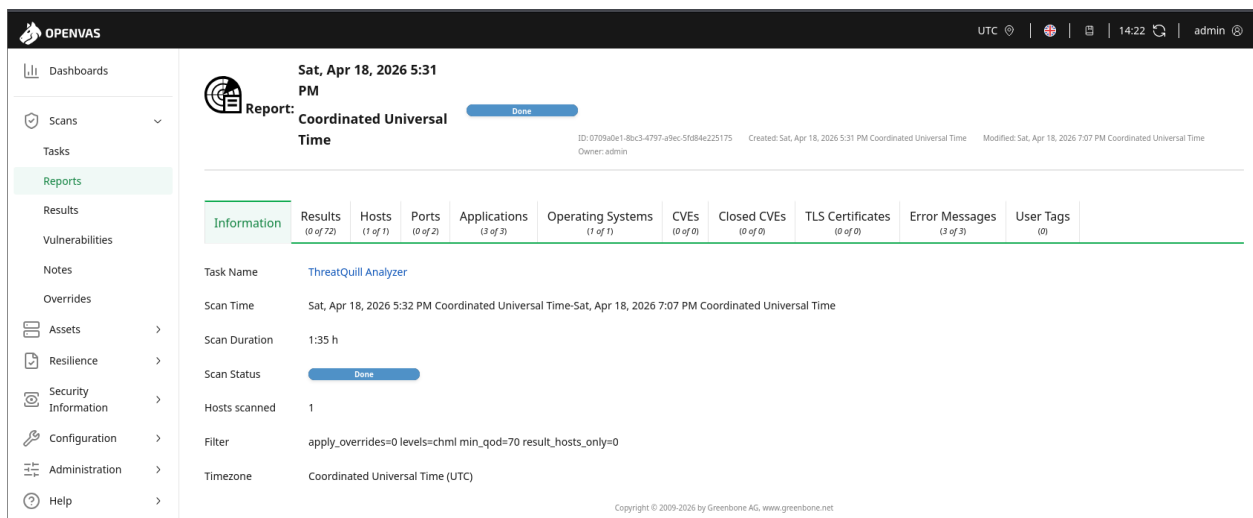


Figure 93: Generated report showing scan details and execution summary

The scanned target system was identified along with its IP address, services, and basic host-level information.

The screenshot displays the OpenVAS web interface. On the left is a sidebar with navigation options: Dashboards, Scans, Tasks, Reports (highlighted), Results, Vulnerabilities, Notes, Overrides, Assets, Resilience, Security Information, Configuration, Administration, and Help. The main content area shows a report titled 'Coordinated Universal Time' with a status of 'Done'. Below the report title, there are tabs for Information, Results, Hosts (selected), Ports, Applications, Operating Systems, CVEs, Closed CVEs, TLS Certificates, Error Messages, and User Tags. The 'Hosts' tab displays a table with columns: IP Address, Hostname, OS, Ports, Apps, Distance, Auth, Start, End, Critical, High, Medium, Low, and Log. The table contains one entry for IP 45.33.32.156, Hostname scanme.nmap.org, OS 0, and Apps 3. The scan was performed on Sat, Apr 18, 2026 5:31 PM Coordinated Universal Time.

Figure 94: Scan results displaying host information and detected services

The OpenVAS scan results were successfully integrated into the ThreatQuill Analyzer dashboard, displaying findings alongside other scanner outputs.

The screenshot shows the ThreatQuill Analyzer dashboard. The top header includes the ThreatQuill logo and the text 'ThreatQuill Analyzer' with 'Logged in as: binni'. A left sidebar contains a 'MENU' with options: Scan Summary, New Scan, Send Email, Reports, Settings, and Help. The main area has a 'Target' field with 'scanme.nmap.org' and buttons for 'Scanning' and 'Clear Results'. Below this is a 'Scan Summary' section with statistics: Total Hosts Scanned: 1, Threats Detected: 1, Total Findings: 3, and Status: Complete. The 'Latest Scan Results' table lists scans from OPENVAS and NESSUS on scanme.nmap.org. The 'Activity Log' on the right shows a timeline of scan events, including Nmap port scans, service detection, and completion messages.

Figure 95: OpenVAS scan results integrated into ThreatQuill Analyzer dashboard

4.5 Login and Dashboard

User authentication is performed by entering valid credentials to access the ThreatQuill Analyzer system.

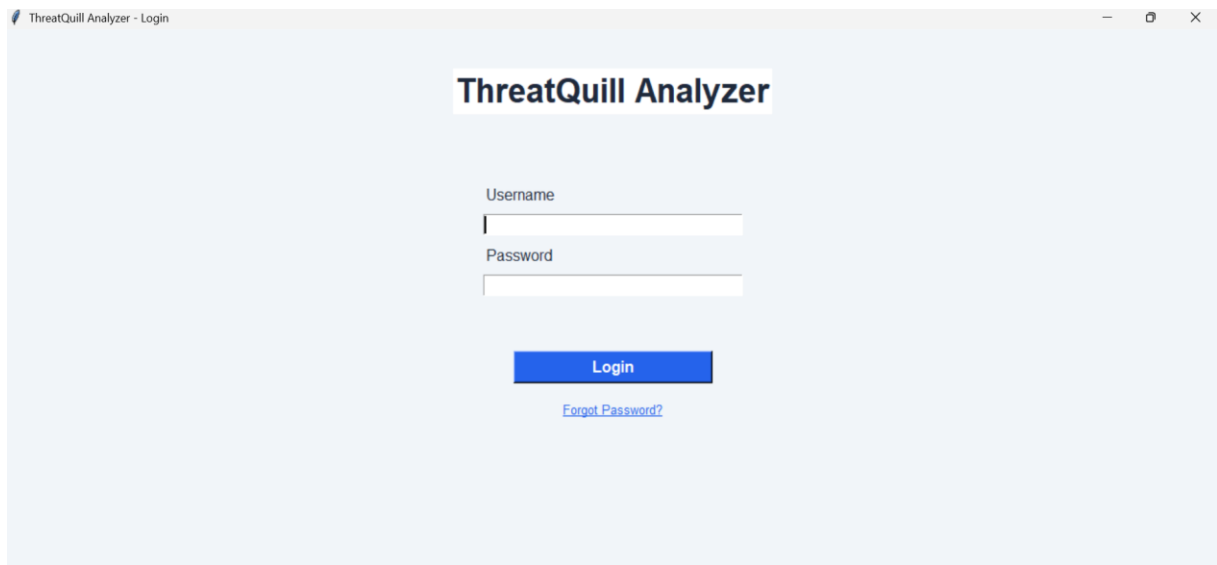


Figure 96: Login interface allowing users to securely enter credentials

The dashboard interface is displayed after system initialization, showing the initial state before scan execution and overall layout of the ThreatQuill Analyzer.

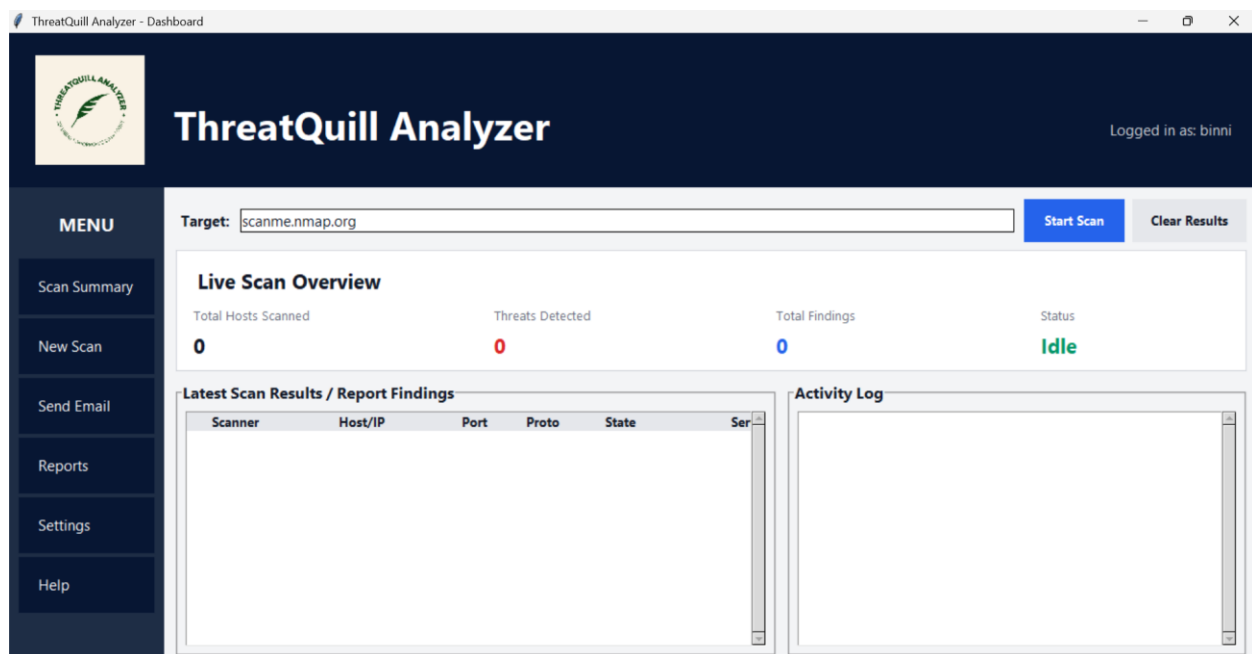


Figure 97: Dashboard displaying system overview

5. Deployment Phase

During the Deployment phase, the entire ThreatQuill Analyzer was tested and implemented as a unified application. All modules such as Nmap, Nikto, Nessus and OpenVAS were integrated and executed via the dashboard interface. The system was experimented by carrying out full scans of target systems, and the outputs were shown in the dashboard. Such aspects like activity logs, scan summary and results table in real-time were confirmed.

The completed scan displays detected threats, total findings, and real-time activity logs from multiple scanners.

ThreatQuill Analyzer - Dashboard

ThreatQuill Analyzer Logged in as: binni

MENU

- Scan Summary
- New Scan
- Send Email
- Reports
- Settings
- Help

Target: Scanning... Clear Results

Live Scan Overview

Total Hosts Scanned	Threats Detected	Total Findings	Status
1	264	314	Complete

Latest Scan Results / Report Findings

Scanner	Host/IP	Port	Proto	State	Ser
NESSUS	scanme.nmap.org		tcp	open	Traceroute
NESSUS	scanme.nmap.org		tcp	open	Target Creden
NESSUS	scanme.nmap.org		tcp	open	SSH SHA-1 HM
NESSUS	scanme.nmap.org		tcp	open	SSH Server Typ
NESSUS	scanme.nmap.org		tcp	open	SSH Password
NESSUS	scanme.nmap.org		tcp	open	SSH Algorithm
NESSUS	scanme.nmap.org		tcp	open	Shor's Harvest
NESSUS	scanme.nmap.org		tcp	open	Service Detect

Activity Log

```
[12:34:48] INFO → Live scan requested for target: scanme.nmap.org
[12:34:48] INFO → start_full_scan called
[12:34:48] INFO → run_all entered
[12:34:48] INFO → [SYSTEM] Starting full scan for target: scanme.nmap.org
[12:34:48] INFO → [SYSTEM] Starting Nmap thread...
[12:34:48] INFO → [SYSTEM] Starting Nikto thread...
[12:34:48] INFO → Nmap function entered
[12:34:48] NMAP → Command: nmap -sS -sV -O --script vuln -T4 --top-ports 1000 --stats-every 5s scanme.nmap.org
[12:34:48] INFO → Nikto function entered
[12:34:48] INFO → [SYSTEM] Starting Nessus thread...
[12:34:48] NMAP → Starting live scan on scanme.nmap.org
[12:34:48] INFO → Nessus function entered
[12:34:48] INFO → [SYSTEM] Starting OpenVAS thread...
```

Figure 98: Final system execution showing integrated scan results on dashboard

A consolidated vulnerability assessment report is generated summarizing findings, severity distribution, and affected hosts.

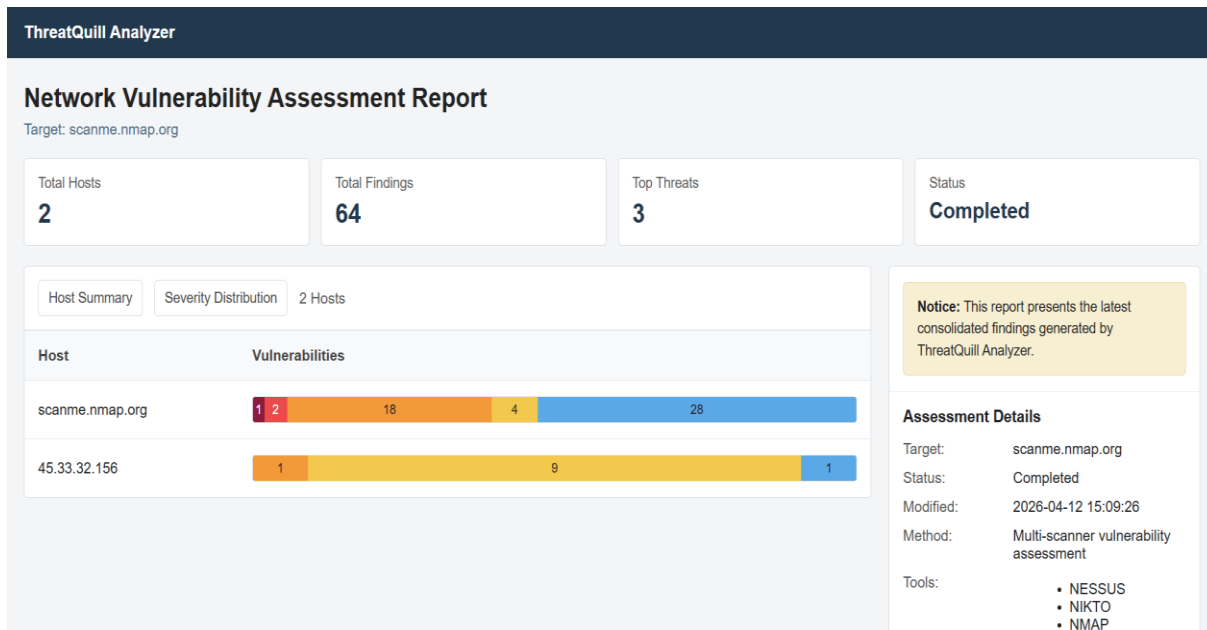


Figure 99: Final report generated displaying vulnerability summary and analysis

A visual summary of the scan results is presented, including severity levels and overall assessment details.

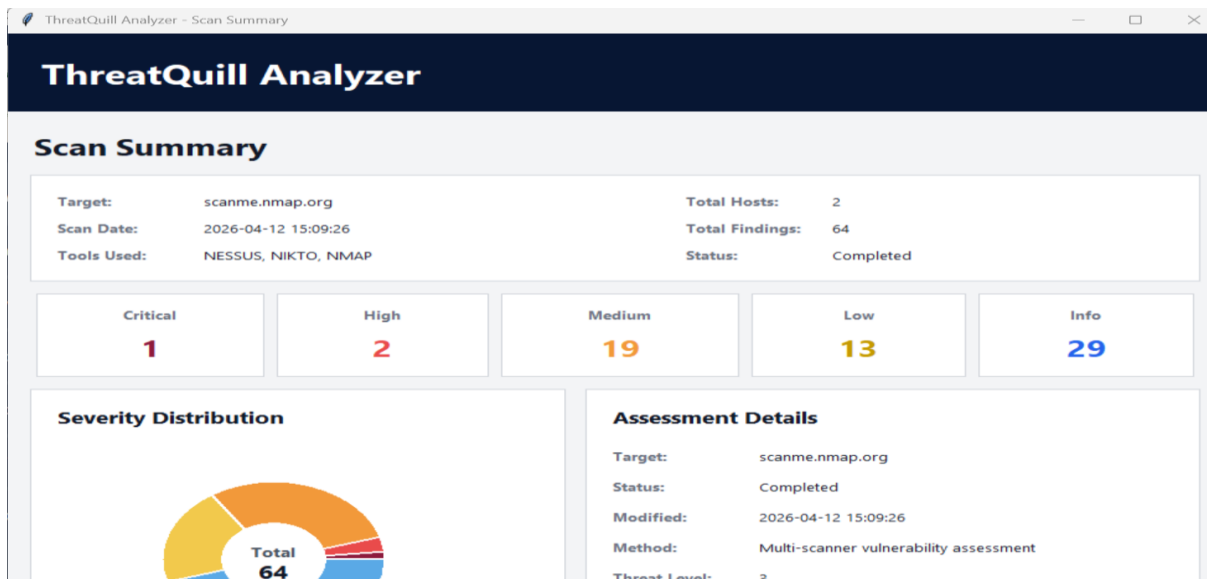
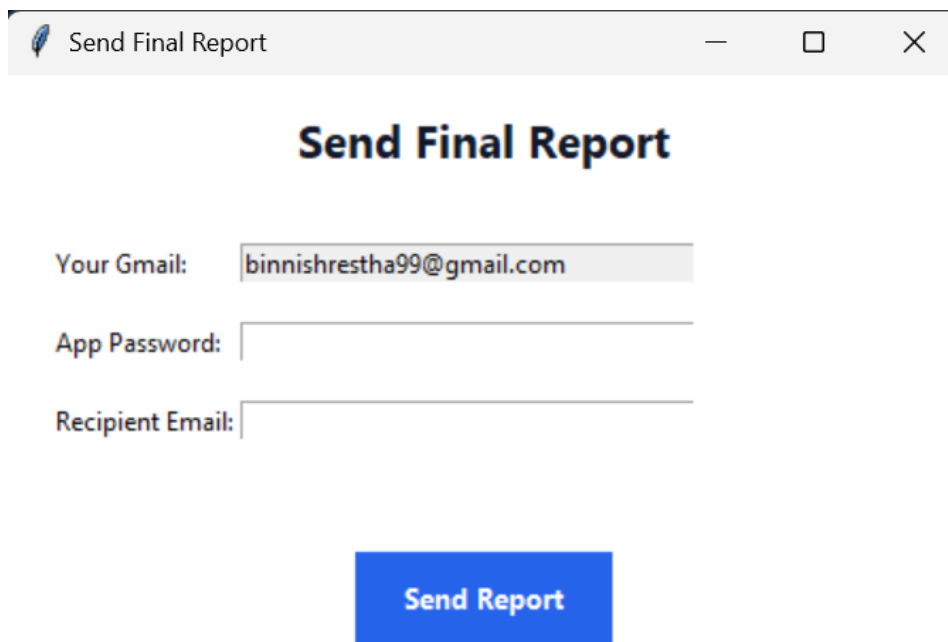


Figure 100: Scan summary showing severity distribution of detected vulnerabilities

The system allows sending the generated vulnerability report via email to a specified recipient.



Send Final Report

Send Final Report

Your Gmail: binnishrestha99@gmail.com

App Password:

Recipient Email:

Send Report

Figure 101: Generated report sent successfully using email functionality

6. Post-Project Phase

During Post-Project phase, the ThreatQuill Analyzer was re-examined following the development to analyze its overall performance and functionality. This system is successfully working due to its ability to combine various vulnerability scanners that include Nmap, Nikto, and Nessus under one platform.

It can scan with precision and present the outcomes in clear and organized format using the dashboard. Other features like log of activities, scan summary and result tables to identify the vulnerabilities and the extent of their vulnerability with ease are added. This evaluation shows that the system meets its main objectives of simplifying vulnerability assessment and improving the efficiency of the scanning process.

3.6.2 Backend Development

The backend of the ThreatQuill Analyzer is developed using Python to manage scanning operations, process results, and generate reports. It integrates multiple vulnerability scanning tools and ensures that all scan data is collected, processed, and presented in a structured way.

Scanner_controller.py

```

scanner_controller.py
1  import threading
2  from scanners.nmap_scanner import run_nmap_live
3  from scanners.nikto_scanner import run_nikto_live
4  from scanners.nessus_scanner import run_nessus_live
5  from scanners.openvas_scanner import run_openvas_live
6  from report_generator import generate_report_data, save_json_report, save_html_report
7
8  all_results = []
9
10
11 def finalize_report(target, dashboard=None):
12     global all_results
13
14     report_data = generate_report_data(target, all_results)
15     json_path = save_json_report(report_data)
16     html_path = save_html_report(report_data)
17
18     if dashboard:
19         dashboard._ui(dashboard.show_report, report_data, html_path)
20         dashboard._ui(
21             dashboard.append_log,
22             f"[SYSTEM] Report generated successfully with {len(all_results)} findings."
23         )
24
25     return report_data, json_path, html_path
26
27
28 def start_full_scan(target, dashboard):
29     global all_results
30     all_results.clear()
31
32     scan_stats = {
33         "hosts": 0,
34         "threats": 0,
35         "findings": 0,
36         "status": "Running"
37     }
38
39     def log_cb(message):
40         dashboard._ui(dashboard.append_log, message)
41

```

Figure 102: Scanner controller module initialized to manage scan execution

```

scanner_controller.py
28 def start_full_scan(target, dashboard):
41
42     def result_cb(result):
43         try:
44             if not isinstance(result, dict):
45                 log_cb("[RESULT ERROR] Invalid result format received.")
46                 return
47
48                 all_results.append(result)
49
50                 scanner = result.get("scanner", "")
51                 host = result.get("host", result.get("target", target))
52                 port = result.get("port", "")
53                 protocol = result.get("protocol", "")
54                 state = result.get("state", "")
55                 service = result.get("service", "")
56
57                 severity = result.get("severity", "")
58                 if not severity:
59                     severity = result.get("risk_level", "Info")
60
61                 dashboard._ui(
62                     dashboard.add_result,
63                     scanner,
64                     host,
65                     str(port),
66                     protocol,
67                     state,
68                     service,
69                     severity
70                 )
71
72                 scan_stats["findings"] = len(all_results)
73
74                 dashboard._ui(
75                     dashboard.update_summary,
76                     scan_stats["hosts"],
77                     scan_stats["threats"],
78                     scan_stats["findings"],
79                     scan_stats["status"]
80                 )
81
82         except Exception as e:
83             log_cb(f"[RESULT ERROR] {e}")
84

```

Figure 103: Functions defined to coordinate execution of multiple scanner

```

scanner_controller.py
28 def start_full_scan(target, dashboard):

85     def summary_cb(hosts, threats, findings, status):
86         try:
87             if hosts is not None:
88                 scan_stats["hosts"] = max(scan_stats["hosts"], hosts)
89
90             if threats is not None:
91                 scan_stats["threats"] = max(scan_stats["threats"], threats)
92
93             if findings is not None:
94                 scan_stats["findings"] = max(scan_stats["findings"], findings)
95
96             if status:
97                 scan_stats["status"] = status
98
99             dashboard.ui(
100                 dashboard.update_summary,
101                 scan_stats["hosts"],
102                 scan_stats["threats"],
103                 max(scan_stats["findings"], len(all_results)),
104                 scan_stats["status"]
105             )
106
107         except Exception as e:
108             log_cb(f"[SUMMARY ERROR] {e}")
109
110     def run_nmap():
111         try:
112             log_cb("[SYSTEM] Nmap scan started.")
113             run_nmap_live(target, log_cb, result_cb, summary_cb, deep_scan=False)
114             log_cb("[SYSTEM] Nmap scan completed.")
115         except Exception as e:
116             log_cb(f"[NMAP ERROR] {e}")
117
118     def run_nikto():
119         try:
120             log_cb("[SYSTEM] Nikto scan started.")
121             run_nikto_live(target, log_cb, result_cb, summary_cb)
122             log_cb("[SYSTEM] Nikto scan completed.")
123         except Exception as e:
124             log_cb(f"[NIKTO ERROR] {e}")
125
126     def run_nessus():
127         try:

```

Figure 104: Logic implemented to trigger Nmap, Nikto, Nessus, and OpenVAS scans

```

scanner_controller.py
28 def start_full_scan(target, dashboard):

126     def run_nessus():
127         try:
128             log_cb("[SYSTEM] Nessus scan started.")
129             run_nessus_live(target, log_cb, result_cb, summary_cb)
130             log_cb("[SYSTEM] Nessus scan completed.")
131         except Exception as e:
132             log_cb(f"[NESSUS ERROR] {e}")
133
134     def run_openvas():
135         try:
136             log_cb("[SYSTEM] OpenVAS scan started.")
137             run_openvas_live(target, log_cb, result_cb, summary_cb)
138             log_cb("[SYSTEM] OpenVAS scan completed.")
139         except Exception as e:
140             log_cb(f"[OPENVAS ERROR] {e}")
141
142     def run_all():
143         try:
144             log_cb("[SYSTEM] Full scan process started.")
145             dashboard._ui(dashboard.update_summary, 0, 0, 0, "Running")
146
147             threads = [
148                 threading.Thread(target=run_nmap, daemon=True),
149                 threading.Thread(target=run_nikto, daemon=True),
150                 threading.Thread(target=run_nessus, daemon=True),
151                 threading.Thread(target=run_openvas, daemon=True)
152             ]
153
154             for thread in threads:
155                 thread.start()
156
157             for thread in threads:
158                 thread.join()
159
160             log_cb("[SYSTEM] All scans completed.")
161             log_cb(f"[SYSTEM] Total collected results: {len(all_results)}")
162
163             scan_stats["findings"] = len(all_results)
164             scan_stats["hosts"] = max(scan_stats["hosts"], 1 if target else 0)
165
166             if len(all_results) > 0:
167                 scan_stats["status"] = "Complete"
168             else:

```

Figure 105: Scan results collected from different scanners during execution


```

scanner_controller.py
28  def start_full_scan(target, dashboard):
142  def run_all():
143      try:
165          if len(all_results) > 0:
166              scan_stats["status"] = "Complete"
167          else:
168              scan_stats["status"] = "No Results"
170
171          dashboard._ui(
172              dashboard.update_summary,
173              scan_stats["hosts"],
174              scan_stats["threats"],
175              scan_stats["findings"],
176              scan_stats["status"]
177          )
178
179          try:
180              finalize_report(target, dashboard)
181          except Exception as e:
182              log_cb(f"[REPORT ERROR] {e}")
183
184      except Exception as e:
185          log_cb(f"[SYSTEM ERROR] {e}")
186          dashboard._ui(
187              dashboard.update_summary,
188              scan_stats["hosts"],
189              scan_stats["threats"],
190              len(all_results),
191              "Failed"
192          )
193
194      finally:
195          dashboard._ui(dashboard.scan_finished)
196
197  try:
198      dashboard._ui(dashboard.append_log, "[SYSTEM] start_full_scan called.")
199      threading.Thread(target=run_all, daemon=True).start()
200  except Exception as e:
201      dashboard._ui(dashboard.append_log, f"[START ERROR] {e}")
202      dashboard._ui(dashboard.scan_finished)
203

```

Figure 106: Combined scan results processed for dashboard display and report generation

Nmap_scanner.py

```

scanners > nmap_scanner.py
1  import re
2  import subprocess
3  from utils.ssh_helper import run_ssh_command
4
5
6  def get_risk_level(port, state, service=""):
7      """
8      Basic risk indicator for open ports/services.
9      This is not real CVSS severity.
10     """
11     if state != "open":
12         return "Low"
13
14     port = str(port)
15     service = service.lower()
16
17     high_risk_ports = {"21", "23", "25", "110", "135", "137", "138", "139", "445", "1433", "1521", "3306", "3389", "5432", "5900"}
18     medium_risk_ports = {"53", "80", "111", "135", "161", "389", "443", "8080", "8443"}
19
20     if port in high_risk_ports:
21         return "High"
22     if port in medium_risk_ports:
23         return "Medium"
24
25     if "ftp" in service or "telnet" in service or "smb" in service or "rdp" in service:
26         return "High"
27     if "http" in service or "ssl" in service or "https" in service:
28         return "Medium"
29
30     return "Low"
31
32
33  def parse_nmap_line(line, current_host):
34      """
35      Parse live Nmap output line by line.
36      Returns:
37          ("host_update", host)
38          ("port_result", result_dict)
39          ("os_result", result_dict)
40          ("script_result", result_dict)
41          ("info_result", result_dict)
42          None
43      """
44
45     host_match = re.search(r"Nmap scan report for (.*?) \([([d\.\-]+)\)", line)
46     if host_match:
47         return ("host_update", host_match.group(2))
48
49     # Example:
50     # Nmap scan report for 192.168.1.10
51     host_match2 = re.search(r"Nmap scan report for ([\w\.\-]+)", line)
52     if host_match2 and "(" not in line:
53         return ("host_update", host_match2.group(1))
54
55     # Example:
56     # 80/tcp open  http
57     # 443/tcp open ssl/http Apache httpd 2.4.54
58     port_match = re.match(r"^(.*)\s*([d+])\s*([w+])\s*([.+])\s*$", line)
59     if port_match:
60         port = port_match.group(1)
61         proto = port_match.group(2)
62         state = port_match.group(3)
63         service = port_match.group(4).strip()
64         risk_level = get_risk_level(port, state, service)
65

```

```

scanners > nmap_scanner.py
33 def parse_nmap_line(line, current_host):
59     if port_match:
66         result = {
67             "scanner": "NMAP",
68             "host": current_host,
69             "port": port,
70             "protocol": proto,
71             "state": state,
72             "service": service,
73             "risk_level": risk_level,
74             "finding": f"Port {port}/{proto} is {state} ({service})"
75         }
76         return ("port_result", result)
77
78     # OS detection line
79     if line.startswith("OS details:"):
80         os_info = line.replace("OS details:", "").strip()
81         result = {
82             "scanner": "NMAP",
83             "host": current_host,
84             "port": "-",
85             "protocol": "-",
86             "state": "detected",
87             "service": "OS Detection",
88             "risk_level": "Info",
89             "finding": f"Detected OS: {os_info}"
90         }
91         return ("os_result", result)
92
93     # Service Info line
94     if line.startswith("Service Info:"):
95         info = line.replace("Service Info:", "").strip()
96         result = {
97             "scanner": "NMAP",
98             "host": current_host,
99             "port": "-",
100            "protocol": "-",
101            "state": "info",
102            "service": "Service Info",
103            "risk_level": "Info",
104            "finding": f"Service Info: {info}"
105        }
106        return ("info_result", result)
107
108    # NSE script output line
109    # Example:
110    # |_http-title: Go ahead and ScanMe!
111    if line.startswith("|_") or line.startswith("|_"):
112        script_output = line.strip()
113        risk_level = "Medium"
114        lower_line = script_output.lower()
115
116        if any(word in lower_line for word in ["vulnerable", "unsafe", "expired", "default", "cve-", "anonymous"]):
117            risk_level = "High"
118
119        result = {
120            "scanner": "NMAP",
121            "host": current_host,
122            "port": "-",
123            "protocol": "-",
124            "state": "script",
125            "service": "NSE Script",
126            "risk_level": risk_level,
127            "finding": script_output
128        }

```

```

scanners > nmap_scanner.py
33 def parse_nmap_line(line, current_host):
111     if line.startswith("|") or line.startswith("|_"):
129         return ("script_result", result)
130
131     return None
132
133
134 def run_nmap_live(target, log_cb, result_cb, summary_cb, deep_scan=True):
135     current_host = target
136     hosts = set()
137     findings = 0
138     threats = 0
139
140     if deep_scan:
141         cmd = (
142             f"nmap -sS -sV -O --script vuln -T4 --top-ports 1000 "
143             f"--stats-every 5s {target}"
144         )
145     else:
146         cmd = f"nmap -sV -T4 --top-ports 1000 --stats-every 5s {target}"
147
148     log_cb(f"[NMAP] Command: {cmd}")
149     log_cb(f"[NMAP] Starting live scan on {target}")
150
151     try:
152         process = run_ssh_command(cmd)
153     except Exception as e:
154         log_cb(f"[NMAP ERROR] Could not start Nmap scan: {e}")
155         summary_cb(0, 0, 0, "Failed")
156         return
157
158     summary_cb(0, 0, 0, "Running")
159
160     try:
161         for raw in process.stdout:
162             line = raw.strip()
163             if not line:
164                 continue
165
166             log_cb(f"[NMAP] {line}")
167             parsed = parse_nmap_line(line, current_host)
168
169             if parsed:
170                 event_type = parsed[0]
171
172                 if event_type == "host_update":
173                     current_host = parsed[1]
174                     hosts.add(current_host)
175
176                 elif event_type in {"port_result", "os_result", "script_result", "info_result"}:
177                     result = parsed[1]
178                     hosts.add(result["host"])
179
180                     if result["risk_level"] in {"Medium", "High"}:
181                         threats += 1
182
183                     findings += 1
184                     result_cb(result)
185
186                 summary_cb(len(hosts), threats, findings, "Running")
187
188             return_code = process.wait()
189
190             if return_code == 0:
191                 log_cb("[NMAP] Scan completed successfully.")

```

```

scanners > nmap_scanner.py
134 def run_nmap_live(target, log_cb, result_cb, summary_cb, deep_scan=True):
160     try:
188         return_code = process.wait()
189
190         if return_code == 0:
191             log_cb("[NMAP] Scan completed successfully.")
192             summary_cb(len(hosts), threats, findings, "Complete")
193         else:
194             log_cb(f"[NMAP] Scan failed with code {return_code}")
195             summary_cb(len(hosts), threats, findings, "Failed")
196
197     except Exception as e:
198         log_cb(f"[NMAP ERROR] Runtime error: {e}")
199         summary_cb(len(hosts), threats, findings, "Failed")
200
201
202 def run_nmap(target, output_file="outputs/nmap.xml", use_ssh=False, deep_scan=True):
203     """
204     Run Nmap and save XML output.
205     If use_ssh=True, it runs on Kali through SSH.
206     Otherwise it runs locally.
207     """
208
209     if deep_scan:
210         cmd_string = f"nmap -sS -sV -O --script vuln -oX {output_file} {target}"
211         cmd_list = ["nmap", "-sS", "-sV", "-O", "--script", "vuln", "-oX", output_file, target]
212     else:
213         cmd_string = f"nmap -sV -oX {output_file} {target}"
214         cmd_list = ["nmap", "-sV", "-oX", output_file, target]
215
216     if use_ssh:
217         process = run_ssh_command(cmd_string)
218         process.wait()
219     else:
220         subprocess.run(cmd_list, check=True)
221
222     return output_file
223

```

Figure 107: Nmap scanner module implemented for detecting open ports and network services

Nikto_scanner.py

```
scanners > nikto_scanner.py
1  from utils.ssh_helper import run_ssh_command
2
3
4  def normalize_target_for_nikto(target):
5      if target.startswith("http://") or target.startswith("https://"):
6          return target
7      return f"http://{target}"
8
9
10 def get_nikto_risk(line):
11     text = line.lower()
12
13     high_keywords = [
14         "outdated",
15         "vulnerable",
16         "sql injection",
17         "xss",
18         "remote code execution",
19         "directory indexing",
20         "admin",
21         "cgi-bin",
22         "exposed",
23         "authentication bypass"
24     ]
25
26     medium_keywords = [
27         "allowed http methods",
28         "header is not present",
29         "cookie",
30         "internal ip",
31         "retrieved",
32         "discloses",
33         "interesting",
34         "entry point"
35     ]
36
37     low_keywords = [
38         "server leaks",
39         "uncommon header"
40     ]
41
42     for word in high_keywords:
43         if word in text:
44             return "High"
45
46     for word in medium_keywords:
47         if word in text:
48             return "Medium"
49
50     for word in low_keywords:
51         if word in text:
52             return "Low"
53
54     return "Medium"
```

```

scanners > nikto_scanner.py
10  def get_nikto_risk(line):
54      return "Medium"
55
56
57  def is_real_nikto_finding(line):
58      """
59      Ignore Nikto informational lines and keep real findings only.
60      """
61      lower_line = line.lower()
62
63      ignore_prefixes = [
64          "+ target ip:",
65          "+ target hostname:",
66          "+ target port:",
67          "+ start time:",
68          "+ end time:",
69          "+ host appears to be up",
70          "+ server:",
71          "+ retrieved",
72          "+ the anti-clickjacking",
73          "+ /robots.txt"
74      ]
75
76      for prefix in ignore_prefixes:
77          if lower_line.startswith(prefix):
78              return False
79
80      if "0 host(s) tested" in lower_line:
81          return False
82
83      return line.startswith("+")
84
85
86  def run_nikto_live(target, log_cb, result_cb, summary_cb):
87      target_url = normalize_target_for_nikto(target)
88
89      findings = 0
90      threats = 0
91      hosts = 1
92      server_name = "http"
93
94      cmd = f"nikto -h {target_url}"
95      process = run_ssh_command(cmd)
96
97      log_cb(f"[NIKTO] Starting Nikto scan on {target_url}")
98      summary_cb(hosts, threats, findings, "Running")
99
100     for raw in process.stdout:
101         line = raw.strip()
102         if not line:
103             continue
104

```

```

scanners > nikto_scanner.py
86  def run_nikto_live(target, log_cb, result_cb, summary_cb):
100      for raw in process.stdout:
104
105          log_cb(f"[NIKTO] {line}")
106
107          lower_line = line.lower()
108
109          # Capture server banner separately
110          if lower_line.startswith("+ server:"):
111              server_name = line.split(":", 1)[1].strip() if ":" in line else "http"
112              continue
113
114          # Only keep real findings
115          if is_real_nikto_finding(line):
116              findings += 1
117              risk_level = get_nikto_risk(line)
118
119              if risk_level in {"Medium", "High", "Critical"}:
120                  threats += 1
121
122              result = {
123                  "scanner": "NIKTO",
124                  "host": target,
125                  "port": "80",
126                  "protocol": "tcp",
127                  "state": "open",
128                  "service": server_name,
129                  "risk_level": risk_level,
130                  "finding": line
131              }
132
133              result_cb(result)
134              summary_cb(hosts, threats, findings, "Running")
135
136      return_code = process.wait()
137
138      if return_code == 0:
139          log_cb("[NIKTO] Scan completed.")
140          summary_cb(hosts, threats, findings, "Complete")
141      else:
142          log_cb(f"[NIKTO] Scan failed with code {return_code}")
143          summary_cb(hosts, threats, findings, "Failed")
144

```

Figure 108: Nikto scanner module implemented for identifying web vulnerabilities in target systems

Nessus_scanner.py

```
scanners > ➤ nessus_scanner.py
1  import requests
2  import urllib3
3
4  urllib3.disable_warnings()
5
6  NESSUS_URL = "https://192.168.153.143:8834"
7  USERNAME = "binni"
8  PASSWORD = "binni@2005"
9
10
11 def get_token():
12     response = requests.post(
13         f"{NESSUS_URL}/session",
14         json={"username": USERNAME, "password": PASSWORD},
15         verify=False,
16         timeout=30
17     )
18     response.raise_for_status()
19     return response.json()["token"]
20
21
22 def get_scan_id(headers, scan_name="ThreatQuill Scan"):
23     response = requests.get(
24         f"{NESSUS_URL}/scans",
25         headers=headers,
26         verify=False,
27         timeout=30
28     )
29     response.raise_for_status()
30
31     scans = response.json().get("scans", [])
32     for scan in scans:
33         if scan.get("name") == scan_name:
34             return scan.get("id")
35
36     raise Exception(f"Scan '{scan_name}' not found.")
37
38
39 def get_scan_details(headers, scan_id):
40     response = requests.get(
41         f"{NESSUS_URL}/scans/{scan_id}",
42         headers=headers,
43         verify=False,
44         timeout=30
45     )
46     response.raise_for_status()
47     return response.json()
48
49
50 def map_severity(severity_num):
51     mapping = {
52         4: "Critical",
53         3: "High",
54         2: "Medium".
```

```

scanners > nessus_scanner.py
50 def map_severity(severity_num):
51     mapping = {
52         2: "Medium",
53         1: "Low",
54         0: "Info"
55     }
56     return mapping.get(severity_num, "Info")
57
58 def run_nessus_live(target, log_cb, result_cb, summary_cb):
59     try:
60         log_cb("[NESSUS] Logging in...")
61         token = get_token()
62
63         headers = {
64             "X-Cookie": f"token={token}",
65             "Content-Type": "application/json"
66         }
67
68         log_cb("[NESSUS] Login successful")
69
70         scan_id = get_scan_id(headers, "ThreatQuill Scan")
71         log_cb(f"[NESSUS] Using scan ID: {scan_id}")
72
73         data = get_scan_details(headers, scan_id)
74         status = data.get("info", {}).get("status", "unknown")
75         log_cb(f"[NESSUS] Current status: {status}")
76
77         if status != "completed":
78             log_cb("[NESSUS] Scan is not completed yet. Please wait for completion in Nessus.")
79             summary_cb(1, 0, 0, "Running")
80             return
81
82         vulns = data.get("vulnerabilities", [])
83
84         findings = 0
85         threats = 0
86
87         for vuln in vulns:
88             findings += 1
89             risk_level = map_severity(vuln.get("severity", 0))
90
91             if risk_level in ["Critical", "High", "Medium"]:
92                 threats += 1
93
94             result = {
95                 "scanner": "NESSUS",
96                 "host": target,
97                 "port": "",
98                 "protocol": "tcp",
99                 "state": "open",
100                 "service": vuln.get("plugin_name", "Nessus Finding"),
101                 "risk_level": risk_level,
102                 "finding": vuln.get("plugin_name", "")
103             }
104
105             result_cb(result)
106
107         log_cb(f"[NESSUS] Retrieved {findings} results successfully")
108         summary_cb(1, threats, findings, "Complete")
109
110     except Exception as e:
111         log_cb(f"[NESSUS ERROR] {e}")
112         summary_cb(1, 0, 0, "Failed")

```

Figure 109: Nessus scanner integrated using API for automated vulnerability scanning and result retrieval

Openvas_scanner.py

```
scanners > openvas_scanner.py
1
2 import subprocess
3 import json
4 import time
5
6
7 def severity_from_cvss(cvss):
8     try:
9         score = float(cvss)
10    except Exception:
11        return "Info"
12
13    if score >= 9.0:
14        return "Critical"
15    elif score >= 7.0:
16        return "High"
17    elif score >= 4.0:
18        return "Medium"
19    elif score > 0:
20        return "Low"
21    return "Info"
22
23
24 def run_openvas_live(target, log_cb, result_cb, summary_cb):
25     """
26     Runs a real OpenVAS helper script on Kali and reads JSON results line by line.
27     This expects your Kali machine to have a script that launches an OpenVAS task
28     and prints JSON findings to stdout.
29     """
30
31     log_cb("[OPENVAS] Starting real OpenVAS scan...")
32
33     command = [
34         "ssh",
35         "kali@192.168.153.143",
36         f"python3 /home/kali/threatquill/openvas_helper.py {target}"
37     ]
38
39     try:
40         process = subprocess.Popen(
41             command,
42             stdout=subprocess.PIPE,
43             stderr=subprocess.STDOUT,
44             text=True,
45             bufsize=1
46         )
47
48         findings_count = 0
49         high_threats = 0
50
51         for raw_line in process.stdout:
52             line = raw_line.strip()
53             if not line:
54                 continue
```

```

scanners > openvas_scanner.py
51     for raw_line in process.stdout:
52         line = raw_line.strip()
53         if not line:
54             continue
55
56         if line.startswith("[LOG]"):
57             log_cb(line)
58             continue
59
60         try:
61             finding = json.loads(line)
62
63             result = {
64                 "scanner": "OPENVAS",
65                 "target": finding.get("target", target),
66                 "host": finding.get("target", target),
67                 "title": finding.get("title", "OpenVAS Finding"),
68                 "description": finding.get("description", ""),
69                 "port": str(finding.get("port", "")),
70                 "protocol": finding.get("protocol", "tcp"),
71                 "state": "open",
72                 "service": finding.get("service", ""),
73                 "severity": finding.get("severity", severity_from_cvss(finding.get("cvss", 0))),
74                 "cvss": finding.get("cvss", 0),
75                 "cve": finding.get("cve", ""),
76                 "solution": finding.get("solution", "")
77             }
78
79             findings_count += 1
80             if result["severity"] in ["High", "Critical"]:
81                 high_threats += 1
82
83             result_cb(result)
84             summary_cb(1, high_threats, findings_count, "Running")
85
86         except json.JSONDecodeError:
87             log_cb(f"[OPENVAS RAW] {line}")
88
89     process.wait()
90
91     if process.returncode == 0:
92         log_cb("[OPENVAS] Real OpenVAS scan completed.")
93         summary_cb(1, high_threats, findings_count, "Complete")
94     else:
95         log_cb(f"[OPENVAS ERROR] Process exited with code {process.returncode}")
96         summary_cb(1, high_threats, findings_count, "Failed")
97
98 except Exception as e:
99     log_cb(f"[OPENVAS ERROR] {e}")

```

Figure 110: OpenVAS scanner module implemented for performing vulnerability scans on target systems

Openvas_praser.py

```
openvas_parser.py
1  # openvas_parser.py
2  import xml.etree.ElementTree as ET
3
4  def parse_openvas_report(file_path):
5      tree = ET.parse(file_path)
6      root = tree.getroot()
7
8      results = []
9
10     for result in root.findall("./result"):
11         data = {
12             "scanner": "OpenVAS",
13             "host": result.findtext("host"),
14             "port": result.findtext("port"),
15             "name": result.findtext("name"),
16             "severity": result.findtext("severity")
17         }
18         results.append(data)
19
20     return results
```

Figure 111: OpenVAS scan results parsed and structured

Ssh_helper.py

```
utils > ssh_helper.py
1
2 import paramiko
3
4
5 class SSHProcess:
6     def __init__(self, ssh, stdout, channel):
7         self.ssh = ssh
8         self.stdout = stdout
9         self.channel = channel
10
11     def wait(self):
12         exit_code = self.channel.recv_exit_status()
13         self.ssh.close()
14         return exit_code
15
16
17 def run_ssh_command(command):
18     ssh = paramiko.SSHClient()
19     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
20
21     ssh.connect(
22         hostname="192.168.153.143",
23         username="kali",
24         password="kalii"
25     )
26
27     stdin, stdout, stderr = ssh.exec_command(command)
28     return SSHProcess(ssh, stdout, stdout.channel)
```

Figure 112: SSH module implemented to execute scans

Database.py

```

database.py
1  import sqlite3
2
3  DB_NAME = "threatquill.db"
4
5
6  def connect_db():
7      return sqlite3.connect(DB_NAME)
8
9
10 def create_table():
11     conn = connect_db()
12     cursor = conn.cursor()
13
14     cursor.execute("""
15         CREATE TABLE IF NOT EXISTS scan_results (
16             id INTEGER PRIMARY KEY AUTOINCREMENT,
17             scanner TEXT,
18             host TEXT,
19             port TEXT,
20             service TEXT,
21             vulnerability TEXT,
22             severity TEXT,
23             timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
24         )
25     """)
26
27     conn.commit()
28     conn.close()
29
30
31 def insert_result(scanner, host, port, service, vulnerability, severity):
32     conn = connect_db()
33     cursor = conn.cursor()
34
35     cursor.execute("""
36         INSERT INTO scan_results (scanner, host, port, service, vulnerability, severity)
37         VALUES (?, ?, ?, ?, ?, ?)
38     """, (scanner, host, port, service, vulnerability, severity))
39
40     conn.commit()
41     conn.close()
42
43
44 def fetch_all_results():
45     conn = connect_db()
46     cursor = conn.cursor()
47
48     cursor.execute("SELECT scanner, host, port, service, vulnerability, severity, timestamp FROM scan_results")
49     rows = cursor.fetchall()
50
51     conn.close()
52     return rows
53
54
55 def count_by_severity(level):
56     conn = connect_db()
57     cursor = conn.cursor()
58
59     cursor.execute("SELECT COUNT(*) FROM scan_results WHERE severity = ?", (level,))
60     count = cursor.fetchone()[0]
61
62     conn.close()
63     return count
64
65
66 def count_all():
67     conn = connect_db()
68     cursor = conn.cursor()
69
70     cursor.execute("SELECT COUNT(*) FROM scan_results")
71     count = cursor.fetchone()[0]
72
73     conn.close()
74     return count

```

Figure 113: Database module developed to store and manage scan

Report_generator.py

```
report_generator.py
1  import os
2  import json
3  from datetime import datetime
4  from collections import Counter, defaultdict
5
6  def normalize_severity(value):
7      if not value:
8          return "Info"
9
10     value = str(value).strip().title()
11     allowed = {"Critical", "High", "Medium", "Low", "Info"}
12
13     if value in allowed:
14         return value
15
16     return "Info"
17
18
19 def get_severity_counts(results):
20     counter = Counter()
21
22     for item in results:
23         risk = normalize_severity(
24             item.get("risk_level", item.get("severity", "Info"))
25         )
26         counter[risk] += 1
27
28     return counter
29
30
31 def group_findings_by_host(results):
32     host_summary = defaultdict(lambda: {
33         "Critical": 0,
34         "High": 0,
35         "Medium": 0,
36         "Low": 0,
37         "Info": 0,
38         "total": 0
39     })
40
41     for item in results:
42         host = item.get("host", item.get("target", "Unknown"))
43         severity = normalize_severity(
44             item.get("risk_level", item.get("severity", "Info"))
45         )
46
47         host_summary[host][severity] += 1
48         host_summary[host]["total"] += 1
49
50     return dict(host_summary)
51
52
```



```
53 def generate_report_data(target, results):
54     severity_counts = get_severity_counts(results)
55     host_summary = group_findings_by_host(results)
56
57     report_data = {
58         "project_name": "ThreatQuill Analyzer",
59         "report_title": "Network Vulnerability Assessment Report",
60         "target": target,
61         "scan_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
62         "tools_used": sorted(list(set(r.get("scanner", "Unknown") for r in results))),
63         "total_findings": len(results),
64         "total_hosts": len(host_summary),
65         "severity_summary": {
66             "Critical": severity_counts.get("Critical", 0),
67             "High": severity_counts.get("High", 0),
68             "Medium": severity_counts.get("Medium", 0),
69             "Low": severity_counts.get("Low", 0),
70             "Info": severity_counts.get("Info", 0),
71         },
72         "host_summary": host_summary,
73         "findings": results
74     }
75
76     return report_data
77
78
79 def save_json_report(report_data, filename="reports/report.json"):
80     os.makedirs("reports", exist_ok=True)
81
82     with open(filename, "w", encoding="utf-8") as f:
83         json.dump(report_data, f, indent=4)
84
85     return filename
86
87
88 def save_html_report(report_data, filename="reports/report.html"):
89     os.makedirs("reports", exist_ok=True)
90
91     severity = report_data["severity_summary"]
92     host_summary = report_data.get("host_summary", {})
93     findings = report_data.get("findings", [])
94
95     total_hosts = report_data.get("total_hosts", 0)
96     total_findings = report_data.get("total_findings", 0)
97     total_threats = severity["Critical"] + severity["High"]
98
99     tools_html = "".join(f"<li>{tool}</li>" for tool in report_data["tools_used"])
100
```

```

101     host_rows = ""
102     sorted_hosts = sorted(
103         host_summary.items(),
104         key=lambda x: x[1]["total"],
105         reverse=True
106     )
107
108     for host, data in sorted_hosts:
109         critical = data["Critical"]
110         high = data["High"]
111         medium = data["Medium"]
112         low = data["Low"]
113         info = data["Info"]
114         total = max(data["total"], 1)
115
116         critical_w = (critical / total) * 100
117         high_w = (high / total) * 100
118         medium_w = (medium / total) * 100
119         low_w = (low / total) * 100
120         info_w = (info / total) * 100
121
122         host_rows += f"""
123         <tr>
124             <td class="host-cell">{host}</td>
125             <td>
126                 <div class="bar-wrap">
127                     <div class="seg critical" style="width:{critical_w:.2f}%>{critical if critical else ""}</div>
128                     <div class="seg high" style="width:{high_w:.2f}%>{high if high else ""}</div>
129                     <div class="seg medium" style="width:{medium_w:.2f}%>{medium if medium else ""}</div>
130                     <div class="seg low" style="width:{low_w:.2f}%>{low if low else ""}</div>
131                     <div class="seg info" style="width:{info_w:.2f}%>{info if info else ""}</div>
132                 </div>
133             </td>
134         </tr>
135         """
136
137     if not host_rows:
138         host_rows = ""
139         <tr>
140             <td class="host-cell">No hosts available</td>
141             <td>
142                 <div class="bar-wrap empty-bar">No scan data available</div>
143             </td>
144         </tr>
145         """
146
147     detailed_rows = ""
148     for idx, item in enumerate(findings, start=1):
149         scanner = item.get("scanner", "")
150         host = item.get("host", item.get("target", ""))
151         port = item.get("port", "")
152         protocol = item.get("protocol", "")

```

```

report_generator.py
88 def save_html_report(report_data, filename="reports/report.html"):
148     for idx, item in enumerate(findings, start=1):
153         state = item.get("state", "")
154         service = item.get("service", "")
155         risk = normalize_severity(item.get("risk_level", item.get("severity", "Info")))
156         description = item.get("description", "No description available.")
157         recommendation = item.get("recommendation", "Review and secure this service.")
158
159         detailed_rows += f"""
160         <tr>
161             <td>{idx}</td>
162             <td>{scanner}</td>
163             <td>{host}</td>
164             <td>{port}/{protocol}</td>
165             <td>{service}</td>
166             <td>{state}</td>
167             <td>{risk}</td>
168             <td>{description}</td>
169             <td>{recommendation}</td>
170         </tr>
171         """
172
173     if not detailed_rows:
174         detailed_rows = """
175         <tr>
176             <td colspan="9">No detailed findings available.</td>
177         </tr>
178         """
179
180     total_chart = max(
181         severity["Critical"] + severity["High"] + severity["Medium"] + severity["Low"] + severity["Info"],
182         1
183     )
184
185     critical_pct = (severity["Critical"] / total_chart) * 100
186     high_pct = (severity["High"] / total_chart) * 100
187     medium_pct = (severity["Medium"] / total_chart) * 100
188     low_pct = (severity["Low"] / total_chart) * 100
189     info_pct = (severity["Info"] / total_chart) * 100
190
191     s1 = critical_pct
192     s2 = s1 + high_pct
193     s3 = s2 + medium_pct
194     s4 = s3 + low_pct
195
196     donut_style = (
197         "background: conic-gradient("
198         f"#8d1b3d 0% {s1:.2f}%, "
199         f"#e94b4b {s1:.2f}% {s2:.2f}%, "
200         f"#f2993a {s2:.2f}% {s3:.2f}%, "
201         f"#f2c94c {s3:.2f}% {s4:.2f}%, "
202         f"#5aa9e6 {s4:.2f}% 100%"
203         ");"
204     )

```

```
205
206     html = f"""
207 <!DOCTYPE html>
208 <html>
209 <head>
210     <meta charset="utf-8">
211     <title>ThreatQuill Analyzer</title>
212     <style>
213         * {{
214             box-sizing: border-box;
215         }}
216
217         body {{
218             margin: 0;
219             font-family: Arial, sans-serif;
220             background: #f4f5f7;
221             color: #202124;
222         }}
223
224         .topbar {{
225             background: #22384c;
226             color: white;
227             padding: 14px 24px;
228             font-size: 16px;
229             font-weight: bold;
230         }}
231
232         .page {{
233             max-width: 1400px;
234             margin: 0 auto;
235             padding: 22px 24px 30px 24px;
236         }}
237
238         .title {{
239             font-size: 24px;
240             font-weight: bold;
241             margin-bottom: 6px;
242             color: #222;
243         }}
244
245         .subtitle {{
246             font-size: 14px;
247             color: #4b6b88;
248             margin-bottom: 18px;
249         }}
250
251         .summary-strip {{
252             display: grid;
253             grid-template-columns: repeat(4, 1fr);
254             gap: 12px;
255             margin-bottom: 18px;
256         }}
```

```

report_generator.py
209 <head>
212 <style>
257
258 .summary-box {{
259     background: white;
260     border: 1px solid #d9dde3;
261     border-radius: 4px;
262     padding: 14px;
263 }}
264
265 .summary-label {{
266     font-size: 13px;
267     color: #555;
268     margin-bottom: 6px;
269 }}
270
271 .summary-value {{
272     font-size: 24px;
273     font-weight: bold;
274     color: #22384c;
275 }}
276
277 .main-grid {{
278     display: grid;
279     grid-template-columns: minmax(0, 1fr) 320px;
280     gap: 18px;
281     align-items: start;
282 }}
283
284 .card {{
285     background: white;
286     border: 1px solid #d9dde3;
287     border-radius: 4px;
288     overflow: hidden;
289 }}
290
291 .filter-row {{
292     display: flex;
293     align-items: center;
294     gap: 12px;
295     padding: 12px 14px;
296     border-bottom: 1px solid #e6e8ec;
297 }}
298
299 .filter-box {{
300     border: 1px solid #d9dde3;
301     background: white;
302     padding: 8px 10px;
303     border-radius: 4px;
304     font-size: 13px;
305     color: #444;
306 }}
307
308 .host-count {{
309     font-size: 14px;
310     color: #555;
311 }}
312
313 table {{
314     width: 100%;
315     border-collapse: collapse;
316 }}
317
318 .summary-table th,
319 .summary-table td {{
320     border-top: 1px solid #e6e8ec;

```

```
report_generator.py
209 <head>
212 <style>
319 .summary-table td {{
320     border-top: 1px solid #e6e8ec;
321     padding: 14px;
322     text-align: left;
323     vertical-align: middle;
324     font-size: 14px;
325 }}
326
327 .summary-table th {{
328     background: #f8f9fb;
329     font-weight: bold;
330     color: #444;
331 }}
332
333 .host-cell {{
334     width: 220px;
335     white-space: nowrap;
336 }}
337
338 .bar-wrap {{
339     width: 100%;
340     height: 24px;
341     display: flex;
342     overflow: hidden;
343     background: #efefef;
344     border: 1px solid #dfdfff;
345     border-radius: 3px;
346 }}
347
348 .empty-bar {{
349     display: flex;
350     align-items: center;
351     justify-content: center;
352     color: #666;
353     font-size: 13px;
354 }}
355
356 .seg {{
357     height: 100%;
358     display: flex;
359     align-items: center;
360     justify-content: center;
361     font-size: 12px;
362     white-space: nowrap;
363     overflow: hidden;
364 }}
365
366 .critical {{
367     background: #8d1b3d;
368     color: white;
369 }}
370
371 .high {{
372     background: #e94b4b;
373     color: white;
374 }}
375
376 .medium {{
377     background: #f2993a;
378     color: #222;
379 }}
380
```

```

report_generator.py
209 <head>
212 <style>

381 .low {{
382     background: #f2c94c;
383     color: #222;
384 }}
385
386 .info {{
387     background: #5aa9e6;
388     color: #222;
389 }}
390
391 .side-card {{
392     background: white;
393     border: 1px solid #d9dde3;
394     border-radius: 4px;
395     overflow: hidden;
396 }}
397
398 .notice {{
399     margin: 14px;
400     padding: 12px 14px;
401     background: #f7efd1;
402     border: 1px solid #e8dca9;
403     border-radius: 4px;
404     font-size: 13px;
405     line-height: 1.5;
406 }}
407
408 .side-section {{
409     padding: 14px;
410     border-top: 1px solid #eceff3;
411 }}
412
413 .side-title {{
414     font-size: 15px;
415     font-weight: bold;
416     margin-bottom: 12px;
417 }}
418
419 .detail-row {{
420     display: grid;
421     grid-template-columns: 105px 1fr;
422     gap: 8px;
423     margin-bottom: 10px;
424     font-size: 14px;
425 }}
426
427 .detail-label {{
428     color: #555;
429 }}
430
431 .donut-wrap {{
432     display: flex;
433     gap: 16px;
434     align-items: center;
435     margin-top: 10px;
436 }}
437
438 .donut {{
439     width: 126px;
440     height: 126px;
441     border-radius: 50%;
442     position: relative;
443     {donut_style}

```

```

report_generator.py
209 <head>
212 <style>
438 .donut {{
444     flex-shrink: 0;
445 }}
446
447 .donut::after {{
448     content: "";
449     width: 62px;
450     height: 62px;
451     border-radius: 50%;
452     background: white;
453     position: absolute;
454     left: 32px;
455     top: 32px;
456 }}
457
458 .legend-item {{
459     display: flex;
460     align-items: center;
461     gap: 8px;
462     margin-bottom: 8px;
463     font-size: 14px;
464 }}
465
466 .dot {{
467     width: 10px;
468     height: 10px;
469     border-radius: 50%;
470     display: inline-block;
471 }}
472
473 .section-title {{
474     margin: 24px 0 12px 0;
475     font-size: 18px;
476     font-weight: bold;
477 }}
478
479 .detail-table {{
480     background: white;
481     border: 1px solid #d9dde3;
482     border-radius: 4px;
483     overflow: hidden;
484 }}
485
486 .detail-table th,
487 .detail-table td {{
488     border-top: 1px solid #e6e8ec;
489     padding: 10px 12px;
490     text-align: left;
491     vertical-align: top;
492     font-size: 13px;
493 }}
494
495 .detail-table th {{
496     background: #f8f9fb;
497     font-weight: bold;
498 }}
499
500 .footer-note {{
501     margin-top: 18px;
502     background: white;
503     border: 1px solid #d9dde3;
504     border-radius: 4px;
505     padding: 14px;

```



```

report_generator.py
209 <head>
212 <style>
500 .footer-note {{
506     font-size: 14px;
507 }}
508
509 ul {{
510     margin: 6px 0 0 18px;
511 }}
512 </style>
513 </head>
514 <body>
515     <div class="topbar">{report_data["project_name"]}</div>
516
517     <div class="page">
518         <div class="title">{report_data["report_title"]}</div>
519         <div class="subtitle">Target: {report_data["target"]}</div>
520
521         <div class="summary-strip">
522             <div class="summary-box">
523                 <div class="summary-label">Total Hosts</div>
524                 <div class="summary-value">{total_hosts}</div>
525             </div>
526
527             <div class="summary-box">
528                 <div class="summary-label">Total Findings</div>
529                 <div class="summary-value">{total_findings}</div>
530             </div>
531
532             <div class="summary-box">
533                 <div class="summary-label">Top Threats</div>
534                 <div class="summary-value">{total_threats}</div>
535             </div>
536
537             <div class="summary-box">
538                 <div class="summary-label">Status</div>
539                 <div class="summary-value" style="font-size:20px;">Completed</div>
540             </div>
541         </div>
542
543         <div class="main-grid">
544             <div class="card">
545                 <div class="filter-row">
546                     <div class="filter-box">Host Summary</div>
547                     <div class="filter-box">Severity Distribution</div>
548                     <div class="host-count">{total_hosts} Hosts</div>
549                 </div>
550
551                 <table class="summary-table">
552                     <tr>
553                         <th class="host-cell">Host</th>
554                         <th>Vulnerabilities</th>
555                     </tr>
556                     {host_rows}
557                 </table>
558             </div>
559
560             <div class="side-card">
561                 <div class="notice">
562                     <strong>Notice:</strong> This report presents the latest consolidated findings generated by ThreatQuill Analyzer.
563                 </div>
564
565                 <div class="side-section">
566                     <div class="side-title">Assessment Details</div>

```

```

report_generator.py
514 <body>
515   <div class="page">
516     <div class="main-grid">
517       <div class="side-card">
518         <div class="side-section">
519           <div class="detail-row">
520             <div class="detail-label">Target:</div>
521             <div>{report_data["target"]}</div>
522           </div>
523           <div class="detail-row">
524             <div class="detail-label">Status:</div>
525             <div>Completed</div>
526           </div>
527           <div class="detail-row">
528             <div class="detail-label">Modified:</div>
529             <div>{report_data["scan_date"]}</div>
530           </div>
531           <div class="detail-row">
532             <div class="detail-label">Method:</div>
533             <div>Multi-scanner vulnerability assessment</div>
534           </div>
535           <div class="detail-row">
536             <div class="detail-label">Tools:</div>
537             <div><ul>{tools_html}</ul></div>
538           </div>
539         </div>
540       <div class="side-section">
541         <div class="side-title">Vulnerabilities</div>
542         <div class="donut-wrap">
543           <div class="donut"></div>
544           <div>
545             <div class="legend-item"><span class="dot" style="background:#8d1b3d;"></span> Critical ({severity["Critical"]})</div>
546             <div class="legend-item"><span class="dot" style="background:#e94b4b;"></span> High ({severity["High"]})</div>
547             <div class="legend-item"><span class="dot" style="background:#f2993a;"></span> Medium ({severity["Medium"]})</div>
548             <div class="legend-item"><span class="dot" style="background:#f2c94c;"></span> Low ({severity["Low"]})</div>
549             <div class="legend-item"><span class="dot" style="background:#5aa9e6;"></span> Info ({severity["Info"]})</div>
550           </div>
551         </div>
552       </div>
553     </div>
554   </div>
555   <div class="section-title">Detailed Findings</div>
556   <table class="detail-table">
557     <tr>
558       <th>#</th>
559       <th>Scanner</th>
560       <th>Host</th>
561       <th>Port</th>
562       <th>Service</th>
563       <th>State</th>
564       <th>Risk</th>
565       <th>Description</th>
566       <th>Recommendation</th>
567     </tr>
568     {detailed_rows}
569   </table>

```

```

report_generator.py
514 <body>
515   <div class="page">
516     <div class="main-grid">
517       <div class="side-card">
518         <div class="side-section">
519           <div>
520             <div class="legend-item"><span class="dot" style="background:#8d1b3d;"></span> Critical {{severity["Critical"]}}</div>
521             <div class="legend-item"><span class="dot" style="background:#e94b4b;"></span> High {{severity["High"]}}</div>
522             <div class="legend-item"><span class="dot" style="background:#f2993a;"></span> Medium {{severity["Medium"]}}</div>
523             <div class="legend-item"><span class="dot" style="background:#f2c94c;"></span> Low {{severity["Low"]}}</div>
524             <div class="legend-item"><span class="dot" style="background:#5aa9e6;"></span> Info {{severity["Info"]}}</div>
525           </div>
526         </div>
527       </div>
528     </div>
529   </div>
530   <div class="section-title">Detailed Findings</div>
531   <table class="detail-table">
532     <tr>
533       <th>#</th>
534       <th>Scanner</th>
535       <th>Host</th>
536       <th>Port</th>
537       <th>Services</th>
538       <th>State</th>
539       <th>Risk</th>
540       <th>Description</th>
541       <th>Recommendation</th>
542     </tr>
543     {{detailed_rows}}
544   </table>
545   <div class="footer-note">
546     <strong>Conclusion:</strong> Prioritize Critical and High findings first, then continue with Medium and Low issues. This report provides a host-based overview together with detailed vulnerability information.
547   </div>
548 </body>
549 </html>
550 ***
551
552 with open(filename, "w", encoding="utf-8") as f:
553     f.write(html)
554
555 return filename
556

```

Figure 114: Report generation module implemented

Email_sender.py

```

1  import os
2  import smtplib
3  from email.message import EmailMessage
4
5
6  def send_report_email(sender_email, app_password, recipient_email, report_path):
7      """
8      Send ThreatQuill Analyzer report by email.
9
10     Args:
11         sender_email (str): Fixed sender Gmail address
12         app_password (str): Gmail App Password
13         recipient_email (str): Receiver email address
14         report_path (str): Path to report file
15     """
16
17     if not sender_email:
18         raise ValueError("Sender email is required.")
19
20     if not app_password:
21         raise ValueError("App password is required.")
22
23     if not recipient_email:
24         raise ValueError("Recipient email is required.")
25
26     if not report_path:
27         raise ValueError("Report path is required.")
28
29     if not os.path.exists(report_path):
30         raise FileNotFoundError(f"Report file not found: {report_path}")
31
32     msg = EmailMessage()
33     msg["Subject"] = "ThreatQuill Analyzer Final Report"
34     msg["From"] = sender_email
35     msg["To"] = recipient_email
36     msg.set_content(
37         "Hello,\n\n"
38         "Please find the attached final report generated by ThreatQuill Analyzer.\n\n"
39         "Regards,\n"
40         "ThreatQuill Analyzer"
41     )
42
43     with open(report_path, "rb") as file:
44         file_data = file.read()
45         file_name = os.path.basename(report_path)
46
47     if file_name.lower().endswith(".html"):
48         maintype = "text"
49         subtype = "html"
50     elif file_name.lower().endswith(".json"):
51         maintype = "application"
52         subtype = "json"
53     elif file_name.lower().endswith(".txt"):
54         maintype = "text"
55         subtype = "plain"
56     else:
57         maintype = "application"
58         subtype = "octet-stream"
59
60     msg.add_attachment(
61         file_data,
62         maintype=maintype,
63         subtype=subtype,
64         filename=file_name
65     )

```

```
email_sender.py
6  def send_report_email(sender_email, app_password, recipient_email, report_path):
65
66
67      try:
68          with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
69              smtp.login(sender_email, app_password)
70              smtp.send_message(msg)
71
72          return True, f"Email sent successfully to {recipient_email}"
73
74      except smtplib.SMTPAuthenticationError:
75          raise Exception("Authentication failed. Please check your Gmail App Password.")
76
77      except smtplib.SMTPRecipientsRefused:
78          raise Exception("Recipient email address was refused by the server.")
79
80      except smtplib.SMTPException as e:
81          raise Exception(f"SMTP error occurred: {e}")
82
83      except Exception as e:
84          raise Exception(f"Failed to send email: {e}")
85
```

Figure 115: Email module implemented

3.6.3 Frontend Development

Login.py

```

login.py
1  import tkinter as tk
2  from tkinter import messagebox
3  import dashboard
4
5  def main():
6      window = tk.Tk()
7      window.title("ThreatQuill Analyzer - Login")
8      window.state("zoomed")
9      window.configure(bg="#f1f5f9")
10
11     username_var = tk.StringVar()
12     password_var = tk.StringVar()
13
14     # ----- title -----
15     title = tk.Label(
16         window,
17         text="ThreatQuill Analyzer",
18         font=("Arial", 26, "bold"),
19         bg="white",
20         fg="#1e293b"
21     )
22     title.pack(pady=40)
23
24     # ----- login frame -----
25     frame = tk.Frame(window, bg="#f1f5f9", padx=30, pady=30)
26     frame.pack()
27
28     # username
29     tk.Label(frame, text="Username", font=("Arial", 12),
30             bg="#f1f5f9", fg="#1e293b").grid(row=0, column=0, sticky="w")
31
32     username_entry = tk.Entry(frame, textvariable=username_var, width=30, font=("Arial", 12))
33     username_entry.grid(row=1, column=0, pady=8)
34
35     # password
36     tk.Label(frame, text="Password", font=("Arial", 12),
37             bg="#f1f5f9", fg="#1e293b").grid(row=2, column=0, sticky="w")
38
39     password_entry = tk.Entry(frame, textvariable=password_var, show="*", width=30, font=("Arial", 12))
40     password_entry.grid(row=3, column=0, pady=8)
41
42     # message
43     message_label = tk.Label(frame, text="", bg="#f1f5f9", fg="red", font=("Arial", 11))
44     message_label.grid(row=4, column=0, pady=5)
45
46     # ----- login function -----
47     def handle_login():
48         user = username_var.get().strip()
49         pwd = password_var.get().strip()
50
51         if user == "" or pwd == "":
52             message_label.config(text="Enter username and password")
53         return
54

```

```

login.py
5   def main():
47   def handle_login():
51       if user == "" or pwd == "":
54           if user == "binni" and pwd == "binnilogin":
55               message_label.config(text="Login successful", fg="green")
56               window.after(500, lambda: open_dashboard(user))
57           else:
58               message_label.config(text="Invalid credentials", fg="red")
59
60   def open_dashboard(user):
61       window.destroy()
62       dashboard.open_dashboard(user)
63
64   # ----- forgot password -----
65   def forgot_password():
66       messagebox.showinfo(
67           "Password Reset",
68           "Please contact the system administrator to reset your password."
69       )
70
71   # ----- buttons -----
72   login_btn = tk.Button(
73       frame,
74       text="Login",
75       command=handle_login,
76       width=20,
77       bg="#2563eb",
78       fg="white",
79       font=("Arial", 12, "bold")
80   )
81   login_btn.grid(row=5, column=0, pady=15)
82
83   forgot_btn = tk.Button(
84       frame,
85       text="Forgot Password?",
86       command=forgot_password,
87       bg="#f1f5f9",
88       fg="#2563eb",
89       bd=0,
90       font=("Arial", 10, "underline"),
91       cursor="hand2"
92   )
93   forgot_btn.grid(row=6, column=0)
94
95   username_entry.focus()
96   window.mainloop()
97
98
99
100  if __name__ == "__main__":
101      main()

```

Figure 116: Login module implemented to authenticate users

Dashboard.py

```

dashboard.py
1  import re
2  import tkinter as tk
3  from tkinter import ttk, messagebox
4  from datetime import datetime
5  import os
6  import webbrowser
7  from scanner_controller import start_full_scan
8  from summary_view import ScanSummaryWindow
9  from email_sender import send_report_email
10
11 def is_valid_target(target: str) -> bool:
12     target = target.strip()
13
14     if not target:
15         return False
16
17     # IPv4 check
18     ipv4_pattern = r"^(?!(\d{1,3}\.){3}\d{1,3})$"
19     if re.match(ipv4_pattern, target):
20         parts = target.split(".")
21         return all(0 <= int(part) <= 255 for part in parts)
22
23     # domain check
24     domain_pattern = r"^(?!-)[A-Za-z0-9-]{1,63}(?!-)(\.[A-Za-z]{2,})+ $"
25     if re.match(domain_pattern, target):
26         return True
27
28     # localhost
29     if target.lower() == "localhost":
30         return True
31
32     return False
33
34
35 class ThreatQuillDashboard:
36     def __init__(self, root, username="binni"):
37         self.root = root
38         self.username = username
39
40         self.latest_report_data = None
41         self.latest_html_path = None
42
43         self.root.title("ThreatQuill Analyzer - Dashboard")
44         self.root.state("zoomed")
45         self.root.configure(bg="#F3F4F6")
46
47         self._build_styles()
48         self._build_layout()
49         self._build_topbar()
50         self._build_sidebar()
51         self._build_main_content()
52         self._bind_sidebar_actions()
53
54     def _ui(self, func, *args):
55         self.root.after(0, lambda: func(*args))
56
57     def _build_styles(self):
58         style = ttk.Style()
59         style.theme_use("default")
60
61         style.configure(
62             "Treeview",
63             font=("Segoe UI", 10),
64             rowheight=26,
65             background="white",
66             ...

```



```

dashboard.py
35 class ThreatQuillDashboard:
57     def _build_styles(self):
61         style.configure(
66             fieldbackground="white",
67             foreground="#111827",
68         )
69         style.configure(
70             "Treeview.Heading",
71             font=("Segoe UI", 10, "bold"),
72             background="#E5E7EB",
73             foreground="#111827",
74             relief="flat",
75         )
76         style.map("Treeview", background=[("selected", "#D8EAFE")])
77
78     def _build_layout(self):
79         self.root.rowconfigure(0, weight=0)
80         self.root.rowconfigure(1, weight=1)
81         self.root.columnconfigure(0, weight=0)
82         self.root.columnconfigure(1, weight=1)
83
84         self.top_bar = tk.Frame(self.root, bg="#071633", height=145)
85         self.top_bar.grid(row=0, column=0, columnspan=2, sticky="nsew")
86         self.top_bar.grid_propagate(False)
87
88         self.sidebar = tk.Frame(self.root, bg="#1E2D45", width=170)
89         self.sidebar.grid(row=1, column=0, sticky="nsew")
90         self.sidebar.grid_propagate(False)
91
92         self.main_area = tk.Frame(self.root, bg="#F3F4F6")
93         self.main_area.grid(row=1, column=1, sticky="nsew", padx=12, pady=12)
94         self.main_area.rowconfigure(0, weight=0)
95         self.main_area.rowconfigure(1, weight=0)
96         self.main_area.rowconfigure(2, weight=1)
97         self.main_area.columnconfigure(0, weight=3)
98         self.main_area.columnconfigure(1, weight=2)
99
100     def _build_topbar(self):
101         left_wrap = tk.Frame(self.top_bar, bg="#071633")
102         left_wrap.pack(side="left", fill="y", padx=18, pady=14)
103
104         logo_frame = tk.Frame(left_wrap, bg="#071633", width=130, height=130)
105         logo_frame.pack(side="left")
106         logo_frame.pack_propagate(False)
107
108         try:
109             logo_path = "logo.png"
110             if not os.path.exists(logo_path):
111                 logo_path = os.path.join("assets", "logo.png")
112
113             self.logo_img = tk.PhotoImage(file=logo_path)
114             self.logo_img = self.logo_img.subsample(3, 3)
115
116             self.logo_label = tk.Label(
117                 logo_frame,
118                 image=self.logo_img,
119                 bg="#071633",
120                 bd=0
121             )
122         except Exception:
123             self.logo_label = tk.Label(
124                 logo_frame,
125                 text="ThreatQuill",
126                 bg="#071633",
127                 fg="white",

```

```

dashboard.py
35 class ThreatQuillDashboard:
100     def _build_topbar(self):
122         except Exception:
123             self.logo_label = tk.Label(
128                 font=("Segoe UI", 14, "bold"),
129             )
130
131             self.logo_label.pack(expand=True)
132
133             title_frame = tk.Frame(left_wrap, bg="#071633")
134             title_frame.pack(side="left", padx=18)
135
136             self.title_label = tk.Label(
137                 title_frame,
138                 text="ThreatQuill Analyzer",
139                 bg="#071633",
140                 fg="white",
141                 font=("Segoe UI", 30, "bold"),
142             )
143             self.title_label.pack(anchor="w", pady=(30, 0))
144
145             right_info = tk.Frame(self.top_bar, bg="#071633")
146             right_info.pack(side="right", padx=24, pady=20)
147
148             self.user_label = tk.Label(
149                 right_info,
150                 text=f"Logged in as: {self.username}",
151                 bg="#071633",
152                 fg="#D1D5DB",
153                 font=("Segoe UI", 11),
154             )
155             self.user_label.pack(anchor="e", pady=(40, 0))
156
157     def _build_sidebar(self):
158         menu_title = tk.Label(
159             self.sidebar,
160             text="MENU",
161             bg="#1E2D45",
162             fg="white",
163             font=("Segoe UI", 14, "bold"),
164         )
165         menu_title.pack(pady=(22, 18))
166
167         self.sidebar_buttons = {}
168
169         for key, text in [
170             ("summary", "Scan Summary"),
171             ("scan", "New Scan"),
172             ("email", "Send Email"),
173             ("reports", "Reports"),
174             ("settings", "Settings"),
175             ("help", "Help"),
176         ]:
177             btn = tk.Button(
178                 self.sidebar,
179                 text=text,
180                 bg="#071633",
181                 fg="white",
182                 activebackground="#2D4263",
183                 activeforeground="white",
184                 relief="flat",
185                 anchor="w",
186                 padx=16,
187                 pady=12,
188                 font=("Segoe UI", 11),

```

```

dashboard.py
35 class ThreatQuillDashboard:
157     def _build_sidebar(self):
176         ]:
177             btn = tk.Button(
189                 cursor="hand2",
190             )
191             btn.pack(fill="x", pady=2, padx=10)
192             self.sidebar_buttons[key] = btn
193
194     def _build_main_content(self):
195         self._build_scan_controls()
196         self._build_summary_card()
197         self._build_results_and_logs()
198
199     def _build_scan_controls(self):
200         controls = tk.Frame(self.main_area, bg="#F3F4F6")
201         controls.grid(row=0, column=0, colspan=2, sticky="ew", pady=(0, 8))
202         controls.columnconfigure(1, weight=1)
203
204         tk.Label(
205             controls,
206             text="Target:",
207             bg="#F3F4F6",
208             fg="#111827",
209             font=("Segoe UI", 11, "bold"),
210         ).grid(row=0, column=0, padx=(2, 8), pady=4, sticky="w")
211
212         self.target_entry = tk.Entry(
213             controls,
214             font=("Segoe UI", 11),
215             bg="white",
216             fg="#111827",
217             relief="solid",
218             bd=1,
219         )
220         self.target_entry.grid(row=0, column=1, sticky="ew", padx=(0, 10))
221         self.target_entry.insert(0, "scanme.nmap.org")
222
223         self.start_scan_btn = tk.Button(
224             controls,
225             text="Start Scan",
226             bg="#2563E8",
227             fg="white",
228             activebackground="#1D4ED8",
229             activeforeground="white",
230             relief="flat",
231             padx=16,
232             pady=8,
233             font=("Segoe UI", 10, "bold"),
234             cursor="hand2",
235             command=self.start_scan_clicked,
236         )
237         self.start_scan_btn.grid(row=0, column=2, padx=(0, 8))
238
239         self.clear_btn = tk.Button(
240             controls,
241             text="Clear Results",
242             bg="#E5E7EB",
243             fg="#111827",
244             activebackground="#CBD5E1",
245             relief="flat",
246             padx=14,
247             pady=8,
248             font=("Segoe UI", 10, "bold"),
249             cursor="hand2",

```

```

dashboard.py
35 class ThreatQuillDashboard:
199     def _build_scan_controls(self):
239         self.clear_btn = tk.Button(
249             cursor="hand2",
250             command=self.clear_results,
251         )
252         self.clear_btn.grid(row=0, column=3)
253
254     def _build_summary_card(self):
255         self.summary_card = tk.Frame(
256             self.main_area,
257             bg="white",
258             highlightthickness=1,
259             highlightbackground="#D1D5DB",
260         )
261         self.summary_card.grid(row=1, column=0, colspan=2, sticky="ew", pady=(0, 10))
262         for i in range(4):
263             self.summary_card.columnconfigure(i, weight=1)
264
265             tk.Label(
266                 self.summary_card,
267                 text="Live Scan Overview",
268                 bg="white",
269                 fg="#111827",
270                 font=("Segoe UI", 16, "bold"),
271             ).grid(row=0, column=0, colspan=4, sticky="w", padx=18, pady=(12, 6))
272
273             self.hosts_value = self._create_stat(0, "Total Hosts Scanned", "0", "#111827")
274             self.threats_value = self._create_stat(1, "Threats Detected", "0", "#DC2626")
275             self.findings_value = self._create_stat(2, "Total Findings", "0", "#2563EB")
276             self.status_value = self._create_stat(3, "Status", "Idle", "#859669")
277
278     def _create_stat(self, col, label_text, value_text, value_color):
279         tk.Label(
280             self.summary_card,
281             text=label_text,
282             bg="white",
283             fg="#6B7280",
284             font=("Segoe UI", 10),
285         ).grid(row=1, column=col, sticky="w", padx=14, pady=(0, 1))
286
287         value = tk.Label(
288             self.summary_card,
289             text=value_text,
290             bg="white",
291             fg=value_color,
292             font=("Segoe UI", 16, "bold"),
293         )
294         value.grid(row=2, column=col, sticky="w", padx=14, pady=(0, 8))
295         return value
296
297     def _build_results_and_logs(self):
298         results_frame = tk.LabelFrame(
299             self.main_area,
300             text="Latest Scan Results / Report Findings",
301             bg="#F3F4F6",
302             fg="#111827",
303             font=("Segoe UI", 12, "bold"),
304             padx=8,
305             pady=8,
306         )
307         results_frame.grid(row=2, column=0, sticky="nsew", padx=(0, 6))
308         results_frame.rowconfigure(0, weight=1)
309         results_frame.columnconfigure(0, weight=1)
310
311         columns = ("scanner", "host", "port", "proto", "state", "service", "severity")
312         self.tree = ttk.Treeview(results_frame, columns=columns, show="headings", height=16)
313
314         headings = {
315             "scanner": "Scanner",
316             "host": "Host/IP",
317             "port": "Port",
318             "proto": "Proto",
319             "state": "State",
320             "service": "Service",
321             "severity": "Severity",
322         }
323

```

```

dashboard.py
35 class ThreatQuillDashboard:
297 def _build_results_and_logs(self):
334     for col in columns:
335         self.tree.heading(col, text=headings[col])
336         self.tree.column(col, width=widths[col], anchor="center")
337
338     self.tree.grid(row=0, column=0, sticky="nsew")
339
340     results_scroll = ttk.Scrollbar(results_frame, orient="vertical", command=self.tree.yview)
341     self.tree.configure(yscrollcommand=results_scroll.set)
342     results_scroll.grid(row=0, column=1, sticky="ns")
343
344     log_frame = tk.LabelFrame(
345         self.main_area,
346         text="Activity Log",
347         bg="#F3F4F6",
348         fg="#111827",
349         font=("Segoe UI", 12, "bold"),
350         padx=8,
351         pady=8,
352     )
353     log_frame.grid(row=2, column=1, sticky="nsew", padx=(6, 0))
354     log_frame.rowconfigure(0, weight=1)
355     log_frame.columnconfigure(0, weight=1)
356
357     self.log_text = tk.Text(
358         log_frame,
359         bg="white",
360         fg="#111827",
361         font=("Consolas", 10),
362         state="disabled",
363         wrap="word",
364         height=18,
365     )
366     self.log_text.grid(row=0, column=0, sticky="nsew")
367
368     log_scroll = ttk.Scrollbar(log_frame, orient="vertical", command=self.log_text.yview)
369     self.log_text.configure(yscrollcommand=log_scroll.set)
370     log_scroll.grid(row=0, column=1, sticky="ns")
371
372     def _bind_sidebar_actions(self):
373         self.sidebar_buttons["scan"].config(command=self.start_scan_clicked)
374         self.sidebar_buttons["reports"].config(command=self.open_reports_view)
375         self.sidebar_buttons["help"].config(command=self.show_help)
376         self.sidebar_buttons["settings"].config(command=self.show_settings)
377         self.sidebar_buttons["email"].config(command=self.show_email_message)
378         self.sidebar_buttons["summary"].config(command=self.show_scan_summary)
379
380     def append_log(self, message):
381         timestamp = datetime.now().strftime("%H:%M:%S")
382
383         if "[NIKTO]" in message:
384             tag = "NIKTO"
385         elif "[NMAP]" in message:
386             tag = "NMAP"
387         elif "[OPENVAS]" in message:
388             tag = "OPENVAS"
389         elif "[NESSUS]" in message:
390             tag = "NESSUS"
391         elif "ERROR" in message:
392             tag = "ERROR"
393         elif "[SYSTEM]" in message:
394             tag = "SYSTEM"
395         else:
396             tag = "INFO"
397
398         clean_msg = (
399             message.replace("[NMAP]", "")
400             .replace("[NIKTO]", "")
401             .replace("[OPENVAS]", "")
402             .replace("[NESSUS]", "")
403             .replace("[DEBUG]", "")
404             .replace("[SYSTEM]", "")
405             .strip()
406         )
407
408         formatted = f"[{timestamp}] {tag:<8} - {clean_msg}"
409
410         self.log_text.configure(state="normal")

```

```

dashboard.py
35 class ThreatQuillDashboard:
380     def append_log(self, message):
411         self.log_text.insert("end", formatted + "\n")
412         self.log_text.see("end")
413         self.log_text.configure(state="disabled")
414
415     def add_result(self, scanner, host, port, proto, state, service, severity):
416         self.tree.insert(
417             "",
418             "end",
419             values=(scanner, host, port, proto, state, service, severity),
420         )
421
422     def clear_results(self):
423         for item in self.tree.get_children():
424             self.tree.delete(item)
425
426         self.log_text.configure(state="normal")
427         self.log_text.delete("1.0", "end")
428         self.log_text.configure(state="disabled")
429
430         self.latest_report_data = None
431         self.latest_html_path = None
432
433         self.update_summary(0, 0, 0, "Idle")
434
435     def update_summary(self, hosts=None, threats=None, findings=None, status=None):
436         if hosts is not None:
437             self.hosts_value.config(text=str(hosts))
438         if threats is not None:
439             self.threats_value.config(text=str(threats))
440         if findings is not None:
441             self.findings_value.config(text=str(findings))
442         if status is not None:
443             self.status_value.config(text=status)
444
445     def show_report(self, report_data, html_path):
446         self.latest_report_data = report_data
447         self.latest_html_path = html_path
448
449         for item in self.tree.get_children():
450             self.tree.delete(item)
451
452         findings = report_data.get("findings", [])
453
454         if not findings:
455             self.append_log("[ERROR] Report generated but no findings were available.")
456             self.update_summary(
457                 hosts=report_data.get("total_hosts", 0),
458                 threats=0,
459                 findings=0,
460                 status="No Report Data"
461             )
462             return
463
464         for finding in findings:
465             host_value = finding.get("target", finding.get("host", ""))
466             port_value = str(finding.get("port", ""))
467             proto_value = finding.get("protocol", "")
468             state_value = finding.get("state", "")
469             service_value = finding.get("service", "")
470             severity_value = finding.get("severity", finding.get("risk_level", "Info"))
471
472             self.tree.insert(
473                 "",
474                 "end",
475                 values=(
476                     finding.get("scanner", ""),
477                     host_value,
478                     port_value,
479                     proto_value,
480                     state_value,
481                     service_value,
482                     severity_value,
483                 ),
484             )
485
486         counts = report_data.get("severity_summary", {})
487         critical = counts.get("Critical", 0)

```

```

dashboard.py
35 class ThreatQuillDashboard:
445 def show_report(self, report_data, html_path):
488     high = counts.get("High", 0)
489     total_threats = critical + high
490
491     self.update_summary(
492         hosts=report_data.get("total_hosts", 1),
493         threats=total_threats,
494         findings=report_data.get("total_findings", len(findings)),
495         status="Report Ready"
496     )
497
498     self.append_log(f"[SYSTEM] Report generated: {html_path}")
499
500 def start_scan_clicked(self):
501     target = self.target_entry.get().strip()
502     if not target:
503         messagebox.showwarning("Missing Target", "Please enter a target IP or domain.")
504         return
505
506     if not is_valid_target(target):
507         messagebox.showwarning("Invalid Target", "Please enter a valid IP address or domain")
508         return
509
510     self.clear_results()
511     self.append_log(f"[SYSTEM] Live scan requested for target: {target}")
512     self.update_summary(0, 0, 0, "Running")
513
514     self.start_scan_btn.config(state="disabled", text="Scanning...")
515     start_full_scan(target, self)
516
517 def scan_finished(self):
518     self.start_scan_btn.config(state="normal", text="Start Scan")
519     self.append_log("[SYSTEM] Scan process finished.")
520
521 def open_reports_view(self):
522     if self.latest_report_data:
523         self.show_report(self.latest_report_data, self.latest_html_path or "")
524         self.append_log("[SYSTEM] Report loaded in dashboard.")
525
526         if self.latest_html_path and os.path.exists(self.latest_html_path):
527             try:
528                 webbrowser.open("file://" + os.path.abspath(self.latest_html_path))
529             except Exception as e:
530                 self.append_log(f"[ERROR] Could not open report automatically: {e}")
531             return
532
533     report_file = os.path.join("reports", "report.html")
534     full_path = os.path.abspath(report_file)
535
536     if not os.path.exists(full_path):
537         messagebox.showwarning("No Report", f"Report file not found:\n{full_path}")
538         return
539
540     try:
541         os.startfile(full_path)
542         self.append_log(f"[SYSTEM] Opened report: {full_path}")
543     except Exception as e:
544         messagebox.showerror("Error", f"Unable to open report.\n\n{e}")
545
546 def focus_summary(self):
547     self.append_log("[SYSTEM] Summary panel is visible on dashboard.")
548
549 def show_help(self):
550     messagebox.showinfo(
551         "Help",
552         "Enter a target and click Start Scan to run live scanners and generate the report."
553     )
554
555 def show_settings(self):
556     messagebox.showinfo("Settings", "Settings panel can be added next.")
557
558 def show_email_message(self):
559     win = tk.Toplevel(self.root)
560     win.title("Send Final Report")
561     win.geometry("450x280")
562     win.configure(bg="white")
563
564     tk.Label(

```

```

dashboard.py X
dashboard.py
35 class ThreatQuillDashboard:
558     def show_email_message(self):
563
564         tk.Label(
565             win,
566             text="Send Final Report",
567             bg="white",
568             fg="#111827",
569             font=("Segoe UI", 16, "bold")
570         ).pack(pady=12)
571
572         form = tk.Frame(win, bg="white")
573         form.pack(fill="both", expand=True, padx=20, pady=10)
574
575         tk.Label(form, text="Your Gmail:", bg="white").grid(row=0, column=0, sticky="w", pady=8)
576         sender_entry = tk.Entry(form, width=35)
577         sender_entry.grid(row=0, column=1, pady=8)
578         sender_entry.insert(0, "binnishrestha99@gmail.com")
579         sender_entry.config(state="readonly")
580
581         tk.Label(form, text="App Password:", bg="white").grid(row=1, column=0, sticky="w", pady=8)
582         password_entry = tk.Entry(form, width=35, show="*")
583         password_entry.grid(row=1, column=1, pady=8)
584
585         tk.Label(form, text="Recipient Email:", bg="white").grid(row=2, column=0, sticky="w", pady=8)
586         recipient_entry = tk.Entry(form, width=35)
587         recipient_entry.grid(row=2, column=1, pady=8)
588
589         def send_now():
590             sender = "binnishrestha99@gmail.com"
591             password = password_entry.get().strip()
592             recipient = recipient_entry.get().strip()
593
594             if not sender or not password or not recipient:
595                 messagebox.showwarning("Missing Data", "Please fill all fields.")
596                 return
597
598             messagebox.showinfo(
599                 "Email Integration",
600                 "Email sending code is ready to connect.\nUncomment send_report_email part after email_sender.py is added."
601             )
602
603         tk.Button(
604             win,
605             text="Send Report",
606             bg="#2563EB",
607             fg="white",
608             relief="flat",
609             padx=18,
610             pady=8,
611             font=("Segoe UI", 10, "bold"),
612             command=send_now
613         ).pack(pady=12)
614
615         def show_scan_summary(self):
616             summary = ScanSummaryWindow(self.root)
617             summary.open()
618
619
620     def open_dashboard(username="binni"):
621         root = tk.Tk()
622         app = ThreatQuillDashboard(root, username=username)
623         root.mainloop()
624
625
626 if __name__ == "__main__":
627     open_dashboard()

```

Figure 117: Dashboard module developed to display scan results

Report_view.py

```

report_view.py
1  import tkinter as tk
2  from tkinter import ttk
3  from collections import Counter
4
5  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
6  from matplotlib.figure import Figure
7
8  from database import fetch_all_results
9
10
11 def open_report_window():
12     rows = fetch_all_results()
13
14     win = tk.Toplevel()
15     win.title("ThreatQuill Report")
16     win.state("zoomed")
17     win.configure(bg="#F3F4F6")
18
19     # ----- Top bar -----
20     top = tk.Frame(win, bg="#071633", height=70)
21     top.pack(fill="x")
22     top.pack_propagate(False)
23
24     tk.Label(
25         top,
26         text="ThreatQuill Analyzer - Report View",
27         bg="#071633",
28         fg="white",
29         font=("Segoe UI", 22, "bold"),
30     ).pack(side="left", padx=20, pady=15)
31
32     # ----- Main body -----
33     body = tk.Frame(win, bg="#F3F4F6")
34     body.pack(fill="both", expand=True, padx=12, pady=12)
35
36     body.columnconfigure(0, weight=3)
37     body.columnconfigure(1, weight=1)
38     body.rowconfigure(0, weight=1)
39
40     # ----- Left side: findings table -----
41     left = tk.LabelFrame(
42         body,
43         text="Detailed Findings",
44         bg="#F3F4F6",
45         fg="#111827",
46         font=("Segoe UI", 12, "bold"),
47         padx=8,
48         pady=8,
49     )
50     left.grid(row=0, column=0, sticky="nsew", padx=(0, 8))
51
52     left.rowconfigure(0, weight=1)
53     left.columnconfigure(0, weight=1)
54
55     columns = ("scanner", "host", "port", "proto", "state", "service", "severity")
56     tree = ttk.Treeview(left, columns=columns, show="headings")
57
58     headings = {
59         "scanner": "Scanner",
60         "host": "Host/IP",
61         "port": "Port",
62         "proto": "Proto",
63         "state": "State",
64         "service": "Service",
65         "severity": "Severity",

```

```

report_view.py
11 def open_report_window():
58     headings = {
66     }
67
68     widths = {
69         "scanner": 110,
70         "host": 170,
71         "port": 70,
72         "proto": 70,
73         "state": 90,
74         "service": 200,
75         "severity": 100,
76     }
77
78     for col in columns:
79         tree.heading(col, text=headings[col])
80         tree.column(col, width=widths[col], anchor="center")
81
82     tree.grid(row=0, column=0, sticky="nsew")
83
84     scroll = ttk.Scrollbar(left, orient="vertical", command=tree.yview)
85     tree.configure(yscrollcommand=scroll.set)
86     scroll.grid(row=0, column=1, sticky="ns")
87
88     # ----- Right side: scan details + chart -----
89     right = tk.Frame(body, bg="#F3F4F6")
90     right.grid(row=0, column=1, sticky="nsew")
91     right.columnconfigure(0, weight=1)
92
93     # Prepare summary data
94     total_findings = len(rows)
95     unique_hosts = len(set(r[1] for r in rows)) if rows else 0
96
97     sev_counter = Counter()
98     scanner_counter = Counter()
99
100    for row in rows:
101        scanner, host, port, proto, state, service, severity = row
102        sev_counter[str(severity).strip().title()] += 1
103        scanner_counter[str(scanner).strip().upper()] += 1
104
105    high_count = sev_counter.get("High", 0)
106    medium_count = sev_counter.get("Medium", 0)
107    low_count = sev_counter.get("Low", 0)
108
109    status = "Completed" if total_findings > 0 else "No Data"
110
111    # Scan details panel
112    details = tk.LabelFrame(
113        right,
114        text="Scan Details",
115        bg="white",
116        fg="#111827",
117        font=("Segoe UI", 12, "bold"),
118        padx=12,
119        pady=12,
120    )
121    details.grid(row=0, column=0, sticky="ew", pady=(0, 10))
122
123    detail_items = [
124        ("Status", status),
125        ("Total Hosts", unique_hosts),
126        ("Total Findings", total_findings),
127        ("High", high_count),
128        ("Medium", medium_count),

```

```

report_view.py
11  def open_report_window():
123     detail_items = [
129         ("Low", low_count),
130     ]
131
132     for label, value in detail_items:
133         rowf = tk.Frame(details, bg="white")
134         rowf.pack(fill="x", pady=4)
135
136         tk.Label(
137             rowf,
138             text=f"{label}:",
139             bg="white",
140             fg="#374151",
141             font=("Segoe UI", 10, "bold"),
142             width=14,
143             anchor="w",
144         ).pack(side="left")
145
146         tk.Label(
147             rowf,
148             text=str(value),
149             bg="white",
150             fg="#111827",
151             font=("Segoe UI", 10),
152             anchor="w",
153         ).pack(side="left")
154
155     # Severity chart panel
156     chart_box = tk.LabelFrame(
157         right,
158         text="Severity Summary",
159         bg="white",
160         fg="#111827",
161         font=("Segoe UI", 12, "bold"),
162         padx=10,
163         pady=10,
164     )
165     chart_box.grid(row=1, column=0, sticky="nsew", pady=(0, 10))
166
167     fig = Figure(figsize=(4, 3), dpi=100)
168     ax = fig.add_subplot(111)
169
170     labels = []
171     sizes = []
172
173     if high_count > 0:
174         labels.append("High")
175         sizes.append(high_count)
176     if medium_count > 0:
177         labels.append("Medium")
178         sizes.append(medium_count)
179     if low_count > 0:
180         labels.append("Low")
181         sizes.append(low_count)
182
183     if sizes:
184         wedges, texts = ax.pie(sizes, startangle=90, wedgeprops=dict(width=0.45))
185         ax.set_title("Findings by Severity", fontsize=10)
186     else:
187         ax.text(0.5, 0.5, "No Data", ha="center", va="center")
188         ax.set_title("Findings by Severity", fontsize=10)
189
190     ax.axis("equal")
191

```

```

report_view.py
11 def open_report_window():
191
192     canvas = FigureCanvasTkAgg(fig, master=chart_box)
193     canvas.draw()
194     canvas.get_tk_widget().pack(fill="both", expand=True)
195
196     legend_box = tk.Frame(chart_box, bg="white")
197     legend_box.pack(fill="x", pady=(8, 0))
198
199     for name, count in [("High", high_count), ("Medium", medium_count), ("Low", low_count)]:
200         tk.Label(
201             legend_box,
202             text=f"{name}: {count}",
203             bg="white",
204             fg="#111827",
205             font=("Segoe UI", 10),
206             anchor="w",
207         ).pack(anchor="w")
208
209     # Scanner summary panel
210     scanner_box = tk.LabelFrame(
211         right,
212         text="Scanner Summary",
213         bg="white",
214         fg="#111827",
215         font=("Segoe UI", 12, "bold"),
216         padx=12,
217         pady=12,
218     )
219     scanner_box.grid(row=2, column=0, sticky="ew")
220
221     if scanner_counter:
222         for scanner_name, count in scanner_counter.items():
223             tk.Label(
224                 scanner_box,
225                 text=f"{scanner_name}: {count} findings",
226                 bg="white",
227                 fg="#111827",
228                 font=("Segoe UI", 10),
229                 anchor="w",
230             ).pack(anchor="w", pady=3)
231     else:
232         tk.Label(
233             scanner_box,
234             text="No scanner data available.",
235             bg="white",
236             fg="#687280",
237             font=("Segoe UI", 10),
238         ).pack(anchor="w")
239
240     # Insert rows into table
241     for row in rows:
242         tree.insert("", "end", values=row)
243

```

Figure 118: Report view module implemented to display detailed vulnerability reports

Summary_view.py

```
summary_view.py
1  import tkinter as tk
2  from tkinter import messagebox
3  import json
4  import os
5
6
7  class ScanSummaryWindow:
8      def __init__(self, parent):
9          self.parent = parent
10
11      def open(self):
12          report_file = os.path.join("reports", "report.json")
13
14          if not os.path.exists(report_file):
15              messagebox.showwarning("No Data", "Run a scan first.")
16              return
17
18          try:
19              with open(report_file, "r", encoding="utf-8") as f:
20                  data = json.load(f)
21          except Exception as e:
22              messagebox.showerror("Error", f"Unable to load scan summary.\n\n{e}")
23              return
24
25          summary = data.get("severity_summary", {})
26          target = data.get("target", "N/A")
27          scan_date = data.get("scan_date", "N/A")
28          tools_used = ", ".join(data.get("tools_used", [])) or "N/A"
29          total_hosts = data.get("total_hosts", 0)
30          total_findings = data.get("total_findings", 0)
31
32          critical = summary.get("Critical", 0)
33          high = summary.get("High", 0)
34          medium = summary.get("Medium", 0)
35          low = summary.get("Low", 0)
36          info = summary.get("Info", 0)
37
38          total = critical + high + medium + low + info
39          if total == 0:
40              total = 1
41
42          win = tk.Toplevel(self.parent)
43          win.title("ThreatQuill Analyzer - Scan Summary")
44          win.geometry("980x680")
```

```

summary_view.py
7   class ScanSummaryWindow:
11  def open(self):
12      win.geometry('800x600')
45      win.configure(bg="#F3F4F6")
46
47      # Header
48      header = tk.Frame(win, bg="#071633", height=90)
49      header.pack(fill="x")
50      header.pack_propagate(False)
51
52      tk.Label(
53          header,
54          text="ThreatQuill Analyzer",
55          bg="#071633",
56          fg="white",
57          font=("Segoe UI", 24, "bold")
58      ).pack(side="left", padx=24, pady=22)
59
60      # Main container
61      main = tk.Frame(win, bg="#F3F4F6")
62      main.pack(fill="both", expand=True, padx=18, pady=18)
63
64      # Title block
65      tk.Label(
66          main,
67          text="Scan Summary",
68          bg="#F3F4F6",
69          fg="#111827",
70          font=("Segoe UI", 20, "bold")
71      ).pack(anchor="w", pady=(0, 12))
72
73      # Details card
74      details_card = tk.Frame(
75          main,
76          bg="white",
77          highlightthickness=1,
78          highlightbackground="#D1D5DB"
79      )
80      details_card.pack(fill="x", pady=(0, 14))
81
82      info_left = tk.Frame(details_card, bg="white")
83      info_left.pack(side="left", fill="both", expand=True, padx=18, pady=14)
84
85      info_right = tk.Frame(details_card, bg="white")
86      info_right.pack(side="right", fill="both", expand=True, padx=18, pady=14)
87
88      def add_info(parent, label, value):
89          row = tk.Frame(parent, bg="white")
90          row.pack(anchor="w", pady=4, fill="x")
91
92          tk.Label(
93              row,
94              text=label,
95              bg="white",

```

```

summary_view.py
7   class ScanSummaryWindow:
11       def open(self):
88           def add_info(parent, label, value):
92               tk.Label(
96                   fg="#687280",
97                   font=("Segoe UI", 10, "bold"),
98                   width=14,
99                   anchor="w"
100              ).pack(side="left")
101
102              tk.Label(
103                  row,
104                  text=value,
105                  bg="white",
106                  fg="#111827",
107                  font=("Segoe UI", 10),
108                  anchor="w"
109              ).pack(side="left")
110
111              add_info(info_left, "Target:", target)
112              add_info(info_left, "Scan Date:", scan_date)
113              add_info(info_left, "Tools Used:", tools_used)
114
115              add_info(info_right, "Total Hosts:", str(total_hosts))
116              add_info(info_right, "Total Findings:", str(total_findings))
117              add_info(info_right, "Status:", "Completed")
118
119              # Severity summary cards
120              sev_row = tk.Frame(main, bg="#F3F4F6")
121              sev_row.pack(fill="x", pady=(0, 14))
122
123              def severity_card(parent, title, value, color):
124                  card = tk.Frame(
125                      parent,
126                      bg="white",
127                      highlightthickness=1,
128                      highlightbackground="#D1D5DB"
129                  )
130                  card.pack(side="left", fill="both", expand=True, padx=5)
131
132                  tk.Label(
133                      card,
134                      text=title,
135                      bg="white",
136                      fg="#687280",
137                      font=("Segoe UI", 10, "bold")
138                  ).pack(pady=(12, 4))
139
140                  tk.Label(
141                      card,
142                      text=str(value),
143                      bg="white",
144                      fg=color,
145                      font=("Segoe UI", 20, "bold")
146                  ).pack(pady=(0, 12))
147
148              severity_card(sev_row, "Critical", critical, "#8d1b3d")
149              severity_card(sev_row, "High", high, "#e94b4b")
150              severity_card(sev_row, "Medium", medium, "#f2993a")
151              severity_card(sev_row, "Low", low, "#c59b00")
152              severity_card(sev_row, "Info", info, "#2563E8")
153
154              # Bottom layout
155              bottom = tk.Frame(main, bg="#F3F4F6")
156              bottom.pack(fill="both", expand=True)

```

```

summary_view.py
7 class ScanSummaryWindow:
11     def open(self):

157
158         chart_card = tk.Frame(
159             bottom,
160             bg="white",
161             highlightthickness=1,
162             highlightbackground="#D1D5DB"
163         )
164         chart_card.pack(side="left", fill="both", expand=True, padx=(0, 8))
165
166         details_panel = tk.Frame(
167             bottom,
168             bg="white",
169             highlightthickness=1,
170             highlightbackground="#D1D5DB"
171         )
172         details_panel.pack(side="right", fill="both", expand=True, padx=(8, 0))
173
174         tk.Label(
175             chart_card,
176             text="Severity Distribution",
177             bg="white",
178             fg="#111827",
179             font=("Segoe UI", 14, "bold")
180         ).pack(anchor="w", padx=18, pady=(16, 8))
181
182         canvas = tk.Canvas(chart_card, width=320, height=260, bg="white", highlightthickness=0)
183         canvas.pack(pady=10)
184
185         # Pie chart
186         cx, cy, r = 140, 130, 90
187         start = 0
188         slices = [
189             ("Critical", critical, "#8d1b3d"),
190             ("High", high, "#e94b4b"),
191             ("Medium", medium, "#f2993a"),
192             ("Low", low, "#f2c94c"),
193             ("Info", info, "#5aa9e6"),
194         ]
195
196         for _, value, color in slices:
197             if value <= 0:
198                 continue
199             extent = (value / total) * 360
200             canvas.create_arc(
201                 cx - r, cy - r, cx + r, cy + r,
202                 start=start,
203                 extent=extent,
204                 fill=color,
205                 outline="white",
206                 width=2
207             )
208             start += extent
209
210         # inner circle
211         canvas.create_oval(cx - 42, cy - 42, cx + 42, cy + 42, fill="white", outline="white")
212         canvas.create_text(cx, cy - 8, text="Total", font=("Segoe UI", 11, "bold"), fill="#485563")
213         canvas.create_text(cx, cy + 14, text=str(total_findings), font=("Segoe UI", 16, "bold"), fill="#111827")
214
215         # legend
216         legend_frame = tk.Frame(chart_card, bg="white")
217         legend_frame.pack(anchor="w", padx=18, pady=(4, 16))
218
219         def legend_item(parent, label, value, color):

```



```

summary_view.py
7 class ScanSummaryWindow:
11     def open(self):
219         def legend_item(parent, label, value, color):
220             row = tk.Frame(parent, bg="white")
221             row.pack(anchor="w", pady=4)
222
223             dot = tk.Canvas(row, width=14, height=14, bg="white", highlightthickness=0)
224             dot.pack(side="left")
225             dot.create_oval(2, 2, 12, 12, fill=color, outline=color)
226
227             tk.Label(
228                 row,
229                 text=f"{label}: {value}",
230                 bg="white",
231                 fg="#111827",
232                 font=("Segoe UI", 10)
233             ).pack(side="left", padx=8)
234
235             legend_item(legend_frame, "Critical", critical, "#8d1b3d")
236             legend_item(legend_frame, "High", high, "#e94b4b")
237             legend_item(legend_frame, "Medium", medium, "#f2993a")
238             legend_item(legend_frame, "Low", low, "#f2c94c")
239             legend_item(legend_frame, "Info", info, "#5aa9e6")
240
241         # right side details
242         tk.Label(
243             details_panel,
244             text="Assessment Details",
245             bg="white",
246             fg="#111827",
247             font=("Segoe UI", 14, "bold")
248         ).pack(anchor="w", padx=18, pady=(16, 12))
249
250         add_box = tk.Frame(details_panel, bg="white")
251         add_box.pack(fill="x", padx=18)
252
253         def detail_line(parent, label, value):
254             row = tk.Frame(parent, bg="white")
255             row.pack(fill="x", pady=6)
256
257             tk.Label(
258                 row,
259                 text=label,
260                 bg="white",
261                 fg="#6B7280",
262                 font=("Segoe UI", 10, "bold"),
263                 width=14,
264                 anchor="w"
265             ).pack(side="left")
266
267             tk.Label(
268                 row,
269                 text=value,
270                 bg="white",
271                 fg="#111827",
272                 font=("Segoe UI", 10),
273                 anchor="w",
274                 justify="left",
275                 wraplength=280
276             ).pack(side="left")
277
278             detail_line(add_box, "Target:", target)
279             detail_line(add_box, "Status:", "Completed")
280             detail_line(add_box, "Modified:", scan_date)
281             detail_line(add_box, "Method:", "Multi-scanner vulnerability assessment")

```

```

summary_view.py
7  class ScanSummaryWindow:
11      def open(self):
253          def detail_line(parent, label, value):
277
278              detail_line(add_box, "Target:", target)
279              detail_line(add_box, "Status:", "Completed")
280              detail_line(add_box, "Modified:", scan_date)
281              detail_line(add_box, "Method:", "Multi-scanner vulnerability assessment")
282              detail_line(add_box, "Threat Level:", str(critical + high))
283
284              notice = tk.Frame(
285                  details_panel,
286                  bg="#F9F2D8",
287                  highlightthickness=1,
288                  highlightbackground="#E5D9A8"
289              )
290              notice.pack(fill="x", padx=18, pady=(18, 0))
291
292              tk.Label(
293                  notice,
294                  text="Notice: This window shows a quick scan summary. Use the Reports section for the full detailed report.",
295                  bg="#F9F2D8",
296                  fg="#374151",
297                  font=("Segoe UI", 10),
298                  wraplength=300,
299                  justify="left"
300              ).pack(anchor="w", padx=12, pady=12)
301

```

Figure 119: Summary view module implemented to display an overview of scan results

Main.py

```

dashboard.py  main.py  X
main.py
1  import dashboard
2
3  if __name__ == "__main__":
4      dashboard.main()

```

Figure 120: Main module implemented to initialize and run the ThreatQuill Analyzer system

3.6.4 Sustainability

ThreatQuill Analyzer is designed based on long-term usability and demand on the market in the sphere of cybersecurity. With the increasing number of cyber threats, organizations require efficient and affordable vulnerability assessment tools (Stallings, 2018). The system offers an economic approach in that it combines various open-source scanners in a single platform eliminating the need to buy costly commercial tools and lessening manual work.

The system is created in a modular and scalable manner, which will enable its future upgrade through the addition of new scanners and better analysis possibilities. Automated scanning and report generation help in saving time and improving efficiency for users (Behl & Behl, 2017). The system is also conducive to the environment by using less paper since reports are done online. In general, the project is sustainable because it is flexible, sustainable and applicable in the real world.

Chapter 4: Testing and Analysis

4.1 Test Plan

4.1.1 Unit Testing Test Plan

Test Case	Objective
Test Case 1	To test whether the system validates empty target input correctly before starting a scan.
Test Case 2	To test whether the system accepts a valid IP address or domain input correctly.
Test Case 3	To test whether the system rejects invalid target input formats correctly.
Test Case 4	To test whether the SSH connection function establishes a successful connection to the Kali Linux machine.
Test Case 5	To test whether the system handles SSH connection failure correctly when the target machine is unreachable.
Test Case 6	To test whether the Nmap result parser extracts host, port, protocol, state, and service correctly from scan output.
Test Case 7	To test whether the risk prioritization function assigns the correct severity level to detected findings.
Test Case 8	To test whether the “Clear Results” button resets the dashboard table, summary values, and activity log correctly.
Test Case 9	To test whether the summary calculation function correctly counts total findings and threats from given results.
Test Case 10	To test whether the report generation module creates the report file correctly after processing scan results.

Table 2: Unit Test Plan

4.1.2 System Testing Test Plan

Test Case	Objective
Test Case 1	To test whether the system performs a complete scan successfully after entering a valid target.
Test Case 2	To test whether the activity log displays real-time scan progress correctly during scan execution.
Test Case 3	To test whether the scan summary section updates correctly after the scan is completed.
Test Case 4	To test whether scan results are displayed correctly in the results table after scan execution.
Test Case 5	To test whether multiple scanners execute together and provide combined results in the dashboard.
Test Case 6	To test whether the system successfully generates and opens the final report after scan execution.

Table 3: System Test Plan

4.1.3 Security Testing Test Plan

Test Case	Objective
Test Case 1	To test whether the system prevents unauthorized access with invalid login credentials.
Test Case 2	To test whether sensitive data such as passwords are not displayed in plain text and password reset is handled securely.
Test Case 3	To test whether the system restricts access to the dashboard without proper authentication.
Test Case 4	To test whether the system handles invalid or malicious input safely without crashing.
Test Case 5	To test whether secure communication is maintained when connecting to external scanners (e.g, via SSH/HTTPS).

Table 4: Security Test Plan

4.2 Unit Testing

Unit testing is performed to check individual components of the system to ensure each function works correctly. In ThreatQuill Analyzer, different modules were tested separately to verify proper functionality. Detailed test cases are given below.

4.2.1 Test Case 1

Test Case 1	
Objective	To test whether the system validates the target input correctly before starting a scan.
Action	The Start Scan button was clicked without entering any target in the input field.
Expected Result	The system should display a warning message and should not start the scan.
Actual Result	The system displayed a warning message stating “Please enter a target IP or domain” and the scan did not start.
Conclusion	Test successful.

Table 5: Unit Testing Test Case 1

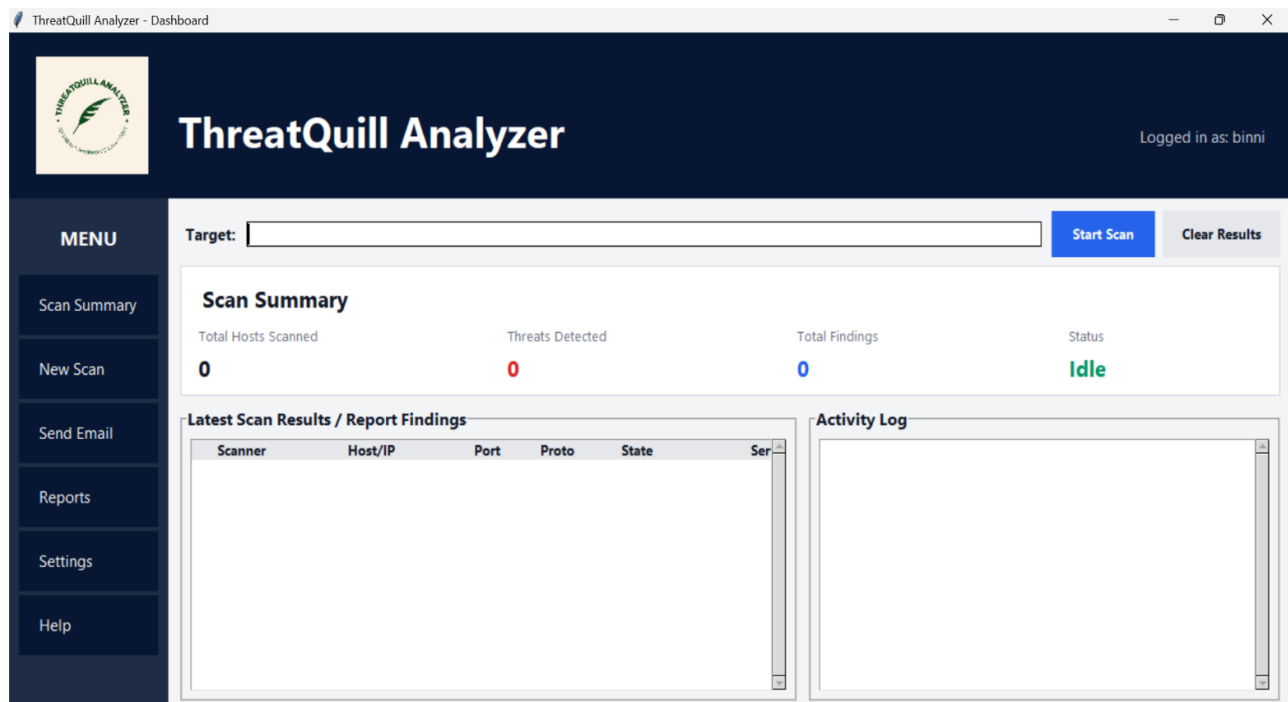


Figure 121: Dashboard with empty target

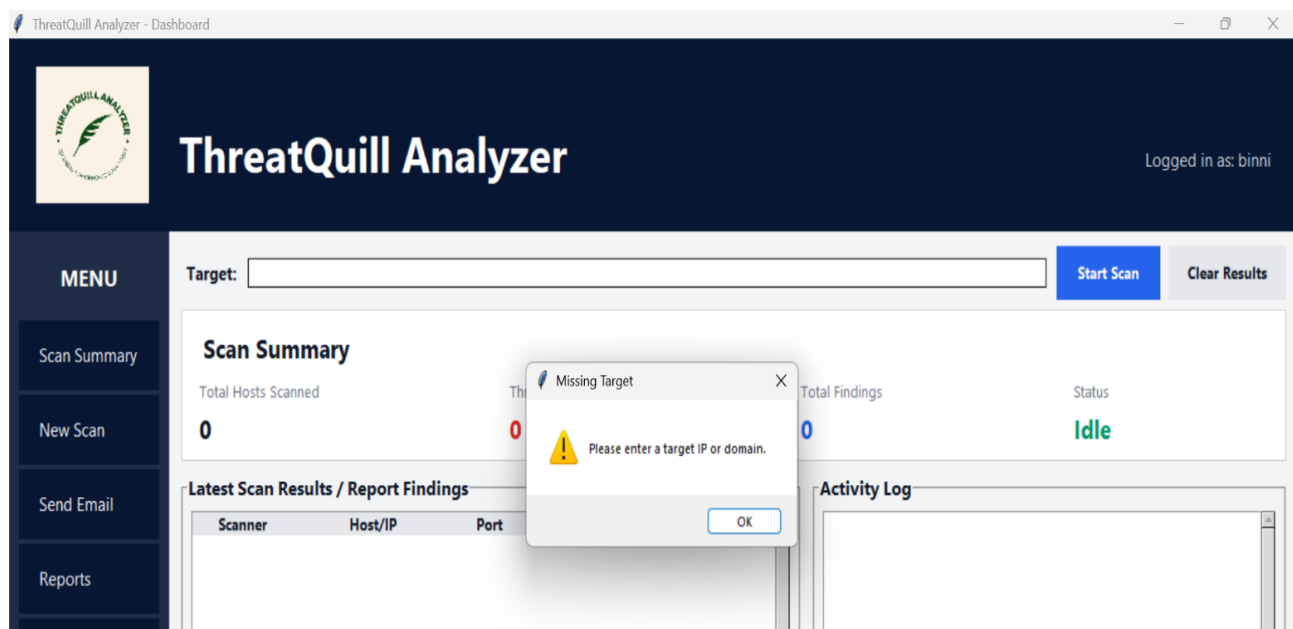


Figure 122: Message shown to enter a target

4.2.2 Test Case 2

Test Case 2	
Objective	To test whether the system accepts a valid IP address or domain input correctly.
Action	A valid target such as “127.0.0.1” or “scanme.nmap.org” was entered in the input field.
Expected Result	The system should accept the input without displaying any validation error.
Actual Result	The system accepted the valid target input successfully without displaying any error message.
Conclusion	Test successful.

Table 6: Unit Testing Test Case 2

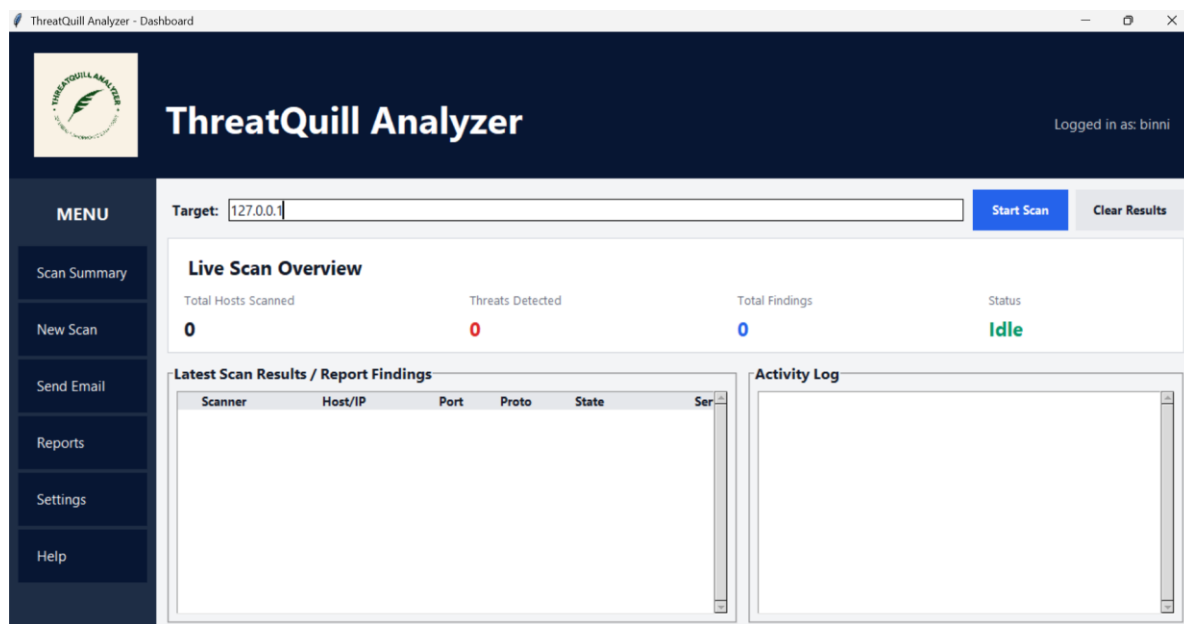


Figure 123: Dashboard in idle state before scan

4.2.3 Test Case 3

Test Case 3	
Objective	To test whether the system rejects invalid target input formats correctly.
Action	An invalid target such as “abc” was entered in the input field and the Start Scan button was clicked.
Expected Result	The system should display a validation error message and should not start the scan.
Actual Result	The system displayed an error message stating “Please enter a valid IP address or domain” and the scan did not start.
Conclusion	Test successful.

Table 7: Unit Testing Test Case 3

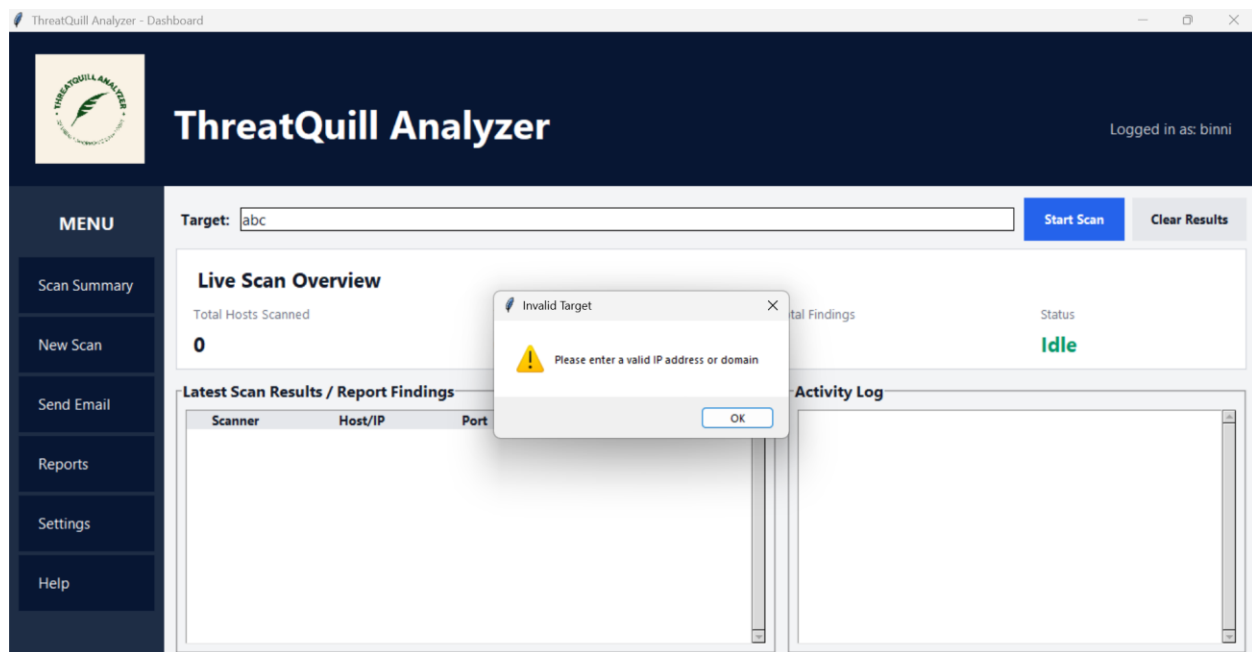


Figure 124: Invalid target entered

4.2.4 Test Case 4

Test Case 4	
Objective	To test whether the SSH connection function successfully connects to the Kali Linux machine.
Action	The SSH connection function was executed using valid credentials and target IP address of the Kali Linux machine.
Expected Result	The system should establish a successful SSH connection and return a valid response from the Kali machine.
Actual Result	The system successfully established the SSH connection and returned the expected response from the Kali Linux machine.
Conclusion	Test successful.

Table 8: Unit Testing Test Case 4

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\ThreatQuill> ssh kali@192.168.153.143
Linux kali 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Apr 12 04:36:22 2026 from 192.168.153.1
(Message from Kali developers)
This is a minimal installation of Kali Linux, you likely
want to install supplementary tools. Learn how:
→ https://www.kali.org/docs/troubleshooting/common-minimum-setup/
(Run: "touch ~/.hushlogin" to hide this message)
(kali@kali)-[~]
$ whoami
kali
(kali@kali)-[~]
$

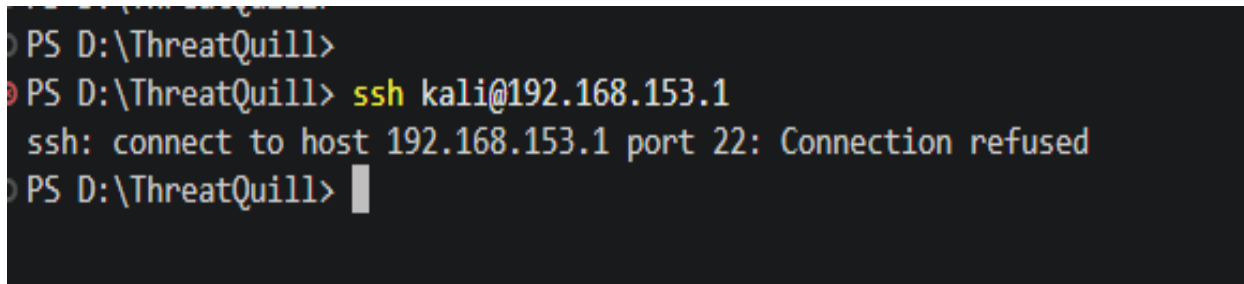
```

Figure 125: SSH connection established successfully

4.2.5 Test Case 5

Test Case 5	
Objective	To test whether the system handles SSH connection failure correctly when the target machine is unreachable or credentials are incorrect.
Action	An incorrect IP address or wrong credentials were used to attempt an SSH connection to the Kali Linux machine.
Expected Result	The system should fail to establish the SSH connection and display an appropriate error message without crashing.
Actual Result	The system failed to establish the SSH connection and displayed an error message indicating connection failure. The system remained stable and did not crash.
Conclusion	Test successful.

Table 9: Unit Testing Test Case 5



```

PS D:\ThreatQuill>
PS D:\ThreatQuill> ssh kali@192.168.153.1
ssh: connect to host 192.168.153.1 port 22: Connection refused
PS D:\ThreatQuill>
  
```

Figure 126: SSH connection failed due to connection error

4.2.6 Test Case 6

Test Case 6	
Objective	To test whether the Nmap result parser correctly extracts host, port, protocol, state, and service from scan output.
Action	A sample Nmap output line (80/tcp open http) was passed to the parser function to extract structured scan details.
Expected Result	The parser should correctly extract host, port, protocol, state, and service values from the raw Nmap output.
Actual Result	The parser successfully extracted the host, port, protocol, state, and service values from the given Nmap output line and returned structured data.
Conclusion	Test successful.

Table 10: Unit Testing Test Case 6

```

PS D:\ThreatQuill>
PS D:\ThreatQuill> python test_nmap.py
Input Line: 80/tcp open http
Parsed Result: ('port_result', {'scanner': 'NMAP', 'host': '127.0.0.1', 'port': '80', 'p
rotocol': 'tcp', 'state': 'open', 'service': 'http', 'risk_level': 'Medium', 'finding':
'Port 80/tcp is open (http)'})
PS D:\ThreatQuill>
PS D:\ThreatQuill>
PS D:\ThreatQuill>

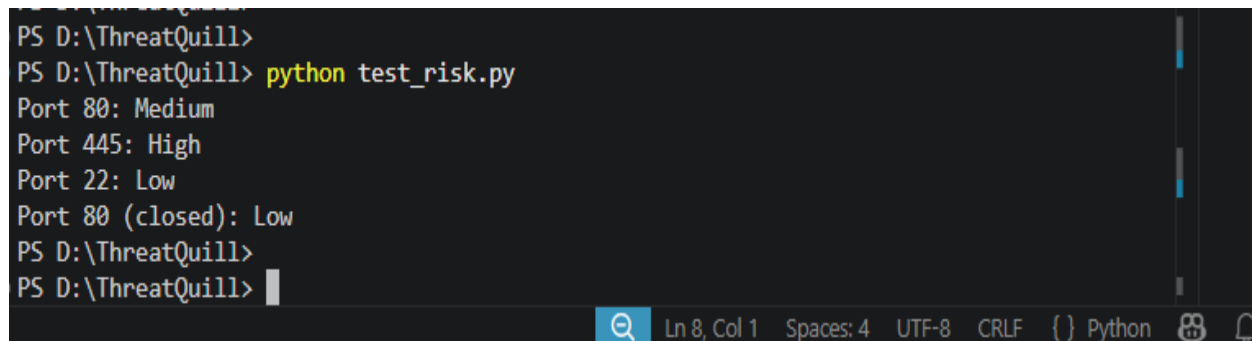
```

Figure 127: Nmap output parsed successfully

4.2.7 Test Case 7

Test Case 7	
Objective	To test whether the system correctly assigns risk levels (High, Medium, Low) based on detected ports and their states.
Action	The risk prioritization function was tested by providing sample inputs such as open ports (eg, port 80, port 445) and observing the assigned risk levels.
Expected Result	The system should assign appropriate risk levels such as High, Medium, or Low based on predefined conditions for each port and state.
Actual Result	The system correctly assigned risk levels such as Medium for port 80 and High for port 445 based on the defined rules.
Conclusion	Test successful.

Table 11: Unit Testing Test Case 7



```

PS D:\ThreatQuill>
PS D:\ThreatQuill> python test_risk.py
Port 80: Medium
Port 445: High
Port 22: Low
Port 80 (closed): Low
PS D:\ThreatQuill>
PS D:\ThreatQuill>

```

Figure 128: Risk levels calculated for scanned ports

4.2.8 Test Case 8

Test Case 8	
Objective	To test whether the “Clear Results” button resets the dashboard data correctly.
Action	Clicked on the “Clear Results” button after viewing the dashboard.
Expected Result	The system should clear scan results, reset summary values to zero, and clear activity logs.
Actual Result	The system successfully cleared all results, reset summary values, and cleared the activity log section.
Conclusion	Test successful.

Table 12: Unit Testing Test Case 8

ThreatQuill Analyzer - Dashboard

Target: Start Scan Clear Results

Scan Summary

Total Hosts Scanned	Threats Detected	Total Findings	Status
1	14	14	Complete

Latest Scan Results

Scanner	Host/IP	Port	Proto	State	Severity
NIKTO	http://scanme.nmap.org	-	-	issue	Server: Apache
NIKTO	http://scanme.nmap.org	-	-	issue	Multiple IPs fo
NIKTO	http://scanme.nmap.org	-	-	issue	No CGI Directo
NIKTO	http://scanme.nmap.org	-	-	issue	ERROR: *** Err
NIKTO	http://scanme.nmap.org	-	-	issue	ERROR: *** Co
NIKTO	http://scanme.nmap.org	-	-	issue	Scan terminate
NIKTO	http://scanme.nmap.org	-	-	issue	End Time:
NIKTO	http://scanme.nmap.org	-	-	issue	1 host(s)

Activity Log

```
[21:31:01] Live scan requested for target: scanme.nmap.org
[21:31:01] [Nessus] Logging into Nessus...
[21:31:01] Starting Nmap scan on scanme.nmap.org
[21:31:01] Starting Nikto scan on http://scanme.nmap.org
[21:31:02] Starting Nmap 7.98 ( https://nmap.org ) at
2026-04-04 11:46 -0400
[21:31:02] - Nikto v2.6.0
[21:31:02]
-----
[21:31:02] [Nessus Error] 401 Client Error: Authorization
required for url: https://localhost:8834/session
[21:31:04] Nmap scan report for scanme.nmap.org
(45.33.32.156)
[21:31:04] Host is up (0.0012s latency).
[21:31:04] Other addresses for scanme.nmap.org (not scanned):
```

Figure 129: Scan execution

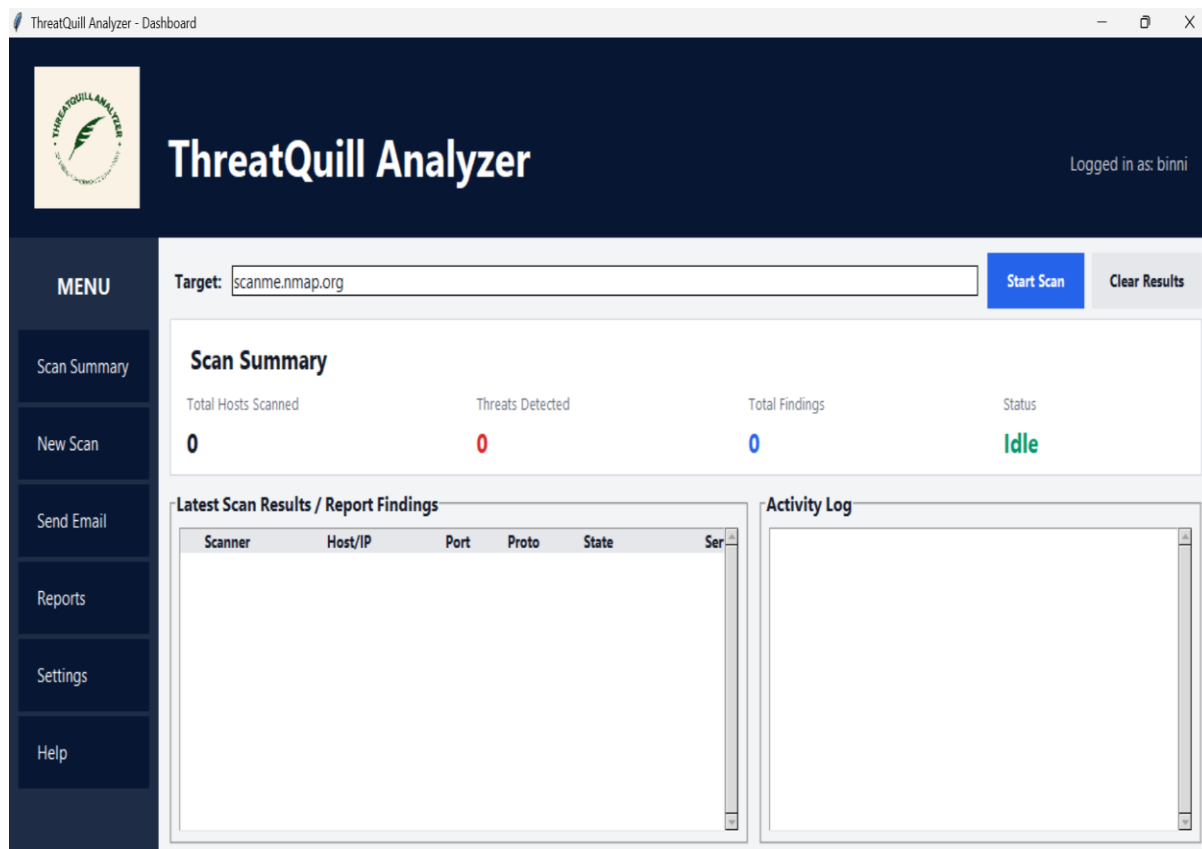


Figure 130: After scan Clear result implementation

4.2.9 Test Case 9

Test Case 9	
Objective	To test whether the system correctly calculates summary values such as total hosts scanned, threats detected, and total findings from scan results.
Action	A scan was performed on a valid target, and the summary section of the dashboard was observed to verify the calculated values.
Expected Result	The system should correctly display the total number of hosts scanned, threats detected, and total findings based on the scan results.
Actual Result	The system correctly displayed the total hosts scanned, number of threats detected, and total findings in the summary section.
Conclusion	Test successful.

Table 13: Unit Testing Test Case 9

ThreatQuill Analyzer - Dashboard

Target: Scanning... Clear Results

Live Scan Overview

Total Hosts Scanned	Threats Detected	Total Findings	Status
1	4	43	Complete

Latest Scan Results / Report Findings

Scanner	Host/IP	Port	Proto	State	Ser
NIKTO	scanme.nmap.org	80	tcp	open	ht
NIKTO	scanme.nmap.org	80	tcp	open	Apache/2.4
NIKTO	scanme.nmap.org	80	tcp	open	Apache/2.4
NMAP	45.33.32.156	22	tcp	open	ssh Open
NMAP	45.33.32.156	80	tcp	open	http Apache
NMAP	45.33.32.156	9929	tcp	open	nping-echo
NMAP	45.33.32.156	31337	tcp	open	tcpwr
NMAP	45.33.32.156	-	-	info	Service

Activity Log

```
[13:25:35] NMAP → 80/tcp open http Apache httpd
2.4.7 ((Ubuntu))
[13:25:35] NMAP → 9929/tcp open nping-echo Nping echo
[13:25:35] NMAP → 31337/tcp open tcpwrapped
[13:25:35] NMAP → Service Info: OS: Linux; CPE:
cpe:/o:linux:linux_kernel
[13:25:35] NMAP → Service detection performed. Please
report any incorrect results at https://nmap.org/submit/ .
[13:25:35] NMAP → Nmap done: 1 IP address (1 host up)
scanned in 140.31 seconds
[13:25:35] NMAP → Scan completed successfully.
[13:25:35] INFO → Nmap function finished
[13:25:35] INFO → Waiting for Nikto thread...
[13:25:47] INFO → [SYSTEM] Opened report:
D:\ThreatQuill\reports\report.html
```

Figure 131: Scan completed with results displayed

4.2.10 Test Case 10

Test Case 10	
Objective	To test whether the system successfully generates the report file after processing scan results.
Action	A scan was performed on a valid target, and the report generation function was triggered. The reports folder was checked for the generated report file.
Expected Result	The system should generate a report file (HTML/JSON) and store it in the reports directory.
Actual Result	The system successfully generated the report file (report.html) in the reports folder, and the report was accessible for viewing.
Conclusion	Test successful.

Table 14: Unit Testing Test Case 10

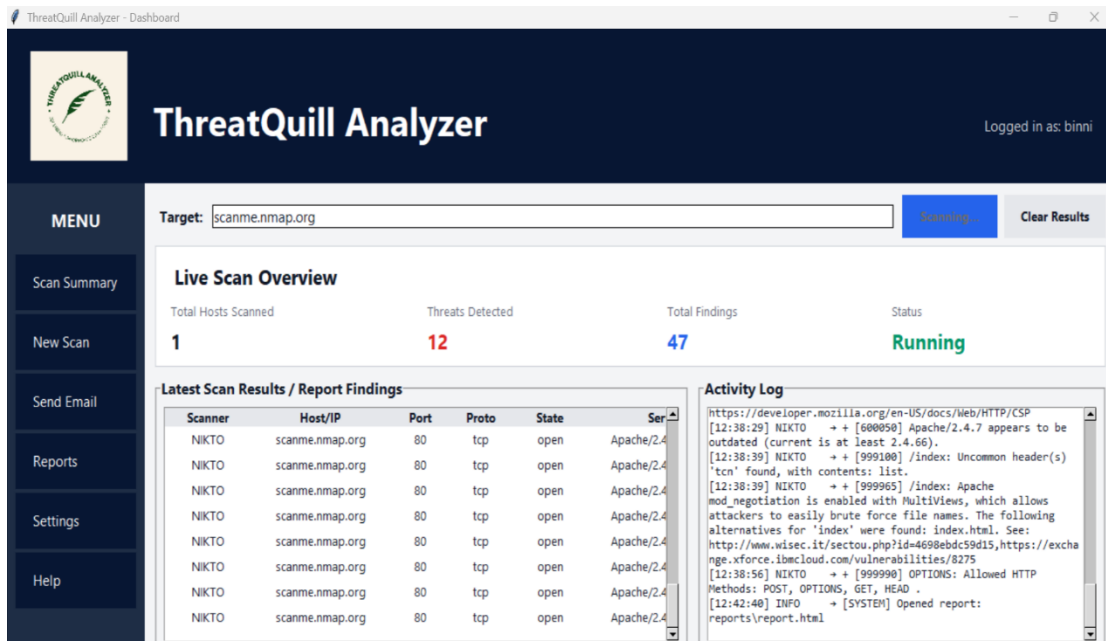


Figure 132: Scan Completed for report generation

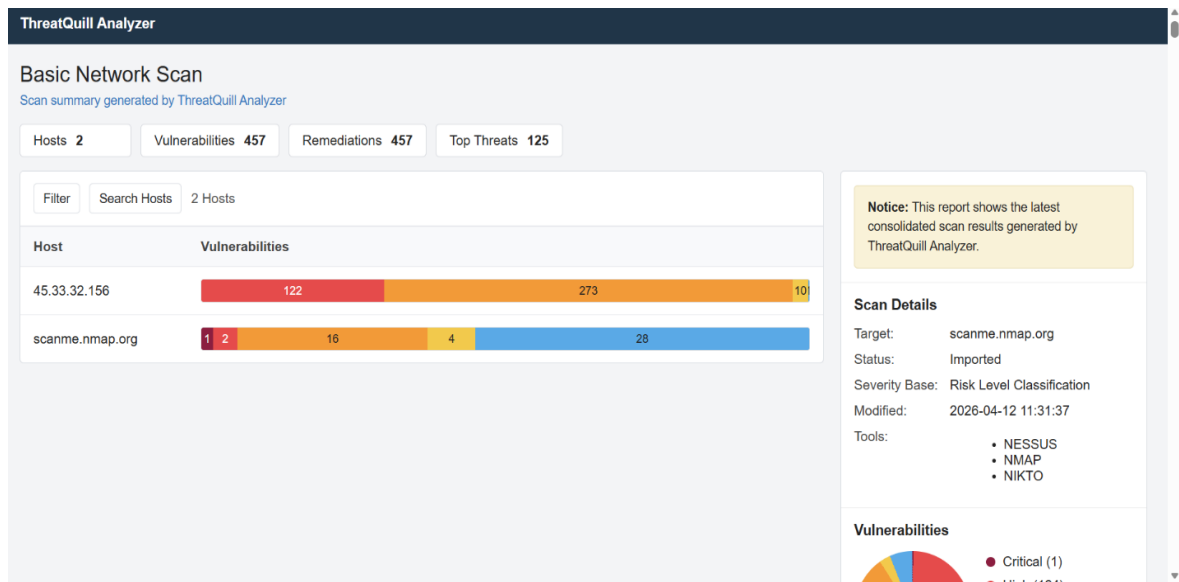


Figure 133: Generated report showing vulnerability summary

4.3 System Testing

System testing was carried out to evaluate the complete system after integrating all modules. This testing ensured that the ThreatQuill Analyzer works correctly as a whole system. Scans were executed using multiple scanners, and the results were displayed on the dashboard. Features such as real-time logs, scan summaries, and result visualization were tested to ensure proper functionality. The report generation feature was also tested to confirm that structured reports are produced correctly.

System testing confirmed that all components are well integrated and the system performs efficiently without errors.

4.3.1 Test Case 1

Test Case 1	
Objective	To test whether the system performs a complete scan successfully after entering a valid target.
Action	A valid target was entered, and the “Start Scan” button was clicked to initiate the scanning process.
Expected Result	The system should execute the scan, display results in the results table, update the scan summary, and show real-time activity logs.
Actual Result	The system successfully executed the scan, displayed results in the results table, updated the summary values, and showed activity logs.
Conclusion	Test successful.

Table 15: System Testing Test Case 1

The screenshot displays the ThreatQuill Analyzer dashboard. At the top, the title 'ThreatQuill Analyzer' is prominent, with a user login 'binni'. A sidebar menu on the left includes options like 'Scan Summary', 'New Scan', 'Send Email', 'Reports', 'Settings', and 'Help'. The main area shows a 'Target' field with 'scanme.nmap.org' and buttons for 'Start Scan' and 'Clear Results'. Below this, a 'Scan Summary' section indicates '1' host scanned, '0' threats detected, '40' total findings, and a 'Complete' status. The 'Latest Scan Results' table lists various services scanned by Nessus, all with a state of 'x1'. The 'Activity Log' on the right provides a detailed timeline of the scan process, from the initial live scan request to the completion of Nmap, Nessus, and Nikto scans.

ThreatQuill Analyzer - Dashboard

ThreatQuill Analyzer

Logged in as: binni

MENU

Scan Summary

New Scan

Send Email

Reports

Settings

Help

Target: scanme.nmap.org

Start Scan

Clear Results

Scan Summary

Total Hosts Scanned: 1

Threats Detected: 0

Total Findings: 40

Status: Complete

Latest Scan Results

Scanner	Host/IP	Port	Proto	State	Ser
NESSUS	scanme.nmap.org	-	-	x1	Traceroute
NESSUS	scanme.nmap.org	-	-	x1	OS Iden
NESSUS	scanme.nmap.org	-	-	x1	OS Fingerpr
NESSUS	scanme.nmap.org	-	-	x1	Network Time
NESSUS	scanme.nmap.org	-	-	x1	Nessus Scan
NESSUS	scanme.nmap.org	-	-	x1	HyperText Tra
NESSUS	scanme.nmap.org	-	-	x1	HTTP Server T
NESSUS	scanme.nmap.org	-	-	x1	HTTP Methods

Activity Log

```
[07:48:42] Live scan requested for target: scanme.nmap.org
[07:48:42] [DEBUG] start_full_scan called
[07:48:42] [DEBUG] run_all entered
[07:48:42] [SYSTEM] Starting full scan for target:
scanme.nmap.org
[07:48:42] [SYSTEM] Starting Nmap thread...
[07:48:42] [SYSTEM] Starting Nikto thread...
[07:48:42] [DEBUG] Nmap function entered
[07:48:42] [SYSTEM] Starting Nessus thread...
[07:48:42] [DEBUG] Nikto function entered
[07:48:42] [DEBUG] Nessus function entered
[07:48:42] [DEBUG] Waiting for Nmap thread...
[07:48:42] [NESSUS] Logging into Nessus...
[07:48:42] [NMAP] Starting live scan on scanme.nmap.org
[07:48:42] [NIKTO] Starting Nikto scan on
http://scanme.nmap.org
```

Figure 134: Complete scan executed successfully

4.3.2 Test Case 2

Test Case 2	
Objective	To test whether the activity log displays real-time scan progress correctly during scan execution.
Action	A valid target was entered, and the “Start Scan” button was clicked. The activity log was observed during the scan process.
Expected Result	The activity log should display real-time updates showing the progress of different scanning modules such as Nmap, Nikto, Nessus, and OpenVAS.
Actual Result	The activity log displayed continuous updates showing the execution of different scanning modules and their progress during the scan.
Conclusion	Test successful.

Table 16: System Testing Test Case 2

The screenshot displays the ThreatQuill Analyzer dashboard. The top header shows the application name and a user login status 'Logged in as: binni'. A sidebar menu on the left includes options like 'Scan Summary', 'New Scan', 'Send Email', 'Reports', 'Settings', and 'Help'. The main content area features a 'Target' input field with 'scanme.nmap.org' and a 'Scanning...' button. Below this, a 'Scan Summary' section provides statistics: 1 Total Hosts Scanned, 1 Threats Detected, 3 Total Findings, and a 'Complete' status. The 'Latest Scan Results' table lists findings from various scanners (OPENVAS, NESSUS) across different ports and protocols. The 'Activity Log' on the right shows real-time output from the scanners, including Nmap's port scan progress and Nessus's service detection results.

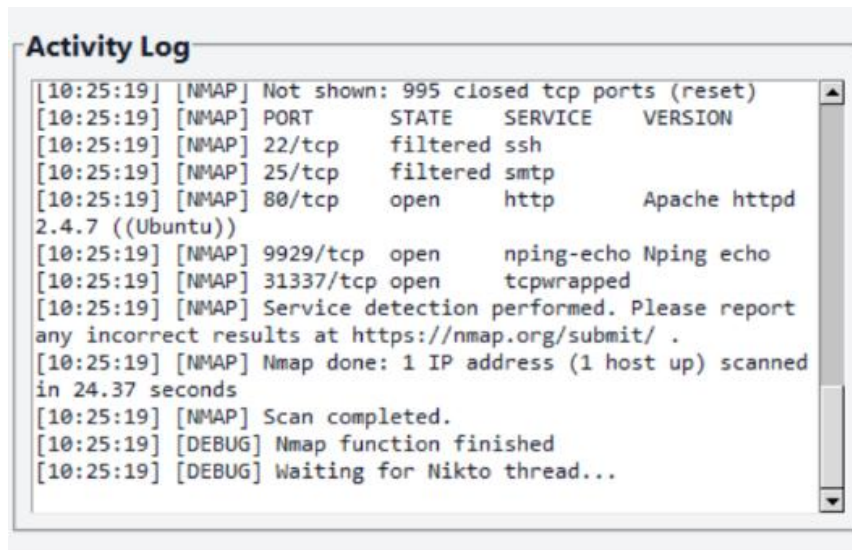
Scanner	Host/IP	Port	Proto	State	Ser
OPENVAS	scanme.nmap.org	80	tcp	open	Outdated A
OPENVAS	scanme.nmap.org	443	tcp	open	Weak SSL C
OPENVAS	scanme.nmap.org	22	tcp	open	SSH Wei
NESSUS	scanme.nmap.org	-	-	x1	Traceroute
NESSUS	scanme.nmap.org	-	-	x1	OS Iden
NESSUS	scanme.nmap.org	-	-	x1	OS Fingerpr
NESSUS	scanme.nmap.org	-	-	x1	Network Time
NESSUS	scanme.nmap.org	-	-	x1	Nessus Scar

```

[10:25:19] [NMAP] Not shown: 995 closed tcp ports (reset)
[10:25:19] [NMAP] PORT STATE SERVICE VERSION
[10:25:19] [NMAP] 22/tcp filtered ssh
[10:25:19] [NMAP] 25/tcp filtered smtp
[10:25:19] [NMAP] 80/tcp open http Apache httpd
2.4.7 ((Ubuntu))
[10:25:19] [NMAP] 9929/tcp open nping-echo Nping echo
[10:25:19] [NMAP] 31337/tcp open tcpwrapped
[10:25:19] [NMAP] Service detection performed. Please report
any incorrect results at https://nmap.org/submit/ .
[10:25:19] [NMAP] Nmap done: 1 IP address (1 host up) scanned
in 24.37 seconds
[10:25:19] [NMAP] Scan completed.
[10:25:19] [DEBUG] Nmap function finished
[10:25:19] [DEBUG] Waiting for Nikto thread...

```

Figure 135: Dashboard showing real time output



The image shows a window titled "Activity Log" with a scrollable text area. The text area contains the following output from an Nmap scan:

```
[10:25:19] [NMAP] Not shown: 995 closed tcp ports (reset)
[10:25:19] [NMAP] PORT      STATE    SERVICE  VERSION
[10:25:19] [NMAP] 22/tcp    filtered ssh
[10:25:19] [NMAP] 25/tcp    filtered smtp
[10:25:19] [NMAP] 80/tcp    open     http      Apache httpd
2.4.7 ((Ubuntu))
[10:25:19] [NMAP] 9929/tcp  open     nping-echo Nping echo
[10:25:19] [NMAP] 31337/tcp open     tcpwrapped
[10:25:19] [NMAP] Service detection performed. Please report
any incorrect results at https://nmap.org/submit/ .
[10:25:19] [NMAP] Nmap done: 1 IP address (1 host up) scanned
in 24.37 seconds
[10:25:19] [NMAP] Scan completed.
[10:25:19] [DEBUG] Nmap function finished
[10:25:19] [DEBUG] Waiting for Nikto thread...
```

Figure 136: Activity log showing real-time update

4.3.3 Test case 3

Test Case 3	
Objective	To test whether the scan summary section updates correctly after the scan is completed.
Action	A valid target was entered, and a scan was performed. After completion, the scan summary section was observed.
Expected Result	The system should update the summary values including total hosts scanned, threats detected, total findings, and status after scan completion.
Actual Result	The system successfully updated the summary values showing total hosts scanned, threats detected, total findings, and status as complete.
Conclusion	Test successful.

Table 17: System Testing Test Case 3

```

EXECUTIVE SUMMARY-----
The ThreatQuill Analyzer performed a vulnerability assessment on scanme.nmap.org using .
A total of 40 findings were identified. The overall risk level was assessed as Info.

SEVERITY SUMMARY-----
Critical: 2
High: 5
Medium: 10
Low: 15
Info: 8

DETAILED FINDINGS -----
1. Open ports detected: 22/tcp (SSH), 80/tcp (HTTP), 9929/tcp (Nping-echo), 31337/tcp (tcpwrapped)
2. HTTP service detected running Apache httpd 2.4.7 with potential
   security misconfigurations.
3. Weak SSL/TLS configurations and missing security headers identified.

RECOMMENDATIONS -----
1. Patch outdated software and services.
2. Close unused ports and restrict unnecessary exposure.
3. Fix weak SSL/TLS configurations if identified.
4. Apply security headers and harden server configuration.
5. Re-scan the target after remediation.

CONCLUSION -----
The assessment identified vulnerabilities and security weaknesses that should be addressed
to improve the overall security posture of the target system.

```

Figure 137: Executive Summary

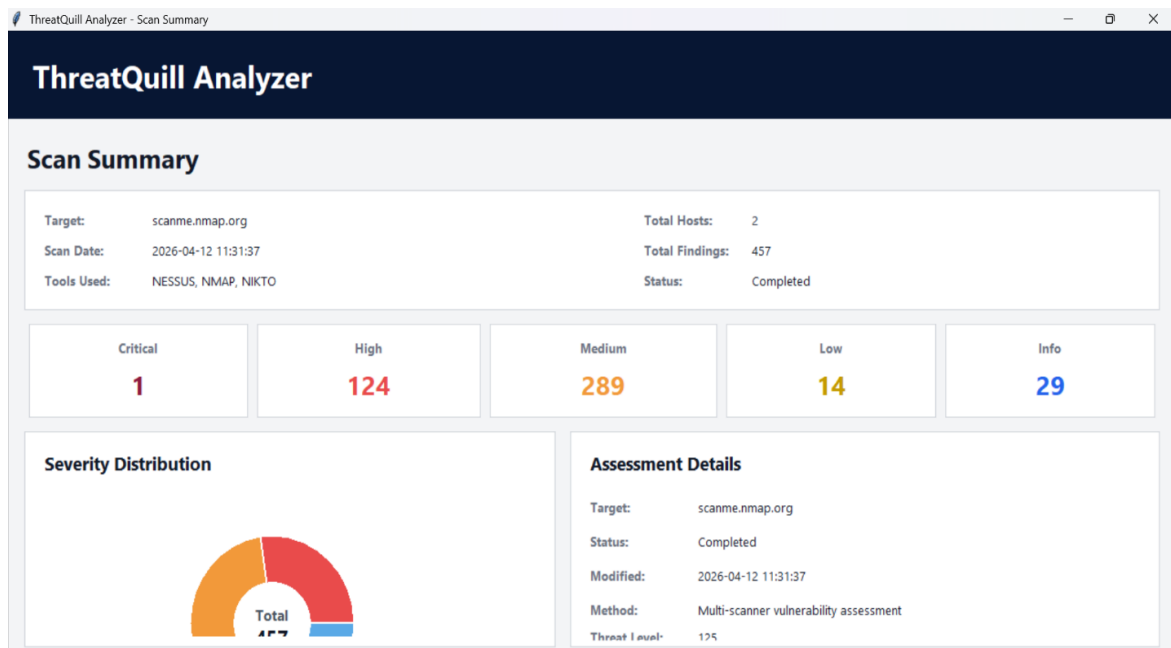


Figure 138: Scan summary updated after completion

4.3.4 Test Case 4

Test Case 4	
Objective	To test whether scan results are displayed correctly in the results table after scan execution.
Action	A valid target was entered, and a scan was performed. After execution, the results table was observed.
Expected Result	The system should display scan results in the results table with details such as scanner name, host/IP, port, protocol, state, and service.
Actual Result	The system successfully displayed scan results in the results table with details including scanner name, host/IP, port, protocol, state, and service.
Conclusion	Test successful.

Table 18: System Testing Test Case 4

ThreatQuill Analyzer - Dashboard

Target: Scanning... Clear Results

Scan Summary

Total Hosts Scanned	Threats Detected	Total Findings	Status
1	1	3	Complete

Latest Scan Results

Scanner	Host/IP	Port	Proto	State	Ser
NIKTO	scanme.nmap.org	80	tcp	open	ht
NIKTO	scanme.nmap.org	80	tcp	open	ht
NIKTO	scanme.nmap.org	80	tcp	open	ht
NMAP	45.33.32.156	22	tcp	filtered	s:
NMAP	45.33.32.156	25	tcp	filtered	sn
NMAP	45.33.32.156	80	tcp	open	http Apache
NMAP	45.33.32.156	9929	tcp	open	nping-echo
NMAP	45.33.32.156	31337	tcp	open	tcpwr

Activity Log

```
[07:37:50] [NMAP] Not shown: 995 closed tcp ports (reset)
[07:37:50] [NMAP] PORT STATE SERVICE VERSION
[07:37:50] [NMAP] 22/tcp filtered ssh
[07:37:50] [NMAP] 25/tcp filtered smtp
[07:37:50] [NMAP] 80/tcp open http Apache httpd
2.4.7 ((Ubuntu))
[07:37:50] [NMAP] 9929/tcp open nping-echo nping echo
[07:37:50] [NMAP] 31337/tcp open tcpwrapped
[07:37:50] [NMAP] Service detection performed. Please report
any incorrect results at https://nmap.org/submit/ .
[07:37:50] [NMAP] Nmap done: 1 IP address (1 host up) scanned
in 20.66 seconds
[07:37:50] [NMAP] Scan completed.
[07:37:50] [DEBUG] Nmap function finished
[07:37:50] [DEBUG] Waiting for Nikto thread...
```

Figure 139: Nmap and Nikto scan result displayed in table

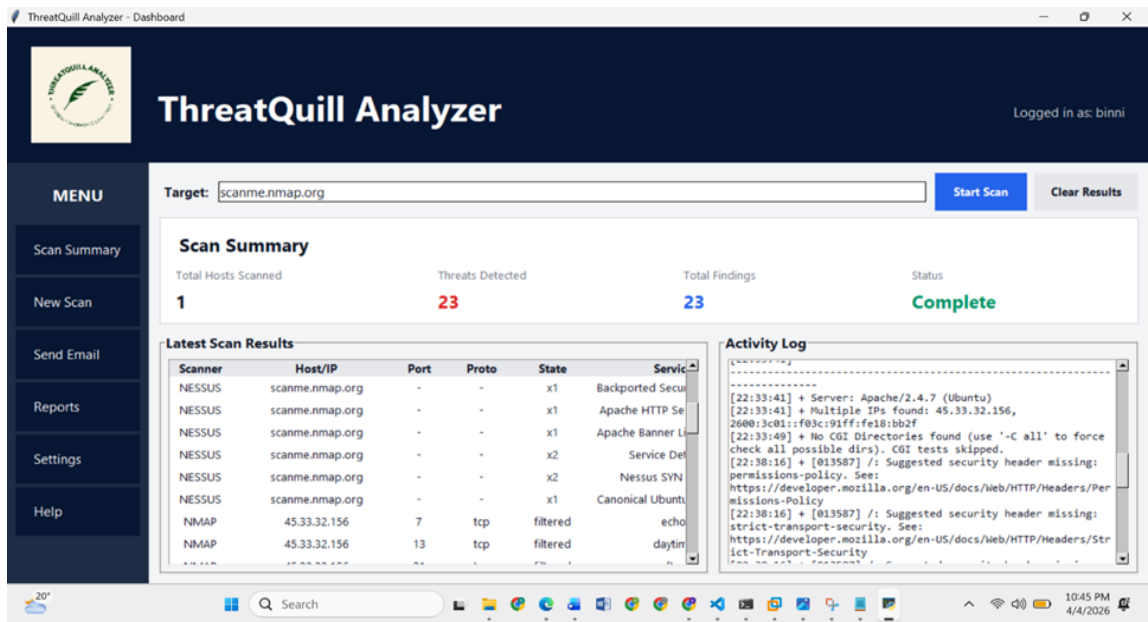


Figure 140: Nessus Scan result

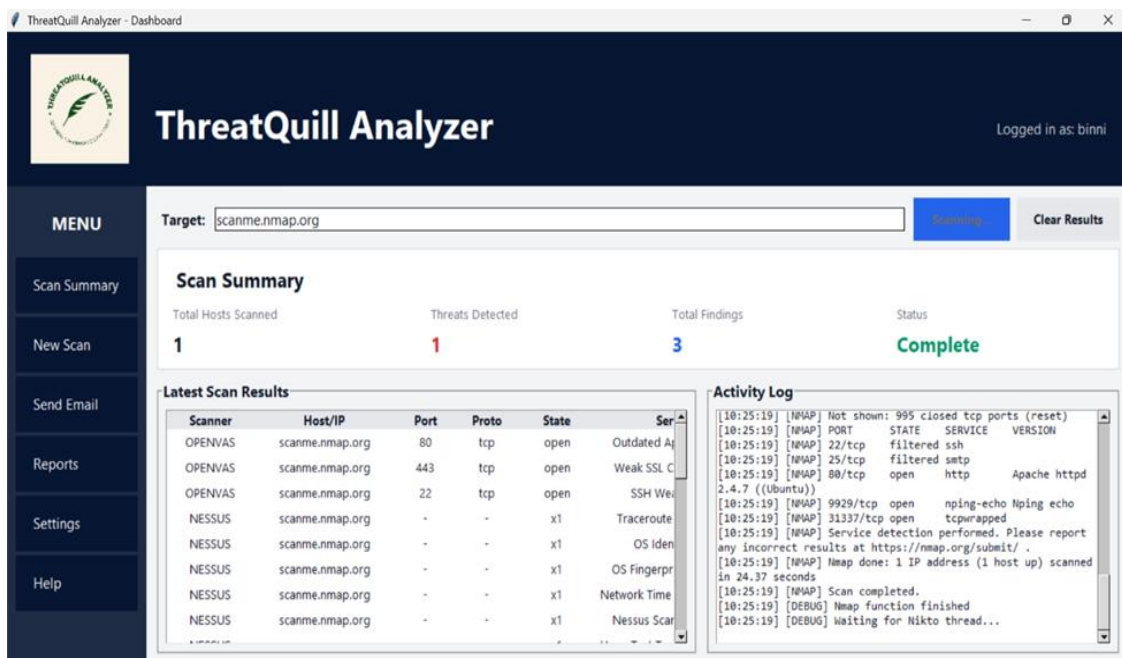


Figure 141: OpenVAS scan result

4.3.5 Test case 5

Test Case 5	
Objective	To test whether multiple scanners execute together and provide combined results in the dashboard.
Action	A valid target was entered, and the scan was initiated. The activity log and results table were observed to verify the execution of multiple scanners.
Expected Result	The system should execute multiple scanners such as Nmap, Nikto, Nessus, and OpenVAS and display their combined results in the dashboard.
Actual Result	The system successfully executed multiple scanners, and the results from different scanners were displayed together in the results table and activity log.
Conclusion	Test successful.

Table 19: System Testing Test Case 5

The screenshot displays the ThreatQuill Analyzer dashboard. The top header shows the application logo and the user is logged in as 'binni'. A sidebar menu on the left includes options like 'Scan Summary', 'New Scan', 'Send Email', 'Reports', 'Settings', and 'Help'. The main content area shows a 'Target' field with 'scanme.nmap.org' and buttons for 'Start Scan' and 'Clear Results'. Below this, a 'Scan Summary' section indicates 1 host scanned, 0 threats detected, 40 total findings, and a 'Complete' status. The 'Latest Scan Results' table lists findings from various scanners (NESSUS, NIKTO) across different ports and protocols. The 'Activity Log' on the right provides a detailed timeline of the scan process, including system messages and scanner-specific actions.

Scanner	Host/IP	Port	Proto	State	Ser
NESSUS	scanme.nmap.org	-	-	x1	Traceroute
NESSUS	scanme.nmap.org	-	-	x1	OS Ident
NESSUS	scanme.nmap.org	-	-	x1	OS Fingerpr
NESSUS	scanme.nmap.org	-	-	x1	Network Time
NESSUS	scanme.nmap.org	-	-	x1	Nessus Scan
NESSUS	scanme.nmap.org	-	-	x1	HyperText Tra
NESSUS	scanme.nmap.org	-	-	x1	HTTP Server Ty
NESSUS	scanme.nmap.org	-	-	x1	HTTP Methods

```

[07:48:42] Live scan requested for target: scanme.nmap.org
[07:48:42] [DEBUG] start_full_scan called
[07:48:42] [DEBUG] run_all entered
[07:48:42] [SYSTEM] Starting full scan for target: scanme.nmap.org
[07:48:42] [SYSTEM] Starting Nmap thread...
[07:48:42] [SYSTEM] Starting Nikto thread...
[07:48:42] [DEBUG] Nmap function entered
[07:48:42] [SYSTEM] Starting Nessus thread...
[07:48:42] [DEBUG] Nikto function entered
[07:48:42] [DEBUG] Nessus function entered
[07:48:42] [DEBUG] Waiting for Nmap thread...
[07:48:42] [NESSUS] Logging into Nessus...
[07:48:42] [NMAP] Starting live scan on scanme.nmap.org
[07:48:42] [NIKTO] Starting Nikto scan on http://scanme.nmap.org
  
```

Figure 142: Multiple scanners executed together

4.3.6 Test case 6

Test Case 6	
Objective	To test whether the system successfully generates and opens the final report after scan execution.
Action	A valid target was entered and the scan was completed. After completion, the Report button was clicked.
Expected Result	The system should generate and open the final report successfully.
Actual Result	The system generated the final report and opened it successfully for viewing.
Conclusion	Test successful.

Table 20: System Testing Test Case 6

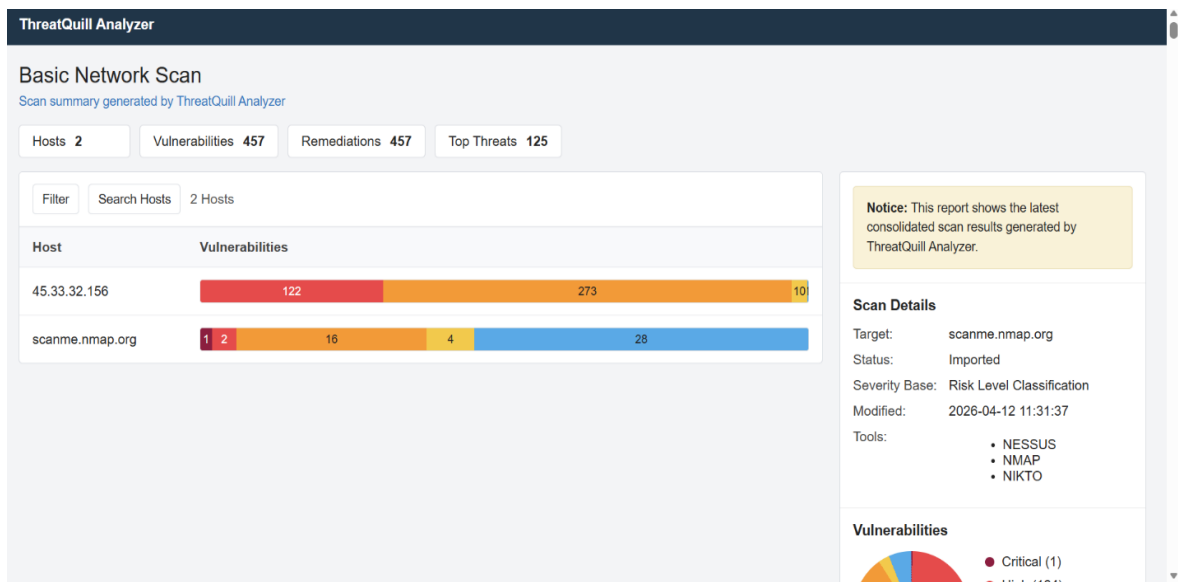


Figure 143: Report generated and opened successfully

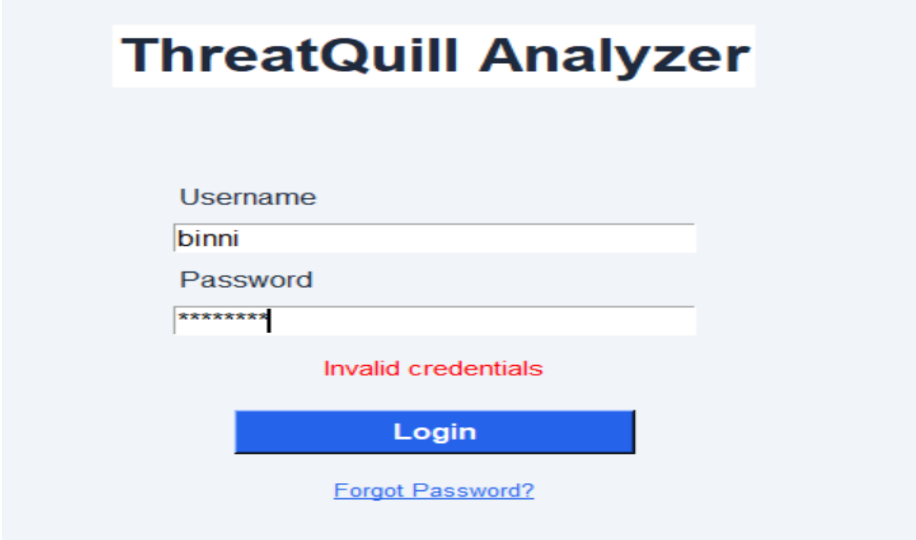
4.4 Security Testing

System testing is the process of testing a complete and integrated system to verify that it meets the specified requirements. It ensures that all components of the system work together correctly and that the system performs as expected in a real-world environment.

4.4.1 Test Case 1

Test Case 1	
Objective	To test whether the system prevents unauthorized access with invalid login credentials.
Action	Entered an invalid username and/or password on the login page and clicked the Login button.
Expected Result	The system should deny access and display an error message such as “Invalid credentials”. The dashboard should not open.
Actual Result	The system denied access and displayed error message “Invalid credentials”. The dashboard was not opened.
Conclusion	Test successful.

Table 21: Security Testing Test Case 1



The screenshot displays the 'ThreatQuill Analyzer' login interface. It features two input fields: 'Username' with the text 'binni' and 'Password' with masked characters '*****'. Below the password field, a red error message 'Invalid credentials' is visible. A blue 'Login' button is positioned below the error message, and a blue link 'Forgot Password?' is located at the bottom of the form.

Figure 144: Invalid login credentials rejected

4.4.2 Test Case 2

Test Case	
Objective	To test whether sensitive data such as passwords are not displayed in plain text and password reset is handled securely.
Action	Entered a password in the login field and clicked on the “Forgot Password?” option to observe system behavior.
Expected Result	The password should remain hidden, and the system should not reveal the password. Instead, it should provide a secure password reset message.
Actual Result	The password was masked using asterisks, and the system displayed a message instructing the user to contact the administrator for password reset.
Conclusion	Test successful.

Table 22: Security Testing Test Case 2

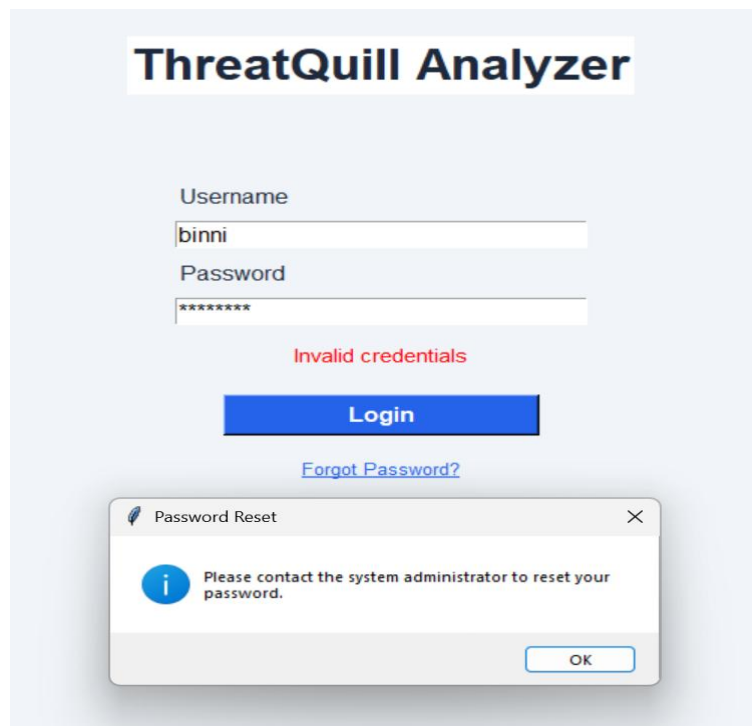
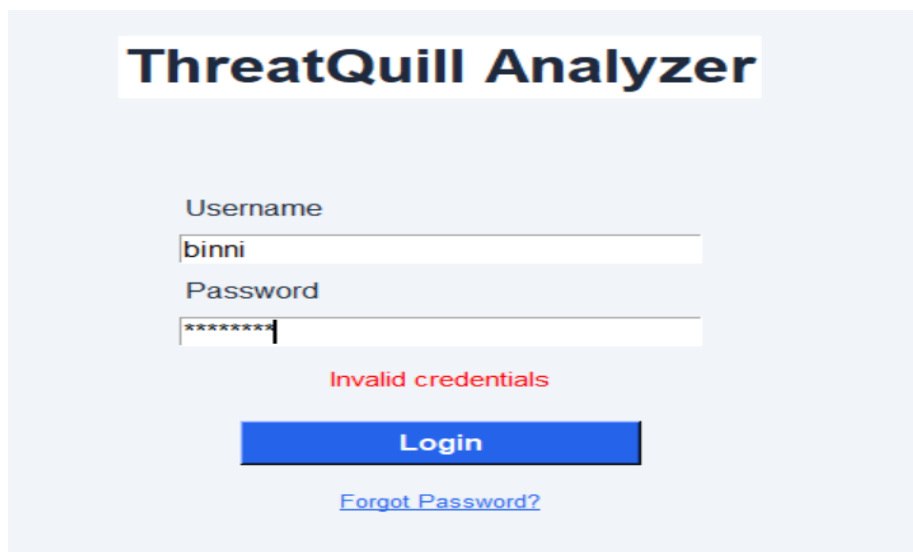


Figure 145: Password input hidden securely

4.4.3 Test Case3

Test Case 3	
Objective	To test whether the system restricts access to the dashboard without proper authentication.
Action	Attempted to access the dashboard directly without logging in or after entering invalid login credentials.
Expected Result	The system should not allow access to the dashboard and should redirect the user to the login page.
Actual Result	The system restricted access to the dashboard and did not allow entry without valid login credentials.
Conclusion	Test successful.

Table 23: Security Testing Test Case 3



The screenshot displays the 'ThreatQuill Analyzer' login interface. It features a title 'ThreatQuill Analyzer' at the top. Below it are two input fields: 'Username' with the text 'binni' and 'Password' with masked characters '*****'. A red error message 'Invalid credentials' is shown below the password field. A blue 'Login' button is positioned below the error message, and a blue link 'Forgot Password?' is at the bottom.

Figure 146: Invalid Credential

4.4.4 Test Case 4

Test Case 4	
Objective	To test whether the system handles invalid or malicious input safely without crashing.
Action	Entered an invalid target value (e.g., @@ or invalid IP) in the target field and clicked the “Start Scan” button.
Expected Result	The system should display an error message and should not start the scan. The system should remain stable without crashing.
Actual Result	The system displayed an “Invalid Target” message and prevented the scan from starting. The system remained stable and did not crash.
Conclusion	Test successful.

Table 24: Security Testing Test Case 4

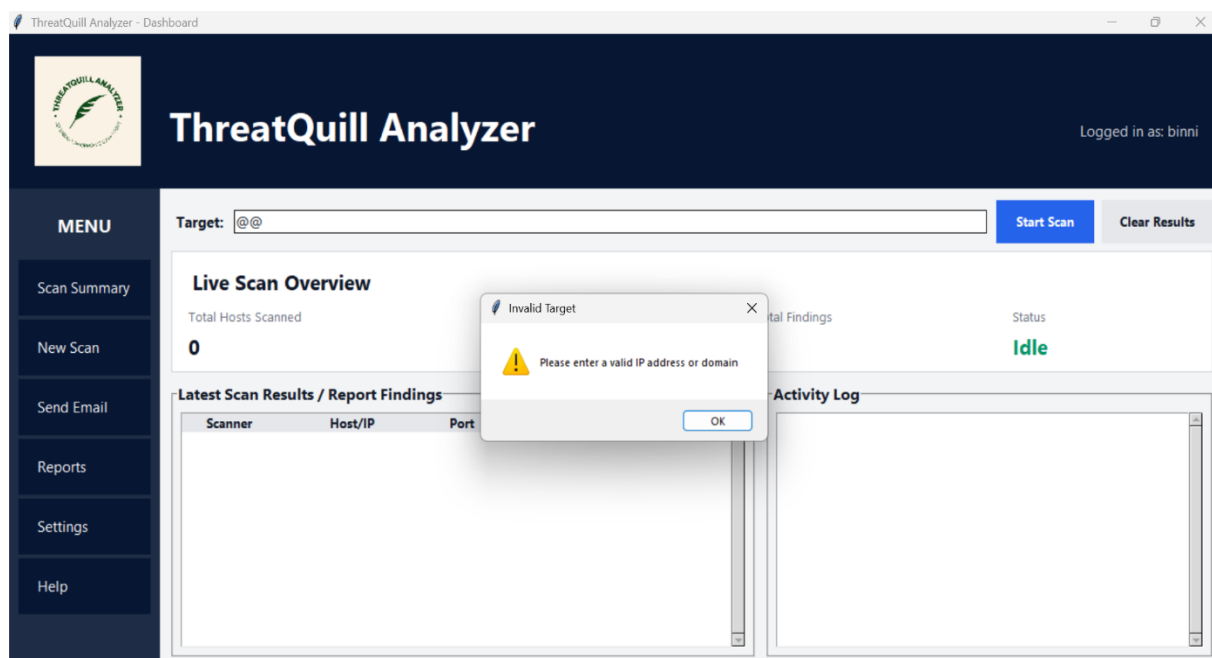


Figure 147: Invalid Target

4.4.5 Test Case 5

Test Case 5	
Objective	To test whether secure communication is maintained when connecting to external scanners (e.g., via SSH/HTTPS).
Action	Initiated a scan which connects to external scanners and observed the connection process through the activity log.
Expected Result	The system should attempt to secure communication using SSH/HTTPS protocols and handle connection failures gracefully without crashing.
Actual Result	The system attempted to establish a secure connection to external scanners. In some cases, connection errors (e.g., SSH timeout) occurred, but the system handled them properly without crashing and continued execution
Conclusion	Test successful.

Table 25: Security Testing Test Case 5

The screenshot displays the ThreatQuill Analyzer dashboard. The top header shows the application name and a user login 'binni'. A sidebar menu on the left includes options like 'Scan Summary', 'New Scan', 'Send Email', 'Reports', 'Settings', and 'Help'. The main content area shows a 'Target' field with 'scanme.nmap.org' and buttons for 'Scanning...' and 'Clear Results'. Below this is a 'Scan Summary' section with metrics: 'Total Hosts Scanned' (1), 'Threats Detected' (2), 'Total Findings' (2), and 'Status' (Complete). A table titled 'Latest Scan Results / Report Findings' lists two entries for the NIKTO scanner on port 80, both with a state of 'open'. To the right, an 'Activity Log' section shows a detailed log of the scan process, including Nmap version information, Nikto version, and OpenVAS raw data showing a connection attempt to 192.168.0.133 on port 22, which timed out.

Figure 148: Secure connection established with scanners

4.5 Critical Analysis

ThreatQuill Analyzer project exemplifies a feasible method of assessing vulnerabilities based on a combination of various scanning tools. The system eases the scanning process, enhances efficiency, and displays results in a well-organized and straightforward fashion. Development and testing are however revealed and some areas of limitations which underscores future development.

4.5.1 Strengths

- Combination of various scanners including Nmap, Nikto, OpenVAS and Nessus under one platform.
- Scans and processing of results, automatic, less work.
- Dashboard is easy to use and presents complicated vulnerability information.
- Live activity feeds to monitor and track.
- Smoother report creation leading to improved decision making.

4.5.2 Weaknesses

- Dependability on external software that influences accuracy and performance.
- There is a risk of scan results have a false positive and false negative.
- Large or complex targets can be time consuming in scanning process.
- Weakly developed advanced data visualization.
- Limited scalability of desktop-based systems.

4.5.3 Opportunities

- Incorporation of other vulnerability testing applications to enhance coverage.
- Increased use of graphs and visual dashboards to improve the user interface.
- Screening of clouds and accessibility.
- Bettering format of report to bias towards professional VAPT criteria.
- Deployed automated scan scheduling capabilities.

4.5.4 Threats

- Quick development of cyber threats that need regular updates on the system.
- Alterations or restrictions on third party scanners.
- Loss of security in case the system is abused in the absence of relevant authorization.
- Network problems that impact scan performance.
- Relying on external APIs (e.g., Nessus) which can cause system failures.

Chapter 5: Conclusion

The ThreatQuill Analyzer is a project that has achieved its goal of producing a multi-scanner vulnerability assessment system that helps to make it easier to locate and evaluate vulnerabilities in security. The system is flexible, and many industry-standard tools like Nmap, Nikto, OpenVAS and Nessus are installed, which enables full and good vulnerability detection within one system.

The system can perform real time scanning, processing scan results quickly and produce structured reports that can guide the users to understand and prioritize security risk. The results of the scan, activity logs, and summary are shown in a clear and organized format on the dashboard interface allowing the user to analyze vulnerabilities effectively. The project also illustrates the translation of ideas of cybersecurity as well as the execution of a systematized software development methodology (DSDM).

Overall, the use of ThreatQuill Analyzer demonstrates that the combination of various scanning tools into a single system can greatly improve efficiency, usability and effectiveness of vulnerability assessment. The project is a good start towards additional improvements and can be extended to suit more advanced cybersecurity needs in real-life settings.

5.1 Legal, Social and Ethical Issues

Through vulnerability assessment tools, there are significant legal, social, and ethical issues in their use. It is critical that the system should be employed in a responsible manner and which is in adherence to applicable legislation and cybersecurity practices.

5.1.1 Legal Issues

Legally, vulnerability scanning should be done only when authorization to the systems is acquired. The Electronic Transactions Act, 2063 (Nepal) defines the unauthorized access of computer systems and networks as a cybercrime which can result in legal repercussions like penalties or prosecution.

We also should securely process the data gathered during the scanning process such as information about the system and any vulnerabilities detected. Mishandling or unnecessary distribution of such data can lead to loss of privacy and lawsuits. Thus, there is a need to adhere to the principles of data protection

and to make sure that any information obtained is employed to fulfill the highest legitimate security interests ((NIST), 2018).

5.1.2 Social Issues

Social implications of using tools such as vulnerability scanners may arise when such tools are not used well. The misuse of the system can engender fear, disorientation or abuse of sensitive information in organizations or communities. As an example, publishing scan results without controlling can result in the loss of the trust of the users.

In addition, scan outcomes can be misunderstood by people of limited technical knowledge, which will result in wrong decisions or unwarranted worries. Thus, the clear and understandable way of presenting the results and encouraging the audience to use them responsibly is a step to enhancing awareness of cybersecurity (Reyes, 2022).

5.1.3 Ethical Issues

The consideration of ethics is critical when utilizing the vulnerability assessment tools. The ThreatQuill Analyzer is a defensive tool, which aims to detect vulnerabilities and enhance the security of systems instead of exploiting them.

Users should be careful that the system is not being misused like unauthorized access, stealing information or disrupting the system. Ethical usage needs to be respectful of privacy of the user, do not disclose any secrets, and that the tool is only utilized with the intention of enhancing security. By adhering to the ethical standards, one will make sure that the system will add to the cybersecurity practices, rather than result in harm (SANS, 2022).

5.2 Advantages

- The combination of various vulnerability scanners within one platform enables great efficiency and coverage.
- The use of automated scanning and result processing saves a lot of time and manual labor.
- The analysis of complicated vulnerability data is made easy by a friendly dashboard interface.
- Detailed scanning records and system activity are easily visible in real-time.
- Formatted generation of reports allows users to learn and prioritize security risks.
- The centralized system will remove the possibility of having to work on various tools individually.
- Offers an affordable package that can be used by higher educational institutions and small organizations.

5.3 Limitations

- Dependency on third-party tools such as Nmap, OpenVAS, and Nessus may affect overall accuracy and performance
- Risks of a vulnerability detecting false positive and false negative.
- Scanning can be expensive with respect to time in the case of large or complicated network environments.
- Weak data visualization capabilities alongside access to limited advanced data visualization capabilities relative to commercial vulnerability assessment tools.
- On-desk implementation limits scalability and access remotely.
- The network conditions and the system resources may impact system performance.
- Needs to be configured and set up correctly with external scanning tools to operate effectively.

5.4 Future Work

5.4.1 Cloud-Based Scanning System

Objective: Increase scalability and accessibility of the system through providing vulnerability scanning with the usage of cloud.

Implementation Plan:

1. Choose an appropriate cloud service provider (AWS, Azure, or Google) according to the cost, availability, and features.
2. Install the ThreatQuill Analyzer in the cloud to aid remote access and scans management.
3. Containerize using technologies like Docker to effectively control the scanners in various environments.
4. Regularly monitor the performance of the monitors and optimization of cloud resources that makes operation in monitors stable and scalable.

5.4.2 Data Visualization Data Dashboard

Objective: Make the user experience smoother and enhance scan results with sophisticated visual-based representations.

Implementation Plan:

1. Include the usage of graphical elements like pie chart, bar chart, and level of severity to the dashboard.
2. Create graphic features to represent the trends in vulnerability and the distribution of severity in a way that is easier to comprehend.
3. Incorporate the filtering and sorting features enabling users to analyze the results of scans better.
4. Streamline the dashboard design, based on usability tests and user reaction, to enhance its comprehensive understanding and engagement.

5.4.3 Automated Scan Timetable System

Objective: Ensuring continuous monitoring of vulnerability scans should be enabled through automatic scheduling of vulnerability scans.

Implementation Plan:

1. Add a time scheduling capability like allowing users to set scan time; this could be every day, week, or once every month.
2. Set up background running, so that no operator is needed to run a scan.
3. Implement some form of notification like email notifications that inform users once they have been scanned.
4. Keep a record of what the scans will be done to achieve an analysis in future.

5.4.4 Enhanced Reporting and VAPT Format

Objective: Enhance the quality of reports by producing more detailed and professional-looking VAPT-type reports.

Implementation Plan:

1. Enhance the content of the report with specific details such as vulnerability descriptions, risk levels and recommended actions on vulnerability.
2. Combine scoring with CVSS, enhance the effectiveness of vulnerabilities prioritization.
3. Allow various formats of output like PDF, HTML and JSON to enhance usability of reports.
4. Better formatting and layout of the report to allow it to be more consistent with professional report of VAPT.

5.4.5 Integration of Additional Scanners

Objective: Enhance vulnerability coverage through incorporating more security scanners.

Implementation Plan:

1. Find other scanning tools that could enhance the level of coverage and functionality of the system.
2. Create individual integration modules to relate to new scanners and their results downloaded.
3. Additional scanners can be standardized so that it is easier to process the outcomes of these scanners into one format.
4. Use deduplication techniques to eliminate duplicate results and better accuracy of results.

6.References

(NIST), N. I. o. S. a. T., 2018. *Technical Guide to Information Security Testing and Assessment*, Gaithersburg: s.n.

Alhassan, J. & al., e., 2021. A Comprehensive Analysis of Vulnerability Assessment Tools. *ResearchGate*, 9(5), pp. 11-13.

Alhassan, J. e. a., 2021. A Comprehensive Analysis of Vulnerability Assessment Tools. *Journal of Cybersecurity and Information Systems*, 7(5), pp. 11-13.

ArtOfTesting, 2023. *Waterfall Model in Software Engineering*. [Online]
Available at: <https://artoftesting.com/waterfall-model>
[Accessed 1 March 2026].

Behl, A. & Behl, K., 2017. *Cybersecurity and Cyberwar: What Everyone Needs to Know*. In: Oxford: Oxford University Press.

Buehring, S., 2026. *DSDM principles: A visual guide*. [Online]
Available at: <https://www.knowledgetrain.co.uk/agile/agile-project-management/agile-project-management-course/dsdm-principles?srsId=AfmBOooQ1ZzNqZ2ZfmiBkB87VC30to9M9VQcbDSveU2UhP87RAbP2GA>
[Accessed 1 March 2026].

Consortium, A. B., 2025. *DSDM Project Framework — Process*. [Online]
Available at: <https://www.agilebusiness.org/dsdm-project-framework/process.html>
[Accessed 29 November 2025].

GeeksforGeeks, 2024. *What is RUP(Rational Unified Process) and its Phases?*. [Online]
Available at: <https://www.geeksforgeeks.org/rup-and-its-phases>
[Accessed 29 12 2025].

GeeksforGeeks, 2025. *Agile Software Development Methodology | Framework, Principles, and Benefits*. [Online]
Available at: <https://www.geeksforgeeks.org/agile-software-development-methodology->

framework

[Accessed 22 November 2025].

Geeksforgeeks, 2025. *Prototyping Model - Software Engineering*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-prototyping-model/>

[Accessed 1 March 2026].

GeeksforGeeks, 2025. *Prototyping Model Software Engineering*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering-prototyping-model>

[Accessed 23 November 2025].

GeeksforGeeks, 2025. *What is Spiral Model in Software Engineering?*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-spiral-model/>

[Accessed 30 November 2025].

Hashemi-Pour, C., 2025. *What is the Spiral Model and How Is It Used? | Definition from TechTarget*. [Online]

Available at: <https://www.techtarget.com/searchsoftwarequality/definition/spiral-model>

[Accessed 28 November 2025].

Kirvan, P., 2024. *What is a Waterfall model? Definition and guide*. [Online]

Available at: <https://www.techtarget.com/searchsoftwarequality/definition/waterfall-model>

[Accessed 26 November 2025].

Laoyan, S., 2025. *What is Agile methodology? (A beginner's guide)*. [Online]

Available at: <https://asana.com/resources/agile-methodology>

[Accessed 30 November 2025].

LizHector, 2025. *What is Rational Unified Process?*. [Online]

Available at: <https://projectskillsmentor.com/project-skills/how-rup-works>

[Accessed 29 November 2025].

Nepal, G. o., 2006. *Electronic Transactions Act, 2063*, Government of Nepal: Kathmandu.

Nvisia, 2020. *The Agile Process 101: Understanding the Benefits of Using Agile Methodology*. [Online]
Available at: <https://www.nvisia.com/insights/agile-methodology>
[Accessed 1 March 2025].

ResearchGate, 2022. *Phases of the RUP Methodology..* [Online]
Available at: https://www.researchgate.net/figure/Phases-of-the-RUP-Methodology_fig1_362504608
[Accessed 1 March 2026].

Reyes, J. F. W. A. P. a. M. M., 2022. An environment-specific prioritization model for information-security vulnerabilities. *Electronics*, 11(9), p. 1334.

SANS, 2022. *Effective Practices for Vulnerability Reporting and Analyst Efficiency*, s.l.: SANS Institute.

Scarfone, K. & M. P., 2007. *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST Publication: s.n.

Scarfone, K. a. M. P., 2007. *uide to Intrusion Detection and Prevention Systems (IDPS)*, s.l.: NIST Special Publication.

Sharma., M., 2025. *Prototyping Model in SDLC*. [Online]
Available at: <https://www.includehelp.com/basics/the-prototyping-model-software-engineering.aspx>
[Accessed 30 November 2025].

Sharma, S. & al., e., 2020. Vulnerability Assessment Techniques and Tools. *International Journal of Computer Science and Information Technologies (IJCSIT)*, 12(4), pp. 5-10.

Sharma, S. e. a., 2020. Vulnerability Assessment Techniques and Tools. *International Journal of Computer Science and Information Technologies (IJCSIT)*, 11(3), pp. 1-5.

Sommestad, T., Ekstedt, M. & Holm, H., 2013. The Cyber Security Modeling Language. *IEEE Security & Privacy*, 11(3), pp. 12-19.

Sovereign, 2025. *Spiral Model Methodology and its Phases*. [Online]
Available at: <https://www.sovereignconsult.com/blog/spiral-model-methodology/>
[Accessed 1 March 2026].

Stallings, W., 2018. Network Security Essentials: Applications and Standards. In: Harlow: Pearson.

Study.com, 2025. *Agile DSDM: Methodology & Project Framework*. [Online]
Available at: <https://study.com/academy/lesson/agile-dsdm-methodology-project-framework.html>
[Accessed March 2026].

TutorialsCampus, 2025. *Waterfall Model in SDLC*. [Online]
Available at: <https://www.tutorialscampus.com/sdlc/waterfall-model.htm>
[Accessed 22 November 2025].

MITRE, 2025. *CVE Program: Vulnerability Data, MITRE Corporation*. [Online]
Available at: <https://cve.mitre.org/>
[Accessed 20 March 2026].

Networks, G., 2023. *Greenbone Vulnerability Management Documentation*. [Online]
[Accessed 20 March 2026].

NIST, 2018. *Importance of Vulnerability Assessment & Continuous Scanning : Technical guide to information security testing and assessment*. [Online]
Available at:
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800115.pdf>
[Accessed March 2026].

NIST, 2018. *Technical guide to information security testing and assessment*. [Online]

Available at:

<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800115.pdf>

[Accessed March 2026].

Pressman, R. & M. B., 2020. *Software Engineering: A Practitioner's Approach*. [Online]

Available at: <https://www.mheducation.com/highered/product/software-engineeringpractitioner-s-approach-pressman-maxim/M9781265862979.html> [Accessed 20 March 2026].

Sitesbay, 2025. *Sitesbay: What is Spiral Model*. [Online]

Available at: <https://www.sitesbay.com/software-engineering/se-spiral-model> [Accessed March 2026].

Tenable, 2025. *Nessus Vulnerability Scanner*. [Online]

Available at: <https://www.sans.org/security-resources/glossary-of-terms/vulnerabilityscanning/>

[Accessed March 2026].

TutorialsCampus, 2025. *Waterfall Model in SDLC*. [Online]

Available at: <https://www.tutorialscampus.com/sdlc/waterfall-model.htm> [Accessed 22 March 2026].

Edgescan, 2025. *Vulnerability Statistics Report*. [Online]

Available at: <https://www.edgescan.com/resources/vulnerability-stats-report/> [Accessed 1 April 2026].

GeeksforGeeks, 2024. *What is RUP(Rational Unified Process) and its Phases?*. [Online]

Available at: <https://www.geeksforgeeks.org/rup-and-its-phases> [Accessed 1 April 2026].

GeeksforGeeks, 2025. *Agile Software Development Methodology | Framework, Principles, and Benefits*. [Online]

Available at: <https://www.geeksforgeeks.org/agile-software-development-methodology->

framework

[Accessed 1 April 2026].

GeeksforGeeks, 2025. *Dynamic Systems Development Method (DSDM)*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/dynamic-systems-development-method-dsdm/>

[Accessed 1 April 2026].

GeeksforGeeks, 2025. *Prototyping Model Software Engineering*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering-prototyping-model>
[Accessed 1 April 2026].

GeeksforGeeks, 2025. *What is Spiral Model in Software Engineering?*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-spiral-model/>
[Accessed 5 April 2026].

Hashemi-Pour, C., 2025. *What is the Spiral Model and How Is It Used? | Definition from TechTarget*. [Online]
Available at: <https://www.techtarget.com/searchsoftwarequality/definition/spiral-model>
[Accessed 5 April 2026].

IBM Security, 2023. *IBM X-Force Threat Intelligence Index 2023*. [Online]
Available at: <https://www.ibm.com/reports/threat-intelligence>
[Accessed 9 April 2026].

Kirvan, P., 2024. *What is a Waterfall model? Definition and guide*. [Online]
Available at: <https://www.techtarget.com/searchsoftwarequality/definition/waterfall-model>
[Accessed 10 April 2026].

Laoyan, S., 2025. *What is Agile methodology? (A beginner's guide)*. [Online]
Available at: <https://asana.com/resources/agile-methodology>
[Accessed 13 April 2026].

LizHector, 2025. *What is Rational Unified Process?*. [Online]

Available at: <https://projectskillsmentor.com/project-skills/how-rup-works>

[Accessed 12 April 2026].

Lvivity, 2025. *SDLC Overview*. [Online]

Available at: <https://lvivity.com/sdlc-overview>

[Accessed 12 April 2026].

Pressman, R. & M. B., 2020. *Software Engineering: A Practitioner's Approach*. [Online]

Available at: <https://www.mheducation.com/highered/product/software-engineering-practitioner-s-approach-pressman-maxim/M9781265862979.html>

[Accessed 12 April 2026].

Qualys, 2022. *Qualys Research Report on Configuration Risks*. [Online]

Available at: <https://www.qualys.com/research/configuration-security-report>

[Accessed 12 April 2026].

Reyes, J., Fuertes, W., Arévalo, P. & Macas, M., 2022. *An Environment-Specific Prioritization Model for Information-Security Vulnerabilities Based on Risk Factor Analysis*. [Online]

Available at: <https://www.mdpi.com/2079-9292/11/9/1334>

[Accessed 19 April 2026].

SANS Institute, 2022. *Effective Practices for Vulnerability Reporting and Analyst Efficiency*. [Online]

Available at: <https://www.sans.org/white-papers/vulnerability-reporting>

[Accessed 17 April 2026].

Sharma., M., 2025. *Prototyping Model in SDLC*. [Online]

Available at: <https://www.includehelp.com/basics/the-prototyping-model-software-engineering.aspx>

[Accessed 3 April 2026].

Tenable, 2025. *Risk-Based Vulnerability Management Principles*. [Online]

Available at: <https://www.tenable.com/principles/risk-based-vulnerability-management->

principles

[Accessed 19 April 2026].

TutorialsCampus, 2025. *Waterfall Model in SDLC*. [Online]

Available at: <https://www.tutorialscampus.com/sdlc/waterfall-model.htm>

[Accessed 12 April 2026].

Vit Lyoshin, 2025. *Dynamic Systems Development Method (DSDM)*. [Online]

Available at: <https://vitlyoshin.com/blog/dynamic-systems-development-method/>

[Accessed 12 April 2026].

7. Appendix

7.1 Background

The additional and supporting information related to the development of the ThreatQuill Analyzer system. It is included to present detailed explanations, technical descriptions, and background concepts that complement the main report. While the main report focuses on the overall design, implementation, and results, the appendix offers extended information that helps in understanding the system more clearly.

This section includes detailed discussions on concepts such as vulnerability assessment, system design approaches, tools and technologies used, and methodology applied during development. It also provides supporting explanations that justify the design decisions and implementation strategies used in the project.

The appendix plays an important role in enhancing the clarity and completeness of the report. It ensures that all technical aspects of the system are properly documented and can be easily understood by readers, evaluators, and other stakeholders. Additionally, it helps in validating the effectiveness of the ThreatQuill Analyzer by providing deeper insights into how the system was developed and how it operates.

7.1.1 Literature Review

Network vulnerability assessment is a key area in cybersecurity that focuses on identifying weaknesses in network systems before they can be exploited by attackers. With the increasing number of cyber threats, organizations are required to regularly assess their systems to detect vulnerabilities such as open ports, outdated software, weak configurations, and insecure services. These vulnerabilities can lead to serious security breaches if not identified and resolved in time.

Various vulnerability scanning tools have been developed to assist in this process. Tools such as Nmap are commonly used for network scanning and port detection, allowing users to identify active hosts and services running on a system. Nikto is widely used for web server scanning, helping to detect misconfigurations and known vulnerabilities in web applications. OpenVAS and Nessus are advanced

vulnerability scanners that provide detailed analysis and reporting of security issues.

Although these tools are effective individually, research shows that relying on a single scanner may not provide complete and accurate results. Different tools use different scanning techniques and databases, which can result in variations in detected vulnerabilities. Additionally, the outputs generated by these tools are often complex and require manual analysis, which can be time-consuming and difficult for users to interpret.

Another challenge identified in the literature is the issue of false positives and false negatives in vulnerability scanning. Some tools may report vulnerabilities that do not actually exist, while others may fail to detect certain security issues. This affects the reliability of vulnerability assessments and makes it difficult for users to prioritize risks effectively.

To address these limitations, there is a growing need for integrated vulnerability assessment systems that combine multiple scanning tools and automate the process of result analysis. Such systems can improve accuracy, reduce manual effort, and provide more meaningful insights into security risks.

The ThreatQuill Analyzer is developed based on these concepts, aiming to integrate multiple scanners into a single system and provide a unified, structured, and user-friendly approach to vulnerability assessment. By automating scanning, analysis, and reporting, the system enhances efficiency and helps users better understand and manage security risks.

7.2 Appendix A: Pre-survey

7.2.1 Pre-Survey Form

ThreatQuillAnalyzer – Pre-Survey Form

This survey is part of the early development of **ThreatQuillAnalyzer**, a tool designed to find and assess vulnerabilities in networks, systems, and web servers. As cyber threats continue to grow, there is a greater need for tools that help users identify security weaknesses and understand risks before they can be exploited.

The survey aims to gather information about how users currently perform vulnerability assessments, the challenges they face, and what they expect from security tools. The feedback will help design **ThreatQuillAnalyzer** to be practical, easy to use, and useful for both technical and non-technical users.

This survey would take about 5–7 minutes.

* Indicates required question

Email *

Your answer

1. What best describes your current role? *

☐ Student

☐ Cybersecurity & Networking Field

☐ IT Professional

☐ Other

2. Do you believe weak network security can lead to serious data or system damage? *

☐ Yes

☐ No

☐ Maybe

3. Have you ever been in a situation where network security was a concern or risk? *

- ☐ Yes
- ☐ No

4. Have you ever used any security-related tool? *

- ☐ Yes
- ☐ No

5. Regular vulnerability scanning helps identify security weaknesses in a network. Is this statement correct? *

- ☐ True
- ☐ False

6. Why do you think network security is important? *

- ☐ To protect data and systems
- ☐ To avoid hacking and cyber attacks
- ☐ It is required by organizations
- ☐ I am not sure why it is important

7. Is it important to prioritize high-risk vulnerabilities first? *

- ☐ Yes
- ☐ No

8. What do you find most challenging about using network security tools? *

- ☐ Understanding technical terms
- ☐ Interpreting scan results
- ☐ Time and effort required
- ☐ I have no experience with such tools

9. Do you think security tools can help reduce cyber attacks? *

- ☐ Yes
- ☐ No

10. How often should an organization check its network for vulnerabilities? *

- ☐ Regularly (weekly/monthly)
- ☐ Only when a problem occurs
- ☐ Only after a security incident

11. Which outcome is most important when ThreatQuillAnalyzer is used? *

- ☐ Identifying all vulnerabilities
- ☐ Prioritizing the most serious risks
- ☐ Providing clear instructions to fix issues
- ☐ Reducing effort and errors in security management

12. If ThreatQuillAnalyzer generates a report after scanning, which format would you prefer? *

- ☐ Simple summary of critical issues
- ☐ Detailed technical report with all findings
- ☐ Visual charts and graphs showing risk levels
- ☐ Step-by-step instructions to fix issues

13. Which feature would make ThreatQuillAnalyzer most useful for you? *

- ☐ Fast scanning for vulnerabilities
- ☐ Easy-to-understand reports
- ☐ Clear instructions to fix problems
- ☐ Rank vulnerabilities by risk level

14. How important is risk prioritization in a network vulnerability tool like ThreatQuillAnalyzer? *

- ☐ Very important
- ☐ Somewhat important
- ☐ Not important

15. What feature or functionality would you most like to see in ThreatQuillAnalyzer? *

Your answer

Submit Clear form

Figure 149: Pre-survey Form

7.2.2 Sample of Filled Pre-Survey Forms

Responses cannot be edited

ThreatQuillAnalyzer – Pre-Survey Form

This survey is part of the early development of **ThreatQuillAnalyzer**, a tool designed to find and assess vulnerabilities in networks, systems, and web servers. As cyber threats continue to grow, there is a greater need for tools that help users identify security weaknesses and understand risks before they can be exploited.

The survey aims to gather information about how users currently perform vulnerability assessments, the challenges they face, and what they expect from security tools. The feedback will help design **ThreatQuillAnalyzer** to be practical, easy to use, and useful for both technical and non-technical users.

This survey would take about 5–7 minutes.

* Indicates required question

Email *

suprina21@gmail.com

1. What best describes your current role? *

☐ Student

☐ Cybersecurity & Networking Field

☒ IT Professional

☐ Other

2. Do you believe weak network security can lead to serious data or system damage? *

☐ Yes

☐ No

☒ Maybe

3. Have you ever been in a situation where network security was a concern or risk? *

☒ Yes

☐ No

4. Have you ever used any security-related tool? *

☒ Yes

☐ No

5. Regular vulnerability scanning helps identify security weaknesses in a network. Is this statement correct? *

☐ True

☒ False

6. Why do you think network security is important? *

☐ To protect data and systems

☐ To avoid hacking and cyber attacks

☒ It is required by organizations

☐ I am not sure why it is important

7. Is it important to prioritize high-risk vulnerabilities first? *

☐ Yes

☒ No

8. What do you find most challenging about using network security tools? *

- ☐ Understanding technical terms
- ☐ Interpreting scan results
- ☐ Time and effort required
- ☒ I have no experience with such tools

9. Do you think security tools can help reduce cyber attacks? *

- ☒ Yes
- ☐ No

10. How often should an organization check its network for vulnerabilities? *

- ☐ Regularly (weekly/monthly)
- ☒ Only when a problem occurs
- ☐ Only after a security incident

11. Which outcome is most important when ThreatQuillAnalyzer is used? *

- ☐ Identifying all vulnerabilities
- ☐ Prioritizing the most serious risks
- ☐ Providing clear instructions to fix issues
- ☒ Reducing effort and errors in security management

12. If ThreatQuillAnalyzer generates a report after scanning, which format would you prefer? *

- ☒ Simple summary of critical issues
- ☐ Detailed technical report with all findings
- ☐ Visual charts and graphs showing risk levels
- ☐ Step-by-step instructions to fix issues

13. Which feature would make ThreatQuillAnalyzer most useful for you? *

- ☐ Fast scanning for vulnerabilities
- ☐ Easy-to-understand reports
- ☒ Clear instructions to fix problems
- ☐ Rank vulnerabilities by risk level

14. How important is risk prioritization in a network vulnerability tool like ThreatQuillAnalyzer? *

- ☒ Very important
- ☐ Somewhat important
- ☐ Not important

15. What feature or functionality would you most like to see in ThreatQuillAnalyzer? *

Automated result

Submitted 12/27/25, 3:59 PM

Figure 150: Pre-survey form Filled

7.3.3 Pre-Survey Result

Most respondents (56.8%) believe that weak network security can lead to serious data or system damage, indicating strong awareness of cybersecurity risks.

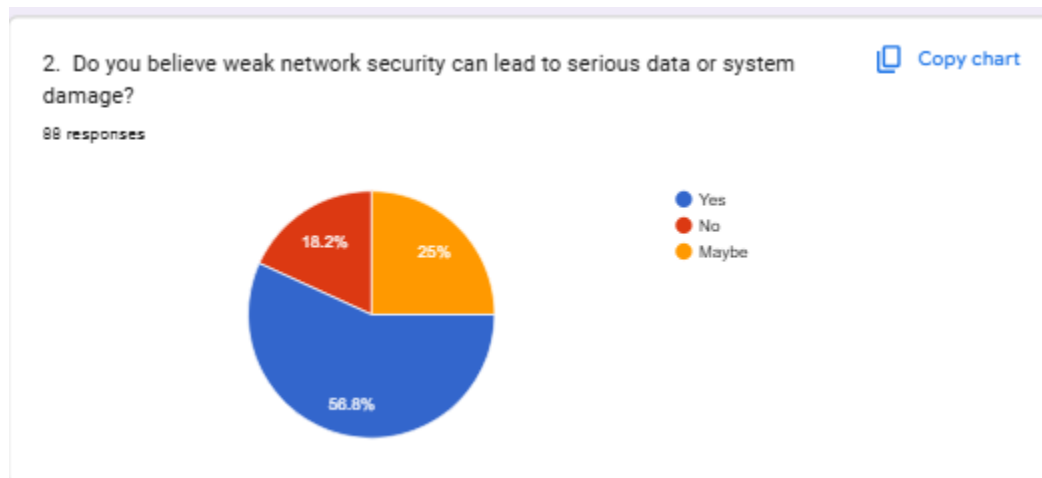


Figure 151: Weak network security impact

A significant portion of respondents (52.3%) have experienced situations where network security was a concern, highlighting real-world exposure to security issues.

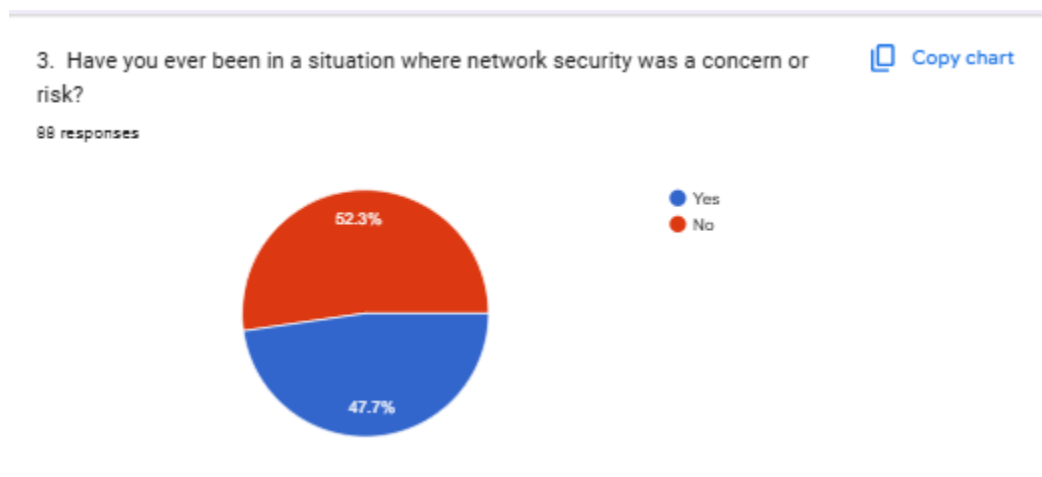


Figure 152: Network security risk experience

Most respondents (64.8%) have used security-related tools, showing a good level of familiarity with cybersecurity practices and technologies.

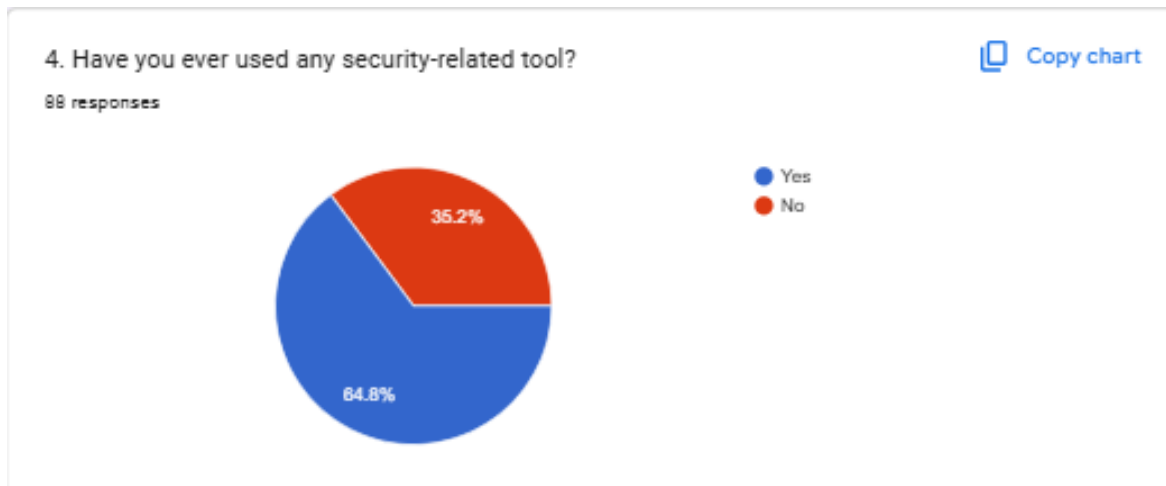


Figure 153: Use of security tools

With 56.8% marking “True” and 43.2% “False,” most respondents agree that regular vulnerability scanning helps in identifying security weaknesses.

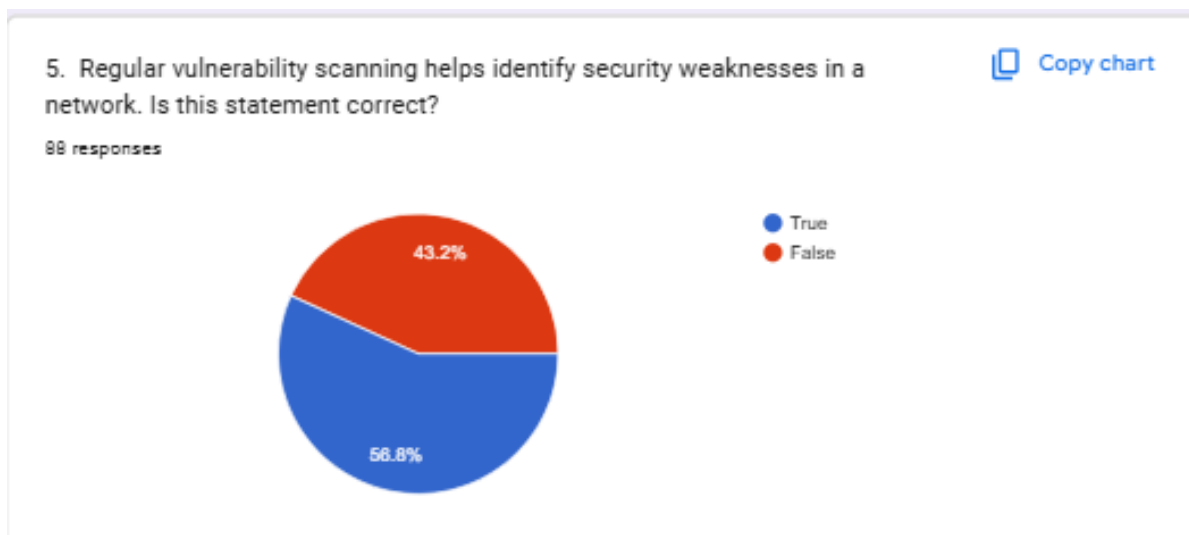


Figure 154: Effectiveness of vulnerability scanning

38.6% believe network security is important to avoid cyber-attacks, 37.5% for protecting data and systems, 19.3% due to organizational requirements, and 13.6% are unsure, showing varied understanding of its importance.

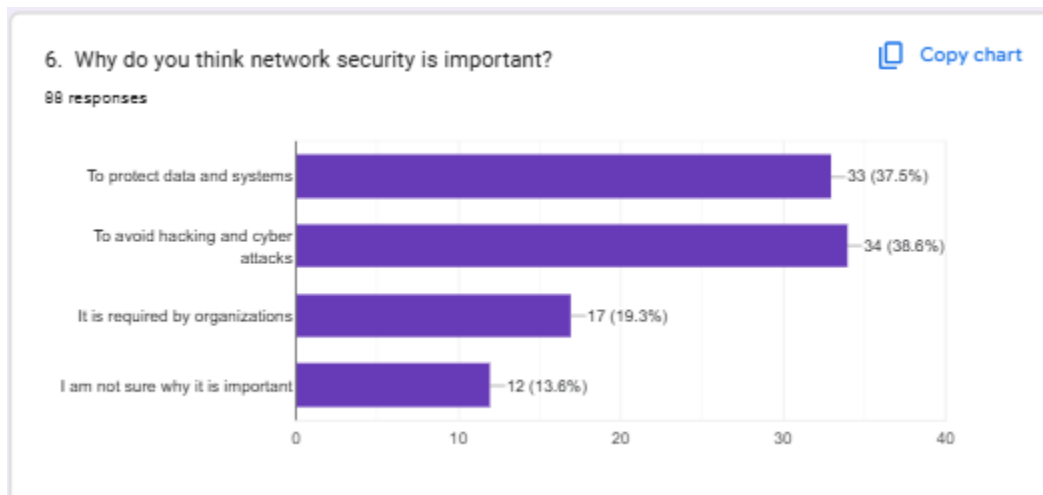


Figure 155: Importance of network security

A total of 59.1% responded “Yes” while 40.9% selected “No,” indicating that more than half support prioritizing high-risk vulnerabilities first.

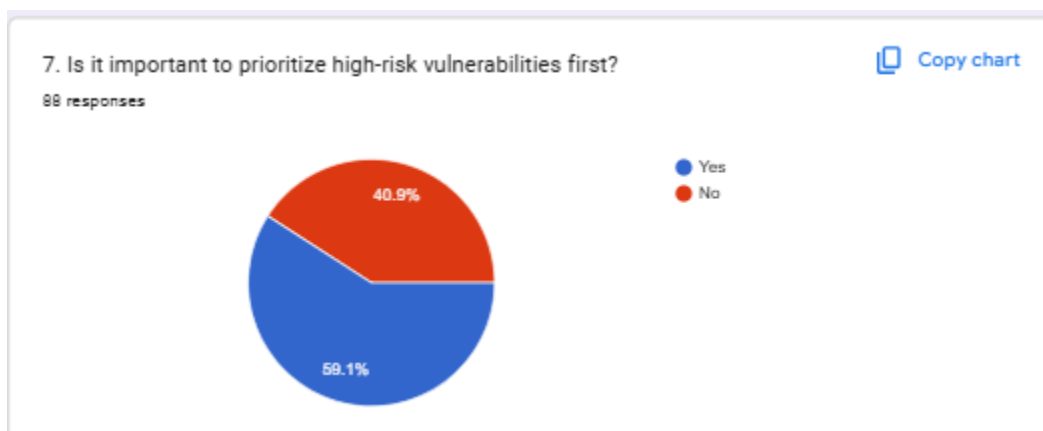


Figure 156: High-risk prioritization

27.3% find understanding technical terms and time consumption challenging, 26.1% lack experience with tools, and 25% struggle with interpreting scan results, highlighting multiple usability challenges.

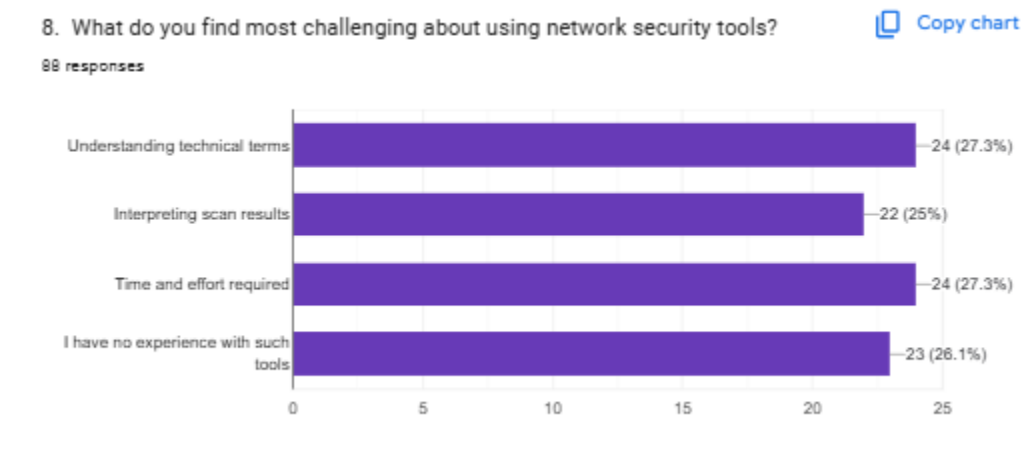


Figure 157: Challenges in using security tools

64.8% believe security tools help reduce cyber-attacks, whereas 35.2% do not, showing overall confidence in security tools.

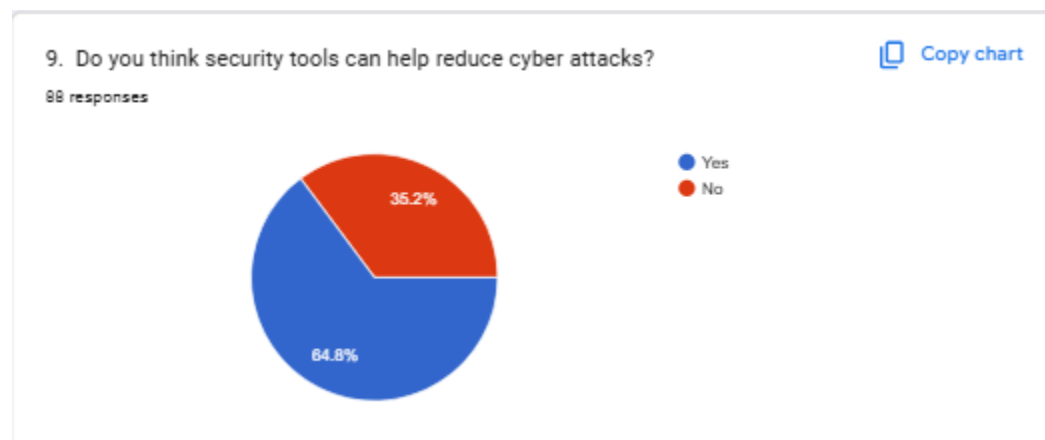


Figure 158: Security tools reducing attacks

54.5% prefer regular vulnerability checks, while 27.3% suggest checking only when problems occur and 18.2% after incidents, indicating a strong preference for proactive security practices.

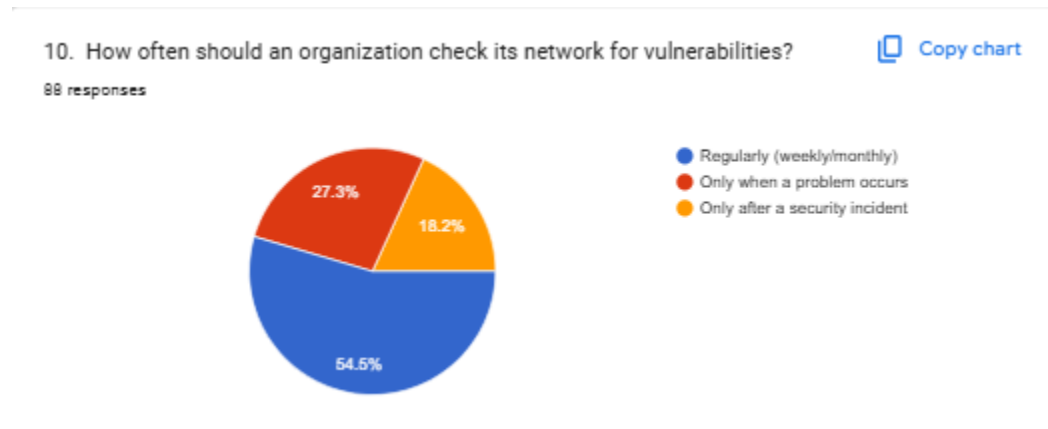


Figure 159: Frequency of vulnerability scanning

37.5% prefer identifying all vulnerabilities, 28.4% focus on prioritizing serious risks, 20.5% want clear fixing instructions, and 19.3% value reduced effort.

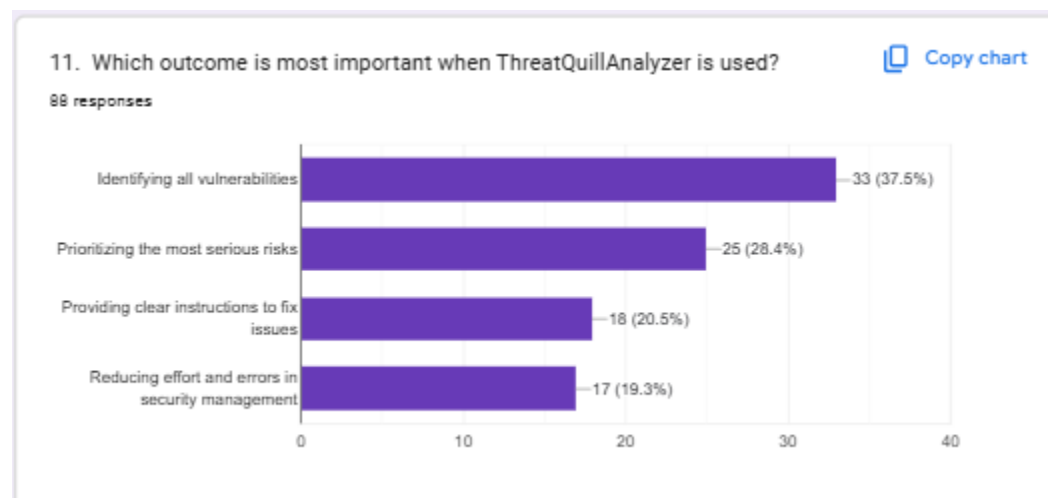


Figure 160: Expected outcomes of system

78.4% prefer a simple summary report, 13.6% want visual charts, while very few prefer detailed reports or step-by-step instructions.

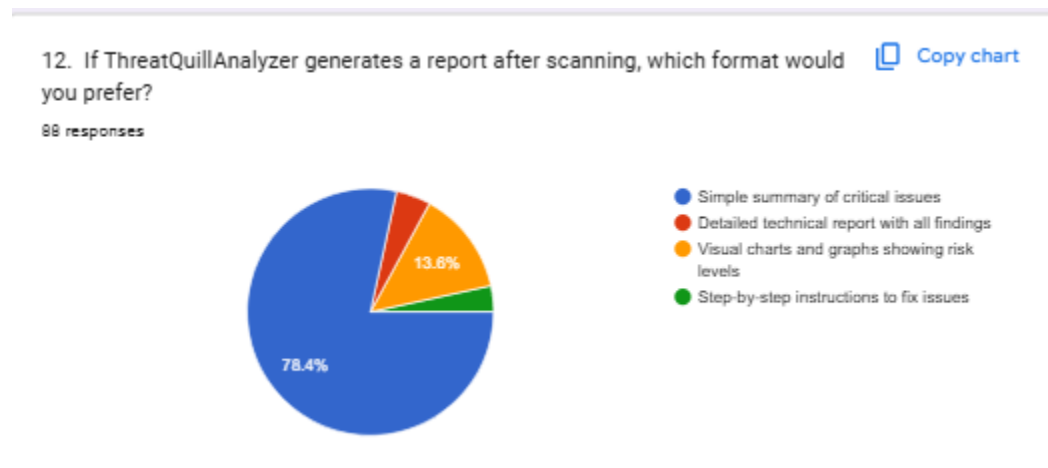


Figure 161: Preferred report format

31.8% prefer fast scanning, 26.1% want easy-to-understand reports, 21.6% want clear instructions, and 20.5% prefer risk-based ranking.

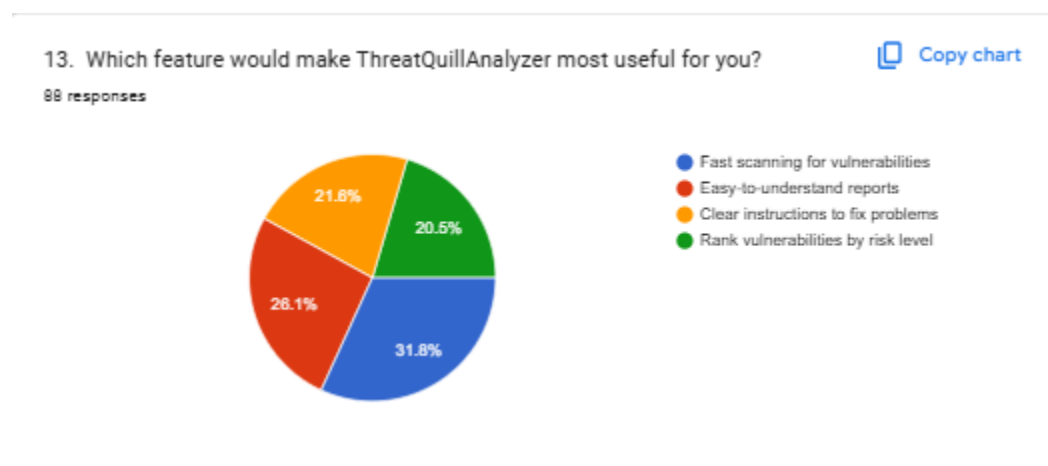


Figure 162: Useful system features

44.3% consider risk prioritization very important, 27.3% somewhat important, and 28.4% not important.

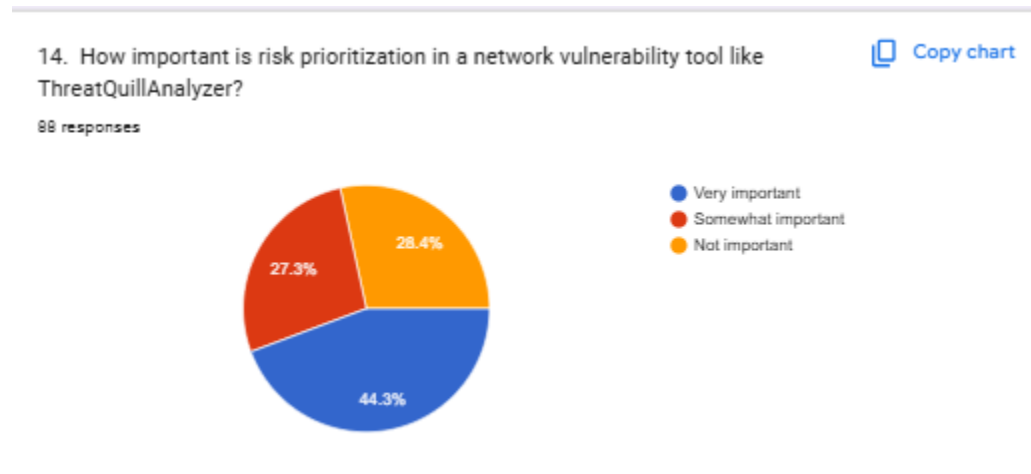


Figure 163: Importance of risk prioritization

Most users prefer features like automated results, easy analysis, risk-based prioritization, real-time monitoring, user-friendly interface, visual graphs, dashboard summary, and detailed reports.

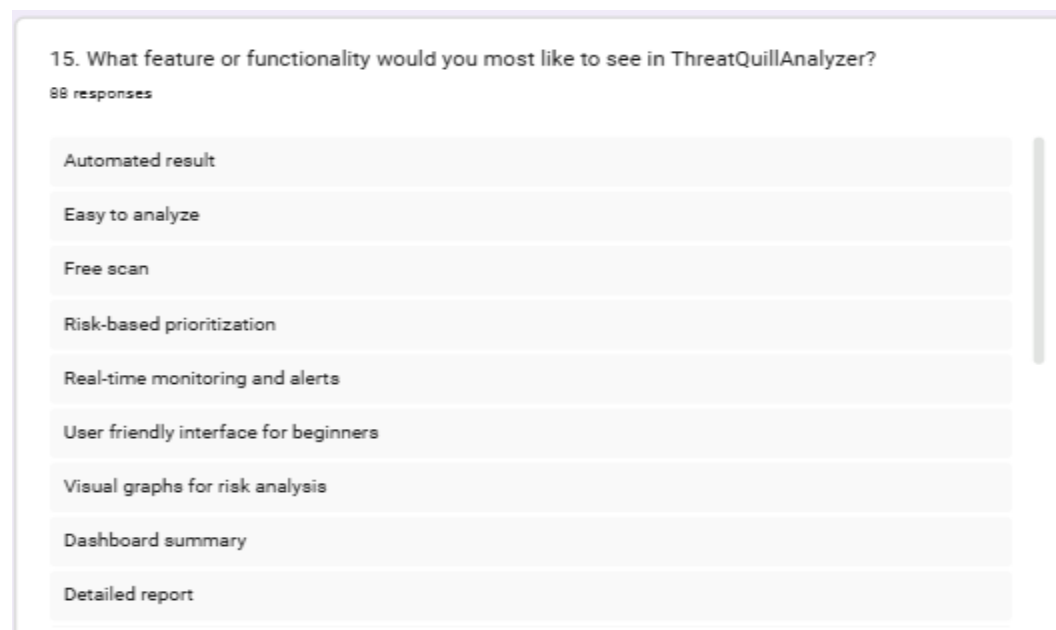


Figure 164: Suggested system improvements

7.3 Appendix B: Post-survey

7.3.1 Post-survey Form

Post Survey Form ThreatQuill Analyzer

This post-survey is conducted to gather user feedback after using the ThreatQuill Analyzer for network vulnerability assessment. It focuses on evaluating the system's usability, scanning performance, result accuracy, and reporting features.

The responses collected will help assess the overall effectiveness of the system and identify key areas for improvement. This feedback will be used to enhance the functionality and usability of the ThreatQuill Analyzer in future development.

binnisthaa@gmail.com [Switch account](#) 

 Not shared

* Indicates required question

Email *

Your answer

1. To what extent does the dashboard handle scan initiation, including target input and execution, effectively? *

☐ With frequent errors or unclear behavior

☐ With minor inconsistencies during setup

☐ Reliably with correct execution flow

☐ Seamlessly with immediate and accurate execution

2. How clear is the visibility of scan progress during execution? *

☐ Lacking real-time feedback

☐ Showing limited or delayed updates

☐ Providing consistent progress indication

☐ Delivering detailed real-time tracking

3. Does the Activity Log provide sufficient information to understand scanner operations? *

- ☐ Providing unclear or missing information
- ☐ Showing partial updates with limited context
- ☐ Presenting clear and useful execution details
- ☐ Offering detailed and real-time operational insights

4. How well are live scan results presented within the dashboard? *

- ☐ Disorganized and difficult to interpret
- ☐ Partially structured with missing details
- ☐ Structured with key scan information
- ☐ Well-organized with complete and consistent data

5. How would you rate navigation across Scan, Summary, and Report modules? *

- ☐ Difficult to manage across sections
- ☐ Functional but lacking smooth transitions
- ☐ Smooth with clear section separation
- ☐ Seamless with intuitive workflow continuity

6. To what extent are multiple scanners effectively integrated within a single workflow? *

- ☐ Unreliable with execution issues
- ☐ Partially integrated with limited coordination
- ☐ Functionally integrated with combined outputs
- ☐ Fully synchronized with unified result handling

7. How stable is the system during multi-scanner execution? *

- ☐ Prone to interruptions or failures
 - ☐ Occasionally unstable under load
 - ☐ Generally stable with minor issues
 - ☐ Fully stable throughout execution
-

8. How would you rate scan execution performance when using multiple tools? *

- ☐ Significantly delayed due to inefficiencies
 - ☐ Moderately slow with noticeable waiting
 - ☐ Acceptable for combined scanning tasks
 - ☐ Efficient with optimized execution time
-

9. To what extent are scan outputs consistent across repeated executions? *

- ☐ Highly inconsistent between runs
 - ☐ Showing occasional variation
 - ☐ Mostly consistent in results
 - ☐ Consistently reliable across scans
-

10. How effectively are duplicate or overlapping findings handled? *

- ☐ Not addressed in results
 - ☐ Minimally handled with repeated entries
 - ☐ Managed with basic filtering
 - ☐ Effectively optimized with clean output
-

11. How accurate are detected services and exposed ports across different scanners? *

- ☐ Frequently inconsistent or missing key detections
- ☐ Partially accurate with noticeable gaps
- ☐ Consistent across scanners with reliable detection
- ☐ Highly precise with detailed service/version identification

12. To what extent are vulnerability findings properly correlated with detected services? *

- ☐ Weak correlation with unrelated or misleading results
- ☐ Limited mapping between services and vulnerabilities
- ☐ Relevant mapping supporting identified exposures
- ☐ Strong correlation with accurate and actionable findings

13. How effective is risk prioritization based on severity levels? *

- ☐ Providing no meaningful prioritization
- ☐ Offering basic classification only
- ☐ Helping identify critical issues
- ☐ Enabling clear and effective prioritization

14. Does severity classification help in understanding system risk? *

- ☐ Failing to clarify risk levels
 - ☐ Offering limited insight
 - ☐ Supporting general understanding
 - ☐ Clearly guiding risk evaluation
-

15. How well are results from multiple scanners consolidated in the report? *

- ☐ Incomplete or poorly combined
- ☐ Partially merged with inconsistencies
- ☐ Properly combined with key details
- ☐ Fully unified with structured findings

16. How clear are vulnerability descriptions in the generated report? *

- ☐ Difficult to interpret or unclear
- ☐ Basic with limited explanation
- ☐ Clear and understandable
- ☐ Detailed with actionable context

17. How useful is the Scan Summary (charts/statistics) for quick analysis? *

- ☐ Not useful for interpretation
- ☐ Providing limited insights
- ☐ Useful for general understanding
- ☐ Highly effective for quick decision-making

18. How would you rate the overall quality of the generated report in a VAPT context? *

- ☐ Not suitable for practical use
- ☐ Suitable only for basic reference
- ☐ Useful for vulnerability assessment
- ☐ Professional and ready for reporting

19. How suitable is ThreatQuill Analyzer for real-world vulnerability assessment? *

☐ Not applicable in real scenarios

☐ Limited to testing environments

☐ Applicable for basic assessments

☐ Suitable for practical security evaluation

20. Which area of improvement would most enhance the system? *

☐ Optimizing scan execution and reducing delays

☐ Enhancing vulnerability detection depth

☐ Improving result analysis and risk prioritization

☐ Upgrading dashboard usability and visualization

21. Please provide any suggestions or improvements you would like to see in the ThreatQuill Analyzer:

Your answer

Submit [Clear form](#)

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms

Figure 165: Post-survey Form

7.3.2 Sample of Filled Post-Survey Forms

Responses cannot be edited

Post Survey Form ThreatQuill Analyzer

This post-survey is conducted to gather user feedback after using the ThreatQuill Analyzer for network vulnerability assessment. It focuses on evaluating the system's usability, scanning performance, result accuracy, and reporting features.

The responses collected will help assess the overall effectiveness of the system and identify key areas for improvement. This feedback will be used to enhance the functionality and usability of the ThreatQuill Analyzer in future development.

* Indicates required question

Email *

ayush7@gmail.com

1. To what extent does the dashboard handle scan initiation, including target input and execution, effectively? *

☐ With frequent errors or unclear behavior

☐ With minor inconsistencies during setup

☒ Reliably with correct execution flow

☐ Seamlessly with immediate and accurate execution

2. How clear is the visibility of scan progress during execution? *

☐ Lacking real-time feedback

☐ Showing limited or delayed updates

☐ Providing consistent progress indication

☒ Delivering detailed real-time tracking

3. Does the Activity Log provide sufficient information to understand scanner operations? *

- ☒ Providing unclear or missing information
- ☐ Showing partial updates with limited context
- ☐ Presenting clear and useful execution details
- ☐ Offering detailed and real-time operational insights

4. How well are live scan results presented within the dashboard? *

- ☐ Disorganized and difficult to interpret
- ☐ Partially structured with missing details
- ☒ Structured with key scan information
- ☐ Well-organized with complete and consistent data

5. How would you rate navigation across Scan, Summary, and Report modules? *

- ☐ Difficult to manage across sections
- ☐ Functional but lacking smooth transitions
- ☐ Smooth with clear section separation
- ☒ Seamless with intuitive workflow continuity

6. To what extent are multiple scanners effectively integrated within a single workflow? *

- ☐ Unreliable with execution issues
- ☐ Partially integrated with limited coordination
- ☐ Functionally integrated with combined outputs
- ☒ Fully synchronized with unified result handling

7. How stable is the system during multi-scanner execution? *

- ☐ Prone to interruptions or failures
- ☐ Occasionally unstable under load
- ☒ Generally stable with minor issues
- ☐ Fully stable throughout execution

8. How would you rate scan execution performance when using multiple tools? *

- ☒ Significantly delayed due to inefficiencies
- ☐ Moderately slow with noticeable waiting
- ☐ Acceptable for combined scanning tasks
- ☐ Efficient with optimized execution time

9. To what extent are scan outputs consistent across repeated executions? *

- ☒ Highly inconsistent between runs
- ☐ Showing occasional variation
- ☐ Mostly consistent in results
- ☐ Consistently reliable across scans

10. How effectively are duplicate or overlapping findings handled? *

- ☐ Not addressed in results
 - ☐ Minimally handled with repeated entries
 - ☐ Managed with basic filtering
 - ☒ Effectively optimized with clean output
-

11. How accurate are detected services and exposed ports across different scanners? *

- ☒ Frequently inconsistent or missing key detections
- ☐ Partially accurate with noticeable gaps
- ☐ Consistent across scanners with reliable detection
- ☐ Highly precise with detailed service/version identification

12. To what extent are vulnerability findings properly correlated with detected services? *

- ☐ Weak correlation with unrelated or misleading results
- ☐ Limited mapping between services and vulnerabilities
- ☒ Relevant mapping supporting identified exposures
- ☐ Strong correlation with accurate and actionable findings

13. How effective is risk prioritization based on severity levels? *

- ☐ Providing no meaningful prioritization
- ☐ Offering basic classification only
- ☒ Helping identify critical issues
- ☐ Enabling clear and effective prioritization

14. Does severity classification help in understanding system risk? *

- ☒ Failing to clarify risk levels
- ☐ Offering limited insight
- ☐ Supporting general understanding
- ☐ Clearly guiding risk evaluation

15. How well are results from multiple scanners consolidated in the report? *

- ☐ Incomplete or poorly combined
- ☐ Partially merged with inconsistencies
- ☒ Properly combined with key details
- ☐ Fully unified with structured findings

16. How clear are vulnerability descriptions in the generated report? *

- ☐ Difficult to interpret or unclear
- ☒ Basic with limited explanation
- ☐ Clear and understandable
- ☐ Detailed with actionable context

17. How useful is the Scan Summary (charts/statistics) for quick analysis? *

- ☐ Not useful for interpretation
- ☐ Providing limited insights
- ☒ Useful for general understanding
- ☐ Highly effective for quick decision-making

18. How would you rate the overall quality of the generated report in a VAPT context? *

- ☐ Not suitable for practical use
- ☐ Suitable only for basic reference
- ☐ Useful for vulnerability assessment
- ☒ Professional and ready for reporting

18. How would you rate the overall quality of the generated report in a VAPT context? *

☐ Not suitable for practical use

☐ Suitable only for basic reference

☐ Useful for vulnerability assessment

☒ Professional and ready for reporting

19. How suitable is ThreatQuill Analyzer for real-world vulnerability assessment? *

☐ Not applicable in real scenarios

☒ Limited to testing environments

☐ Applicable for basic assessments

☐ Suitable for practical security evaluation

20. Which area of improvement would most enhance the system? *

☐ Optimizing scan execution and reducing delays

☐ Enhancing vulnerability detection depth

☐ Improving result analysis and risk prioritization

☒ Upgrading dashboard usability and visualization

21. Please provide any suggestions or improvements you would like to see in the ThreatQuill Analyzer:

Provide detailed vulnerability reports
.....

Submitted 4/18/26, 4:41 PM

Figure 166: Filled Post-survey Form

7.3.3 Post-survey Result

The survey collected responses from 53 participants, providing a diverse set of user feedback.

Email
53 responses
priya31@gmail.con
ayush7@gmail.com
suprina21@gmail.com
ramasth9@gmailcom
prabin21@gmail.com
shantishrestha@gmail.com
pushpaarti246@gmail.com
Chhetrinikita1717@gmail.com

Figure 167: Filled Emails

Most users reported that the dashboard handles scan initiation effectively, with many experiencing smooth and accurate execution.

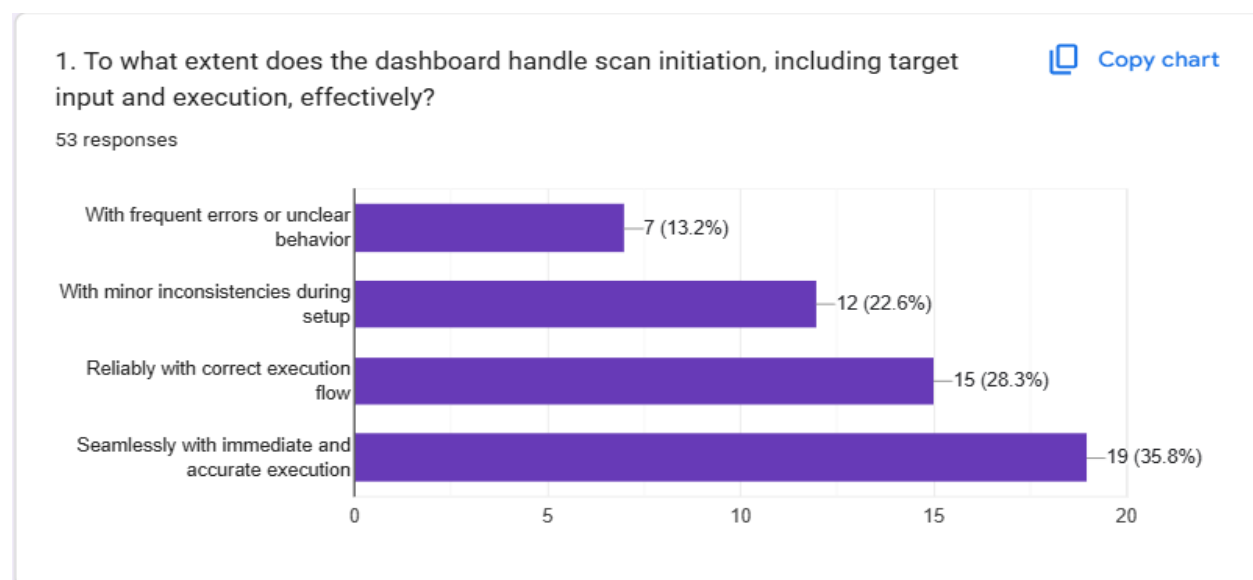


Figure 168: Dashboard scan execution effectiveness

Most users found the scan progress visibility clear, with strong support for real-time tracking.

2. How clear is the visibility of scan progress during execution?

 Copy chart

53 responses

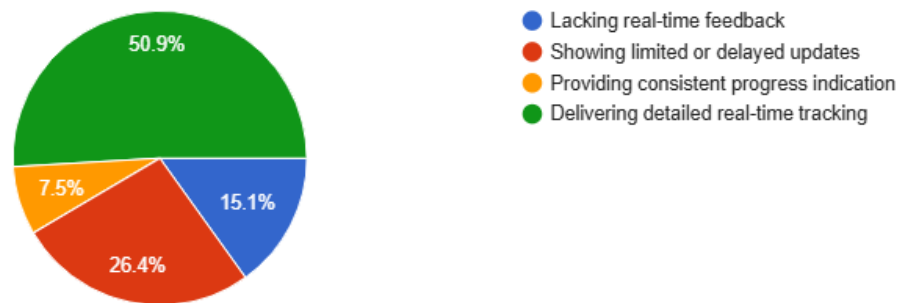


Figure 169: Visibility of scan progress

Most respondents agreed that the Activity Log provides clear and useful information about scanner operations.

3. Does the Activity Log provide sufficient information to understand scanner operations?

 Copy chart

53 responses

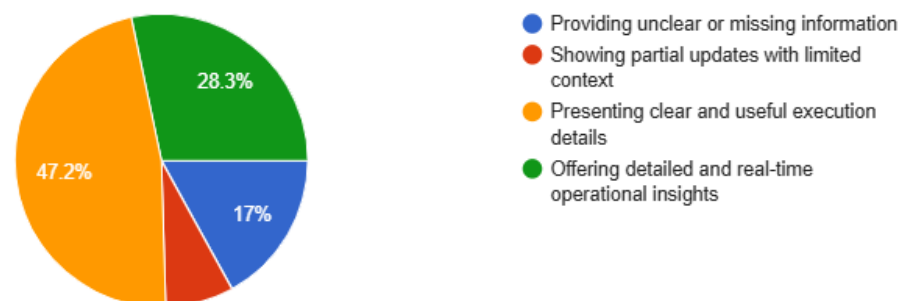


Figure 170: Activity log clarity

Live scan results are generally presented in a structured manner, though some users find them difficult to interpret.

4. How well are live scan results presented within the dashboard?

 Copy chart

53 responses

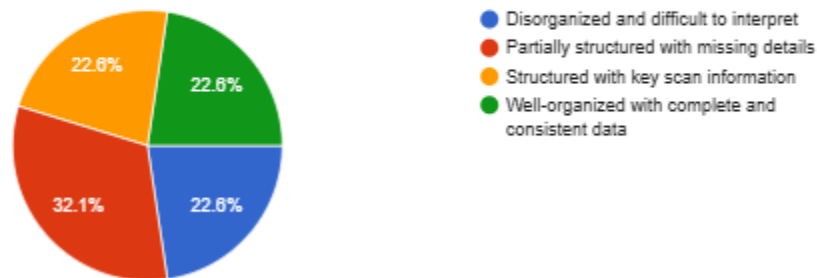


Figure 171: Presentation of scan results

Users showed mixed feedback on navigation, with many finding it intuitive while others noted minor usability issues.

5. How would you rate navigation across Scan, Summary, and Report modules?

 Copy chart

53 responses

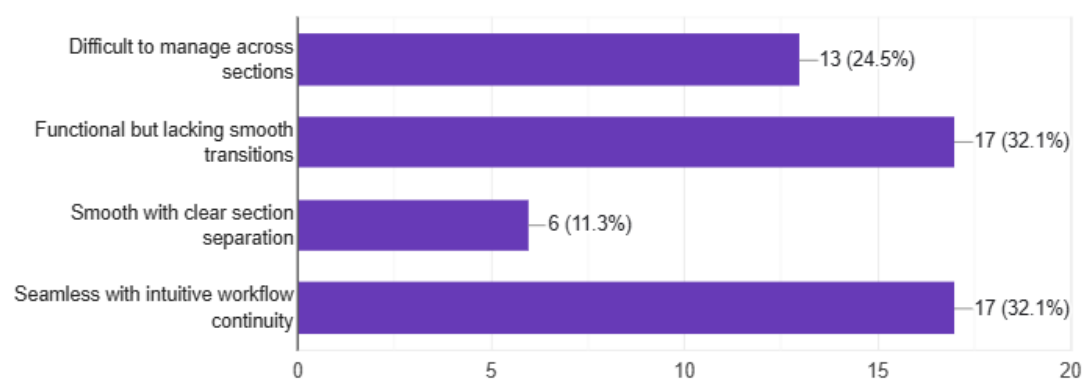


Figure 172: Navigation across modules

Multiple scanners are moderately well integrated, with combined outputs but not always fully synchronized.

6. To what extent are multiple scanners effectively integrated within a single workflow?

 [Copy chart](#)

53 responses

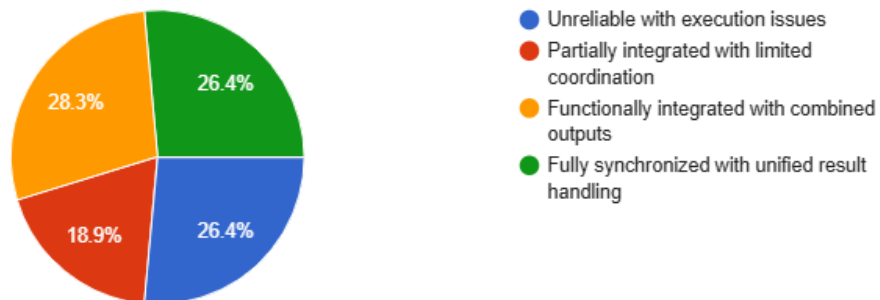


Figure 173: Multi-scanner integration effectiveness:

The system is generally stable during multi-scanner execution, with only occasional instability reported.

7. How stable is the system during multi-scanner execution?

 [Copy chart](#)

53 responses

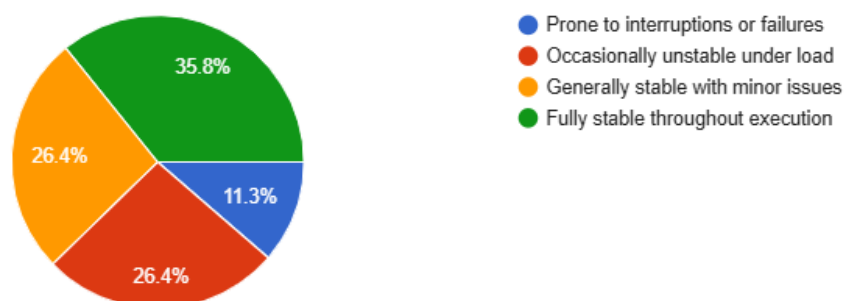


Figure 174: System stability during scans

Scan execution performance is most efficient, although some users experience delays when using multiple tools.

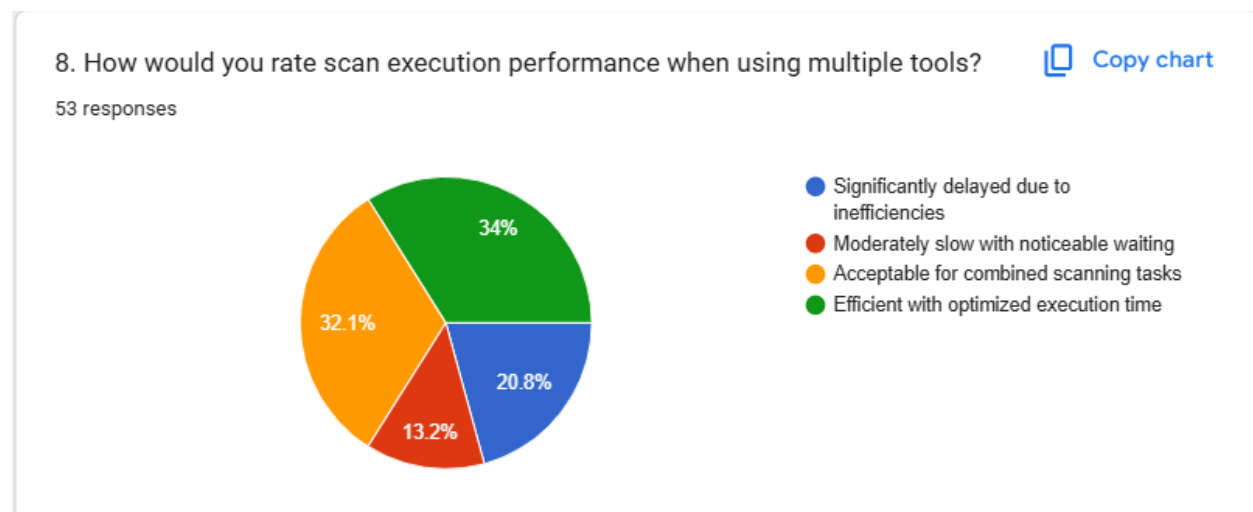


Figure 175: Scan execution performance

Scan outputs show mixed consistency, with some users experiencing variations while others report reliable results.

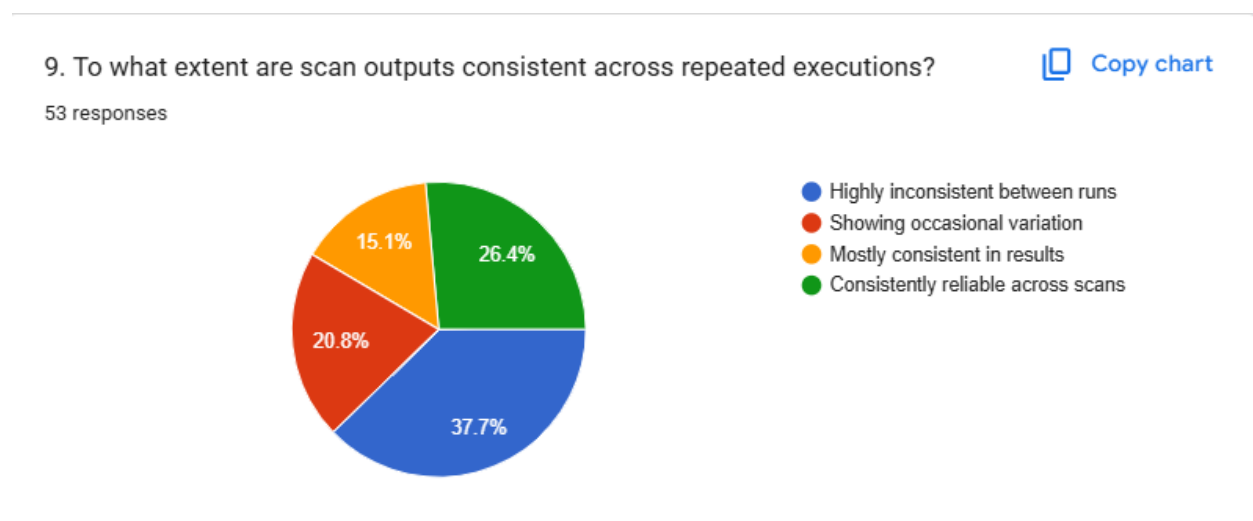


Figure 176: Consistency of scan results

Duplicate findings are handled moderately well, though improvements are needed to ensure cleaner and optimized results.

10. How effectively are duplicate or overlapping findings handled?

 [Copy chart](#)

53 responses

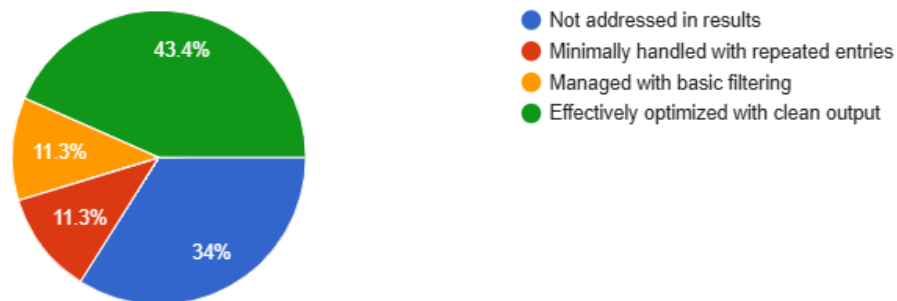


Figure 177: Handling of duplicate findings

Detection accuracy across scanners is generally good, but some inconsistencies and gaps are still observed.

11. How accurate are detected services and exposed ports across different scanners?

 [Copy chart](#)

53 responses

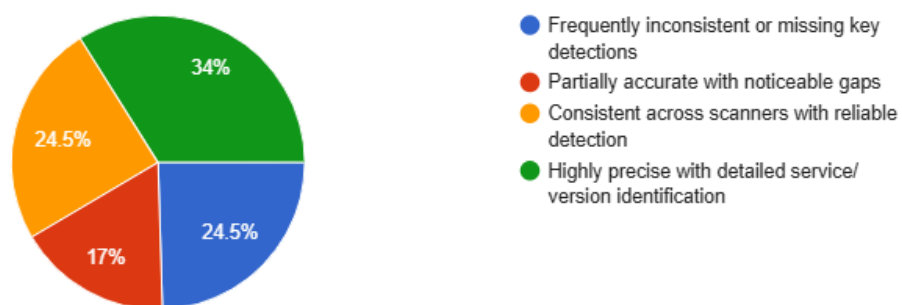


Figure 178: Accuracy of detected services and ports

Most users find a strong correlation between vulnerabilities and detected services, supporting accurate analysis.

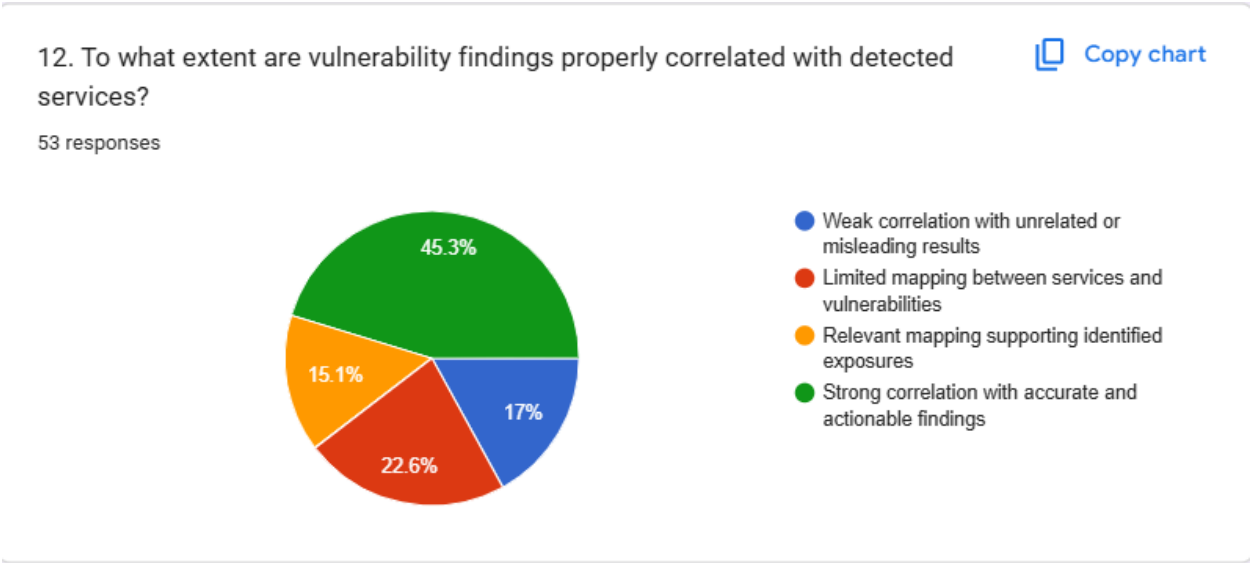


Figure 179: Correlation of vulnerabilities with services

Risk prioritization is considered effective by many users, helping in identifying critical issues clearly.

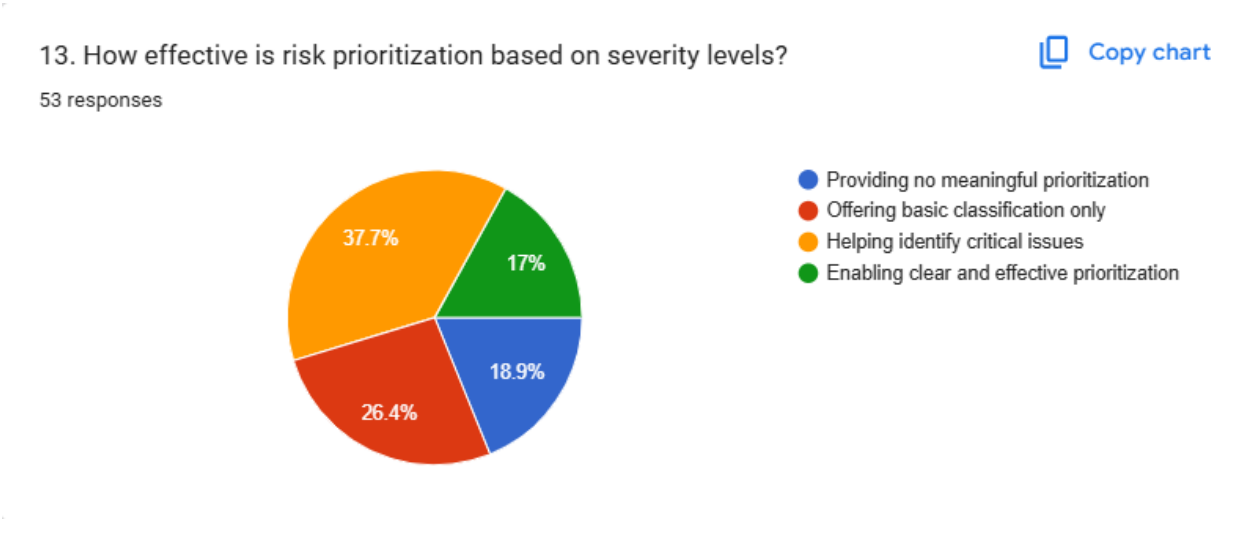


Figure 180: Effectiveness of risk prioritization

Severity classification is useful in understanding system risk, with many users finding it helpful for evaluation.

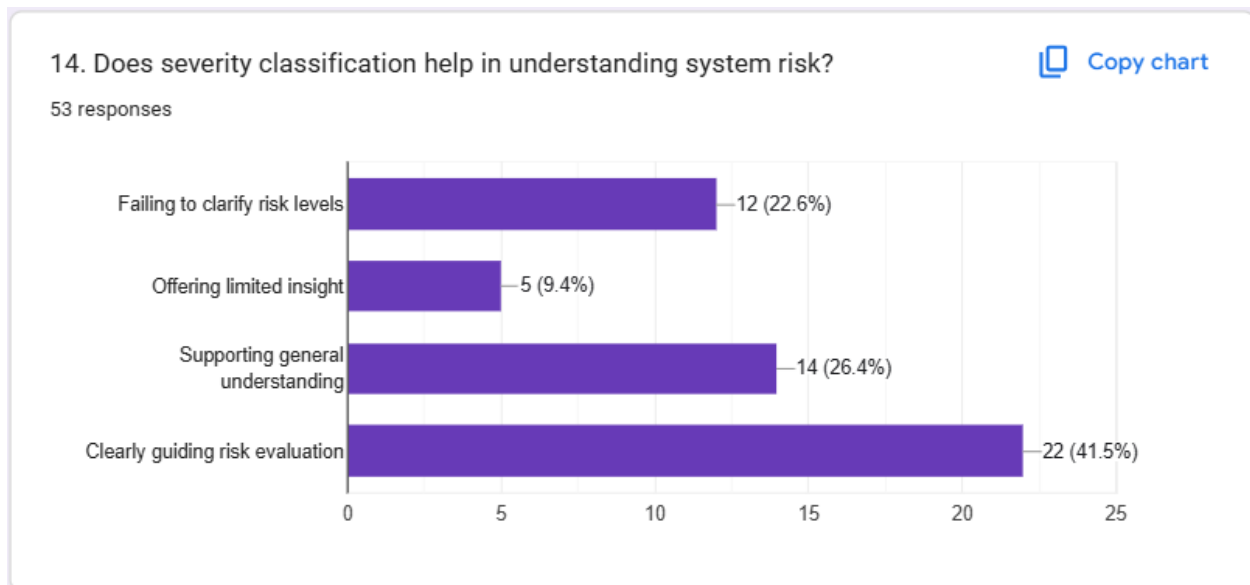


Figure 181: Understanding risk through severity levels

Results from multiple scanners are partially well consolidated, though full integration can be improved.

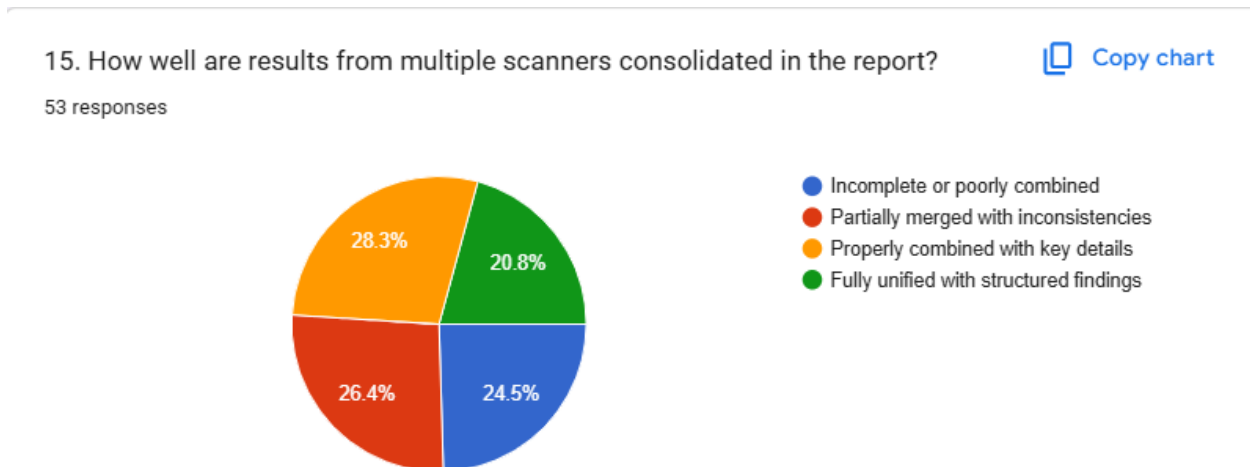


Figure 182: Consolidation of multi-scanner results

Vulnerability descriptions are mostly clear and understandable, with some users expecting more detailed explanations.

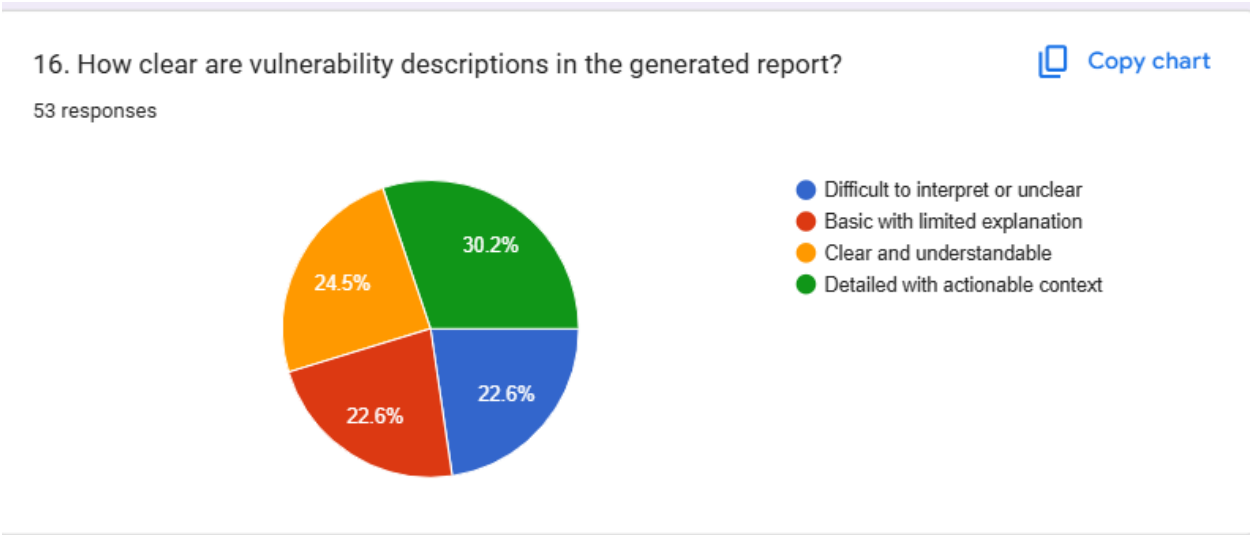


Figure 183: Clarity of vulnerability descriptions

The scan summary is effective for quick analysis, supporting faster understanding and decision-making.

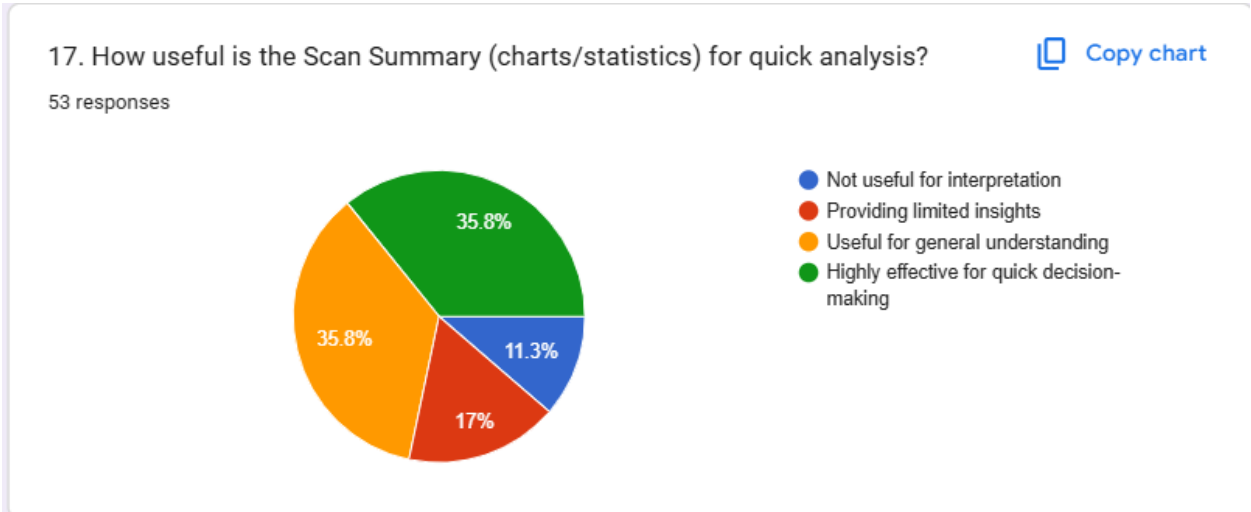


Figure 184: Usefulness of scan summary

The overall report quality is acceptable for assessment, though improvements are needed for professional use.

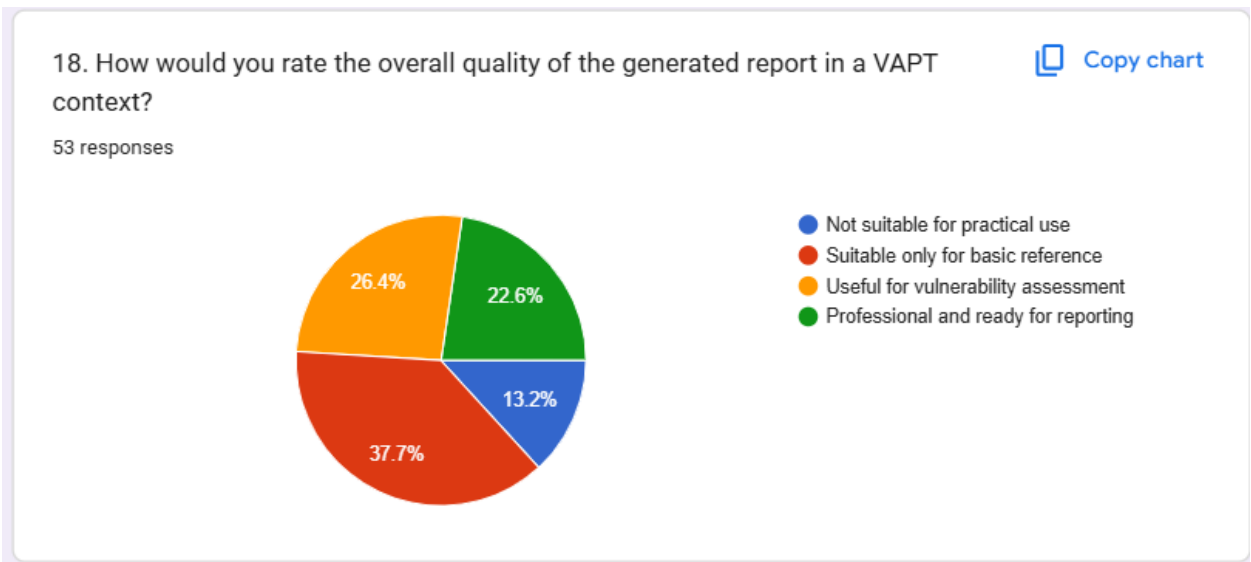


Figure 185: Overall report quality

The system is suitable for basic vulnerability assessment, with limited use in real-world scenarios.

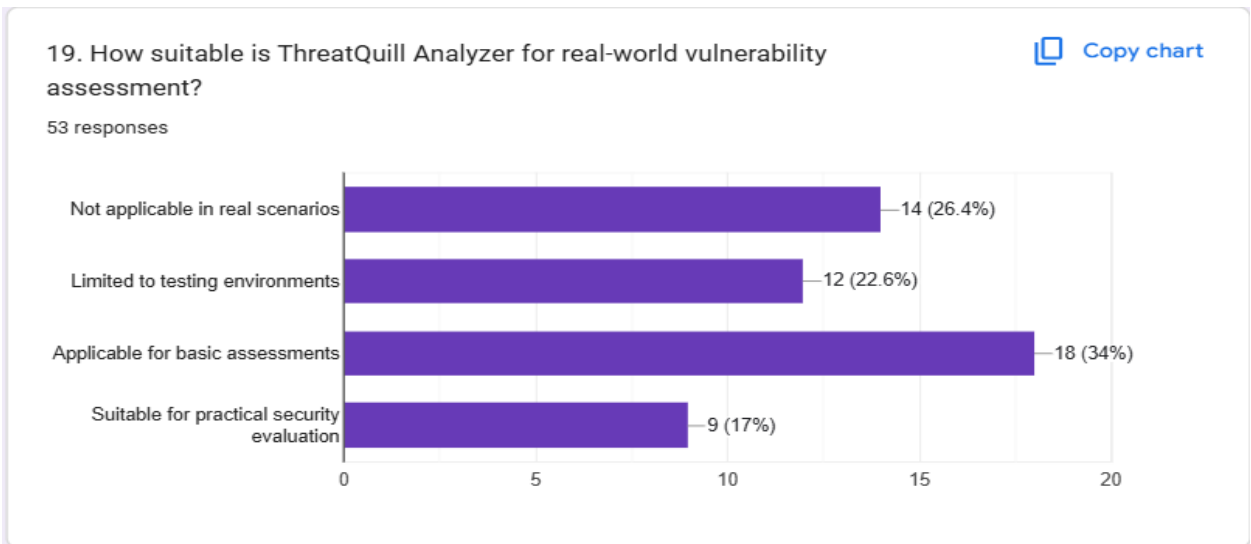


Figure 186: Suitability for real-world use

Users suggest improving performance, detection depth, and dashboard usability to enhance the system.

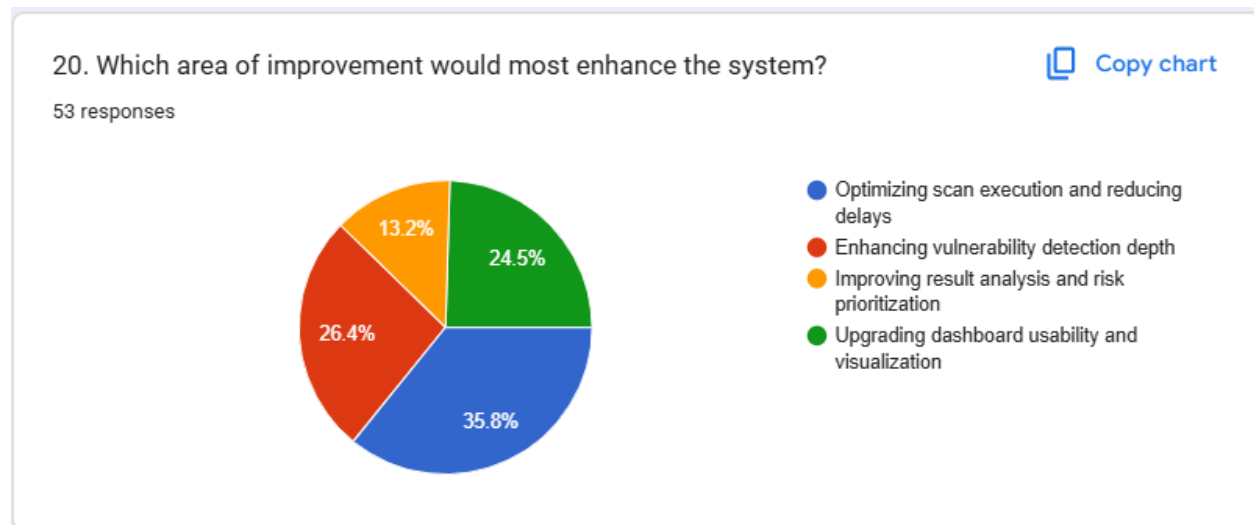


Figure 187: Areas for system improvement

User feedback highlights the need for better filtering, faster scans, improved reporting, and enhanced visualization.

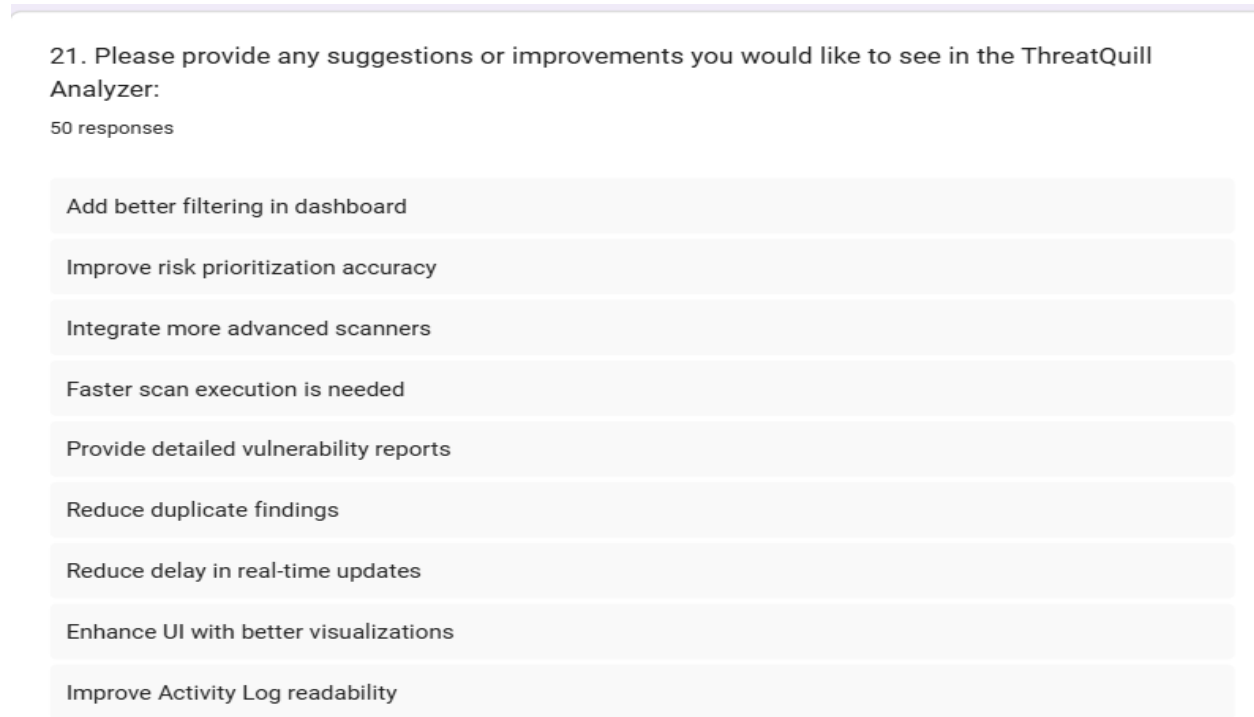


Figure 188: User suggestions and feedback

7.4 Appendix C: Codes

7.4.1 Backend Code

Scanner_controller.py

The scanner_controller.py module acts as the central controller of the system, managing the execution of multiple scanners such as Nmap, Nikto, OpenVAS, and Nessus. It initiates scans based on user input and coordinates the overall scanning process. The module collects and processes results from different scanners and forwards them to the dashboard and report generator. It ensures smooth integration and real-time updating of scan results within the system.

```
scanner_controller.py
1  import threading
2  from scanners.nmap_scanner import run_nmap_live
3  from scanners.nikto_scanner import run_nikto_live
4  from scanners.nessus_scanner import run_nessus_live
5  from scanners.openvas_scanner import run_openvas_live
6  from report_generator import generate_report_data, save_json_report, save_html_report
7
8  all_results = []
9
10
11 def finalize_report(target, dashboard=None):
12     global all_results
13
14     report_data = generate_report_data(target, all_results)
15     json_path = save_json_report(report_data)
16     html_path = save_html_report(report_data)
17
18     if dashboard:
19         dashboard._ui(dashboard.show_report, report_data, html_path)
20         dashboard._ui(
21             dashboard.append_log,
22             f"[SYSTEM] Report generated successfully with {len(all_results)} findings."
23         )
24
25     return report_data, json_path, html_path
26
27
28 def start_full_scan(target, dashboard):
29     global all_results
30     all_results.clear()
31
32     scan_stats = {
33         "hosts": 0,
34         "threats": 0,
35         "findings": 0,
36         "status": "Running"
37     }
38
39     def log_cb(message):
40         dashboard._ui(dashboard.append_log, message)
41
```

```
scanner_controller.py
28 def start_full_scan(target, dashboard):
41
42     def result_cb(result):
43         try:
44             if not isinstance(result, dict):
45                 log_cb("[RESULT ERROR] Invalid result format received.")
46                 return
47
48                 all_results.append(result)
49
50                 scanner = result.get("scanner", "")
51                 host = result.get("host", result.get("target", target))
52                 port = result.get("port", "")
53                 protocol = result.get("protocol", "")
54                 state = result.get("state", "")
55                 service = result.get("service", "")
56
57                 severity = result.get("severity", "")
58                 if not severity:
59                     severity = result.get("risk_level", "Info")
60
61                 dashboard.ui(
62                     dashboard.add_result,
63                     scanner,
64                     host,
65                     str(port),
66                     protocol,
67                     state,
68                     service,
69                     severity
70                 )
71
72                 scan_stats["findings"] = len(all_results)
73
74                 dashboard.ui(
75                     dashboard.update_summary,
76                     scan_stats["hosts"],
77                     scan_stats["threats"],
78                     scan_stats["findings"],
79                     scan_stats["status"]
80                 )
81
82         except Exception as e:
83             log_cb(f"[RESULT ERROR] {e}")
84
```

```
scanner_controller.py
28 def start_full_scan(target, dashboard):

85     def summary_cb(hosts, threats, findings, status):
86         try:
87             if hosts is not None:
88                 scan_stats["hosts"] = max(scan_stats["hosts"], hosts)
89
90             if threats is not None:
91                 scan_stats["threats"] = max(scan_stats["threats"], threats)
92
93             if findings is not None:
94                 scan_stats["findings"] = max(scan_stats["findings"], findings)
95
96             if status:
97                 scan_stats["status"] = status
98
99             dashboard.ui(
100                 dashboard.update_summary,
101                 scan_stats["hosts"],
102                 scan_stats["threats"],
103                 max(scan_stats["findings"], len(all_results)),
104                 scan_stats["status"]
105             )
106
107         except Exception as e:
108             log_cb(f"[SUMMARY ERROR] {e}")
109
110     def run_nmap():
111         try:
112             log_cb("[SYSTEM] Nmap scan started.")
113             run_nmap_live(target, log_cb, result_cb, summary_cb, deep_scan=False)
114             log_cb("[SYSTEM] Nmap scan completed.")
115         except Exception as e:
116             log_cb(f"[NMAP ERROR] {e}")
117
118     def run_nikto():
119         try:
120             log_cb("[SYSTEM] Nikto scan started.")
121             run_nikto_live(target, log_cb, result_cb, summary_cb)
122             log_cb("[SYSTEM] Nikto scan completed.")
123         except Exception as e:
124             log_cb(f"[NIKTO ERROR] {e}")
125
126     def run_nessus():
127         try:
```

```
scanner_controller.py
28 def start_full_scan(target, dashboard):

126     def run_nessus():
127         try:
128             log_cb("[SYSTEM] Nessus scan started.")
129             run_nessus_live(target, log_cb, result_cb, summary_cb)
130             log_cb("[SYSTEM] Nessus scan completed.")
131         except Exception as e:
132             log_cb(f"[NESSUS ERROR] {e}")
133
134     def run_openvas():
135         try:
136             log_cb("[SYSTEM] OpenVAS scan started.")
137             run_openvas_live(target, log_cb, result_cb, summary_cb)
138             log_cb("[SYSTEM] OpenVAS scan completed.")
139         except Exception as e:
140             log_cb(f"[OPENVAS ERROR] {e}")
141
142     def run_all():
143         try:
144             log_cb("[SYSTEM] Full scan process started.")
145             dashboard._ui(dashboard.update_summary, 0, 0, 0, "Running")
146
147             threads = [
148                 threading.Thread(target=run_nmap, daemon=True),
149                 threading.Thread(target=run_nikto, daemon=True),
150                 threading.Thread(target=run_nessus, daemon=True),
151                 threading.Thread(target=run_openvas, daemon=True)
152             ]
153
154             for thread in threads:
155                 thread.start()
156
157             for thread in threads:
158                 thread.join()
159
160             log_cb("[SYSTEM] All scans completed.")
161             log_cb(f"[SYSTEM] Total collected results: {len(all_results)}")
162
163             scan_stats["findings"] = len(all_results)
164             scan_stats["hosts"] = max(scan_stats["hosts"], 1 if target else 0)
165
166             if len(all_results) > 0:
167                 scan_stats["status"] = "Complete"
168             else:
```



```

scanner_controller.py
28  def start_full_scan(target, dashboard):
142  def run_all():
143      try:
165          if len(all_results) > 0:
166              scan_stats["status"] = "Complete"
167          else:
168              scan_stats["status"] = "No Results"
170
171          dashboard._ui(
172              dashboard.update_summary,
173              scan_stats["hosts"],
174              scan_stats["threats"],
175              scan_stats["findings"],
176              scan_stats["status"]
177          )
178
179          try:
180              finalize_report(target, dashboard)
181          except Exception as e:
182              log_cb(f"[REPORT ERROR] {e}")
183
184      except Exception as e:
185          log_cb(f"[SYSTEM ERROR] {e}")
186          dashboard._ui(
187              dashboard.update_summary,
188              scan_stats["hosts"],
189              scan_stats["threats"],
190              len(all_results),
191              "Failed"
192          )
193
194      finally:
195          dashboard._ui(dashboard.scan_finished)
196
197  try:
198      dashboard._ui(dashboard.append_log, "[SYSTEM] start_full_scan called.")
199      threading.Thread(target=run_all, daemon=True).start()
200  except Exception as e:
201      dashboard._ui(dashboard.append_log, f"[START ERROR] {e}")
202      dashboard._ui(dashboard.scan_finished)
203

```

Figure 189: Controls and coordinates execution of all scanners

Nmap_scanner.py

The nmap_scanner.py module is responsible for performing network scanning using the Nmap tool. It executes scan commands and collects information such as open ports, services, and host status. The module processes and parses the scan output into a structured format. The results are then sent to the controller for integration.

```
scanners > nmap_scanner.py
1 import re
2 import subprocess
3 from utils.ssh_helper import run_ssh_command
4
5
6 def get_risk_level(port, state, service=""):
7     """
8     Basic risk indicator for open ports/services.
9     This is not real CVSS severity.
10    """
11    if state != "open":
12        return "Low"
13
14    port = str(port)
15    service = service.lower()
16
17    high_risk_ports = {"21", "23", "25", "110", "135", "137", "138", "139", "445", "1433", "1521", "3306", "3389", "5432", "5900"}
18    medium_risk_ports = {"53", "80", "111", "135", "161", "389", "443", "8080", "8443"}
19
20    if port in high_risk_ports:
21        return "High"
22    if port in medium_risk_ports:
23        return "Medium"
24
25    if "ftp" in service or "telnet" in service or "smb" in service or "rdp" in service:
26        return "High"
27    if "http" in service or "ssl" in service or "https" in service:
28        return "Medium"
29
30    return "Low"
31
32
33 def parse_nmap_line(line, current_host):
34     """
35     Parse live Nmap output line by line.
36     Returns:
37     ("host_update", host)
38     ("port_result", result_dict)
39     ("os_result", result_dict)
40     ("script_result", result_dict)
41     ("info_result", result_dict)
42     None
43     """
44
45    host_match = re.search(r"Nmap scan report for (.?) \([([d\.\-]+)\]", line)
46    if host_match:
47        return ("host_update", host_match.group(2))
48
49    # Example:
50    # Nmap scan report for 192.168.1.10
51    host_match2 = re.search(r"Nmap scan report for ([\w\.\-]+)", line)
52    if host_match2 and (" not in line:
53        return ("host_update", host_match2.group(1))
54
55    # Example:
56    # 80/tcp open http
57    # 443/tcp open ssl/http Apache httpd 2.4.54
58    port_match = re.match(r"^\s*([d+]/[w+]/s+([w+]/s+(.+?))\s*$", line)
59    if port_match:
60        port = port_match.group(1)
61        proto = port_match.group(2)
62        state = port_match.group(3)
63        service = port_match.group(4).strip()
64        risk_level = get_risk_level(port, state, service)
65
66    return
```

```

scanners > nmap_scanner.py
33 def parse_nmap_line(line, current_host):
59     if port_match:
66         result = {
67             "scanner": "NMAP",
68             "host": current_host,
69             "port": port,
70             "protocol": proto,
71             "state": state,
72             "service": service,
73             "risk_level": risk_level,
74             "finding": f"Port {port}/{proto} is {state} ({service})"
75         }
76         return ("port_result", result)
77
78     # OS detection line
79     if line.startswith("OS details:"):
80         os_info = line.replace("OS details:", "").strip()
81         result = {
82             "scanner": "NMAP",
83             "host": current_host,
84             "port": "-",
85             "protocol": "-",
86             "state": "detected",
87             "service": "OS Detection",
88             "risk_level": "Info",
89             "finding": f"Detected OS: {os_info}"
90         }
91         return ("os_result", result)
92
93     # Service Info line
94     if line.startswith("Service Info:"):
95         info = line.replace("Service Info:", "").strip()
96         result = {
97             "scanner": "NMAP",
98             "host": current_host,
99             "port": "-",
100            "protocol": "-",
101            "state": "info",
102            "service": "Service Info",
103            "risk_level": "Info",
104            "finding": f"Service Info: {info}"
105        }
106        return ("info_result", result)
107
108    # NSE script output line
109    # Example:
110    # |_http-title: Go ahead and ScanMe!
111    if line.startswith("|_") or line.startswith("|_"):
112        script_output = line.strip()
113        risk_level = "Medium"
114        lower_line = script_output.lower()
115
116        if any(word in lower_line for word in ["vulnerable", "unsafe", "expired", "default", "cve-", "anonymous"]):
117            risk_level = "High"
118
119        result = {
120            "scanner": "NMAP",
121            "host": current_host,
122            "port": "-",
123            "protocol": "-",
124            "state": "script",
125            "service": "NSE Script",
126            "risk_level": risk_level,
127            "finding": script_output
128        }

```

```

scanners > nmap_scanner.py
33 def parse_nmap_line(line, current_host):
111     if line.startswith("|") or line.startswith("|_"):
129         return ("script_result", result)
130
131     return None
132
133
134 def run_nmap_live(target, log_cb, result_cb, summary_cb, deep_scan=True):
135     current_host = target
136     hosts = set()
137     findings = 0
138     threats = 0
139
140     if deep_scan:
141         cmd = (
142             f"nmap -sS -sV -O --script vuln -T4 --top-ports 1000 "
143             f"--stats-every 5s {target}"
144         )
145     else:
146         cmd = f"nmap -sV -T4 --top-ports 1000 --stats-every 5s {target}"
147
148     log_cb(f"[NMAP] Command: {cmd}")
149     log_cb(f"[NMAP] Starting live scan on {target}")
150
151     try:
152         process = run_ssh_command(cmd)
153     except Exception as e:
154         log_cb(f"[NMAP ERROR] Could not start Nmap scan: {e}")
155         summary_cb(0, 0, 0, "Failed")
156         return
157
158     summary_cb(0, 0, 0, "Running")
159
160     try:
161         for raw in process.stdout:
162             line = raw.strip()
163             if not line:
164                 continue
165
166             log_cb(f"[NMAP] {line}")
167             parsed = parse_nmap_line(line, current_host)
168
169             if parsed:
170                 event_type = parsed[0]
171
172                 if event_type == "host_update":
173                     current_host = parsed[1]
174                     hosts.add(current_host)
175
176                 elif event_type in {"port_result", "os_result", "script_result", "info_result"}:
177                     result = parsed[1]
178                     hosts.add(result["host"])
179
180                     if result["risk_level"] in {"Medium", "High"}:
181                         threats += 1
182
183                     findings += 1
184                     result_cb(result)
185
186                 summary_cb(len(hosts), threats, findings, "Running")
187
188         return_code = process.wait()
189
190         if return_code == 0:
191             log_cb("[NMAP] Scan completed successfully.")

```

```

scanners > nmap_scanner.py
134 def run_nmap_live(target, log_cb, result_cb, summary_cb, deep_scan=True):
160     try:
188         return_code = process.wait()
189
190         if return_code == 0:
191             log_cb("[NMAP] Scan completed successfully.")
192             summary_cb(len(hosts), threats, findings, "Complete")
193         else:
194             log_cb(f"[NMAP] Scan failed with code {return_code}")
195             summary_cb(len(hosts), threats, findings, "Failed")
196
197     except Exception as e:
198         log_cb(f"[NMAP ERROR] Runtime error: {e}")
199         summary_cb(len(hosts), threats, findings, "Failed")
200
201
202 def run_nmap(target, output_file="outputs/nmap.xml", use_ssh=False, deep_scan=True):
203     """
204     Run Nmap and save XML output.
205     If use_ssh=True, it runs on Kali through SSH.
206     Otherwise it runs locally.
207     """
208
209     if deep_scan:
210         cmd_string = f"nmap -sS -sV -O --script vuln -oX {output_file} {target}"
211         cmd_list = ["nmap", "-sS", "-sV", "-O", "--script", "vuln", "-oX", output_file, target]
212     else:
213         cmd_string = f"nmap -sV -oX {output_file} {target}"
214         cmd_list = ["nmap", "-sV", "-oX", output_file, target]
215
216     if use_ssh:
217         process = run_ssh_command(cmd_string)
218         process.wait()
219     else:
220         subprocess.run(cmd_list, check=True)
221
222     return output_file
223

```

Figure 190: Performs network scanning and detects open ports

Nikto_scanner.py

The nikto_scanner.py module performs web vulnerability scanning using the Nikto tool. It identifies potential security issues in web servers such as misconfigurations and outdated software. The module executes scan commands and processes the output into a structured format. The results are then passed to the controller for integration and display in the system.

```
scanners > nikto_scanner.py
1  from utils.ssh_helper import run_ssh_command
2
3
4  def normalize_target_for_nikto(target):
5      if target.startswith("http://") or target.startswith("https://"):
6          return target
7      return f"http://{target}"
8
9
10 def get_nikto_risk(line):
11     text = line.lower()
12
13     high_keywords = [
14         "outdated",
15         "vulnerable",
16         "sql injection",
17         "xss",
18         "remote code execution",
19         "directory indexing",
20         "admin",
21         "cgi-bin",
22         "exposed",
23         "authentication bypass"
24     ]
25
26     medium_keywords = [
27         "allowed http methods",
28         "header is not present",
29         "cookie",
30         "internal ip",
31         "retrieved",
32         "discloses",
33         "interesting",
34         "entry point"
35     ]
36
37     low_keywords = [
38         "server leaks",
39         "uncommon header"
40     ]
41
42     for word in high_keywords:
43         if word in text:
44             return "High"
45
46     for word in medium_keywords:
47         if word in text:
48             return "Medium"
49
50     for word in low_keywords:
51         if word in text:
52             return "Low"
53
54     return "Medium"
```

```
scanners > nikto_scanner.py
10 def get_nikto_risk(line):
54     return "Medium"
55
56
57 def is_real_nikto_finding(line):
58     """
59     Ignore Nikto informational lines and keep real findings only.
60     """
61     lower_line = line.lower()
62
63     ignore_prefixes = [
64         "+ target ip:",
65         "+ target hostname:",
66         "+ target port:",
67         "+ start time:",
68         "+ end time:",
69         "+ host appears to be up",
70         "+ server:",
71         "+ retrieved",
72         "+ the anti-clickjacking",
73         "+ /robots.txt"
74     ]
75
76     for prefix in ignore_prefixes:
77         if lower_line.startswith(prefix):
78             return False
79
80     if "0 host(s) tested" in lower_line:
81         return False
82
83     return line.startswith("+")
84
85
86 def run_nikto_live(target, log_cb, result_cb, summary_cb):
87     target_url = normalize_target_for_nikto(target)
88
89     findings = 0
90     threats = 0
91     hosts = 1
92     server_name = "http"
93
94     cmd = f"nikto -h {target_url}"
95     process = run_ssh_command(cmd)
96
97     log_cb(f"[NIKTO] Starting Nikto scan on {target_url}")
98     summary_cb(hosts, threats, findings, "Running")
99
100     for raw in process.stdout:
101         line = raw.strip()
102         if not line:
103             continue
104
```

```

scanners > nikto_scanner.py
86  def run_nikto_live(target, log_cb, result_cb, summary_cb):
100      for raw in process.stdout:
104
105          log_cb(f"[NIKTO] {line}")
106
107          lower_line = line.lower()
108
109          # Capture server banner separately
110          if lower_line.startswith("+ server:"):
111              server_name = line.split(":", 1)[1].strip() if ":" in line else "http"
112              continue
113
114          # Only keep real findings
115          if is_real_nikto_finding(line):
116              findings += 1
117              risk_level = get_nikto_risk(line)
118
119              if risk_level in {"Medium", "High", "Critical"}:
120                  threats += 1
121
122              result = {
123                  "scanner": "NIKTO",
124                  "host": target,
125                  "port": "80",
126                  "protocol": "tcp",
127                  "state": "open",
128                  "service": server_name,
129                  "risk_level": risk_level,
130                  "finding": line
131              }
132
133              result_cb(result)
134              summary_cb(hosts, threats, findings, "Running")
135
136      return_code = process.wait()
137
138      if return_code == 0:
139          log_cb("[NIKTO] Scan completed.")
140          summary_cb(hosts, threats, findings, "Complete")
141      else:
142          log_cb(f"[NIKTO] Scan failed with code {return_code}")
143          summary_cb(hosts, threats, findings, "Failed")
144

```

Figure 191: Scans web servers for vulnerabilities

Nessus_scanner.py

The nessus_scanner.py module integrates the Nessus vulnerability scanner using its API. It initiates scans, retrieves scan results, and processes vulnerability data including severity levels. The module ensures secure communication with the Nessus server. The processed results are sent to the controller for further analysis and display.

```
scanners > nessus_scanner.py
1  import requests
2  import urllib3
3
4  urllib3.disable_warnings()
5
6  NESSUS_URL = "https://192.168.153.143:8834"
7  USERNAME = "binni"
8  PASSWORD = "binni@2005"
9
10
11 def get_token():
12     response = requests.post(
13         f"{NESSUS_URL}/session",
14         json={"username": USERNAME, "password": PASSWORD},
15         verify=False,
16         timeout=30
17     )
18     response.raise_for_status()
19     return response.json()["token"]
20
21
22 def get_scan_id(headers, scan_name="ThreatQuill Scan"):
23     response = requests.get(
24         f"{NESSUS_URL}/scans",
25         headers=headers,
26         verify=False,
27         timeout=30
28     )
29     response.raise_for_status()
30
31     scans = response.json().get("scans", [])
32     for scan in scans:
33         if scan.get("name") == scan_name:
34             return scan.get("id")
35
36     raise Exception(f"Scan '{scan_name}' not found.")
37
38
39 def get_scan_details(headers, scan_id):
40     response = requests.get(
41         f"{NESSUS_URL}/scans/{scan_id}",
42         headers=headers,
43         verify=False,
44         timeout=30
45     )
46     response.raise_for_status()
47     return response.json()
48
49
50 def map_severity(severity_num):
51     mapping = {
52         4: "Critical",
53         3: "High",
54         2: "Medium".
```

```

scanners > nessus_scanner.py
50 def map_severity(severity_num):
51     mapping = {
52         2: "Medium",
53         1: "Low",
54         0: "Info"
55     }
56     return mapping.get(severity_num, "Info")
57
58 def run_nessus_live(target, log_cb, result_cb, summary_cb):
59     try:
60         log_cb("[NESSUS] Logging in...")
61         token = get_token()
62
63         headers = {
64             "X-Cookie": f"token={token}",
65             "Content-Type": "application/json"
66         }
67
68         log_cb("[NESSUS] Login successful")
69
70         scan_id = get_scan_id(headers, "ThreatQuill Scan")
71         log_cb(f"[NESSUS] Using scan ID: {scan_id}")
72
73         data = get_scan_details(headers, scan_id)
74         status = data.get("info", {}).get("status", "unknown")
75         log_cb(f"[NESSUS] Current status: {status}")
76
77         if status != "completed":
78             log_cb("[NESSUS] Scan is not completed yet. Please wait for completion in Nessus.")
79             summary_cb(1, 0, 0, "Running")
80             return
81
82         vulns = data.get("vulnerabilities", [])
83
84         findings = 0
85         threats = 0
86
87         for vuln in vulns:
88             findings += 1
89             risk_level = map_severity(vuln.get("severity", 0))
90
91             if risk_level in ["Critical", "High", "Medium"]:
92                 threats += 1
93
94             result = {
95                 "scanner": "NESSUS",
96                 "host": target,
97                 "port": "",
98                 "protocol": "tcp",
99                 "state": "open",
100                 "service": vuln.get("plugin_name", "Nessus Finding"),
101                 "risk_level": risk_level,
102                 "finding": vuln.get("plugin_name", "")
103             }
104
105             result_cb(result)
106
107         log_cb(f"[NESSUS] Retrieved {findings} results successfully")
108         summary_cb(1, threats, findings, "Complete")
109
110     except Exception as e:
111         log_cb(f"[NESSUS ERROR] {e}")
112         summary_cb(1, 0, 0, "Failed")
113

```

Figure 192: Integrates Nessus for automated vulnerability scanning

Openvas_scanner.py

The openvas_scanner.py module performs vulnerability scanning using the OpenVAS (GVM) framework. It connects to the OpenVAS service, executes scan tasks, and retrieves vulnerability findings. The module processes the scan results into a structured format. These results are then forwarded to the controller for integration and visualization.

```
scanners > openvas_scanner.py
1
2 import subprocess
3 import json
4 import time
5
6
7 def severity_from_cvss(cvss):
8     try:
9         score = float(cvss)
10    except Exception:
11        return "Info"
12
13    if score >= 9.0:
14        return "Critical"
15    elif score >= 7.0:
16        return "High"
17    elif score >= 4.0:
18        return "Medium"
19    elif score > 0:
20        return "Low"
21    return "Info"
22
23
24 def run_openvas_live(target, log_cb, result_cb, summary_cb):
25     """
26     Runs a real OpenVAS helper script on Kali and reads JSON results line by line.
27     This expects your Kali machine to have a script that launches an OpenVAS task
28     and prints JSON findings to stdout.
29     """
30
31     log_cb("[OPENVAS] Starting real OpenVAS scan...")
32
33     command = [
34         "ssh",
35         "kali@192.168.153.143",
36         f"python3 /home/kali/threatquill/openvas_helper.py {target}"
37     ]
38
39     try:
40         process = subprocess.Popen(
41             command,
42             stdout=subprocess.PIPE,
43             stderr=subprocess.STDOUT,
44             text=True,
45             bufsize=1
46         )
47
48         findings_count = 0
49         high_threats = 0
50
51         for raw_line in process.stdout:
52             line = raw_line.strip()
53             if not line:
54                 continue
```

```

scanners > openvas_scanner.py
51     for raw_line in process.stdout:
52         line = raw_line.strip()
53         if not line:
54             continue
55
56         if line.startswith("[LOG]"):
57             log_cb(line)
58             continue
59
60         try:
61             finding = json.loads(line)
62
63             result = {
64                 "scanner": "OPENVAS",
65                 "target": finding.get("target", target),
66                 "host": finding.get("target", target),
67                 "title": finding.get("title", "OpenVAS Finding"),
68                 "description": finding.get("description", ""),
69                 "port": str(finding.get("port", "")),
70                 "protocol": finding.get("protocol", "tcp"),
71                 "state": "open",
72                 "service": finding.get("service", ""),
73                 "severity": finding.get("severity", severity_from_cvss(finding.get("cvss", 0))),
74                 "cvss": finding.get("cvss", 0),
75                 "cve": finding.get("cve", ""),
76                 "solution": finding.get("solution", "")
77             }
78
79             findings_count += 1
80             if result["severity"] in ["High", "Critical"]:
81                 high_threats += 1
82
83             result_cb(result)
84             summary_cb(1, high_threats, findings_count, "Running")
85
86         except json.JSONDecodeError:
87             log_cb(f"[OPENVAS RAW] {line}")
88
89     process.wait()
90
91     if process.returncode == 0:
92         log_cb("[OPENVAS] Real OpenVAS scan completed.")
93         summary_cb(1, high_threats, findings_count, "Complete")
94     else:
95         log_cb(f"[OPENVAS ERROR] Process exited with code {process.returncode}")
96         summary_cb(1, high_threats, findings_count, "Failed")
97
98 except Exception as e:
99     log_cb(f"[OPENVAS ERROR] {e}")

```

Figure 193: Executes OpenVAS scans on target systems

Openvas_praser.py

The openvas_parser.py module processes raw OpenVAS scan results and extracts key details such as vulnerabilities and severity levels. It converts the data into a structured format for further use in the system.

```
openvas_parser.py
1  # openvas_parser.py
2  import xml.etree.ElementTree as ET
3
4  def parse_openvas_report(file_path):
5      tree = ET.parse(file_path)
6      root = tree.getroot()
7
8      results = []
9
10     for result in root.findall("./result"):
11         data = {
12             "scanner": "OpenVAS",
13             "host": result.findtext("host"),
14             "port": result.findtext("port"),
15             "name": result.findtext("name"),
16             "severity": result.findtext("severity")
17         }
18         results.append(data)
19
20     return results
```

Figure 194: Parses OpenVAS results into structured data

Ssh_helper.py

The ssh_helper.py module enables secure communication between the dashboard and the Kali Linux system using SSH. It is used to remotely execute scanning commands and retrieve outputs.

```
utils > ssh_helper.py
1
2 import paramiko
3
4
5 class SSHProcess:
6     def __init__(self, ssh, stdout, channel):
7         self.ssh = ssh
8         self.stdout = stdout
9         self.channel = channel
10
11     def wait(self):
12         exit_code = self.channel.recv_exit_status()
13         self.ssh.close()
14         return exit_code
15
16
17 def run_ssh_command(command):
18     ssh = paramiko.SSHClient()
19     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
20
21     ssh.connect(
22         hostname="192.168.153.143",
23         username="kali",
24         password="kalii"
25     )
26
27     stdin, stdout, stderr = ssh.exec_command(command)
28     return SSHProcess(ssh, stdout, stdout.channel)
```

Figure 195: Manages secure SSH communication with Kali Linux

Database.py

The database.py module manages storage and retrieval of scan results and system data. It ensures that information is stored in an organized and accessible format.

```

database.py
1  import sqlite3
2
3  DB_NAME = "threatquill.db"
4
5
6  def connect_db():
7      return sqlite3.connect(DB_NAME)
8
9
10 def create_table():
11     conn = connect_db()
12     cursor = conn.cursor()
13
14     cursor.execute("""
15         CREATE TABLE IF NOT EXISTS scan_results (
16             id INTEGER PRIMARY KEY AUTOINCREMENT,
17             scanner TEXT,
18             host TEXT,
19             port TEXT,
20             service TEXT,
21             vulnerability TEXT,
22             severity TEXT,
23             timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
24         )
25     """)
26
27     conn.commit()
28     conn.close()
29
30
31 def insert_result(scanner, host, port, service, vulnerability, severity):
32     conn = connect_db()
33     cursor = conn.cursor()
34
35     cursor.execute("""
36         INSERT INTO scan_results (scanner, host, port, service, vulnerability, severity)
37         VALUES (?, ?, ?, ?, ?, ?)
38     """, (scanner, host, port, service, vulnerability, severity))
39
40     conn.commit()
41     conn.close()
42
43
44 def fetch_all_results():
45     conn = connect_db()
46     cursor = conn.cursor()
47
48     cursor.execute("SELECT scanner, host, port, service, vulnerability, severity, timestamp FROM scan_results")
49     rows = cursor.fetchall()
50
51     conn.close()
52     return rows
53
54
55 def count_by_severity(level):
56     conn = connect_db()
57     cursor = conn.cursor()
58
59     cursor.execute("SELECT COUNT(*) FROM scan_results WHERE severity = ?", (level,))
60     count = cursor.fetchone()[0]
61
62     conn.close()
63     return count
64
65
66 def count_all():
67     conn = connect_db()
68     cursor = conn.cursor()
69
70     cursor.execute("SELECT COUNT(*) FROM scan_results")
71     count = cursor.fetchone()[0]
72
73     conn.close()
74     return count

```

Figure 196: Stores and manages scan results in the database

Report_generator.py

The report_generator.py module generates structured vulnerability reports based on scan results. It formats the data into readable reports such as HTML or JSON for analysis and sharing.

```
report_generator.py
1  import os
2  import json
3  from datetime import datetime
4  from collections import Counter, defaultdict
5
6  def normalize_severity(value):
7      if not value:
8          return "Info"
9
10     value = str(value).strip().title()
11     allowed = {"Critical", "High", "Medium", "Low", "Info"}
12
13     if value in allowed:
14         return value
15
16     return "Info"
17
18
19  def get_severity_counts(results):
20     counter = Counter()
21
22     for item in results:
23         risk = normalize_severity(
24             item.get("risk_level", item.get("severity", "Info"))
25         )
26         counter[risk] += 1
27
28     return counter
29
30
31  def group_findings_by_host(results):
32     host_summary = defaultdict(lambda: {
33         "Critical": 0,
34         "High": 0,
35         "Medium": 0,
36         "Low": 0,
37         "Info": 0,
38         "total": 0
39     })
40
41     for item in results:
42         host = item.get("host", item.get("target", "Unknown"))
43         severity = normalize_severity(
44             item.get("risk_level", item.get("severity", "Info"))
45         )
46
47         host_summary[host][severity] += 1
48         host_summary[host]["total"] += 1
49
50     return dict(host_summary)
51
52
```



```
53 def generate_report_data(target, results):
54     severity_counts = get_severity_counts(results)
55     host_summary = group_findings_by_host(results)
56
57     report_data = {
58         "project_name": "ThreatQuill Analyzer",
59         "report_title": "Network Vulnerability Assessment Report",
60         "target": target,
61         "scan_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
62         "tools_used": sorted(list(set(r.get("scanner", "Unknown") for r in results))),
63         "total_findings": len(results),
64         "total_hosts": len(host_summary),
65         "severity_summary": {
66             "Critical": severity_counts.get("Critical", 0),
67             "High": severity_counts.get("High", 0),
68             "Medium": severity_counts.get("Medium", 0),
69             "Low": severity_counts.get("Low", 0),
70             "Info": severity_counts.get("Info", 0),
71         },
72         "host_summary": host_summary,
73         "findings": results
74     }
75
76     return report_data
77
78
79 def save_json_report(report_data, filename="reports/report.json"):
80     os.makedirs("reports", exist_ok=True)
81
82     with open(filename, "w", encoding="utf-8") as f:
83         json.dump(report_data, f, indent=4)
84
85     return filename
86
87
88 def save_html_report(report_data, filename="reports/report.html"):
89     os.makedirs("reports", exist_ok=True)
90
91     severity = report_data["severity_summary"]
92     host_summary = report_data.get("host_summary", {})
93     findings = report_data.get("findings", [])
94
95     total_hosts = report_data.get("total_hosts", 0)
96     total_findings = report_data.get("total_findings", 0)
97     total_threats = severity["Critical"] + severity["High"]
98
99     tools_html = "".join(f"<li>{tool}</li>" for tool in report_data["tools_used"])
100
```

```

101     host_rows = ""
102     sorted_hosts = sorted(
103         host_summary.items(),
104         key=lambda x: x[1]["total"],
105         reverse=True
106     )
107
108     for host, data in sorted_hosts:
109         critical = data["Critical"]
110         high = data["High"]
111         medium = data["Medium"]
112         low = data["Low"]
113         info = data["Info"]
114         total = max(data["total"], 1)
115
116         critical_w = (critical / total) * 100
117         high_w = (high / total) * 100
118         medium_w = (medium / total) * 100
119         low_w = (low / total) * 100
120         info_w = (info / total) * 100
121
122         host_rows += f"""
123         <tr>
124             <td class="host-cell">{host}</td>
125             <td>
126                 <div class="bar-wrap">
127                     <div class="seg critical" style="width:{critical_w:.2f}%>{critical if critical else ""}</div>
128                     <div class="seg high" style="width:{high_w:.2f}%>{high if high else ""}</div>
129                     <div class="seg medium" style="width:{medium_w:.2f}%>{medium if medium else ""}</div>
130                     <div class="seg low" style="width:{low_w:.2f}%>{low if low else ""}</div>
131                     <div class="seg info" style="width:{info_w:.2f}%>{info if info else ""}</div>
132                 </div>
133             </td>
134         </tr>
135         """
136
137     if not host_rows:
138         host_rows = ""
139         <tr>
140             <td class="host-cell">No hosts available</td>
141             <td>
142                 <div class="bar-wrap empty-bar">No scan data available</div>
143             </td>
144         </tr>
145         """
146
147     detailed_rows = ""
148     for idx, item in enumerate(findings, start=1):
149         scanner = item.get("scanner", "")
150         host = item.get("host", item.get("target", ""))
151         port = item.get("port", "")
152         protocol = item.get("protocol", "")

```

```

report_generator.py
88 def save_html_report(report_data, filename="reports/report.html"):
148     for idx, item in enumerate(findings, start=1):
153         state = item.get("state", "")
154         service = item.get("service", "")
155         risk = normalize_severity(item.get("risk_level", item.get("severity", "Info")))
156         description = item.get("description", "No description available.")
157         recommendation = item.get("recommendation", "Review and secure this service.")
158
159         detailed_rows += f"""
160         <tr>
161             <td>{idx}</td>
162             <td>{scanner}</td>
163             <td>{host}</td>
164             <td>{port}/{protocol}</td>
165             <td>{service}</td>
166             <td>{state}</td>
167             <td>{risk}</td>
168             <td>{description}</td>
169             <td>{recommendation}</td>
170         </tr>
171         """
172
173     if not detailed_rows:
174         detailed_rows = """
175         <tr>
176             <td colspan="9">No detailed findings available.</td>
177         </tr>
178         """
179
180     total_chart = max(
181         severity["Critical"] + severity["High"] + severity["Medium"] + severity["Low"] + severity["Info"],
182         1
183     )
184
185     critical_pct = (severity["Critical"] / total_chart) * 100
186     high_pct = (severity["High"] / total_chart) * 100
187     medium_pct = (severity["Medium"] / total_chart) * 100
188     low_pct = (severity["Low"] / total_chart) * 100
189     info_pct = (severity["Info"] / total_chart) * 100
190
191     s1 = critical_pct
192     s2 = s1 + high_pct
193     s3 = s2 + medium_pct
194     s4 = s3 + low_pct
195
196     donut_style = (
197         "background: conic-gradient("
198         f"#8d1b3d 0% {s1:.2f}%, "
199         f"#e94b4b {s1:.2f}% {s2:.2f}%, "
200         f"#f2993a {s2:.2f}% {s3:.2f}%, "
201         f"#f2c94c {s3:.2f}% {s4:.2f}%, "
202         f"#5aa9e6 {s4:.2f}% 100%"
203         ");"
204     )

```

```
205
206     html = f"""
207 <!DOCTYPE html>
208 <html>
209 <head>
210     <meta charset="utf-8">
211     <title>ThreatQuill Analyzer</title>
212     <style>
213         * {{
214             box-sizing: border-box;
215         }}
216
217         body {{
218             margin: 0;
219             font-family: Arial, sans-serif;
220             background: #f4f5f7;
221             color: #202124;
222         }}
223
224         .topbar {{
225             background: #22384c;
226             color: white;
227             padding: 14px 24px;
228             font-size: 16px;
229             font-weight: bold;
230         }}
231
232         .page {{
233             max-width: 1400px;
234             margin: 0 auto;
235             padding: 22px 24px 30px 24px;
236         }}
237
238         .title {{
239             font-size: 24px;
240             font-weight: bold;
241             margin-bottom: 6px;
242             color: #222;
243         }}
244
245         .subtitle {{
246             font-size: 14px;
247             color: #4b6b88;
248             margin-bottom: 18px;
249         }}
250
251         .summary-strip {{
252             display: grid;
253             grid-template-columns: repeat(4, 1fr);
254             gap: 12px;
255             margin-bottom: 18px;
256         }}
```

```

report_generator.py
209 <head>
212 <style>
257
258 .summary-box {{
259     background: white;
260     border: 1px solid #d9dde3;
261     border-radius: 4px;
262     padding: 14px;
263 }}
264
265 .summary-label {{
266     font-size: 13px;
267     color: #555;
268     margin-bottom: 6px;
269 }}
270
271 .summary-value {{
272     font-size: 24px;
273     font-weight: bold;
274     color: #22384c;
275 }}
276
277 .main-grid {{
278     display: grid;
279     grid-template-columns: minmax(0, 1fr) 320px;
280     gap: 18px;
281     align-items: start;
282 }}
283
284 .card {{
285     background: white;
286     border: 1px solid #d9dde3;
287     border-radius: 4px;
288     overflow: hidden;
289 }}
290
291 .filter-row {{
292     display: flex;
293     align-items: center;
294     gap: 12px;
295     padding: 12px 14px;
296     border-bottom: 1px solid #e6e8ec;
297 }}
298
299 .filter-box {{
300     border: 1px solid #d9dde3;
301     background: white;
302     padding: 8px 10px;
303     border-radius: 4px;
304     font-size: 13px;
305     color: #444;
306 }}
307
308 .host-count {{
309     font-size: 14px;
310     color: #555;
311 }}
312
313 table {{
314     width: 100%;
315     border-collapse: collapse;
316 }}
317
318 .summary-table th,
319 .summary-table td {{
320     border-top: 1px solid #e6e8ec;

```

```
report_generator.py
209 <head>
212 <style>
319 .summary-table td {{
320     border-top: 1px solid #e6e8ec;
321     padding: 14px;
322     text-align: left;
323     vertical-align: middle;
324     font-size: 14px;
325 }}
326
327 .summary-table th {{
328     background: #f8f9fb;
329     font-weight: bold;
330     color: #444;
331 }}
332
333 .host-cell {{
334     width: 220px;
335     white-space: nowrap;
336 }}
337
338 .bar-wrap {{
339     width: 100%;
340     height: 24px;
341     display: flex;
342     overflow: hidden;
343     background: #efefef;
344     border: 1px solid #dfdfff;
345     border-radius: 3px;
346 }}
347
348 .empty-bar {{
349     display: flex;
350     align-items: center;
351     justify-content: center;
352     color: #666;
353     font-size: 13px;
354 }}
355
356 .seg {{
357     height: 100%;
358     display: flex;
359     align-items: center;
360     justify-content: center;
361     font-size: 12px;
362     white-space: nowrap;
363     overflow: hidden;
364 }}
365
366 .critical {{
367     background: #8d1b3d;
368     color: white;
369 }}
370
371 .high {{
372     background: #e94b4b;
373     color: white;
374 }}
375
376 .medium {{
377     background: #f2993a;
378     color: #222;
379 }}
380
```

```
report_generator.py
209 <head>
212 <style>

381 .low {{
382     background: #f2c94c;
383     color: #222;
384 }}
385
386 .info {{
387     background: #5aa9e6;
388     color: #222;
389 }}
390
391 .side-card {{
392     background: white;
393     border: 1px solid #d9dde3;
394     border-radius: 4px;
395     overflow: hidden;
396 }}
397
398 .notice {{
399     margin: 14px;
400     padding: 12px 14px;
401     background: #f7efd1;
402     border: 1px solid #e8dca9;
403     border-radius: 4px;
404     font-size: 13px;
405     line-height: 1.5;
406 }}
407
408 .side-section {{
409     padding: 14px;
410     border-top: 1px solid #eceff3;
411 }}
412
413 .side-title {{
414     font-size: 15px;
415     font-weight: bold;
416     margin-bottom: 12px;
417 }}
418
419 .detail-row {{
420     display: grid;
421     grid-template-columns: 105px 1fr;
422     gap: 8px;
423     margin-bottom: 10px;
424     font-size: 14px;
425 }}
426
427 .detail-label {{
428     color: #555;
429 }}
430
431 .donut-wrap {{
432     display: flex;
433     gap: 16px;
434     align-items: center;
435     margin-top: 10px;
436 }}
437
438 .donut {{
439     width: 126px;
440     height: 126px;
441     border-radius: 50%;
442     position: relative;
443     {donut_style}
```

```

report_generator.py
209 <head>
212 <style>
438 .donut {{
444     flex-shrink: 0;
445 }}
446
447 .donut::after {{
448     content: "";
449     width: 62px;
450     height: 62px;
451     border-radius: 50%;
452     background: white;
453     position: absolute;
454     left: 32px;
455     top: 32px;
456 }}
457
458 .legend-item {{
459     display: flex;
460     align-items: center;
461     gap: 8px;
462     margin-bottom: 8px;
463     font-size: 14px;
464 }}
465
466 .dot {{
467     width: 10px;
468     height: 10px;
469     border-radius: 50%;
470     display: inline-block;
471 }}
472
473 .section-title {{
474     margin: 24px 0 12px 0;
475     font-size: 18px;
476     font-weight: bold;
477 }}
478
479 .detail-table {{
480     background: white;
481     border: 1px solid #d9dde3;
482     border-radius: 4px;
483     overflow: hidden;
484 }}
485
486 .detail-table th,
487 .detail-table td {{
488     border-top: 1px solid #e6e8ec;
489     padding: 10px 12px;
490     text-align: left;
491     vertical-align: top;
492     font-size: 13px;
493 }}
494
495 .detail-table th {{
496     background: #f8f9fb;
497     font-weight: bold;
498 }}
499
500 .footer-note {{
501     margin-top: 18px;
502     background: white;
503     border: 1px solid #d9dde3;
504     border-radius: 4px;
505     padding: 14px;

```



```

report_generator.py
209 <head>
212 <style>
500 .footer-note {{
506     font-size: 14px;
507 }}
508
509 ul {{
510     margin: 6px 0 0 18px;
511 }}
512 </style>
513 </head>
514 <body>
515     <div class="topbar">{report_data["project_name"]}</div>
516
517     <div class="page">
518         <div class="title">{report_data["report_title"]}</div>
519         <div class="subtitle">Target: {report_data["target"]}</div>
520
521         <div class="summary-strip">
522             <div class="summary-box">
523                 <div class="summary-label">Total Hosts</div>
524                 <div class="summary-value">{total_hosts}</div>
525             </div>
526
527             <div class="summary-box">
528                 <div class="summary-label">Total Findings</div>
529                 <div class="summary-value">{total_findings}</div>
530             </div>
531
532             <div class="summary-box">
533                 <div class="summary-label">Top Threats</div>
534                 <div class="summary-value">{total_threats}</div>
535             </div>
536
537             <div class="summary-box">
538                 <div class="summary-label">Status</div>
539                 <div class="summary-value" style="font-size:20px;">Completed</div>
540             </div>
541         </div>
542
543         <div class="main-grid">
544             <div class="card">
545                 <div class="filter-row">
546                     <div class="filter-box">Host Summary</div>
547                     <div class="filter-box">Severity Distribution</div>
548                     <div class="host-count">{total_hosts} Hosts</div>
549                 </div>
550
551                 <table class="summary-table">
552                     <tr>
553                         <th class="host-cell">Host</th>
554                         <th>Vulnerabilities</th>
555                     </tr>
556                     {host_rows}
557                 </table>
558             </div>
559
560             <div class="side-card">
561                 <div class="notice">
562                     <strong>Notice:</strong> This report presents the latest consolidated findings generated by ThreatQuill Analyzer.
563                 </div>
564
565                 <div class="side-section">
566                     <div class="side-title">Assessment Details</div>

```

```

report_generator.py
514 <body>
515   <div class="page">
516     <div class="main-grid">
517       <div class="side-card">
518         <div class="side-section">
519           <div class="detail-row">
520             <div class="detail-label">Target:</div>
521             <div>{report_data["target"]}</div>
522           </div>
523           <div class="detail-row">
524             <div class="detail-label">Status:</div>
525             <div>Completed</div>
526           </div>
527           <div class="detail-row">
528             <div class="detail-label">Modified:</div>
529             <div>{report_data["scan_date"]}</div>
530           </div>
531           <div class="detail-row">
532             <div class="detail-label">Method:</div>
533             <div>Multi-scanner vulnerability assessment</div>
534           </div>
535           <div class="detail-row">
536             <div class="detail-label">Tools:</div>
537             <div><ul>{tools_html}</ul></div>
538           </div>
539         </div>
540       <div class="side-section">
541         <div class="side-title">Vulnerabilities</div>
542         <div class="donut-wrap">
543           <div class="donut"></div>
544           <div>
545             <div class="legend-item"><span class="dot" style="background:#8d1b3d;"></span> Critical ({severity["Critical"]})</div>
546             <div class="legend-item"><span class="dot" style="background:#e94b4b;"></span> High ({severity["High"]})</div>
547             <div class="legend-item"><span class="dot" style="background:#f2993a;"></span> Medium ({severity["Medium"]})</div>
548             <div class="legend-item"><span class="dot" style="background:#f2c94c;"></span> Low ({severity["Low"]})</div>
549             <div class="legend-item"><span class="dot" style="background:#5aa9e6;"></span> Info ({severity["Info"]})</div>
550           </div>
551         </div>
552       </div>
553     </div>
554   </div>
555   <div class="section-title">Detailed Findings</div>
556   <table class="detail-table">
557     <tr>
558       <th>#</th>
559       <th>Scanner</th>
560       <th>Host</th>
561       <th>Port</th>
562       <th>Service</th>
563       <th>State</th>
564       <th>Risk</th>
565       <th>Description</th>
566       <th>Recommendation</th>
567     </tr>
568     {detailed_rows}
569   </table>

```

```

report_generator.py
514 <body>
515   <div class="page">
516     <div class="main-grid">
517       <div class="side-card">
518         <div class="side-section">
519           <div>
520             <div class="legend-item"><span class="dot" style="background:#8d1b3d;"></span> Critical {{severity["Critical"]}}</div>
521             <div class="legend-item"><span class="dot" style="background:#e94b4b;"></span> High {{severity["High"]}}</div>
522             <div class="legend-item"><span class="dot" style="background:#f2993a;"></span> Medium {{severity["Medium"]}}</div>
523             <div class="legend-item"><span class="dot" style="background:#f2c94c;"></span> Low {{severity["Low"]}}</div>
524             <div class="legend-item"><span class="dot" style="background:#5aa9e6;"></span> Info {{severity["Info"]}}</div>
525           </div>
526         </div>
527       </div>
528     </div>
529   </div>
530   <div class="section-title">Detailed Findings</div>
531   <table class="detail-table">
532     <tr>
533       <th>#</th>
534       <th>Scanner</th>
535       <th>Host</th>
536       <th>Port</th>
537       <th>Services</th>
538       <th>State</th>
539       <th>Risk</th>
540       <th>Description</th>
541       <th>Recommendation</th>
542     </tr>
543     {{detailed_rows}}
544   </table>
545   <div class="footer-note">
546     <strong>Conclusion:</strong> Prioritize Critical and High findings first, then continue with Medium and Low issues. This report provides a host-based overview together with detailed vulnerability information.
547   </div>
548 </body>
549 </html>
550 ***
551
552 with open(filename, "w", encoding="utf-8") as f:
553     f.write(html)
554
555 return filename
556

```

Figure 197: Generates structured vulnerability reports

Email_sender.py

The email_sender.py module is responsible for sending generated vulnerability reports to users via email. It securely handles email credentials and ensures successful delivery of reports to the specified recipient.

```

email_sender.py
1  import os
2  import smtplib
3  from email.message import EmailMessage
4
5
6  def send_report_email(sender_email, app_password, recipient_email, report_path):
7      """
8      Send ThreatQuill Analyzer report by email.
9
10     Args:
11         sender_email (str): Fixed sender Gmail address
12         app_password (str): Gmail App Password
13         recipient_email (str): Receiver email address
14         report_path (str): Path to report file
15     """
16
17     if not sender_email:
18         raise ValueError("Sender email is required.")
19
20     if not app_password:
21         raise ValueError("App password is required.")
22
23     if not recipient_email:
24         raise ValueError("Recipient email is required.")
25
26     if not report_path:
27         raise ValueError("Report path is required.")
28
29     if not os.path.exists(report_path):
30         raise FileNotFoundError(f"Report file not found: {report_path}")
31
32     msg = EmailMessage()
33     msg["Subject"] = "ThreatQuill Analyzer Final Report"
34     msg["From"] = sender_email
35     msg["To"] = recipient_email
36     msg.set_content(
37         "Hello,\n\n"
38         "Please find the attached final report generated by ThreatQuill Analyzer.\n\n"
39         "Regards,\n"
40         "ThreatQuill Analyzer"
41     )
42
43     with open(report_path, "rb") as file:
44         file_data = file.read()
45         file_name = os.path.basename(report_path)
46
47     if file_name.lower().endswith(".html"):
48         maintype = "text"
49         subtype = "html"
50     elif file_name.lower().endswith(".json"):
51         maintype = "application"
52         subtype = "json"
53     elif file_name.lower().endswith(".txt"):
54         maintype = "text"
55         subtype = "plain"
56     else:
57         maintype = "application"
58         subtype = "octet-stream"
59
60     msg.add_attachment(
61         file_data,
62         maintype=maintype,
63         subtype=subtype,
64         filename=file_name
65     )

```

```
email_sender.py
6  def send_report_email(sender_email, app_password, recipient_email, report_path):
65
66
67      try:
68          with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
69              smtp.login(sender_email, app_password)
70              smtp.send_message(msg)
71
72              return True, f"Email sent successfully to {recipient_email}"
73
74      except smtplib.SMTPAuthenticationError:
75          raise Exception("Authentication failed. Please check your Gmail App Password.")
76
77      except smtplib.SMTPRecipientsRefused:
78          raise Exception("Recipient email address was refused by the server.")
79
80      except smtplib.SMTPException as e:
81          raise Exception(f"SMTP error occurred: {e}")
82
83      except Exception as e:
84          raise Exception(f"Failed to send email: {e}")
85
```

Figure 198: Sends generated reports via email

7.4.2 Frontend Development

Login.py

The login.py module provides the user authentication interface for the system. It allows users to enter credentials and ensures secure access before opening the main dashboard.

```
login.py
1  import tkinter as tk
2  from tkinter import messagebox
3  import dashboard
4
5  def main():
6      window = tk.Tk()
7      window.title("ThreatQuill Analyzer - Login")
8      window.state("zoomed")
9      window.configure(bg="#f1f5f9")
10
11     username_var = tk.StringVar()
12     password_var = tk.StringVar()
13
14     # ----- title -----
15     title = tk.Label(
16         window,
17         text="ThreatQuill Analyzer",
18         font=("Arial", 26, "bold"),
19         bg="white",
20         fg="#1e293b"
21     )
22     title.pack(pady=40)
23
24     # ----- login frame -----
25     frame = tk.Frame(window, bg="#f1f5f9", padx=30, pady=30)
26     frame.pack()
27
28     # username
29     tk.Label(frame, text="Username", font=("Arial", 12),
30             bg="#f1f5f9", fg="#1e293b").grid(row=0, column=0, sticky="w")
31
32     username_entry = tk.Entry(frame, textvariable=username_var, width=30, font=("Arial", 12))
33     username_entry.grid(row=1, column=0, pady=8)
34
35     # password
36     tk.Label(frame, text="Password", font=("Arial", 12),
37             bg="#f1f5f9", fg="#1e293b").grid(row=2, column=0, sticky="w")
38
39     password_entry = tk.Entry(frame, textvariable=password_var, show="*", width=30, font=("Arial", 12))
40     password_entry.grid(row=3, column=0, pady=8)
41
42     # message
43     message_label = tk.Label(frame, text="", bg="#f1f5f9", fg="red", font=("Arial", 11))
44     message_label.grid(row=4, column=0, pady=5)
45
46     # ----- login function -----
47     def handle_login():
48         user = username_var.get().strip()
49         pwd = password_var.get().strip()
50
51         if user == "" or pwd == "":
52             message_label.config(text="Enter username and password")
53         return
54
```

```

login.py
5   def main():
47   def handle_login():
51       if user == "" or pwd == "":
54           if user == "binni" and pwd == "binnilogin":
55               message_label.config(text="Login successful", fg="green")
56               window.after(500, lambda: open_dashboard(user))
57           else:
58               message_label.config(text="Invalid credentials", fg="red")
59
60   def open_dashboard(user):
61       window.destroy()
62       dashboard.open_dashboard(user)
63
64   # ----- forgot password -----
65   def forgot_password():
66       messagebox.showinfo(
67           "Password Reset",
68           "Please contact the system administrator to reset your password."
69       )
70
71   # ----- buttons -----
72   login_btn = tk.Button(
73       frame,
74       text="Login",
75       command=handle_login,
76       width=20,
77       bg="#2563eb",
78       fg="white",
79       font=("Arial", 12, "bold")
80   )
81   login_btn.grid(row=5, column=0, pady=15)
82
83   forgot_btn = tk.Button(
84       frame,
85       text="Forgot Password?",
86       command=forgot_password,
87       bg="#f1f5f9",
88       fg="#2563eb",
89       bd=0,
90       font=("Arial", 10, "underline"),
91       cursor="hand2"
92   )
93   forgot_btn.grid(row=6, column=0)
94
95   username_entry.focus()
96   window.mainloop()
97
98
99
100  if __name__ == "__main__":
101      main()

```

Figure 199: Handles user authentication and login interface


```

dashboard.py
35 class ThreatQuillDashboard:
57     def _build_styles(self):
61         style.configure(
66             fieldbackground="white",
67             foreground="#111827",
68         )
69         style.configure(
70             "Treeview.Heading",
71             font=("Segoe UI", 10, "bold"),
72             background="#E5E7EB",
73             foreground="#111827",
74             relief="flat",
75         )
76         style.map("Treeview", background=[("selected", "#D8EAFE")])
77
78     def _build_layout(self):
79         self.root.rowconfigure(0, weight=0)
80         self.root.rowconfigure(1, weight=1)
81         self.root.columnconfigure(0, weight=0)
82         self.root.columnconfigure(1, weight=1)
83
84         self.top_bar = tk.Frame(self.root, bg="#071633", height=145)
85         self.top_bar.grid(row=0, column=0, columnspan=2, sticky="nsew")
86         self.top_bar.grid_propagate(False)
87
88         self.sidebar = tk.Frame(self.root, bg="#1E2D45", width=170)
89         self.sidebar.grid(row=1, column=0, sticky="nsew")
90         self.sidebar.grid_propagate(False)
91
92         self.main_area = tk.Frame(self.root, bg="#F3F4F6")
93         self.main_area.grid(row=1, column=1, sticky="nsew", padx=12, pady=12)
94         self.main_area.rowconfigure(0, weight=0)
95         self.main_area.rowconfigure(1, weight=0)
96         self.main_area.rowconfigure(2, weight=1)
97         self.main_area.columnconfigure(0, weight=3)
98         self.main_area.columnconfigure(1, weight=2)
99
100     def _build_topbar(self):
101         left_wrap = tk.Frame(self.top_bar, bg="#071633")
102         left_wrap.pack(side="left", fill="y", padx=18, pady=14)
103
104         logo_frame = tk.Frame(left_wrap, bg="#071633", width=130, height=130)
105         logo_frame.pack(side="left")
106         logo_frame.pack_propagate(False)
107
108         try:
109             logo_path = "logo.png"
110             if not os.path.exists(logo_path):
111                 logo_path = os.path.join("assets", "logo.png")
112
113             self.logo_img = tk.PhotoImage(file=logo_path)
114             self.logo_img = self.logo_img.subsample(3, 3)
115
116             self.logo_label = tk.Label(
117                 logo_frame,
118                 image=self.logo_img,
119                 bg="#071633",
120                 bd=0
121             )
122         except Exception:
123             self.logo_label = tk.Label(
124                 logo_frame,
125                 text="ThreatQuill",
126                 bg="#071633",
127                 fg="white",

```

```

dashboard.py
35 class ThreatQuillDashboard:
100     def _build_topbar(self):
122         except Exception:
123             self.logo_label = tk.Label(
128                 font=("Segoe UI", 14, "bold"),
129             )
130
131             self.logo_label.pack(expand=True)
132
133             title_frame = tk.Frame(left_wrap, bg="#071633")
134             title_frame.pack(side="left", padx=18)
135
136             self.title_label = tk.Label(
137                 title_frame,
138                 text="ThreatQuill Analyzer",
139                 bg="#071633",
140                 fg="white",
141                 font=("Segoe UI", 30, "bold"),
142             )
143             self.title_label.pack(anchor="w", pady=(30, 0))
144
145             right_info = tk.Frame(self.top_bar, bg="#071633")
146             right_info.pack(side="right", padx=24, pady=20)
147
148             self.user_label = tk.Label(
149                 right_info,
150                 text=f"Logged in as: {self.username}",
151                 bg="#071633",
152                 fg="#D1D5DB",
153                 font=("Segoe UI", 11),
154             )
155             self.user_label.pack(anchor="e", pady=(40, 0))
156
157     def _build_sidebar(self):
158         menu_title = tk.Label(
159             self.sidebar,
160             text="MENU",
161             bg="#1E2D45",
162             fg="white",
163             font=("Segoe UI", 14, "bold"),
164         )
165         menu_title.pack(pady=(22, 18))
166
167         self.sidebar_buttons = {}
168
169         for key, text in [
170             ("summary", "Scan Summary"),
171             ("scan", "New Scan"),
172             ("email", "Send Email"),
173             ("reports", "Reports"),
174             ("settings", "Settings"),
175             ("help", "Help"),
176         ]:
177             btn = tk.Button(
178                 self.sidebar,
179                 text=text,
180                 bg="#071633",
181                 fg="white",
182                 activebackground="#2D4263",
183                 activeforeground="white",
184                 relief="flat",
185                 anchor="w",
186                 padx=16,
187                 pady=12,
188                 font=("Segoe UI", 11),

```

```

dashboard.py
35 class ThreatQuillDashboard:
157     def _build_sidebar(self):
176         ]:
177             btn = tk.Button(
189                 cursor="hand2",
190             )
191             btn.pack(fill="x", pady=2, padx=10)
192             self.sidebar_buttons[key] = btn
193
194     def _build_main_content(self):
195         self._build_scan_controls()
196         self._build_summary_card()
197         self._build_results_and_logs()
198
199     def _build_scan_controls(self):
200         controls = tk.Frame(self.main_area, bg="#F3F4F6")
201         controls.grid(row=0, column=0, columnspan=2, sticky="ew", pady=(0, 8))
202         controls.columnconfigure(1, weight=1)
203
204         tk.Label(
205             controls,
206             text="Target:",
207             bg="#F3F4F6",
208             fg="#111827",
209             font=("Segoe UI", 11, "bold"),
210         ).grid(row=0, column=0, padx=(2, 8), pady=4, sticky="w")
211
212         self.target_entry = tk.Entry(
213             controls,
214             font=("Segoe UI", 11),
215             bg="white",
216             fg="#111827",
217             relief="solid",
218             bd=1,
219         )
220         self.target_entry.grid(row=0, column=1, sticky="ew", padx=(0, 10))
221         self.target_entry.insert(0, "scanme.nmap.org")
222
223         self.start_scan_btn = tk.Button(
224             controls,
225             text="Start Scan",
226             bg="#2563E8",
227             fg="white",
228             activebackground="#1D4ED8",
229             activeforeground="white",
230             relief="flat",
231             padx=16,
232             pady=8,
233             font=("Segoe UI", 10, "bold"),
234             cursor="hand2",
235             command=self.start_scan_clicked,
236         )
237         self.start_scan_btn.grid(row=0, column=2, padx=(0, 8))
238
239         self.clear_btn = tk.Button(
240             controls,
241             text="Clear Results",
242             bg="#E5E7EB",
243             fg="#111827",
244             activebackground="#CBD5E1",
245             relief="flat",
246             padx=14,
247             pady=8,
248             font=("Segoe UI", 10, "bold"),
249             cursor="hand2",

```

```

35 class ThreatQuillDashboard:
199     def _build_scan_controls(self):
239         self.clear_btn = tk.Button(
249             cursor="hand2",
250             command=self.clear_results,
251         )
252         self.clear_btn.grid(row=0, column=3)
253
254     def _build_summary_card(self):
255         self.summary_card = tk.Frame(
256             self.main_area,
257             bg="white",
258             highlightthickness=1,
259             highlightbackground="#D1D5DB",
260         )
261         self.summary_card.grid(row=1, column=0, colspan=2, sticky="ew", pady=(0, 10))
262         for i in range(4):
263             self.summary_card.columnconfigure(i, weight=1)
264
265         tk.Label(
266             self.summary_card,
267             text="Live Scan Overview",
268             bg="white",
269             fg="#111827",
270             font=("Segoe UI", 16, "bold"),
271         ).grid(row=0, column=0, colspan=4, sticky="w", padx=18, pady=(12, 6))
272
273         self.hosts_value = self._create_stat(0, "Total Hosts Scanned", "0", "#111827")
274         self.threats_value = self._create_stat(1, "Threats Detected", "0", "#DC2626")
275         self.findings_value = self._create_stat(2, "Total Findings", "0", "#2563EB")
276         self.status_value = self._create_stat(3, "Status", "Idle", "#59669")
277
278     def _create_stat(self, col, label_text, value_text, value_color):
279         tk.Label(
280             self.summary_card,
281             text=label_text,
282             bg="white",
283             fg="#6B7280",
284             font=("Segoe UI", 10),
285         ).grid(row=1, column=col, sticky="w", padx=14, pady=(0, 1))
286
287         value = tk.Label(
288             self.summary_card,
289             text=value_text,
290             bg="white",
291             fg=value_color,
292             font=("Segoe UI", 16, "bold"),
293         )
294         value.grid(row=2, column=col, sticky="w", padx=14, pady=(0, 8))
295         return value
296
297     def _build_results_and_logs(self):
298         results_frame = tk.LabelFrame(
299             self.main_area,
300             text="Latest Scan Results / Report Findings",
301             bg="#F3F4F6",
302             fg="#111827",
303             font=("Segoe UI", 12, "bold"),
304             padx=8,
305             pady=8,
306         )
307         results_frame.grid(row=2, column=0, sticky="nsew", padx=(0, 6))
308         results_frame.rowconfigure(0, weight=1)
309         results_frame.columnconfigure(0, weight=1)
310
311         columns = ("scanner", "host", "port", "proto", "state", "service", "severity")
312         self.tree = ttk.Treeview(results_frame, columns=columns, show="headings", height=16)
313
314         headings = {
315             "scanner": "Scanner",
316             "host": "Host/IP",
317             "port": "Port",
318             "proto": "Proto",
319             "state": "State",
320             "service": "Service",
321             "severity": "Severity",
322         }
323

```

```

dashboard.py
35 class ThreatQuillDashboard:
297 def _build_results_and_logs(self):
334     for col in columns:
335         self.tree.heading(col, text=headings[col])
336         self.tree.column(col, width=widths[col], anchor="center")
337
338     self.tree.grid(row=0, column=0, sticky="nsew")
339
340     results_scroll = ttk.Scrollbar(results_frame, orient="vertical", command=self.tree.yview)
341     self.tree.configure(yscrollcommand=results_scroll.set)
342     results_scroll.grid(row=0, column=1, sticky="ns")
343
344     log_frame = tk.LabelFrame(
345         self.main_area,
346         text="Activity Log",
347         bg="#F3F4F6",
348         fg="#111827",
349         font=("Segoe UI", 12, "bold"),
350         padx=8,
351         pady=8,
352     )
353     log_frame.grid(row=2, column=1, sticky="nsew", padx=(6, 0))
354     log_frame.rowconfigure(0, weight=1)
355     log_frame.columnconfigure(0, weight=1)
356
357     self.log_text = tk.Text(
358         log_frame,
359         bg="white",
360         fg="#111827",
361         font=("Consolas", 10),
362         state="disabled",
363         wrap="word",
364         height=18,
365     )
366     self.log_text.grid(row=0, column=0, sticky="nsew")
367
368     log_scroll = ttk.Scrollbar(log_frame, orient="vertical", command=self.log_text.yview)
369     self.log_text.configure(yscrollcommand=log_scroll.set)
370     log_scroll.grid(row=0, column=1, sticky="ns")
371
372     def _bind_sidebar_actions(self):
373         self.sidebar_buttons["scan"].config(command=self.start_scan_clicked)
374         self.sidebar_buttons["reports"].config(command=self.open_reports_view)
375         self.sidebar_buttons["help"].config(command=self.show_help)
376         self.sidebar_buttons["settings"].config(command=self.show_settings)
377         self.sidebar_buttons["email"].config(command=self.show_email_message)
378         self.sidebar_buttons["summary"].config(command=self.show_scan_summary)
379
380     def append_log(self, message):
381         timestamp = datetime.now().strftime("%H:%M:%S")
382
383         if "[NIKTO]" in message:
384             tag = "NIKTO"
385         elif "[NMAP]" in message:
386             tag = "NMAP"
387         elif "[OPENVAS]" in message:
388             tag = "OPENVAS"
389         elif "[NESSUS]" in message:
390             tag = "NESSUS"
391         elif "ERROR" in message:
392             tag = "ERROR"
393         elif "[SYSTEM]" in message:
394             tag = "SYSTEM"
395         else:
396             tag = "INFO"
397
398         clean_msg = (
399             message.replace("[NMAP]", "")
400             .replace("[NIKTO]", "")
401             .replace("[OPENVAS]", "")
402             .replace("[NESSUS]", "")
403             .replace("[DEBUG]", "")
404             .replace("[SYSTEM]", "")
405             .strip()
406         )
407
408         formatted = f"[{timestamp}] {tag:<8} - {clean_msg}"
409
410         self.log_text.configure(state="normal")

```

```

dashboard.py
35 class ThreatQuillDashboard:
380     def append_log(self, message):
411         self.log_text.insert("end", formatted + "\n")
412         self.log_text.see("end")
413         self.log_text.configure(state="disabled")
414
415     def add_result(self, scanner, host, port, proto, state, service, severity):
416         self.tree.insert(
417             "",
418             "end",
419             values=(scanner, host, port, proto, state, service, severity),
420         )
421
422     def clear_results(self):
423         for item in self.tree.get_children():
424             self.tree.delete(item)
425
426         self.log_text.configure(state="normal")
427         self.log_text.delete("1.0", "end")
428         self.log_text.configure(state="disabled")
429
430         self.latest_report_data = None
431         self.latest_html_path = None
432
433         self.update_summary(0, 0, 0, "Idle")
434
435     def update_summary(self, hosts=None, threats=None, findings=None, status=None):
436         if hosts is not None:
437             self.hosts_value.config(text=str(hosts))
438         if threats is not None:
439             self.threats_value.config(text=str(threats))
440         if findings is not None:
441             self.findings_value.config(text=str(findings))
442         if status is not None:
443             self.status_value.config(text=status)
444
445     def show_report(self, report_data, html_path):
446         self.latest_report_data = report_data
447         self.latest_html_path = html_path
448
449         for item in self.tree.get_children():
450             self.tree.delete(item)
451
452         findings = report_data.get("findings", [])
453
454         if not findings:
455             self.append_log("[ERROR] Report generated but no findings were available.")
456             self.update_summary(
457                 hosts=report_data.get("total_hosts", 0),
458                 threats=0,
459                 findings=0,
460                 status="No Report Data"
461             )
462             return
463
464         for finding in findings:
465             host_value = finding.get("target", finding.get("host", ""))
466             port_value = str(finding.get("port", ""))
467             proto_value = finding.get("protocol", "")
468             state_value = finding.get("state", "")
469             service_value = finding.get("service", "")
470             severity_value = finding.get("severity", finding.get("risk_level", "Info"))
471
472             self.tree.insert(
473                 "",
474                 "end",
475                 values=(
476                     finding.get("scanner", ""),
477                     host_value,
478                     port_value,
479                     proto_value,
480                     state_value,
481                     service_value,
482                     severity_value,
483                 ),
484             )
485
486         counts = report_data.get("severity_summary", {})
487         critical = counts.get("Critical", 0)

```

```

dashboard.py
35 class ThreatQuillDashboard:
445 def show_report(self, report_data, html_path):
488     high = counts.get("High", 0)
489     total_threats = critical + high
490
491     self.update_summary(
492         hosts=report_data.get("total_hosts", 1),
493         threats=total_threats,
494         findings=report_data.get("total_findings", len(findings)),
495         status="Report Ready"
496     )
497
498     self.append_log(f"[SYSTEM] Report generated: {html_path}")
499
500 def start_scan_clicked(self):
501     target = self.target_entry.get().strip()
502     if not target:
503         messagebox.showwarning("Missing Target", "Please enter a target IP or domain.")
504         return
505
506     if not is_valid_target(target):
507         messagebox.showwarning("Invalid Target", "Please enter a valid IP address or domain")
508         return
509
510     self.clean_results()
511     self.append_log(f"[SYSTEM] Live scan requested for target: {target}")
512     self.update_summary(0, 0, 0, "Running")
513
514     self.start_scan_btn.config(state="disabled", text="Scanning...")
515     start_full_scan(target, self)
516
517 def scan_finished(self):
518     self.start_scan_btn.config(state="normal", text="Start Scan")
519     self.append_log("[SYSTEM] Scan process finished.")
520
521 def open_reports_view(self):
522     if self.latest_report_data:
523         self.show_report(self.latest_report_data, self.latest_html_path or "")
524         self.append_log("[SYSTEM] Report loaded in dashboard.")
525
526         if self.latest_html_path and os.path.exists(self.latest_html_path):
527             try:
528                 webbrowser.open("file://" + os.path.abspath(self.latest_html_path))
529             except Exception as e:
530                 self.append_log(f"[ERROR] Could not open report automatically: {e}")
531             return
532
533     report_file = os.path.join("reports", "report.html")
534     full_path = os.path.abspath(report_file)
535
536     if not os.path.exists(full_path):
537         messagebox.showwarning("No Report", f"Report file not found:\n{full_path}")
538         return
539
540     try:
541         os.startfile(full_path)
542         self.append_log(f"[SYSTEM] Opened report: {full_path}")
543     except Exception as e:
544         messagebox.showerror("Error", f"Unable to open report.\n\n{e}")
545
546 def focus_summary(self):
547     self.append_log("[SYSTEM] Summary panel is visible on dashboard.")
548
549 def show_help(self):
550     messagebox.showinfo(
551         "Help",
552         "Enter a target and click Start Scan to run live scanners and generate the report."
553     )
554
555 def show_settings(self):
556     messagebox.showinfo("Settings", "Settings panel can be added next.")
557
558 def show_email_message(self):
559     win = tk.Toplevel(self.root)
560     win.title("Send Final Report")
561     win.geometry("450x280")
562     win.configure(bg="white")
563
564     tk.Label(

```



```

dashboard.py X
dashboard.py
35 class ThreatQuillDashboard:
558     def show_email_message(self):
563
564         tk.Label(
565             win,
566             text="Send Final Report",
567             bg="white",
568             fg="#111827",
569             font=("Segoe UI", 16, "bold")
570         ).pack(pady=12)
571
572         form = tk.Frame(win, bg="white")
573         form.pack(fill="both", expand=True, padx=20, pady=10)
574
575         tk.Label(form, text="Your Gmail:", bg="white").grid(row=0, column=0, sticky="w", pady=8)
576         sender_entry = tk.Entry(form, width=35)
577         sender_entry.grid(row=0, column=1, pady=8)
578         sender_entry.insert(0, "binnishrestha99@gmail.com")
579         sender_entry.config(state="readonly")
580
581         tk.Label(form, text="App Password:", bg="white").grid(row=1, column=0, sticky="w", pady=8)
582         password_entry = tk.Entry(form, width=35, show="*")
583         password_entry.grid(row=1, column=1, pady=8)
584
585         tk.Label(form, text="Recipient Email:", bg="white").grid(row=2, column=0, sticky="w", pady=8)
586         recipient_entry = tk.Entry(form, width=35)
587         recipient_entry.grid(row=2, column=1, pady=8)
588
589         def send_now():
590             sender = "binnishrestha99@gmail.com"
591             password = password_entry.get().strip()
592             recipient = recipient_entry.get().strip()
593
594             if not sender or not password or not recipient:
595                 messagebox.showwarning("Missing Data", "Please fill all fields.")
596                 return
597
598             messagebox.showinfo(
599                 "Email Integration",
600                 "Email sending code is ready to connect.\nUncomment send_report_email part after email_sender.py is added."
601             )
602
603         tk.Button(
604             win,
605             text="Send Report",
606             bg="#2563EB",
607             fg="white",
608             relief="flat",
609             padx=18,
610             pady=8,
611             font=("Segoe UI", 10, "bold"),
612             command=send_now
613         ).pack(pady=12)
614
615         def show_scan_summary(self):
616             summary = ScanSummaryWindow(self.root)
617             summary.open()
618
619         def open_dashboard(username="binni"):
620             root = tk.Tk()
621             app = ThreatQuillDashboard(root, username=username)
622             root.mainloop()
623
624
625
626 if __name__ == "__main__":
627     open_dashboard()

```

Figure 200: Displays scan results, activity logs, and system status

Report_view.py

The report_view.py module displays the generated vulnerability report within the system. It allows users to view detailed scan results in a structured and readable format.

```

report_view.py
1  import tkinter as tk
2  from tkinter import ttk
3  from collections import Counter
4
5  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
6  from matplotlib.figure import Figure
7
8  from database import fetch_all_results
9
10
11 def open_report_window():
12     rows = fetch_all_results()
13
14     win = tk.Toplevel()
15     win.title("ThreatQuill Report")
16     win.state("zoomed")
17     win.configure(bg="#F3F4F6")
18
19     # ----- Top bar -----
20     top = tk.Frame(win, bg="#071633", height=70)
21     top.pack(fill="x")
22     top.pack_propagate(False)
23
24     tk.Label(
25         top,
26         text="ThreatQuill Analyzer - Report View",
27         bg="#071633",
28         fg="white",
29         font=("Segoe UI", 22, "bold"),
30     ).pack(side="left", padx=20, pady=15)
31
32     # ----- Main body -----
33     body = tk.Frame(win, bg="#F3F4F6")
34     body.pack(fill="both", expand=True, padx=12, pady=12)
35
36     body.columnconfigure(0, weight=3)
37     body.columnconfigure(1, weight=1)
38     body.rowconfigure(0, weight=1)
39
40     # ----- Left side: findings table -----
41     left = tk.LabelFrame(
42         body,
43         text="Detailed Findings",
44         bg="#F3F4F6",
45         fg="#111827",
46         font=("Segoe UI", 12, "bold"),
47         padx=8,
48         pady=8,
49     )
50     left.grid(row=0, column=0, sticky="nsew", padx=(0, 8))
51
52     left.rowconfigure(0, weight=1)
53     left.columnconfigure(0, weight=1)
54
55     columns = ("scanner", "host", "port", "proto", "state", "service", "severity")
56     tree = ttk.Treeview(left, columns=columns, show="headings")
57
58     headings = {
59         "scanner": "Scanner",
60         "host": "Host/IP",
61         "port": "Port",
62         "proto": "Proto",
63         "state": "State",
64         "service": "Service",
65         "severity": "Severity",

```

```

report_view.py
11 def open_report_window():
58     headings = {
66     }
67
68     widths = {
69         "scanner": 110,
70         "host": 170,
71         "port": 70,
72         "proto": 70,
73         "state": 90,
74         "service": 200,
75         "severity": 100,
76     }
77
78     for col in columns:
79         tree.heading(col, text=headings[col])
80         tree.column(col, width=widths[col], anchor="center")
81
82     tree.grid(row=0, column=0, sticky="nsew")
83
84     scroll = ttk.Scrollbar(left, orient="vertical", command=tree.yview)
85     tree.configure(yscrollcommand=scroll.set)
86     scroll.grid(row=0, column=1, sticky="ns")
87
88     # ----- Right side: scan details + chart -----
89     right = tk.Frame(body, bg="#F3F4F6")
90     right.grid(row=0, column=1, sticky="nsew")
91     right.columnconfigure(0, weight=1)
92
93     # Prepare summary data
94     total_findings = len(rows)
95     unique_hosts = len(set(r[1] for r in rows)) if rows else 0
96
97     sev_counter = Counter()
98     scanner_counter = Counter()
99
100    for row in rows:
101        scanner, host, port, proto, state, service, severity = row
102        sev_counter[str(severity).strip().title()] += 1
103        scanner_counter[str(scanner).strip().upper()] += 1
104
105    high_count = sev_counter.get("High", 0)
106    medium_count = sev_counter.get("Medium", 0)
107    low_count = sev_counter.get("Low", 0)
108
109    status = "Completed" if total_findings > 0 else "No Data"
110
111    # Scan details panel
112    details = tk.LabelFrame(
113        right,
114        text="Scan Details",
115        bg="white",
116        fg="#111827",
117        font=("Segoe UI", 12, "bold"),
118        padx=12,
119        pady=12,
120    )
121    details.grid(row=0, column=0, sticky="ew", pady=(0, 10))
122
123    detail_items = [
124        ("Status", status),
125        ("Total Hosts", unique_hosts),
126        ("Total Findings", total_findings),
127        ("High", high_count),
128        ("Medium", medium_count),

```

```

report_view.py
11  def open_report_window():
123     detail_items = [
129         ("Low", low_count),
130     ]
131
132     for label, value in detail_items:
133         rowf = tk.Frame(details, bg="white")
134         rowf.pack(fill="x", pady=4)
135
136         tk.Label(
137             rowf,
138             text=f"{label}:",
139             bg="white",
140             fg="#374151",
141             font=("Segoe UI", 10, "bold"),
142             width=14,
143             anchor="w",
144         ).pack(side="left")
145
146         tk.Label(
147             rowf,
148             text=str(value),
149             bg="white",
150             fg="#111827",
151             font=("Segoe UI", 10),
152             anchor="w",
153         ).pack(side="left")
154
155     # Severity chart panel
156     chart_box = tk.LabelFrame(
157         right,
158         text="Severity Summary",
159         bg="white",
160         fg="#111827",
161         font=("Segoe UI", 12, "bold"),
162         padx=10,
163         pady=10,
164     )
165     chart_box.grid(row=1, column=0, sticky="nsew", pady=(0, 10))
166
167     fig = Figure(figsize=(4, 3), dpi=100)
168     ax = fig.add_subplot(111)
169
170     labels = []
171     sizes = []
172
173     if high_count > 0:
174         labels.append("High")
175         sizes.append(high_count)
176     if medium_count > 0:
177         labels.append("Medium")
178         sizes.append(medium_count)
179     if low_count > 0:
180         labels.append("Low")
181         sizes.append(low_count)
182
183     if sizes:
184         wedges, texts = ax.pie(sizes, startangle=90, wedgeprops=dict(width=0.45))
185         ax.set_title("Findings by Severity", fontsize=10)
186     else:
187         ax.text(0.5, 0.5, "No Data", ha="center", va="center")
188         ax.set_title("Findings by Severity", fontsize=10)
189
190     ax.axis("equal")
191

```

```

report_view.py
11 def open_report_window():
191
192     canvas = FigureCanvasTkAgg(fig, master=chart_box)
193     canvas.draw()
194     canvas.get_tk_widget().pack(fill="both", expand=True)
195
196     legend_box = tk.Frame(chart_box, bg="white")
197     legend_box.pack(fill="x", pady=(8, 0))
198
199     for name, count in [("High", high_count), ("Medium", medium_count), ("Low", low_count)]:
200         tk.Label(
201             legend_box,
202             text=f"{name}: {count}",
203             bg="white",
204             fg="#111827",
205             font=("Segoe UI", 10),
206             anchor="w",
207         ).pack(anchor="w")
208
209     # Scanner summary panel
210     scanner_box = tk.LabelFrame(
211         right,
212         text="Scanner Summary",
213         bg="white",
214         fg="#111827",
215         font=("Segoe UI", 12, "bold"),
216         padx=12,
217         pady=12,
218     )
219     scanner_box.grid(row=2, column=0, sticky="ew")
220
221     if scanner_counter:
222         for scanner_name, count in scanner_counter.items():
223             tk.Label(
224                 scanner_box,
225                 text=f"{scanner_name}: {count} findings",
226                 bg="white",
227                 fg="#111827",
228                 font=("Segoe UI", 10),
229                 anchor="w",
230             ).pack(anchor="w", pady=3)
231     else:
232         tk.Label(
233             scanner_box,
234             text="No scanner data available.",
235             bg="white",
236             fg="#687280",
237             font=("Segoe UI", 10),
238         ).pack(anchor="w")
239
240     # Insert rows into table
241     for row in rows:
242         tree.insert("", "end", values=row)
243

```

Figure 201: Shows detailed vulnerability reports to the user

Summary_view.py

The summary_view.py module presents a summarized view of scan results. It shows key information such as total findings, threats, and severity levels for quick analysis.

```
summary_view.py
1  import tkinter as tk
2  from tkinter import messagebox
3  import json
4  import os
5
6
7  class ScanSummaryWindow:
8      def __init__(self, parent):
9          self.parent = parent
10
11      def open(self):
12          report_file = os.path.join("reports", "report.json")
13
14          if not os.path.exists(report_file):
15              messagebox.showwarning("No Data", "Run a scan first.")
16              return
17
18          try:
19              with open(report_file, "r", encoding="utf-8") as f:
20                  data = json.load(f)
21          except Exception as e:
22              messagebox.showerror("Error", f"Unable to load scan summary.\n\n{e}")
23              return
24
25          summary = data.get("severity_summary", {})
26          target = data.get("target", "N/A")
27          scan_date = data.get("scan_date", "N/A")
28          tools_used = ", ".join(data.get("tools_used", [])) or "N/A"
29          total_hosts = data.get("total_hosts", 0)
30          total_findings = data.get("total_findings", 0)
31
32          critical = summary.get("Critical", 0)
33          high = summary.get("High", 0)
34          medium = summary.get("Medium", 0)
35          low = summary.get("Low", 0)
36          info = summary.get("Info", 0)
37
38          total = critical + high + medium + low + info
39          if total == 0:
40              total = 1
41
42          win = tk.Toplevel(self.parent)
43          win.title("ThreatQuill Analyzer - Scan Summary")
44          win.geometry("980x680")
```

```

summary_view.py
7 class ScanSummaryWindow:
11 def open(self):
12     win.geometry("800x600")
45     win.configure(bg="#F3F4F6")
46
47     # Header
48     header = tk.Frame(win, bg="#071633", height=90)
49     header.pack(fill="x")
50     header.pack_propagate(False)
51
52     tk.Label(
53         header,
54         text="ThreatQuill Analyzer",
55         bg="#071633",
56         fg="white",
57         font=("Segoe UI", 24, "bold")
58     ).pack(side="left", padx=24, pady=22)
59
60     # Main container
61     main = tk.Frame(win, bg="#F3F4F6")
62     main.pack(fill="both", expand=True, padx=18, pady=18)
63
64     # Title block
65     tk.Label(
66         main,
67         text="Scan Summary",
68         bg="#F3F4F6",
69         fg="#111827",
70         font=("Segoe UI", 20, "bold")
71     ).pack(anchor="w", pady=(0, 12))
72
73     # Details card
74     details_card = tk.Frame(
75         main,
76         bg="white",
77         highlightthickness=1,
78         highlightbackground="#D1D5DB"
79     )
80     details_card.pack(fill="x", pady=(0, 14))
81
82     info_left = tk.Frame(details_card, bg="white")
83     info_left.pack(side="left", fill="both", expand=True, padx=18, pady=14)
84
85     info_right = tk.Frame(details_card, bg="white")
86     info_right.pack(side="right", fill="both", expand=True, padx=18, pady=14)
87
88     def add_info(parent, label, value):
89         row = tk.Frame(parent, bg="white")
90         row.pack(anchor="w", pady=4, fill="x")
91
92         tk.Label(
93             row,
94             text=label,
95             bg="white",

```

```

summary_view.py
7   class ScanSummaryWindow:
11       def open(self):
88           def add_info(parent, label, value):
92               tk.Label(
96                   fg="#687280",
97                   font=("Segoe UI", 10, "bold"),
98                   width=14,
99                   anchor="w"
100              ).pack(side="left")
101
102              tk.Label(
103                  row,
104                  text=value,
105                  bg="white",
106                  fg="#111827",
107                  font=("Segoe UI", 10),
108                  anchor="w"
109              ).pack(side="left")
110
111          add_info(info_left, "Target:", target)
112          add_info(info_left, "Scan Date:", scan_date)
113          add_info(info_left, "Tools Used:", tools_used)
114
115          add_info(info_right, "Total Hosts:", str(total_hosts))
116          add_info(info_right, "Total Findings:", str(total_findings))
117          add_info(info_right, "Status:", "Completed")
118
119          # Severity summary cards
120          sev_row = tk.Frame(main, bg="#F3F4F6")
121          sev_row.pack(fill="x", pady=(0, 14))
122
123          def severity_card(parent, title, value, color):
124              card = tk.Frame(
125                  parent,
126                  bg="white",
127                  highlightthickness=1,
128                  highlightbackground="#D1D5DB"
129              )
130              card.pack(side="left", fill="both", expand=True, padx=5)
131
132              tk.Label(
133                  card,
134                  text=title,
135                  bg="white",
136                  fg="#687280",
137                  font=("Segoe UI", 10, "bold")
138              ).pack(pady=(12, 4))
139
140              tk.Label(
141                  card,
142                  text=str(value),
143                  bg="white",
144                  fg=color,
145                  font=("Segoe UI", 20, "bold")
146              ).pack(pady=(0, 12))
147
148          severity_card(sev_row, "Critical", critical, "#8d1b3d")
149          severity_card(sev_row, "High", high, "#e94b4b")
150          severity_card(sev_row, "Medium", medium, "#f2993a")
151          severity_card(sev_row, "Low", low, "#c59b00")
152          severity_card(sev_row, "Info", info, "#2563e8")
153
154          # Bottom layout
155          bottom = tk.Frame(main, bg="#F3F4F6")
156          bottom.pack(fill="both", expand=True)

```



```

summary_view.py
7 class ScanSummaryWindow:
11     def open(self):

157
158         chart_card = tk.Frame(
159             bottom,
160             bg="white",
161             highlightthickness=1,
162             highlightbackground="#D1D5DB"
163         )
164         chart_card.pack(side="left", fill="both", expand=True, padx=(0, 8))
165
166         details_panel = tk.Frame(
167             bottom,
168             bg="white",
169             highlightthickness=1,
170             highlightbackground="#D1D5DB"
171         )
172         details_panel.pack(side="right", fill="both", expand=True, padx=(8, 0))
173
174         tk.Label(
175             chart_card,
176             text="Severity Distribution",
177             bg="white",
178             fg="#111827",
179             font=("Segoe UI", 14, "bold")
180         ).pack(anchor="w", padx=18, pady=(16, 8))
181
182         canvas = tk.Canvas(chart_card, width=320, height=260, bg="white", highlightthickness=0)
183         canvas.pack(pady=10)
184
185         # Pie chart
186         cx, cy, r = 140, 130, 90
187         start = 0
188         slices = [
189             ("Critical", critical, "#8d1b3d"),
190             ("High", high, "#e94b4b"),
191             ("Medium", medium, "#f2993a"),
192             ("Low", low, "#f2c94c"),
193             ("Info", info, "#5aa9e6"),
194         ]
195
196         for _, value, color in slices:
197             if value <= 0:
198                 continue
199             extent = (value / total) * 360
200             canvas.create_arc(
201                 cx - r, cy - r, cx + r, cy + r,
202                 start=start,
203                 extent=extent,
204                 fill=color,
205                 outline="white",
206                 width=2
207             )
208             start += extent
209
210         # inner circle
211         canvas.create_oval(cx - 42, cy - 42, cx + 42, cy + 42, fill="white", outline="white")
212         canvas.create_text(cx, cy - 8, text="Total", font=("Segoe UI", 11, "bold"), fill="#485563")
213         canvas.create_text(cx, cy + 14, text=str(total_findings), font=("Segoe UI", 16, "bold"), fill="#111827")
214
215         # legend
216         legend_frame = tk.Frame(chart_card, bg="white")
217         legend_frame.pack(anchor="w", padx=18, pady=(4, 16))
218
219         def legend_item(parent, label, value, color):

```

```

summary_view.py
7   class ScanSummaryWindow:
11   def open(self):
219   def legend_item(parent, label, value, color):
220       row = tk.Frame(parent, bg="white")
221       row.pack(anchor="w", pady=4)
222
223       dot = tk.Canvas(row, width=14, height=14, bg="white", highlightthickness=0)
224       dot.pack(side="left")
225       dot.create_oval(2, 2, 12, 12, fill=color, outline=color)
226
227       tk.Label(
228           row,
229           text=f"{label}: {value}",
230           bg="white",
231           fg="#111827",
232           font=("Segoe UI", 10)
233       ).pack(side="left", padx=8)
234
235       legend_item(legend_frame, "Critical", critical, "#8d1b3d")
236       legend_item(legend_frame, "High", high, "#e94b4b")
237       legend_item(legend_frame, "Medium", medium, "#f2993a")
238       legend_item(legend_frame, "Low", low, "#f2c94c")
239       legend_item(legend_frame, "Info", info, "#5aa9e6")
240
241   # right side details
242   tk.Label(
243       details_panel,
244       text="Assessment Details",
245       bg="white",
246       fg="#111827",
247       font=("Segoe UI", 14, "bold")
248   ).pack(anchor="w", padx=18, pady=(16, 12))
249
250   add_box = tk.Frame(details_panel, bg="white")
251   add_box.pack(fill="x", padx=18)
252
253   def detail_line(parent, label, value):
254       row = tk.Frame(parent, bg="white")
255       row.pack(fill="x", pady=6)
256
257       tk.Label(
258           row,
259           text=label,
260           bg="white",
261           fg="#6B7280",
262           font=("Segoe UI", 10, "bold"),
263           width=14,
264           anchor="w"
265       ).pack(side="left")
266
267       tk.Label(
268           row,
269           text=value,
270           bg="white",
271           fg="#111827",
272           font=("Segoe UI", 10),
273           anchor="w",
274           justify="left",
275           wraplength=280
276       ).pack(side="left")
277
278       detail_line(add_box, "Target:", target)
279       detail_line(add_box, "Status:", "Completed")
280       detail_line(add_box, "Modified:", scan_date)
281       detail_line(add_box, "Method:", "Multi-scanner vulnerability assessment")

```

```

summary_view.py
7  class ScanSummaryWindow:
11      def open(self):
253      def detail_line(parent, label, value):
277
278      detail_line(add_box, "Target:", target)
279      detail_line(add_box, "Status:", "Completed")
280      detail_line(add_box, "Modified:", scan_date)
281      detail_line(add_box, "Method:", "Multi-scanner vulnerability assessment")
282      detail_line(add_box, "Threat Level:", str(critical + high))
283
284      notice = tk.Frame(
285          details_panel,
286          bg="#F9F2D8",
287          highlightthickness=1,
288          highlightbackground="#E5D9A8"
289      )
290      notice.pack(fill="x", padx=18, pady=(18, 0))
291
292      tk.Label(
293          notice,
294          text="Notice: This window shows a quick scan summary. Use the Reports section for the full detailed report.",
295          bg="#F9F2D8",
296          fg="#374151",
297          font=("Segoe UI", 10),
298          wraplength=300,
299          justify="left"
300      ).pack(anchor="w", padx=12, pady=12)
301

```

Figure 202: Displays summary of scan results and severity levels

Main.py

The main.py module acts as the entry point of the system. It initializes the application and launches the dashboard interface for user interaction.

```

dashboard.py  main.py  X
main.py
1  import dashboard
2
3  if __name__ == "__main__":
4      dashboard.main()

```

Figure 203: Initializes and runs the ThreatQuill Analyzer system

7.5 Appendix D: Designs

7.5.1 Work Breakdown Structure

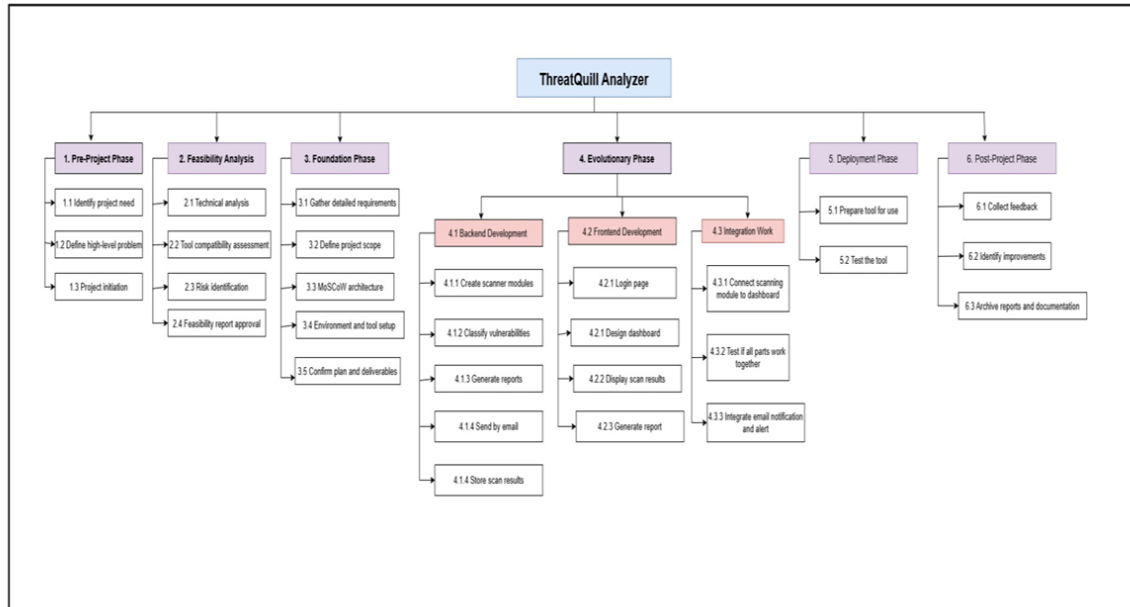


Figure 204: WBS

7.5.2 Gantt Chart

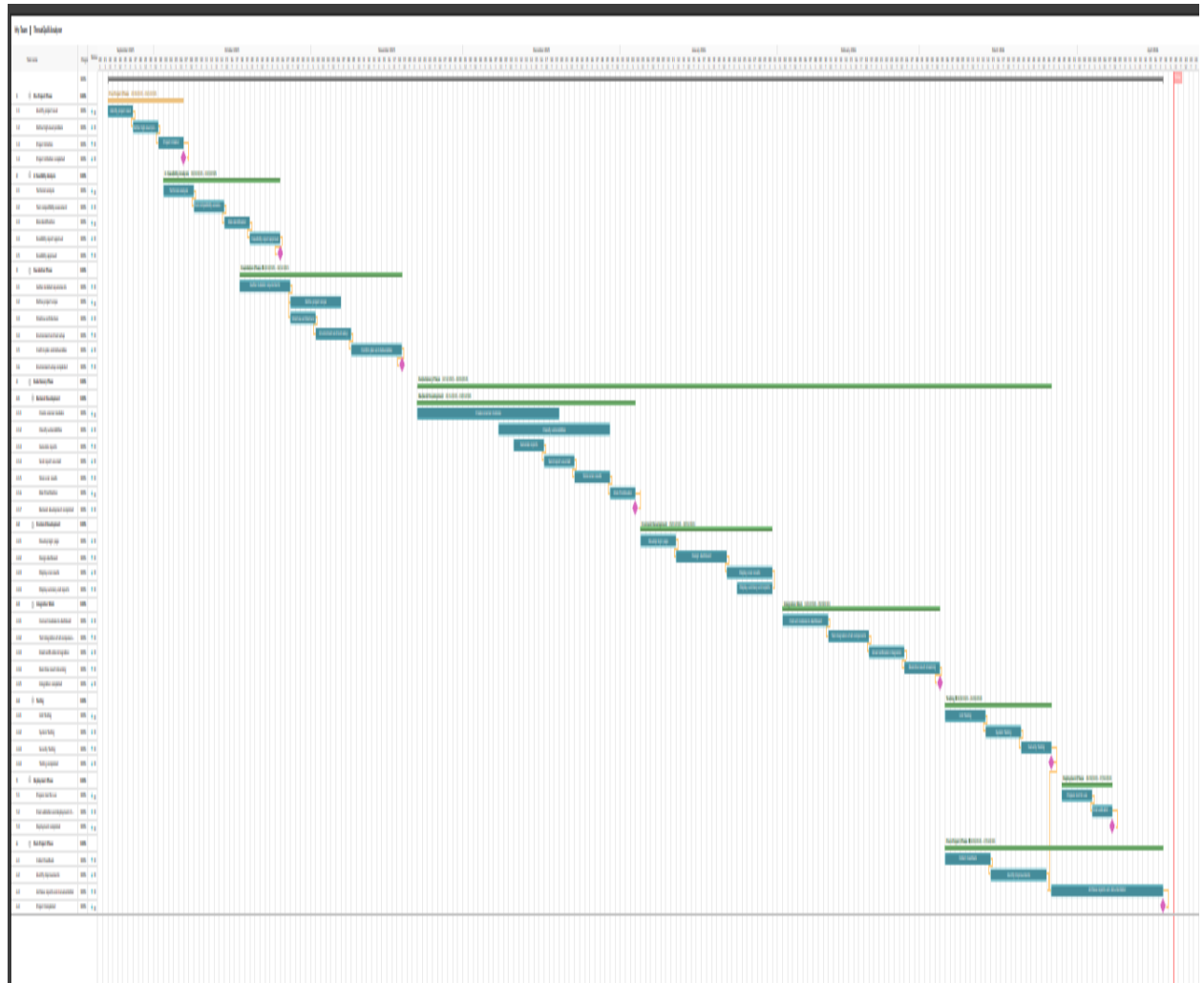


Figure 205: Gantt Chart

For more information [Gantt Chart](#)

7.5.3 Milestone



Figure 206: Project Milestone

7.5.4 Mind Map

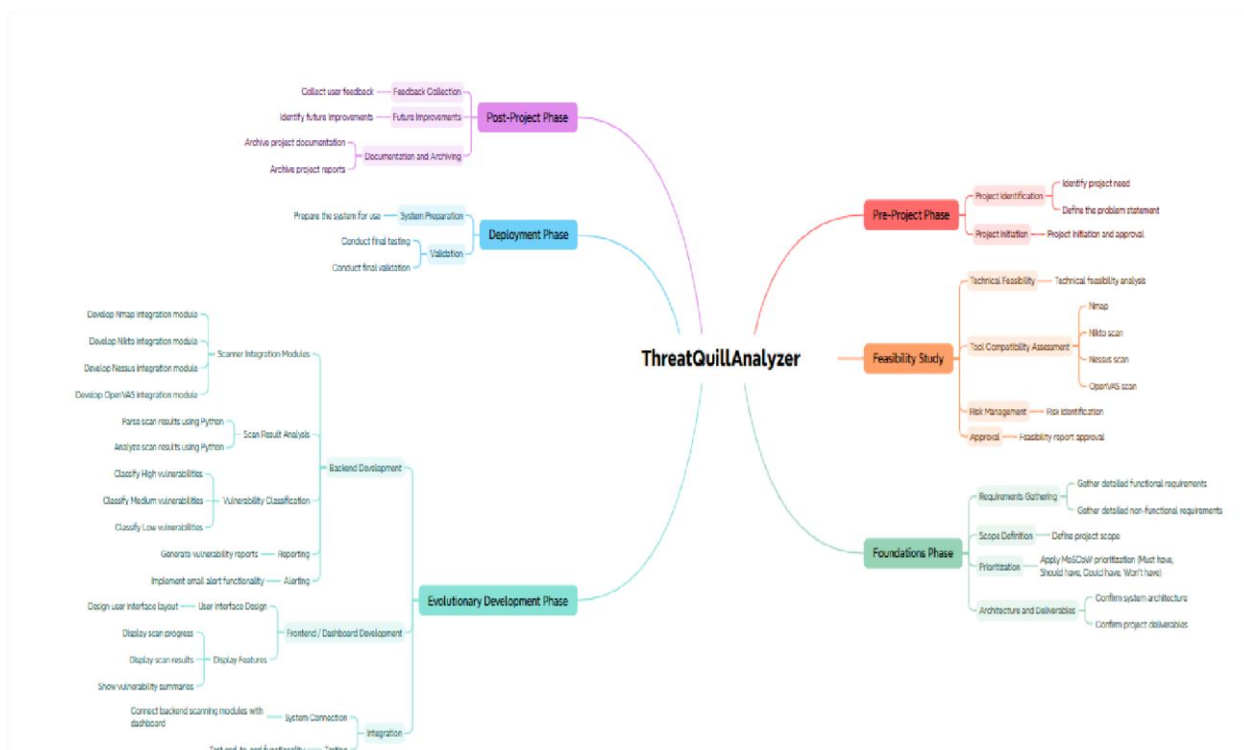


Figure 207: Mind Map

7.6 Appendix E: Screenshot of the System

The login interface enables users to securely access the ThreatQuill Analyzer system by entering valid credentials. It ensures controlled access to the system and provides a simple and user-friendly authentication mechanism.

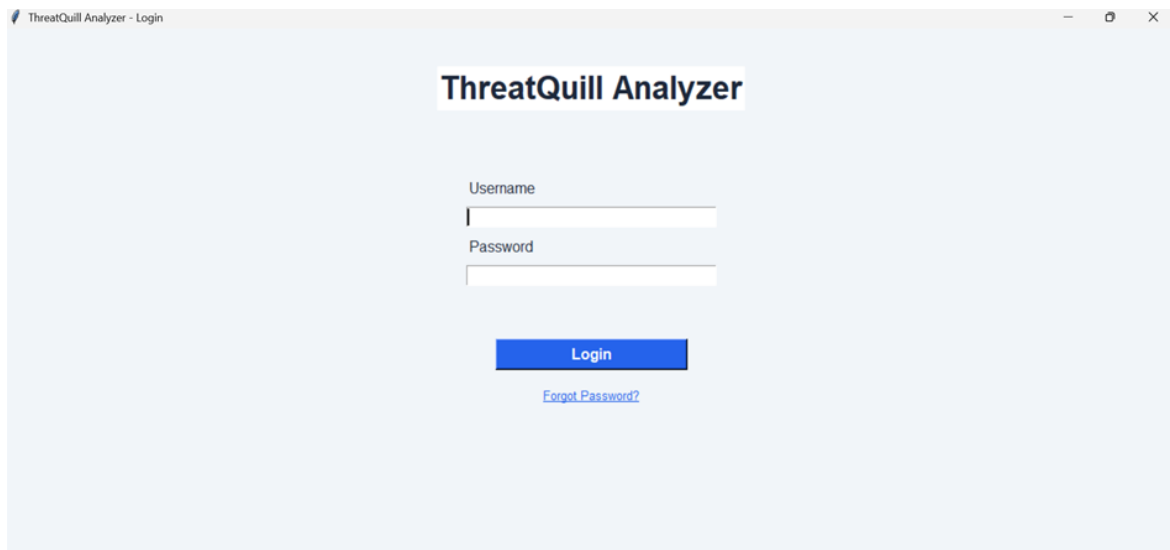


Figure 208: Login Page

The dashboard interface is displayed after successful login and provides options to initiate vulnerability scans. It includes sections for scan overview, results table, and activity logs, allowing users to monitor system status in real time.

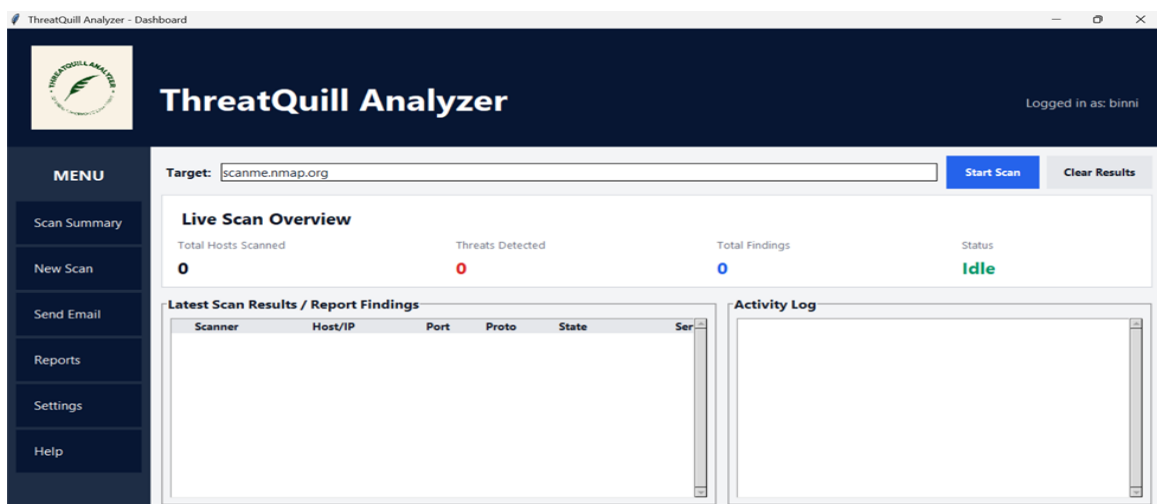


Figure 209: login after Dashboard Page

The dashboard displays the results after scan execution, including detected vulnerabilities, host information, and detailed activity logs. It provides real-time updates and a structured view of scan outputs from multiple integrated scanners.

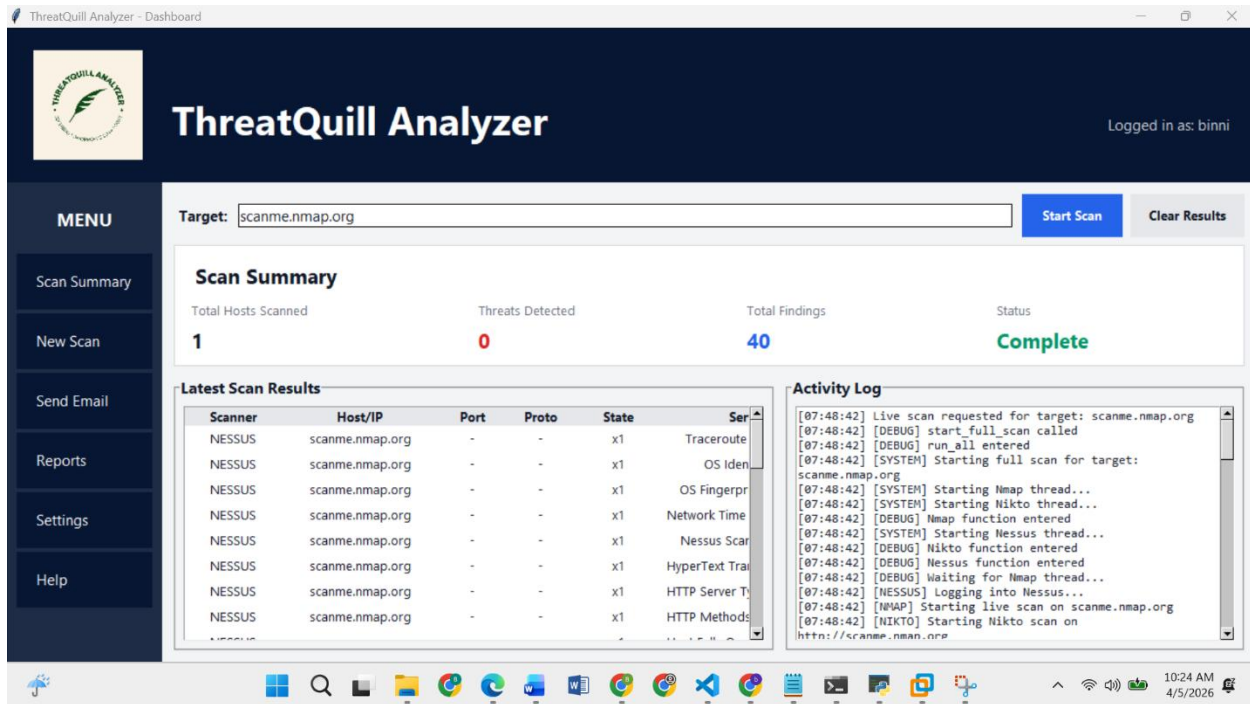


Figure 210: Dashboard after scanning completed

The report interface presents consolidated vulnerability results in a structured format. It includes detailed findings, severity levels, and host-based analysis, enabling users to review and understand security risks effectively.

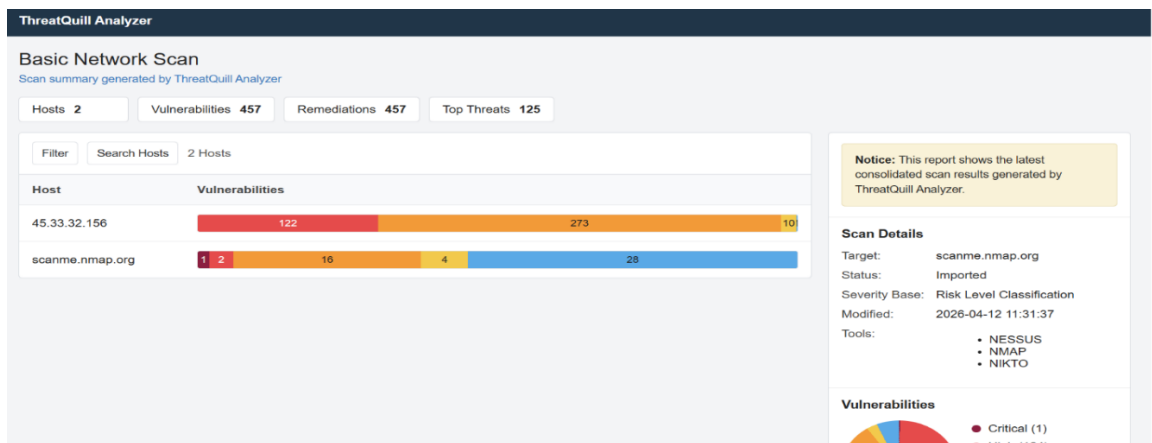


Figure 211: Report opened

The scan summary interface provides an overview of the completed scan, including total hosts, vulnerabilities, and severity distribution. It visually represents risk levels and helps with quick assessment of the overall system security status.

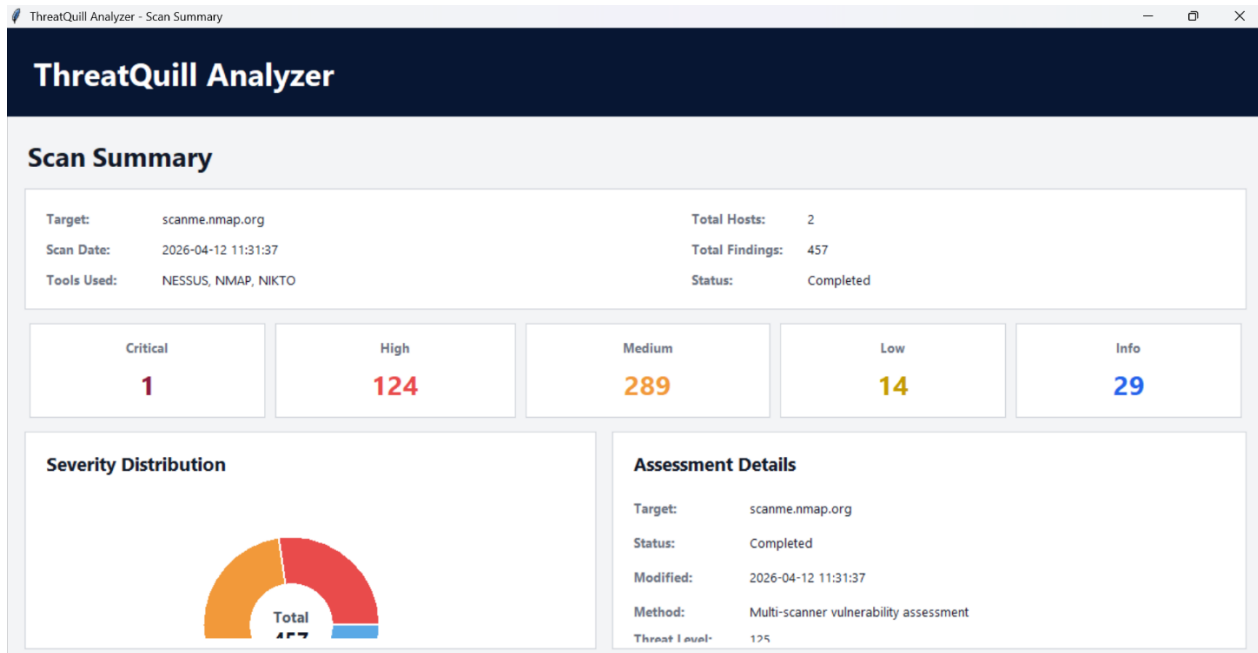


Figure 212: Scan Summary

The email interface allows users to send the generated vulnerability report directly to a recipient. It requires sender credentials and supports secure sharing of scan results for further analysis or documentation.

The screenshot shows the 'Send Final Report' window. It contains the following fields and a button:

- Your Gmail:** binnishrestha99@gmail.com
- App Password:** [Empty field]
- Recipient Email:** [Empty field]
- Send Report** (Blue button)

Figure 213: Email sender